

Εθνικο Μετσοβιο Πολυτεχνειο
Σχολη Ηλεκτρολογων Μηχανικων και Μηχανικων Υπολογιστων
Τομεας Ηλεκτρικων Βιομηχανικων Διαταξεων και Συστηματων Αποφασεων

Δημιουργία ενός κατανεμημένου συστήματος με χρήση Kubernetes, Kafka και Datalake για τη διαχείριση δεδομένων από IoT συσκευές

Διπλωματικη Εργασια

του

ΘΕΟΔΩΡΟΥ ΜΠΕΡΕΤΣΟΥ

Επιβλέπων: Ευάγγελος Μαρινάκης
Επίκουρος Καθηγητής ΕΜΠ

Αθήνα, Ιούλιος 2026



Εθνικο Μετσοβιο Πολυτεχνειο
Σχολη Ηλεκτρολογων Μηχανικων και Μηχανικων Υπολογιστων
Τομεας Ηλεκτρικων Βιομηχανικων Διαταξεων και Συστηματων Αποφασεων

Δημιουργία ενός κατανεμημένου συστήματος με χρήση Kubernetes, Kafka και Datalake για τη διαχείριση δεδομένων από IoT συσκευές

Διπλωματικη Εργασια

του

ΘΕΟΔΩΡΟΥ ΜΠΕΡΕΤΣΟΥ

Επιβλέπων: Ευάγγελος Μαρινάκης
Επίκουρος Καθηγητής ΕΜΠ

Εγκρίθηκε από την τριμελή εξεταστική επιτροπή την 9 Ιουλίου 2026.

(Υπογραφή)	*(Υπογραφή)*	*(Υπογραφή)*
.....	.....	.....
Ευάγγελος Μαρινάκης	Δημήτριος Ασκούνης	Ευάγγελος Σπηλιώτης
Επίκουρος Καθηγητής ΕΜΠ	Καθηγητής ΕΜΠ	Επίκουρος Καθηγητής ΕΜΠ

Αθήνα, Ιούλιος 2026



Εθνικο Μετσοβιο Πολυτεχνειο
Σχολη Ηλεκτρολογων Μηχανικων και Μηχανικων Υπολογιστων
Τομεας Ηλεκτρικων Βιομηχανικων Διαταξεων και Συστηματων Αποφασεων

Θεόδωρος Μπερέτσος

Διπλωματούχος Ηλεκτρολόγος Μηχανικός και Μηχανικός Υπολογιστών ΕΜΠ
Εργαστήριο Συστημάτων Αποφάσεων και Διοίκησης ΕΜΠ

Σημείωμα Πνευματικής Ιδιοκτησίας

Copyright © Θεόδωρος Γ. Μπερέτσος, 2026
Με επιφύλαξη παντός δικαιώματος. All rights reserved.

Κανένα μέρος της παρούσας Διπλωματικής Εργασίας δεν επιτρέπεται να αναπαραχθεί, αποθηκευτεί ή διανεμηθεί για εμπορικούς σκοπούς χωρίς την προηγούμενη έγγραφη άδεια του συγγραφέα. Η ανατύπωση, αποθήκευση ή διανομή για μη κερδοσκοπικούς, εκπαιδευτικούς ή ερευνητικούς σκοπούς επιτρέπεται, υπό την προϋπόθεση ότι αναφέρεται η πηγή προέλευσης και διατηρείται το παρόν σημείωμα. Ερωτήματα σχετικά με εμπορική χρήση αυτής της Διπλωματικής Εργασίας θα πρέπει να απευθύνονται στον συγγραφέα.

Οι απόψεις και τα συμπεράσματα που εκφράζονται στο παρόν έγγραφο ανήκουν αποκλειστικά στον συγγραφέα και δεν θα πρέπει να ερμηνεύονται ως εκπροσωπώντας τις επίσημες θέσεις του Εθνικού Μετσόβιου Πολυτεχνείου.

στους γονείς και την αδελφή μου

Περίληψη

Η παρούσα διπλωματική εργασία πραγματεύεται το σχεδιασμό, την υλοποίηση και την πειραματική αξιολόγηση μιας κατανεμημένης υποδομής διαχείρισης τηλεμετρικών δεδομένων από διασυνδεδεμένες συσκευές μέσω Internet of Things (IoT) σε περιβάλλον Kubernetes, ακολουθώντας αρχιτεκτονική τύπου 'Data Lakehouse'. Η ταχεία εξάπλωση των συσκευών IoT δημιουργεί συνεχείς ροές δεδομένων που απαιτούν κλιμακούμενη επεξεργασία σε πραγματικό χρόνο, ιστορική αποθήκευση και αποδοτική ανάλυση αυτών, απαιτήσεις που δύσκολα ικανοποιούνται από παραδοσιακές μονολιθικές αρχιτεκτονικές.

Στο θεωρητικό μέρος εξετάζεται η μετάβαση από την κλασική εικονικοποίηση έως τα containers και τα συστήματα ενορχήστρωσης containers όπως το Kubernetes, καθώς και οι βασικές τεχνολογίες ροών γεγονότων (Apache Kafka, Apache Flink) και αποθήκευσης τύπου data lake (Apache Paimon). Παρουσιάζεται επίσης η μηχανή κατανεμημένων ερωτημάτων Trino, η στοίβα παρακολούθησης και παρατηρησιμότητας Prometheus και η πλατφόρμα οπτικοποίησης Grafana.

Στο πρακτικό μέρος περιγράφονται λεπτομερώς ο αρχιτεκτονικός σχεδιασμός της υποδομής στη δημόσια νεφοϋπολογιστική πλατφόρμα Microsoft Azure και η υλοποίηση αυτής μέσω υποδομής ως κώδικα (Infrastructure as Code). Επίσης παρουσιάζεται ολόκληρη η αλυσίδα ροής δεδομένων, η οποία περιλαμβάνει την προσομοίωση τηλεμετρίας, την εισαγωγή των δεδομένων σε Kafka topics, την επεξεργασία των δεδομένων μέσω Flink jobs, την μόνιμη αποθήκευση σε πίνακες Paimon, τα αναλυτικά ερωτήματα SQL μέσω Trino, την παρακολούθηση της υποδομής μέσω Prometheus και την οπτικοποίηση των δεδομένων μέσω Grafana dashboards. Η αξιολόγηση πραγματοποιείται με σενάρια κλιμακούμενου φόρτου, από 250 έως 87.500 μηνύματα ανά δευτερόλεπτο, μετρώντας ρυθμό μηνυμάτων, καθυστέρηση από άκρο σε άκρο, χρόνους ερωτημάτων, κατανάλωση πόρων (επεξεργαστή, μνήμης, δικτύου, αποθηκευτικού χώρου) και κόστος λειτουργίας. Τέλος, αναλύονται στρατηγικές βελτιστοποίησης πόρων, αυτόματης κλιμάκωσης και λειτουργικές οδηγίες, ενώ εντοπίζονται τα πρακτικά όρια κλιμάκωσης και οι περιορισμοί του συστήματος.

Λέξεις Κλειδιά

Internet of Things, IoT, Kubernetes, Data Lakehouse, Cloud, Microsoft Azure, Apache Kafka, Apache Flink, Apache Paimon, Trino, Prometheus, Grafana, Ροές Δεδομένων, Ροές Γεγονότων, Επεξεργασία Ροών, Παρατηρησιμότητα, Υποδομή ως Κώδικας, Κλιμάκωση, Βελτιστοποίηση Πόρων, Βελτιστοποίηση Κόστους

Abstract

This diploma thesis addresses the design, implementation, and experimental evaluation of a distributed IoT telemetry management infrastructure on Kubernetes, following a Data Lakehouse architecture. The rapid proliferation of connected devices generates continuous data streams that demand scalable real-time processing, long-term storage, and efficient analytics, all of which traditional monolithic architectures struggle to provide.

The theoretical part examines the evolution from classical virtualization to containers and orchestration systems (Kubernetes), as well as the core event-streaming technologies (Apache Kafka, Apache Flink) and data lake storage (Apache Paimon). The distributed query engine Trino, the monitoring and observability stack Prometheus, and the visualization platform Grafana are also presented.

The practical part describes in detail the architectural design of the cloud infrastructure on Microsoft Azure and the implementation via Infrastructure as Code. The full data pipeline is also presented, encompassing IoT telemetry simulation, ingestion into Kafka topics, data processing via Flink jobs, persistent storage in Paimon tables, SQL analytics via Trino, infrastructure monitoring via Prometheus, and data visualization through Grafana dashboards. The evaluation is conducted with scalable load scenarios (from 250 to 87,500 messages per second), measuring message throughput, end-to-end latency, query response times, resource consumption (CPU, memory, network, storage), and operational cost. Finally, resource optimization strategies, auto-scaling approaches, and operational guidelines are analysed, while the practical scaling limits and system bottlenecks are identified.

Keywords

Internet of Things, IoT, Kubernetes, Data Lakehouse, Cloud, Microsoft Azure, Apache Kafka, Apache Flink, Apache Paimon, Trino, Prometheus, Grafana, Data Streaming, Event Streaming, Stream Processing, Observability, Infrastructure as Code, Scalability, Resource Optimization, Cost Optimization

Ευχαριστίες

Η παρούσα διπλωματική εργασία σηματοδοτεί για μένα το κλείσιμο μιας μακράς και ιδιαίτερης διαδρομής στη Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών του Εθνικού Μετσόβιου Πολυτεχνείου. Η πορεία αυτή δεν υπήρξε ευθύγραμμη αλλά αντίθετα περιλάμβανε παύσεις, δυσκολίες, αλλαγές και περιόδους επαναπροσδιορισμού. Για αυτόν ακριβώς τον λόγο, η ολοκλήρωση της παρούσας εργασίας έχει για μένα ιδιαίτερη σημασία, καθώς αποτελεί όχι μόνο την κατάληξη των σπουδών μου, αλλά και μια προσωπική δικαίωση ύστερα από μια μακρόχρονη προσπάθεια.

Θα ήθελα αρχικά να ευχαριστήσω θερμά τον καθηγητή Βαγγέλη Μαρινάκη για την εμπιστοσύνη του και για την ευκαιρία που μου έδωσε να εκπονήσω την εργασία αυτή στο εργαστήριο.

Ιδιαίτερες ευχαριστίες οφείλω στον Φίλιππο Σερέπα για τη στήριξη, την καθοδήγηση και κυρίως για την υπομονή του καθ' όλη τη διάρκεια της εργασίας. Η συμβολή του ήταν καθοριστική. Ευχαριστώ επίσης θερμά τον Γιάννη Παπία για τη βοήθεια και την καθοδήγησή του, που συνέβαλαν ουσιαστικά στην ολοκλήρωση της προσπάθειας αυτής.

Ευχαριστώ ακόμη τον Βαγγέλη Κούκη από το CSLab καθώς και όσους προϊσταμένους είχα στη μέχρι τώρα επαγγελματική μου καριέρα, γιατί υπήρξαν από τους ανθρώπους που με βοήθησαν ουσιαστικά να χτίσω τις βάσεις, και να διαμορφώσω τον τεχνικό τρόπο σκέψης με τον οποίο προσεγγίζω σήμερα τα συστήματα υπολογιστών και τις νέες τεχνολογίες.

Παράλληλα, νιώθω ιδιαίτερα ευγνώμων για όσα μου προσέφερε αυτή η σχολή όλα αυτά τα χρόνια, όχι μόνο σε επίπεδο γνώσεων, αλλά και μέσα από τους ανθρώπους, τις εμπειρίες, τις αξίες και τις οπτικές που διαμόρφωσαν τον χαρακτήρα μου και την επαγγελματική μου πορεία. Μέρος αυτής της διαδρομής με οδήγησε και στον επαγγελματικό δρόμο που ακολουθώ σήμερα.

Τέλος, θα ήθελα να εκφράσω την πιο βαθιά μου ευγνωμοσύνη προς την οικογένειά μου, και ιδιαίτερα προς τους γονείς μου και την αδελφή μου, για την αδιάκοπη στήριξη, την αγάπη, την κατανόηση και την πίστη τους σε μένα όλα αυτά τα χρόνια. Η παρουσία τους ήταν σταθερή και πολύτιμη σε κάθε στάδιο αυτής της πορείας, και σε αυτούς αφιερώνω την παρούσα εργασία.

Αθήνα, Ιούλιος 2026

Θεόδωρος Μπερέτσος

Περιεχόμενα

Περίληψη **7**

Abstract **9**

Ευχαριστίες **11**

1 Εισαγωγή **29**
1.1 Αντικείμενο και πρόβλημα προς επίλυση 29
1.2 Κίνητρο και στόχοι 30
1.3 Ερευνητικά ερωτήματα 30
1.4 Σχετικές εργασίες και τοποθέτηση 31
1.5 Συνεισφορά της εργασίας 34
1.6 Δομή της εργασίας 34

I Θεωρητικό Μέρος **35**

2 Από το Virtualization στα Containers **37**
2.1 Virtualization και χρήση του στο cloud 37
2.2 Virtual machines και hypervisor 38
2.3 Περιορισμοί του virtualization για σύγχρονα συστήματα microservices . . . 38
2.4 Containers και διαφοροποίηση από τα VMs 39
2.5 Container images, container registries και ο κύκλος ζωής τους 41
2.6 Περιβάλλον εκτέλεσης container και βασικά ζητήματα ασφάλειας 42

3 Συστήματα Ορχήστρωσης Containers και Kubernetes **43**
3.1 Ανάγκη συστημάτων ορχήστρωσης containers και βασικές λειτουργίες . . . 43
3.2 Kubernetes βασικές έννοιες και αρχιτεκτονική 44
3.2.1 Συστατικά του Kubernetes 45
3.2.2 Πρόσθετα δικτύωσης cluster 46
3.2.3 Υπηρεσίες (services) και δημοσίευση εφαρμογών 46
3.2.4 Πόροι φόρτου εργασίας (workloads) 47
3.3 Κατανεμημένα συστήματα 48
3.3.1 Θεμελιώδεις αρχές 48
3.3.2 Προκλήσεις συντονισμού και συνέπειας 49
3.3.3 Ο ρόλος της ορχήστρωσης στη διαχείριση κατανεμημένων πόρων . . 49

4 Ροές Δεδομένων IoT και Αρχιτεκτονική 'Data Lakehouse' 51
4.1 IoT και χαρακτηριστικά τηλεμετρίας . . . 51
4.2 Συλλογή δεδομένων και ροές γεγονότων (event streaming) . . . 55
4.3 Apache Kafka, βασικές έννοιες (topics, partitions, brokers, replication) . . . 55
4.4 Apache Flink, επεξεργασία ροών δεδομένων . . . 57
4.4.1 Αρχιτεκτονική . . . 57
4.4.2 Διαχείριση κατάστασης (state) . . . 59
4.4.3 Χρονικές σημασιολογίες . . . 63
4.4.4 Παράθυρα . . . 65
4.4.5 Στρώματα API . . . 65
4.4.6 Κλιμακωσιμότητα και απόδοση . . . 66
4.5 Δεδομένα τύπου 'data lakehouse' . . . 67
4.5.1 Από τα data warehouses στις data lakes και την αρχιτεκτονική data lakehouse . . . 67
4.5.2 Apache Paimon, αρχιτεκτονική και ενοποιημένη αποθήκευση . . . 68
4.5.3 Βασικές έννοιες, αρχεία, snapshots και partitions . . . 70
4.5.4 Λειτουργίες αρχείων και διαχείριση μικρών αρχείων . . . 72
4.5.5 Εγγυήσεις συνέπειας και έλεγχος ταυτοχρονισμού . . . 74
4.5.6 Catalog . . . 75
4.5.7 Οργάνωση δεδομένων σε στρώματα . . . 75
4.5.8 Ιστορικότητα και χρονική μεταβολή . . . 75

5 Παρακολούθηση, Παρατηρησιμότητα και Ανάλυση Ερωτημάτων 77
5.1 Βασικές αρχές παρακολούθησης και παρατηρησιμότητας . . . 77
5.1.1 Four Golden Signals . . . 78
5.1.2 Μοντέλα παρακολούθησης τύπου push και pull . . . 79
5.2 Prometheus, συλλογή metrics χρονοσειράς . . . 79
5.2.1 Αρχιτεκτονική Prometheus . . . 79
5.2.2 Μοντέλο δεδομένων και τύποι metrics . . . 80
5.2.3 Γλώσσα ερωτημάτων PromQL . . . 80
5.2.4 Ανακάλυψη υπηρεσιών στο Kubernetes . . . 81
5.2.5 kube-prometheus-stack, ολοκληρωμένη λύση παρακολούθησης cluster . . . 81
5.3 Metrics των Flink, Kafka και Kubernetes . . . 82
5.3.1 Flink metrics . . . 82
5.3.2 Kafka metrics . . . 82
5.3.3 Kubernetes metrics . . . 82
5.4 Trino, μηχανή ερωτημάτων SQL . . . 83
5.4.1 Κατάλογοι δεδομένων . . . 83
5.4.2 Μοντέλο εκτέλεσης . . . 83
5.4.3 Βελτιστοποίηση ερωτημάτων . . . 83
5.5 Οπτικοποίηση και Grafana dashboards . . . 84
5.5.1 Βασικά χαρακτηριστικά . . . 84

5.5.2 Παρακολούθηση επίδοσης υποδομής και ανάλυση δεδομένων εφαρμογής . . . 84
5.6 Ολοκληρωμένη λύση, από τη ροή δεδομένων στην οπτικοποίηση . . . 85

II Πρακτικό Μέρος 87

6 Αρχιτεκτονικός Σχεδιασμός 89
6.1 Σχεδιασμός Υποδομής στο Azure . . . 90
6.1.1 Tenant . . . 90
6.1.2 Subscription . . . 90
6.1.3 Ομαδοποίηση πόρων και περιοχές φιλοξενίας (Resource Group / Region) . . . 90
6.1.4 Azure Kubernetes Service και node pools . . . 90
6.1.5 Azure Virtual Network, υποδίκτυα και πρόσβαση στο cluster . . . 91
6.1.6 Έκθεση υπηρεσιών μέσω Azure Load Balancer . . . 91
6.1.7 Αποθήκευση δεδομένων τύπου object, Azure Data Lake Storage Gen2 και σύστημα αρχείων τύπου ABFS . . . 92
6.1.8 Azure Container Registry . . . 92
6.1.9 Αποθήκευση τύπου block, Azure Managed Disks για Persistent Volume Claims . . . 93
6.1.10 Εκτίμηση κόστους σε παρόχους Public Cloud . . . 93
6.1.10.1 Αναλυτική κοστολόγηση μέγιστης διαμόρφωσης στο Azure 93
6.1.10.2 Συγκριτική εκτίμηση κόστους ανά ζεύγος βαθμίδων στο Azure . . . 95
6.1.10.3 Συγκριτική εκτίμηση μέγιστης διαμόρφωσης σε AWS και GCP . . . 97
6.1.10.4 Συγκριτική εκτίμηση κόστους ανά ζεύγος βαθμίδων σε AWS και GCP . . . 98
6.2 Σχεδιασμός Εφαρμογής . . . 99
6.2.1 Ροή δεδομένων . . . 99
6.2.2 Σχεδιασμός Kafka . . . 100
6.2.3 Σχεδιασμός ροών επεξεργασίας στο Flink . . . 100
6.2.4 Σχεδιασμός Paimon και οργάνωση πινάκων σε 3 στρώματα partition δεδομένων ανά πηγή . . . 101
6.2.5 Σχεδιασμός Trino (κατάλογοι, σχήματα, βασικά ερωτήματα KPIs) . . . 101
6.2.6 Σχεδιασμός Grafana dashboards και ρυθμός ανανέωσής τους . . . 102

7 Υλοποίηση 103
7.1 Υλοποίηση υποδομής στο Azure μέσω Terraform . . . 103
7.1.1 Terraform modules . . . 103
7.1.2 Κεντρικό αρχείο Terraform και αρχείο παραμέτρων . . . 104
7.2 Υλοποίηση Kubernetes cluster . . . 104
7.2.1 Namespaces . . . 104

7.2.2 Αποθήκευση κλειδιών (Secrets) και ρύθμιση μόνιμου αποθηκευτικού χώρου (PersistentVolumes) . . . 105
7.2.3 Δίκτυο και επικοινωνία υπηρεσιών . . . 105
7.3 Εγκατάσταση Kafka και αρχικοποίηση topics . . . 106
7.4 Εγκατάσταση Flink, jobs εισαγωγής και συγκεντρώσεων . . . 106
7.4.1 Εξειδικευμένο Docker image . . . 106
7.4.2 Αρχείο ρυθμίσεων Flink . . . 107
7.4.3 Διαμόρφωση των JobManager και TaskManagers . . . 107
7.4.4 Job εισαγωγής streaming δεδομένων . . . 108
7.4.5 Ωριαία συγκέντρωση δεδομένων . . . 110
7.4.6 Δεκάλεπτη συγκέντρωση δεδομένων . . . 110
7.5 Ρύθμιση Paimon και αποθήκευση σε ADLS . . . 111
7.5.1 Αρχείο αρχικοποίησης Paimon αντικειμένων . . . 111
7.5.2 Αρχείο εκκίνησης πινάκων Paimon . . . 111
7.6 Διαμόρφωση Trino και σύνδεση με Paimon . . . 113
7.6.1 Αρχείο ρυθμίσεων Trino . . . 113
7.6.2 Εγκατάσταση Trino . . . 113
7.7 Διαμόρφωση Grafana . . . 114
7.7.1 Εγκατάσταση datasource και εξωτερική πρόσβαση . . . 114
7.7.2 Αυτόματη εισαγωγή πηγών δεδομένων μέσω βοηθητικού (sidecar) container . . . 115
7.7.3 Αυτόματη εισαγωγή πινάκων ελέγχου . . . 115
7.8 Ρυθμίσεις Prometheus . . . 116
7.9 Προσομοιωτής παραγωγής IoT δεδομένων . . . 117
7.10 Κατασκευή εξειδικευμένων Docker images . . . 118
7.10.1 Flink image . . . 119
7.10.2 Trino image . . . 119
7.10.3 Image προσομοιωτή παραγωγής IoT δεδομένων . . . 119
7.11 Pipelines αυτοματοποίησης CI/CD με GitHub Actions . . . 120
7.11.1 Εγκατάσταση υποδομής μέσω Terraform . . . 121
7.11.2 Ανάπτυξη εφαρμογών . . . 121
7.11.3 Κατασκευή images IoT προσομοιωτή . . . 122
7.11.4 Κατασκευή image Trino-Paimon . . . 122
7.11.5 Διαγραφή υποδομής . . . 122
7.12 Προκλήσεις υλοποίησης . . . 122
7.12.1 Εντοπισμός ελάχιστου συνόλου βιβλιοθηκών Flink για σύνδεση Paimon-ADLS . . . 123
7.12.2 Συνδυασμός Paimon και Trino . . . 123

8 Δοκιμές και Αξιολόγηση 125
8.1 Πειραματικό περιβάλλον και παραδοχές . . . 125
8.1.1 Υλικό (hardware) . . . 125
8.1.2 Λογισμικό . . . 126

8.1.3 Παραδοχές . . . 126
8.2 Σενάρια φόρτου . . . 127
8.2.1 Ροή δεδομένων υπό αξιολόγηση . . . 127
8.2.2 Μεθοδολογία δημιουργίας σεναρίων . . . 127
8.2.3 Διφασική εκτέλεση πειράματος ομαδοποιώντας τα μηνύματα των παραγωγών . . . 130
8.2.4 Ορισμός βαθμίδων . . . 131
8.3 Ερωτήματα Αναφοράς Trino . . . 131
8.4 Δείκτες απόδοσης . . . 133
8.4.1 Ρυθμός μηνυμάτων ανά δευτερόλεπτο . . . 133
8.4.2 Καθυστέρηση άκρο-σε-άκρο (end-to-end latency) . . . 133
8.4.3 Παρατήρηση δικτύου . . . 134
8.5 Αποτελέσματα ρυθμού μηνυμάτων και κλιμάκωσης . . . 134
8.5.1 Συνοπτικά αποτελέσματα . . . 134
8.5.2 Ρυθμός μηνυμάτων ανά βαθμίδα . . . 135
8.6 Δείκτες απόδοσης ερωτημάτων Trino . . . 136
8.6.1 Χρόνοι ερωτημάτων αναφοράς στη Nonbatched φάση . . . 136
8.6.2 Χρόνοι ερωτημάτων αναφοράς στη batched φάση . . . 137
8.6.3 Ανάλυση κλιμάκωσης ερωτημάτων . . . 137
8.7 Καθυστέρηση από άκρο σε άκρο . . . 138
8.8 Κατανάλωση πόρων ανά υποσύστημα . . . 139
8.8.1 Σύγκριση Kafka CPU μεταξύ μη ομαδοποιημένων και ομαδοποιημένων παραγωγών . . . 140
8.8.2 Μνήμη και αποθηκευτικός χώρος Kafka . . . 141
8.8.3 Flink . . . 141
8.8.4 Trino . . . 145
8.8.5 Go παραγωγοί IoT δεδομένων . . . 145
8.8.6 Azure Storage Account metrics . . . 145
8.9 Σταθερότητα checkpoints Flink . . . 148
8.10 Οριακές περιπτώσεις και αστοχίες . . . 149
8.10.1 Ταυτόχρονες αστοχίες checkpoint στη βαθμίδα T7 . . . 149
8.10.2 Αστοχίες checkpoint, failover και κορεσμός στην υπο-φάση N9 . . . 149
8.10.3 Κορεσμός παραγωγού και αποτυχία Q3 στη βαθμίδα N10 . . . 150
8.11 Συζήτηση αποτελεσμάτων και εντοπισμός bottlenecks . . . 150
8.11.1 Ανάλυση κλιμάκωσης . . . 150
8.11.2 Εντοπισμός bottlenecks . . . 150
8.11.3 Σύγκριση μη ομαδοποιημένων και ομαδοποιημένων παραγωγών . . . 151
8.11.4 Χαρτογράφηση κόστους . . . 152

9 Βελτιστοποίηση Πόρων, Εφαρμογής και Κόστους 153
9.1 Ρύθμιση αίτησης κατώτατων και ανώτατων ορίων πόρων και επίδρασή τους στη σταθερότητα της εφαρμογής . . . 153
9.1.1 Requests και limits . . . 153

9.1.2 Ρυθμίσεις requests και limits στην τρέχουσα υλοποίηση 154
9.1.3 Πρόταση βελτιστοποίησης των requests και limits 154
9.2 Στρατηγικές κλιμάκωσης και προτάσεις αυτοματοποίησης 155
9.2.1 Οριζόντια κλιμάκωση 155
9.2.2 Κατακόρυφη κλιμάκωση 155
9.2.3 Αυτόματη κλιμάκωση cluster 156
9.3 Επίδραση ρυθμού ανανέωσης των Grafana dashboards στο Trino 156
9.3.1 Πρόβλημα 156
9.3.2 Αναμενόμενη επίδραση 157
9.3.3 Σύσταση 157
9.4 Οδηγίες λειτουργίας 157
9.4.1 Διάρκεια ζωής δεδομένων και διαχείριση στιγμιοτύπων 157
9.4.2 Παρακολούθηση και ειδοποιήσεις 157
9.4.3 Ασφάλεια 158
9.4.4 Εκτίμηση κόστους και βελτιστοποίηση δαπανών 158

III Επίλογος 159

10 Συμπεράσματα και Μελλοντική Εργασία 161
10.1 Απαντήσεις στα ερευνητικά ερωτήματα 161
10.1.1 Ισορροπία απόδοσης και κόστους 161
10.1.2 Επίδραση επιλογών υποδομής στην καθυστέρηση από άκρο σε άκρο 161
10.1.3 Όρια κλιμάκωσης και αστοχίες 162
10.2 Συμπεράσματα και μαθήματα από την υλοποίηση 163
10.2.1 Κύρια συμπεράσματα 163
10.2.2 Μετατόπιση bottleneck από εισαγωγή σε ανάλυση 164
10.2.3 Η αναλογία (ratio) Kafka/Paimon ως δείκτης υγείας 164
10.2.4 Οριοθέτηση αναλυτικών ερωτημάτων Trino 164
10.2.5 Μεθοδολογική συνεισφορά 165
10.3 Μελλοντικές επεκτάσεις 165
10.3.1 Αυτόματη κλιμάκωση πλήρους στοίβας 165
10.3.2 Βελτιωμένη παρακολούθηση 165
10.3.3 Ασφάλεια και συμμόρφωση 166
10.3.4 Επέκταση σε πραγματικά δεδομένα IoT 166
10.3.5 Μετάβαση στην SaaS πλατφόρμα Microsoft Fabric 166
10.3.6 Εναλλακτικές αρχιτεκτονικές 167

Παραρτήματα 169

Α΄ Υποδομή Terraform 171
Α΄.1 Σύνθεση modules 171
Α΄.2 AKS module 172

Β΄ Dockerfiles και κώδικας προσομοιωτή IoT **175**
Β΄.1 Dockerfile - Flink με Paimon και Kafka . . . 175
Β΄.2 Dockerfile - Προσομοιωτής IoT σε Go . . . 175
Β΄.3 Προσομοιωτής IoT - δομές δεδομένων . . . 176
Β΄.4 Προσομοιωτής IoT - βρόχος homeSimulator . . . 177
Β΄.5 Kafka Writer - ρυθμίσεις . . . 178
Β΄.5.1 Μη ομαδοποιημένη αποστολή (Unbatched Writer) . . . 178
Β΄.5.2 Ομαδοποιημένη αποστολή (Batched Writer) . . . 178

Γ΄ Kubernetes Deployments, ConfigMaps, Helm και SQL **179**
Γ΄.1 Kafka StatefulSet και αρχικοποίηση topics . . . 179
Γ΄.2 JobManager deployment σε Flink Session Mode . . . 180
Γ΄.3 Flink ConfigMap - σημεία ελέγχου και μετρικές . . . 181
Γ΄.4 Flink SQL - αρχικοποίηση καταλόγου Paimon (`init.sql`) . . . 182
Γ΄.5 Flink SQL - δημιουργία πινάκων Paimon (`bootstrap.sql`) . . . 183
Γ΄.6 Flink SQL - εισαγωγή Kafka → Paimon . . . 184
Γ΄.7 Flink SQL - ωριαία συγκέντρωση . . . 185
Γ΄.8 Kubernetes CronJob - ωριαία batch συγκέντρωση . . . 186
Γ΄.9 Τιμές Helm για Trino - κατάλογος Paimon . . . 187
Γ΄.10 Τιμές Helm για kube-prometheus-stack . . . 188
Γ΄.11 Σχήματα πινάκων Paimon . . . 189
Γ΄.11.1 Ακατέργαστος πίνακας (sensor_data_raw) . . . 189
Γ΄.11.2 Πίνακας ωριαίων συγκεντρώσεων (sensor_data_hourly) . . . 189
Γ΄.12 Deployment προσομοιωτή IoT (iot-sim-producers.yaml) . . . 190

Δ΄ Σωληνώσεις (Pipelines) CI/CD - GitHub Actions **193**
Δ΄.1 `apps-ci.yml` - Εγκατάσταση εφαρμογών . . . 193
Δ΄.2 Διαγράμματα βημάτων διοχετεύσεων . . . 194

Ε΄ Ερωτήματα αναφοράς - Trino benchmark queries **197**

ΣΤ΄ Σενάριο ενορχήστρωσης πειραμάτων **201**
ΣΤ΄.1 Παράμετροι και σταθερές . . . 201
ΣΤ΄.2 Επικύρωση πριν την εκτέλεση . . . 202
ΣΤ΄.3 Επαναφορά περιβάλλοντος . . . 203
ΣΤ΄.4 Βρόχος εκτέλεσης ανά βαθμίδα . . . 204
ΣΤ΄.5 Συλλογή μετρικών . . . 205
ΣΤ΄.6 Σουίτα σημείων αναφοράς Trino . . . 207

Συντομογραφίες - Αρκτικόλεξα - Ακρωνύμια **209**

Απόδοση ξενόγλωσσων όρων **213**

Βιβλιογραφία **236**

Κατάλογος Σχημάτων

6.1 Κατανομή εκτιμώμενου μηνιαίου κόστους Azure ανά κατηγορία πόρων (σύνολο €1.240,58). 95

8.1 Κυκλική σύνοψη του πειραματικού πρωτοκόλλου 129
8.2 Αναμενόμενος έναντι παρατηρούμενου RPS ανά βαθμίδα φόρτου 135
8.3 Χρόνοι ερωτημάτων Trino (p95) ανά βαθμίδα, μη ομαδοποιημένοι παραγωγοί 137
8.4 Χρόνοι ερωτημάτων Trino (p95) ανά βαθμίδα, ομαδοποιημένοι παραγωγοί . . 138
8.5 Καθυστέρηση από άκρο σε άκρο (E2E) ανά βαθμίδα 139
8.6 Κατανάλωση CPU Kafka (σύνολο 2 brokers) ανά βαθμίδα 142
8.7 Κατανάλωση πόρων ανά βαθμίδα σε έναν Kafka broker σε επίπεδο CPU, μνήμης και εισερχόμενης δικτυακής κίνησης. 143
8.8 Χρήση CPU των δύο Flink TaskManagers ανά κλίμακα. 144
8.9 Εισερχόμενη δικτυακή κίνηση (Net RX) των δύο Flink TaskManagers ανά κλίμακα 144
8.10 Μέσος και μέγιστος χρόνος κύκλου των Go παραγωγών ανά κλίμακα (μη ομαδοποιημένοι) 146
8.11 Δικτυακή κίνηση Azure Storage Account (ADLS) ανά βαθμίδα 146
8.12 Μετρήσεις Azure Storage Account ανά βαθμίδα 148

Κατάλογος Εικόνων

2.1 Σύγκριση bare-metal (Τύπος 1) και hosted (Τύπος 2) hypervisors. 39

2.2 Αρχιτεκτονική application containers (Docker) και system containers (LXC/LXD). 40

2.3 Κύκλος ζωής image container, από την κατασκευή έως την εκτέλεση. 42

3.1 Αρχιτεκτονική cluster Kubernetes. Επίπεδο ελέγχου (αριστερά) και worker-nodes (δεξιά). 45

3.2 Η αφαίρεση Service με παράδειγμα τύπου ClusterIP 47

4.1 Γενική αρχιτεκτονική του IoT, από τα φυσικά αντικείμενα και τους αισθητήρες, μέσω δικτυακής μεταφοράς, ως την επεξεργασία στο cloud και τις εφαρμογές. 52

4.2 Έξυπνη βιομηχανία και έξυπνη παραγωγή. Η σχέση μεταξύ Industry 4.0, IoT, αυτοματοποίησης και ψηφιακού μετασχηματισμού στον βιομηχανικό τομέα. 54

4.3 Δομή συστοιχίας Apache Flink. Ο JobManager συντονίζει την εκτέλεση, ενώ οι TaskManagers εκτελούν τις εργασίες της εφαρμογής. Ο πελάτης (Client) υποβάλλει τον γράφο εκτέλεσης. 58

4.4 Μηχανισμός διαμοιρασμού υποδοχών εργασίας (slot sharing) στο Apache Flink. Εργασίες διαφορετικών τελεστών μοιράζονται την ίδια υποδοχή, μεγιστοποιώντας τη χρήση πόρων. 59

4.5 Αλυσιδωτή σύνδεση τελεστών (operator chaining) στο Apache Flink. Υποεργασίες τελεστών συγχωνεύονται σε ενιαίες εργασίες, μειώνοντας την επιβάρυνση επικοινωνίας. 60

4.6 Partition κατάστασης στο Apache Flink. Η κατάσταση κατανέμεται μαζί με τα κλειδιά της ροής, εξασφαλίζοντας τοπική πρόσβαση χωρίς επιβάρυνση συναλλαγών. 61

4.7 Φράγματα σημείων ελέγχου (checkpoint barriers) σε ροή δεδομένων Flink. Τα φράγματα εισάγονται στις πηγές και ρέουν μαζί με τις εγγραφές, χωρίζοντας τα δεδομένα σε snapshots. 62

4.8 Μηχανισμός σημείων ελέγχου στον Apache Flink. Κατά την ανάκτηση το Flink επαναφέρει την κατάσταση και τις θέσεις (offsets) των πηγών από το τελευταίο ολοκληρωμένο snapshot. 62

4.9 Σχέση μεταξύ χρόνου γεγονότος (event time) και χρόνου επεξεργασίας (processing time) στο Apache Flink. Ο χρόνος γεγονότος πηγάζει από τα δεδομένα, ενώ ο χρόνος επεξεργασίας από το ρολόι του node. 63

4.10 Watermarks σε ροή δεδομένων εκτός σειράς (out-of-order) στο Apache Flink. Τα watermarks δηλώνουν ότι όλα τα γεγονότα μέχρι ένα timestamp θα έπρεπε να έχουν φθάσει. 64
4.11 Watermarks σε παράλληλες ροές δεδομένων Flink. Κάθε πηγή παράγει ανεξάρτητα watermarks, ενώ τελεστές με πολλαπλές εισόδους χρησιμοποιούν το ελάχιστο μεταξύ τους. 64
4.12 Tumbling windows στο Apache Flink, μη επικαλυπτόμενα, σταθερού μήκους. 66
4.13 Αρχιτεκτονική του Apache Paimon. Streaming και κατά παρτίδες ανάγνωση/εγγραφή, αποθήκευση αρχείων σε στήλες σε σύστημα αρχείων, μεταδεδομένα σε αρχεία δηλώσεων και δομή LSM tree για πίνακες με πρωτεύον κλειδί. 69
4.14 Ιεραρχική δομή αρχείων ενός πίνακα Apache Paimon. Τα snapshots αναφέρονται σε λίστες δηλώσεων, οι οποίες με τη σειρά τους περιγράφουν τα αρχεία δεδομένων σε κάθε partition και κάδο. 71
4.15 Δομή LSM tree στο Apache Paimon. Οι εγγραφές ταξινομούνται σε πολλαπλά sorted runs εντός κάθε bucket, ενώ η compaction τα συγχωνεύει περιοδικά. 72
4.16 Streaming εγγραφή Apache Paimon μέσω Flink. Το Paimon Sink αποθηκεύει δεδομένα σε LSM tree στο heap, ο CompactManager ενεργοποιεί ασύγχρονα το compaction και ο Committer Operator δημιουργεί snapshots κατά τα checkpoints του Flink. 73

5.1 Τα τρία θεμελιώδη στοιχεία της παρατηρησιμότητας (κόκκινο) και επιπλέον δεδομένα που θεωρούνται ζωτικά από διάφορους παρόχους (πράσινο). 78
5.2 Αρχιτεκτονική του Prometheus. Ο κεντρικός server συλλέγει metrics από εξωτερικές εργασίες και exporters, αποθηκεύει χρονοσειρές και ενεργοποιεί ειδοποιήσεις μέσω του Alertmanager. 80

6.1 Αρχιτεκτονικός σχεδιασμός υποδομής Azure και Kubernetes 89

7.1 Διασύνδεση υπηρεσιών στο AKS cluster της εργασίας 105
7.2 Τοπολογία Flink cluster 108
7.3 Διάγραμμα ακολουθίας μηνυμάτων 109
7.4 Μοντέλο δεδομένων 112
7.5 Τοπολογία Trino cluster 114
7.6 Παράδειγμα IoT dashboard στο Grafana το οποίο παρουσιάζει τη ροή ακατέργαστων μετρήσεων. 115
7.7 Παράδειγμα IoT dashboard στο Grafana το οποίο παρουσιάζει τους δείκτες κατανάλωσης ενέργειας. 116
7.8 Παράδειγμα IoT dashboard στο Grafana το οποίο παρουσιάζει την ανάλυση συμπεριφοράς συσκευών. 116
7.9 Ροές εργασίας GitHub Actions 120

8.1 Πίνακας ελέγχου kube-prometheus-stack (Compute Resources / Cluster) . 140
8.2 Πίνακας ελέγχου kube-prometheus-stack (Compute Resources / Pod) . . . 141

Δ΄.1 Σειρά βημάτων του pipeline `apps-ci.yml` από τη σύνδεση στο Azure έως το deployment των εφαρμογών του συστήματος 195
Δ΄.2 Σειρά βημάτων του pipeline `infra-ci.yml` για την επαλήθευση και εφαρμογή της Terraform 196

Κατάλογος Πινάκων

5.1 Βασικά Kafka metrics που συλλέγονται μέσω JMX exporter. 82

6.1 Εκτίμηση μηνιαίου κόστους Azure μέσω Azure Pricing Calculator 94
6.2 Ανάλυση κόστους ADLS ανά κατηγορία λειτουργιών (μηνιαία βάση, Hot tier). 94
6.3 Εκτίμηση μηνιαίου κόστους Azure ανά ζεύγος βαθμίδων 96
6.4 Αντιστοίχιση υπηρεσιών μεταξύ Azure, AWS και GCP για την αρχιτεκτονική της εργασίας. 97
6.5 Συγκριτική εκτίμηση μηνιαίου κόστους μέγιστης διαμόρφωσης σε Azure, AWS και GCP 98
6.6 Αντιστοίχιση user pool VMs ανά ζεύγος βαθμίδων στους τρεις παρόχους. . . . 98
6.7 Συγκριτική εκτίμηση μηνιαίου κόστους ανά ζεύγος βαθμίδων σε Azure, AWS και GCP 99
6.8 Namespaces του Kubernetes cluster και οι αντίστοιχες εφαρμογές που φιλοξενούν. 99

7.1 JARs εκτέλεσης που ενσωματώνονται στο image Flink. 107
7.2 Βασικές ρυθμίσεις ConfigMap flink-config. 107
7.3 Εξειδικευμένα images Docker του συστήματος. 119

8.1 Υλικό υποδομής υπολογιστικού AKS cluster. 125
8.2 Λογισμικές συνιστώσες της πλατφόρμας. 126
8.3 Αποτελέσματα βαθμονόμησης παραγωγού IoT δεδομένων 130
8.4 Βαθμίδες πειραμάτων φόρτου. 131
8.5 Συνοπτικά αποτελέσματα ανά βαθμίδα φόρτου για τις δύο στρατηγικές παραγωγών 134
8.6 Ρυθμός μηνυμάτων (RPS) και κατάσταση κορεσμού παραγωγών ανά κλίμακα. 136
8.7 Χρόνος ερωτημάτων Trino (p95, σε δευτερόλεπτα) ανά κλίμακα, μη ομαδοποιημένοι παραγωγοί. 136
8.8 Χρόνος ερωτημάτων Trino (p95, σε δευτερόλεπτα) ανά κλίμακα, ομαδοποιημένοι παραγωγοί. 137
8.9 Σύγκριση καθυστέρησης E2E μεταξύ μη ομαδοποιημένων και ομαδοποιημένων παραγωγών 140
8.10 Μέσοι πόροι του Kafka broker-0 ανά κλίμακα (παράθυρο 30 min, μη ομαδοποιημένοι παραγωγοί). 142
8.11 Μέσοι πόροι των Flink TM-1 και TM-2 ανά κλίμακα στους μη ομαδοποιημένους παραγωγούς. 143

8.12 Μέσος και μέγιστος χρόνος κύκλου των Go παραγωγών ανά κλίμακα (μη ομαδοποιημένοι παραγωγοί). 145
8.13 Μετρήσεις Azure Storage Account ανά κλίμακα (30 min, μη ομαδοποιημένοι παραγωγοί). 147
8.14 Μετρήσεις Azure Storage Account ανά κλίμακα (30 min, ομαδοποιημένοι παραγωγοί). 147
8.15 Σημεία ελέγχου Flink ανά κλίμακα (παράθυρο 30 min). 148
8.16 Σύγκριση μη ομαδοποιημένων (N) και ομαδοποιημένων (B) παραγωγών. . . . 152

9.1 Κατηγορίες QoS στο Kubernetes ως προς requests και limits. 153
9.2 Κάτω όρια (requests) και άνω όρια (limits) πόρων ανά υπηρεσία. 154
9.3 Προτεινόμενες βελτιστοποιήσεις ορίων πόρων ανά υπηρεσία 155
9.4 Λειτουργίες του VPA. 156

Κεφάλαιο 1

Εισαγωγή

Η παρούσα εργασία πραγματεύεται τη σχεδίαση και υλοποίηση μιας σύγχρονης υποδομής ροών δεδομένων συσκευών Internet of Things (IoT) σε περιβάλλον Kubernetes, με έμφαση σε αρχιτεκτονική τύπου data lakehouse, ξεκινώντας από την προσομοίωση παραγωγής δεδομένων φτάνοντας έως την ανάλυση και την οπτικοποίηση αυτών. Η ταχεία εξάπλωση των συσκευών IoT σε τομείς όπως η ενεργειακή διαχείριση, τα έξυπνα σπίτια, οι έξυπνες πόλεις και η βιομηχανική παραγωγή [1, 2, 3, 4, 5] δημιουργεί τεράστιους όγκους τηλεμετρικών δεδομένων που απαιτούν αξιόπιστη, κλιμακούμενη και οικονομικά αποδοτική υποδομή για την επεξεργασία τους. Παραδοσιακές αρχιτεκτονικές αποθήκευσης, σχεδιασμένες για στατικές αναλύσεις κατά παρτίδες (batches), αδυνατούν να αντιμετωπίσουν τις αυξημένες ανάγκες ανάλυσης σε πραγματικό χρόνο. Η αρχιτεκτονική data lakehouse [6] συνδυάζει τα πλεονεκτήματα της λίμνης δεδομένων (data lake) και της αποθήκης δεδομένων (data warehouse) σε ένα ενοποιημένο επίπεδο, ενώ η χρήση τεχνολογιών μικροϋπηρεσιών (microservices) και ορχήστρωσης σε containers [7] επιτρέπει τη δυναμική κατανομή υπολογιστικών πόρων με ελάχιστη χειροκίνητη παρέμβαση.

1.1 Αντικείμενο και πρόβλημα προς επίλυση

Το αντικείμενο της εργασίας είναι η αξιόπιστη και κλιμακούμενη διαχείριση τηλεμετρικών ροών, η επίτευξη χαμηλής καθυστέρησης και η διατήρηση ιστορικότητας με αποδοτικό κόστος. Συγκεκριμένα, αντιμετωπίζεται το πρόβλημα της ενοποίησης δεδομένων που προέρχονται από ετερογενείς αισθητήρες ή κεντρικούς μετρητές ενέργειας ανά εγκατάσταση και επιμέρους αισθητήρες σε επίπεδο τελικής συσκευής (πρίζα, μηχανή, κινητήρας, γραμμή παραγωγής κ.ά.) σε μία κοινή πλατφόρμα ανάλυσης [8, 9]. Στη βιβλιογραφία επισημαίνεται ότι η ανομοιογένεια των πηγών, η μεταβλητότητα του ρυθμού παραγωγής δεδομένων και η ανάγκη διατήρησης πλήρους ιστορικότητας αποτελούν σημαντικές προκλήσεις κατά τη σχεδίαση ενός data lake [10, 11, 12]. Επιπρόσθετα, η ανάγκη διασφάλισης ελάχιστης καθυστέρησης (end-to-end latency) από το σημείο παραγωγής των δεδομένων μέχρι την αποθήκευση και την οπτικοποίηση αυτών, καθιστά αναγκαία τη χρήση τεχνολογιών επεξεργασίας ροής (stream processing) αντί κλασικού μοντέλου επεξεργασίας κατά παρτίδες [13]. Παράλληλα, η απαίτηση για ελαστική κλιμάκωση από εκατοντάδες έως εκατοντάδες χιλιάδες εικονικές εγκαταστάσεις (όπως κατοικίες, βιομηχανικές μονάδες, εργοστάσια, κ.ά.) και αντίστοιχου όγκου παραγόμενων μηνυμάτων, ο οποίος φτάνει έως εκατομμύρια μηνύματα

ανά δευτερόλεπτο (msg/s) καθώς και η ανάγκη ελέγχου κόστους σε δημόσιους νεφοϋπολογιστικούς (public cloud) πόρους [14, 15] επιβάλλουν μία αρχιτεκτονική που βασίζεται σε microservices φιλοξενούμενα σε containers και επιτρέπει τόσο οριζόντια κλιμάκωση όσο και χρήση πόρων κατά απαίτηση (spot instances).

1.2 Κίνητρο και στόχοι

Κίνητρο αποτελεί η ανάγκη για πλατφόρμες που υποστηρίζουν πραγματικού χρόνου δεδομένα, υψηλές ροές γεγονότων και ευέλικτη επεξεργασία και ανάλυση. Καθώς ο αριθμός των συνδεδεμένων συσκευών αυξάνεται εκθετικά [16, 17, 18], οι οργανισμοί αντιμετωπίζουν αυξανόμενη πίεση για εξαγωγή αξίας άμεσα από τα δεδομένα που παράγονται από αυτές, χωρίς να εισάγουν δυσανάλογο κόστος. Τα κλασικά αρχιτεκτονικά πρότυπα, όπως η αρχιτεκτονική Lambda [19], η οποία διαχωρίζει την επεξεργασία σε ένα στρώμα παρτίδων (batch layer) και ένα στρώμα ροής (speed layer), εισάγουν πολυπλοκότητα στη συντήρηση, καθώς απαιτούν διπλό σύστημα για τις ροές πραγματικού χρόνου και τις αναλύσεις κατά παρτίδες. Η αρχιτεκτονική data lakehouse που υιοθετείται στην παρούσα εργασία στοχεύει στην εξάλειψη αυτού του διπλασιασμού, παρέχοντας ένα ενοποιημένο στρώμα αποθήκευσης [6].

Οι συγκεκριμένοι στόχοι της εργασίας συνοψίζονται στα εξής:

1. Σχεδιασμός μιας πλήρους διαδρομής δεδομένων (data pipeline) φιλοξενούμενης σε Kubernetes, ξεκινώντας από την παραγωγή τηλεμετρικών δεδομένων έως την οπτικοποίησή τους μέσω Grafana dashboards [20].

2. Αξιοποίηση τεχνολογιών ανοιχτού κώδικα, οι οποίες είναι το Kafka [21], το Flink [22], το Paimon [23] και το Trino [24], για τη δημιουργία αρχιτεκτονικής data lakehouse χωρίς εξάρτηση από λύσεις κλειστού κώδικα και κοστοβόρων αδειοδοτήσεων.

3. Ανάπτυξη της υποδομής στο public cloud του Microsoft Azure χρησιμοποιώντας Infrastructure as Code (IaC) [25] σε γλώσσα προγραμματισμού Terraform [26], ώστε η υλοποίηση να είναι εύκολα αναπαράξιμη και ελεγχόμενη.

4. Πειραματική αξιολόγηση υπό κλιμακούμενο φόρτο, με σενάρια που κυμαίνονται από 50 έως 17.500 εικονικές εγκαταστάσεις, οι οποίες προσομοιώνουν 250 έως 87.500 msg/s, υπό διφασική στρατηγική παραγωγής μηνυμάτων (ομαδοποιημένη και μη), και αξιολόγηση δεικτών απόδοσης, κατανάλωσης πόρων και κόστους.

1.3 Ερευνητικά ερωτήματα

Η εργασία αυτή αποσκοπεί πρωτίστως στον προσδιορισμό της αρχιτεκτονικής εκείνης που επιτυγχάνει ικανοποιητική ισορροπία μεταξύ απόδοσης επεξεργασίας (throughput) και λειτουργικού κόστους σε public cloud περιβάλλοντα, όταν χρησιμοποιούνται αποκλειστικά τεχνολογίες ανοιχτού κώδικα πάνω στην πλατφόρμα Kubernetes.

Ένας δεύτερος κρίσιμος άξονας αφορά τον βαθμό στον οποίο οι επιλογές υποδομής, όπως, πρώτον, το πλήθος και το μέγεθος των κόμβων (nodes) μιας συστάδας (cluster) και,

δεύτερον, οι ρυθμίσεις ελάχιστων και μέγιστων ορίων πόρων επεξεργαστή και μνήμης (requests και limits), επηρεάζουν την καθυστέρηση από άκρο σε άκρο (end-to-end latency) της ροής δεδομένων.

Επιπλέον, η έρευνα επικεντρώνεται στον εντοπισμό των πρακτικών ορίων κλιμάκωσης της προτεινόμενης αρχιτεκτονικής, και σε ποιο σημείο εμφανίζονται αστοχίες (bottlenecks) που επηρεάζουν τη σταθερότητα του συστήματος.

Τα ζητήματα αυτά αντικατοπτρίζουν τις κεντρικές προκλήσεις που αναδεικνύονται στη σχετική βιβλιογραφία [27, 28], ιδιαίτερα στο πεδίο των κατανεμημένων συστημάτων δεδομένων IoT.

1.4 Σχετικές εργασίες και τοποθέτηση

Η σχετική βιβλιογραφία γύρω από αξιολόγηση αρχιτεκτονικών IoT με ροές δεδομένων δεν συγκλίνει σε μια ενιαία προηγούμενη εργασία που να συνδυάζει Kubernetes, Apache Kafka, Apache Flink, ανοιχτό μορφότυπο πινάκων τύπου lakehouse (table format), αποθήκευση δεδομένων ως αντικειμένων (object storage), κατανεμημένη ανάλυση μέσω Γλώσσας Δομημένων Ερωτημάτων (Structured Query Language, SQL) [29] και παρατηρησιμότητα υπό φόρτο τηλεμετρίας. Αντιθέτως, διασπάται σε τρεις γειτνιάζοντες άξονες, δηλαδή σε αξιολογήσεις end-to-end πλατφορμών ροών IoT επί Kubernetes, σε επίσημα benchmarks επεξεργασίας ροών IoT και σε benchmarks αποθήκευσης και ερωτημάτων χρονοσειρών. Παρακάτω παρουσιάζονται οκτώ εργασίες με την υψηλότερη μεθοδολογική ή τοπολογική επικάλυψη με την παρούσα διπλωματική, μαζί με σύντομη αποτίμηση της σχέσης τους με το πειραματικό μας πλαίσιο.

Στο Theodolite των Henning και Hasselbring [30], που δημοσιεύτηκε το 2021, προτείνεται ένα πλαίσιο μέτρησης κλιμάκωσης για κατανεμημένες μηχανές ροών (Apache Flink, Kafka Streams) πάνω σε Kubernetes clusters, με σενάρια χρήσης που προέρχονται από βιομηχανικά IoT φορτία ανάλυσης κατανάλωσης ισχύος. Η κεντρική τους μεθοδολογία ορίζει τη βιώσιμη ρυθμοαπόδοση (sustainable throughput) με κριτήριο στόχου επιπέδου υπηρεσίας (Service Level Objective, SLO) [31, 32] επί δεικτών υγείας του Kafka, προσέγγιση συμβατή με τη δική μας λογική σταδιακής κλιμάκωσης φόρτου και ελέγχου σταθερότητας. Η παρούσα εργασία διαφέρει στο ότι εκτείνει την αλυσίδα έως το επίπεδο αποθήκευσης (Apache Paimon επί Azure Data Lake Storage, ADLS) και την αναλυτική επεξεργασία (Trino), ενώ διατηρεί την ίδια λογική διφασικής εκτέλεσης ανά βαθμίδα φόρτου.

Σε μεταγενέστερη δημοσίευσή τους το 2024, οι ίδιοι συγγραφείς [33], επεκτείνουν το Theodolite, σε cloud-native μικροϋπηρεσίες και αξιολογούν τα Apache Flink, Kafka Streams, Hazelcast Jet [34] και Apache Beam [35] σε υψηλή κλίμακα πάνω σε Google Cloud Kubernetes, με τα ίδια σενάρια Βιομηχανικού Διαδικτύου των Πραγμάτων (Industrial IoT, IIoT) [36] κατανάλωσης ισχύος. Η συγκεκριμένη εργασία τους, αποτελεί το πιο ώριμο μεθοδολογικό πρότυπο για benchmarks σε υποδομές που φιλοξενούνται σε Kubernetes που εκτελούν επεξεργασία ροών οδηγούμενη από IoT φορτία και τεκμηριώνει τη χρήση αυτοματισμού πειραμάτων βασισμένου σε Kubernetes Operators [37]. Στην παρούσα

διπλωματική, αντί συγκριτικής αξιολόγησης πολλών μηχανών ροών, εστιάζουμε σε μία ολοκληρωμένη αλυσίδα Kafka, Flink, Paimon και Trino και μελετούμε τη συμπεριφορά της σε διαφορετικές στρατηγικές παραγωγής, ομαδοποιημένη και μη.

Στην εργασία των Geldenhuys κ.ά. [38] το 2021 περιγράφεται το Dependable IoT Stream Processing, μια πλατφόρμα ανοιχτού κώδικα για επεξεργασία ροών IoT πάνω σε Kubernetes, χτισμένη γύρω από Apache Kafka, Apache Flink και Prometheus, με στόχο την παρακολούθηση και τον έλεγχο αστικών υποδομών. Πρόκειται για την πλησιέστερη αρχιτεκτονικά αναφορά, καθώς συνδέει ρητά τη ροή IoT, την ανάπτυξη σε Kubernetes, την ανοχή σφαλμάτων και την αξιολόγηση πόρων σε ένα ενιαίο σύστημα. Η παρούσα διπλωματική επεκτείνει αυτή τη βάση προσθέτοντας Paimon table format, αποθήκευση σε object storage (ADLS) και κατανεμημένη SQL ανάλυση (Trino), δηλαδή τα δύο επίπεδα που λείπουν από την αναφορά εκείνη ώστε να ολοκληρωθεί ένα lakehouse.

Οι Shukla κ.ά. εισήγαγαν το 2017 το RIoTBench [39], ένα θεμελιώδες ακαδημαϊκό benchmark suite για distributed stream processing systems σε IoT φορτία, το οποίο ορίζει 27 μικρο-εργασίες (micro-tasks) και τέσσερις εφαρμογές τύπου εξαγωγής-μετασχηματισμού-φόρτωσης (Extract-Transform-Load, ETL) [28], στατιστικής σύνοψης, προβλεπτικής ανάλυσης και εκπαίδευσης μοντέλου, οι οποίες εκτελούνται πάνω σε Apache Storm [40] με δεδομένα από έργα έξυπνων πόλεων (smart-city) και έξυπνης υγείας (smart-health). Η εστίασή του σε σαρώσεις ρυθμού φόρτου, καθυστέρηση end-to-end και χρήση πόρων ανά εικονική μηχανή νέφους ταυτίζεται μεθοδολογικά με τα δικά μας πειράματα κλιμάκωσης ανά βαθμίδα. Διαφοροποιούμαστε στο ότι αξιολογούμε Flink επί Kubernetes αντί Storm επί Υποδομής ως Υπηρεσίας (Infrastructure as a Service, IaaS) [41] και επιπρόσθετα συμπεριλαμβάνουμε επίπεδο αποθήκευσης τύπου lakehouse.

Σε ανάλυση των Pöss κ.ά. [42] το 2018 αναδεικνύεται το TPCx-IoT ως το πρώτο βιομηχανικό πρότυπο benchmark για συστήματα πύλης IoT, με σενάριο συνεχούς εισροής μετρήσεων από σταθμούς ηλεκτρικής ενέργειας και ταυτόχρονα ερωτήματα αναζήτησης. Το μοντέλο δεδομένων (αναγνωριστικά συσκευής, όνομα αισθητήρα, χρονοσφραγίδα και μέτρηση) βρίσκεται δομικά κοντά στο σχήμα της δικής μας τηλεμετρίας έξυπνων μετρητών και πριζών, ενώ το metric IoTps, δηλαδή μετρήσεις IoT ανά δευτερόλεπτο, προσφέρει τυποποιημένο λεξιλόγιο για τη συζήτηση ρυθμοαπόδοσης και SLO. Στην παρούσα διπλωματική εργασία, αξιοποιούμε το ίδιο πνεύμα διαρκούς εισροής με αναλυτικά ερωτήματα, εκτελούμενο όμως πάνω σε ολοκληρωμένη lakehouse αλυσίδα και όχι μόνο σε επίπεδο πύλης.

Οι Jerzak και Ziekow συγκρότησαν το 2014 το DEBS Grand Challenge [43] γύρω από έξυπνα σπίτια, το οποίο εμφανίζει την πιο άμεση τοπολογική αντιστοίχιση με την προσέγγιση της παρούσας εργασίας. Το σύνολο δεδομένων κατασκευάζεται από πραγματικές γερμανικές οικιακές εγκαταστάσεις και αποτελείται από μετρήσεις έξυπνων πριζών (smart plugs) σε ιδιωτικές κατοικίες, με τοπολογία πολύ κοντινή στη δική μας προσομοίωση. Τα δύο επίσημα ερωτήματα της πρόκλησης, βραχυπρόθεσμη πρόβλεψη φορτίου και στατιστικά πραγματικού

χρόνου, αποτελούν ακαδημαϊκό πρότυπο για τα δικά μας ερωτήματα ομαδοποίησης, παραθύρων και ποσοστημορίων. Η αξιολόγηση της παρούσας εργασίας, διαφοροποιείται στο ότι γίνεται πάνω σε σύγχρονη lakehouse στοίβα και με ελεγχόμενα κλιμακούμενο συνθετικό φόρτο.

Το ESPBench, που προτάθηκε από τους Hesse κ.ά. [44] το 2021, τοποθετείται ρητά σε σενάρια τέταρτης βιομηχανικής επανάστασης (Industry 4.0) και IoT και αξιολογεί τα Spark Structured Streaming [45], Apache Flink και Hazelcast Jet (μέσω ερωτημάτων Apache Beam [35]), διαβάζοντας από Kafka και γράφοντας σε σχεσιακή βάση δεδομένων. Προσφέρει ώριμη υποδομή μέτρησης end-to-end latency και επικύρωσης αποτελεσμάτων, η οποία ταιριάζει με τη δική μας ανάγκη να μετρηθεί καθυστέρηση από τον παραγωγό μέχρι τον αναλυτικό πίνακα. Στη δική μας περίπτωση, η σχεσιακή βάση δεδομένων αντικαθίσταται από συνδυασμό Apache Paimon και Trino πάνω από object storage, διατηρώντας ωστόσο το ίδιο σχήμα ροής με streaming εισροή και αναλυτικά ερωτήματα.

Στο SciTS των Mostafa κ.ά. [46], που δημοσιεύτηκε το 2022, συγκρίνονται συστήματα αποθήκευσης χρονοσειρών όπως το ClickHouse [47], η InfluxDB [48], η TimescaleDB [49] και η PostgreSQL [50] υπό βαριά εισαγωγή και πέντε αναλυτικά ερωτήματα που εκτείνονται από κλιμάκωση εισαγωγής έως στατιστικές συνόψεις. Αποτελεί την πιο ευθυγραμμισμένη μεθοδολογικά αναφορά για το αποθηκευτικό σκέλος της πλατφόρμας μας, καθώς εξετάζει τη συμπεριφορά της εισαγωγής σε σχέση με το αυξανόμενο μέγεθος βάσης και τον χρόνο απόκρισης ερωτημάτων υπό συνεχή εγγραφή. Διαφοροποιούμαστε στο ότι το επίπεδο αποθήκευσης δεν αποτελεί κάθετη βάση χρονοσειρών (Time Series Database, TSDB) αλλά lakehouse με ανοιχτό table format επί αποθήκευσης αντικειμένων, οπότε τα αντίστοιχα metrics ερμηνεύονται ως κόστος ανά εγγραφή σε ADLS.

Πέραν των στενά συγκρίσιμων πειραματικών εργασιών, αξίζει αναφορά η προηγούμενη διπλωματική του Νικόλαου Μπέλλου [51] που εκπονήθηκε το 2024 στον ίδιο τομέα της σχολής και αφορά σχεδίαση και υλοποίηση συστήματος επεξεργασίας δεδομένων IoT με κατανεμημένες τεχνολογίες πάνω σε Kubernetes, χωρίς ωστόσο εστίαση σε πειραματική αξιολόγηση κλιμακούμενου φόρτου ροής.

Συνοψίζοντας τα παραπάνω, η παρούσα διπλωματική δανείζεται από κάθε εργασία ένα διακριτό στοιχείο. Από το Theodolite υιοθετείται η λογική διφασικής εκτέλεσης ανά βαθμίδα και η χρήση δεικτών υγείας του Kafka, από το DEBS 2014 η τοπολογία έξυπνων πριζών σε ιδιωτικές κατοικίες, από το TPCx-IoT το λεξιλόγιο metrics IoTps και SLO, από το RIoTBench η ιδέα κλιμάκωσης φόρτου σε διακριτές βαθμίδες, από το ESPBench η μεθοδολογία μέτρησης καθυστέρησης από άκρο σε άκρο, από το SciTS τα αναλυτικά metrics αποθήκευσης χρονοσειρών και από τους Geldenhuys κ.ά. η αρχιτεκτονική βάση Kafka, Flink και Prometheus επί Kubernetes. Στο σημείο όπου η βιβλιογραφία παραμένει ελλιπής, δηλαδή σε μια ενιαία αξιολόγηση μορφότυπου πινάκων τύπου lakehouse (Apache Paimon) πάνω σε αντικειμενοστραφή αποθήκευση και αναλυτικού επιπέδου Trino με ταυτόχρονη ροή IoT, εντοπίζεται το βασικό σημείο συνεισφοράς της παρούσας εργασίας.

1.5 Συνεισφορά της εργασίας

Η συνεισφορά της εργασίας εντοπίζεται σε τρεις βασικούς άξονες. Πρώτον, παρέχεται τεκμηριωμένος σχεδιασμός υποδομής στο Azure, με πλήρη αποτύπωση των υποδομικών πόρων στο cloud και εκτίμηση του κόστους τους, ώστε ο σχεδιασμός να λειτουργεί ως αναπαράξιμο πρότυπο αναφοράς για αντίστοιχες εφαρμογές. Δεύτερον, υλοποιείται ολοκληρωμένη ροή IoT μέσω αλυσίδας εφαρμογών ανοιχτού κώδικα η οποία φιλοξενείται αποκλειστικά σε Kubernetes, αξιοποιώντας τρία στρώματα διατήρησης δεδομένων σε κάθε πηγή, δηλαδή ακατέργαστα δεδομένα (raw), επεξεργασμένα και οργανωμένα δεδομένα (curated) και συγκεντρωτικά δεδομένα (aggregated), τα οποία αντιστοιχούν στην αρχιτεκτονική medallion που έχει αποκτήσει αυξανόμενο ενδιαφέρον στη βιβλιογραφία [52]. Τρίτον, διενεργείται πειραματική αξιολόγηση με κλιμακούμενο φόρτο, εκτελώντας σενάρια δέκα βαθμίδων, από 50 έως 17.500 προσομοιούμενες εγκαταστάσεις (φόρτος που αντιστοιχεί σε 250 έως 87.500 msg/s), και αποτυπώνονται δείκτες ρυθμού μηνυμάτων, καθυστέρησης, κατανάλωσης πόρων και κόστους, με αντιπαραβολή πραγματικής κατανάλωσης και θεωρητικής εκτίμησης.

1.6 Δομή της εργασίας

Το υπόλοιπο κείμενο οργανώνεται ως εξής. Στο Μέρος I (Θεωρητικό Μέρος), το Κεφάλαιο 2 εξετάζει τη μετάβαση από την εικονικοποίηση στα containers, παρέχοντας το απαραίτητο τεχνολογικό υπόβαθρο. Το Κεφάλαιο 3 εστιάζει στα συστήματα ενορχήστρωσης containers και ειδικότερα το Kubernetes, ενώ το Κεφάλαιο 4 καλύπτει τις ροές δεδομένων IoT και την αρχιτεκτονική data lakehouse, αναλύοντας τις τεχνολογίες Kafka, Flink και Paimon. Το Κεφάλαιο 5 καλύπτει την παρακολούθηση, παρατηρησιμότητα και ανάλυση ερωτημάτων, παρουσιάζοντας τις τεχνολογίες Prometheus, Trino και Grafana. Στο Μέρος II (Πρακτικό Μέρος), το Κεφάλαιο 6 παρουσιάζει τον αρχιτεκτονικό σχεδιασμό τόσο σε επίπεδο υποδομής όσο και εφαρμογής, το Κεφάλαιο 7 τα βήματα υλοποίησης της λύσης, το Κεφάλαιο 8 τις δοκιμές και την αξιολόγησή τους και το Κεφάλαιο 9 τη βελτιστοποίηση πόρων και κόστους. Τέλος, στο Μέρος III (Επίλογος), στο Κεφάλαιο 10 συνοψίζονται τα συμπεράσματα και προτείνονται κατευθύνσεις επέκτασης της εργασίας.

Μέρος I

Θεωρητικό Μέρος

Κεφάλαιο 2

Από το Virtualization στα Containers

Στο κεφάλαιο αυτό παρουσιάζονται οι βασικές έννοιες της εικονικοποίησης (virtualization) και η μετάβαση στα containers ως θεμέλιο για σύγχρονες αρχιτεκτονικές μικροϋπηρεσιών (microservices). Η κατανόηση αυτών των τεχνολογιών αποτελεί προϋπόθεση για τη μελέτη των κατανεμημένων συστημάτων που αναπτύσσονται στα επόμενα κεφάλαια, καθώς ολόκληρη η προτεινόμενη αρχιτεκτονική επεξεργασίας ροών δεδομένων εκτελείται πάνω σε containers που φιλοξενούνται σε Kubernetes.

2.1 Virtualization και χρήση του στο cloud

Το virtualization αποτελεί θεμελιώδη τεχνολογία στη σύγχρονη πληροφορική, επιτρέποντας την αφαιρετική αναπαράσταση υπολογιστικών πόρων (όπως επεξεργαστή, μνήμης, αποθηκευτικού χώρου, δικτύου) ανεξάρτητα από τον φυσικό εξοπλισμό. Η βασική ιδέα του διατυπώθηκε ήδη από τη δεκαετία του 1970, όταν οι Popek και Goldberg [53] όρισαν τις τυπικές απαιτήσεις για μία αρχιτεκτονική που υποστηρίζει virtualization τρίτης γενιάς. Η θεμελίωση αυτή οδήγησε σταδιακά στην ανάπτυξη ανώριμων αρχικά, αλλά ολοένα πιο ώριμων τεχνολογιών virtualization.

Στις σύγχρονες υπηρεσίες cloud, το virtualization αξιοποιείται ως ο κεντρικός μηχανισμός πολυενοικίασης (multi-tenancy). Πάροχοι public cloud όπως το Microsoft Azure [54], το Amazon Web Services (AWS) [55] και το Google Cloud Platform (GCP) [56] χρησιμοποιούν virtualization για να εκτελούν πολλαπλούς φόρτους εργασίας πελατών πάνω σε κοινή φυσική υποδομή, μεγιστοποιώντας τον βαθμό αξιοποίησης του εξοπλισμού [14]. Το virtualization διασφαλίζει δύο θεμελιώδεις ιδιότητες. Αυτές είναι, πρώτον, η απομόνωση (isolation) μεταξύ χρηστών και των περιβαλλόντων τους, ώστε ο φόρτος ενός πελάτη να μην επηρεάζει τους υπόλοιπους, και δεύτερον, η ελαστικότητα (elasticity), δηλαδή τη δυνατότητα δυναμικής κατανομής και αποδέσμευσης πόρων ανάλογα με τη ζήτηση [57]. Επιπλέον, το public cloud μετατρέπει τις κεφαλαιουχικές δαπάνες (Capital Expenditure, CapEx), δηλαδή τις εφάπαξ επενδύσεις σε φυσική υποδομή (διακομιστές, αποθηκευτικά συστήματα, δικτυακός εξοπλισμός), σε λειτουργικές δαπάνες (Operational Expenditure, OpEx), δηλαδή σε περιοδικό κόστος ανάλογο της πραγματικής χρήσης (pay-as-you-go). Η μετατόπιση αυτή μειώνει τον χρηματοοικονομικό κίνδυνο, καθώς οι οργανισμοί δεν χρειάζεται να δεσμεύσουν εκ των προτέρων κεφάλαια για υποδομή αβέβαιης ζήτησης [41].

Επιπρόσθετα, το virtualization απλοποιεί τη διαχείριση υποδομών μεγάλης κλίμακας,

καθώς οι εικονικές μηχανές (virtual machines, VMs) μπορούν να δημιουργηθούν, να μεταφερθούν μεταξύ φυσικών εξυπηρετητών (migration) και να ανακτηθούν μετά από αστοχία, με τρόπο αυτοματοποιημένο. Η δυνατότητα αυτή αξιοποιείται εκτενώς στη διαχείριση clusters, όπου πόροι πολλών φυσικών μηχανών παρουσιάζονται ως ένα ενοποιημένο σύνολο, κάτι που αποτελεί τη βάση για τα σύγχρονα συστήματα ενορχήστρωσης.

2.2 Virtual machines και hypervisor

Το VM αποτελεί ένα αυτόνομο λογισμικό περιβάλλον που προσομοιώνει πλήρη υπολογιστικό σύστημα, περιλαμβάνοντας εικονικό επεξεργαστή, μνήμη, δίσκο και δικτυακή διεπαφή. Κάθε VM εκτελεί το δικό της λειτουργικό σύστημα (guest OS), πλήρως ανεξάρτητο από το λειτουργικό του υποκείμενου εξυπηρετητή (host).

Ο hypervisor είναι το στρώμα λογισμικού που μεσολαβεί μεταξύ του φυσικού εξοπλισμού και των εικονικών μηχανών, αναλαμβάνοντας τη δρομολόγηση και απομόνωση πόρων. Διακρίνονται δύο κύριοι τύποι:

- Bare-metal hypervisor. Εκτελείται απευθείας πάνω στον φυσικό εξοπλισμό, χωρίς ενδιάμεσο λειτουργικό σύστημα. Παραδείγματα αποτελούν ο Xen [58], ο VMware ESXi [59] και ο Microsoft Hyper-V [60]. Ο τύπος αυτός προσφέρει κατά κανόνα χαμηλότερη επιβάρυνση (overhead) και υψηλότερη απόδοση, καθώς ο hypervisor ελέγχει απευθείας τον εξοπλισμό.

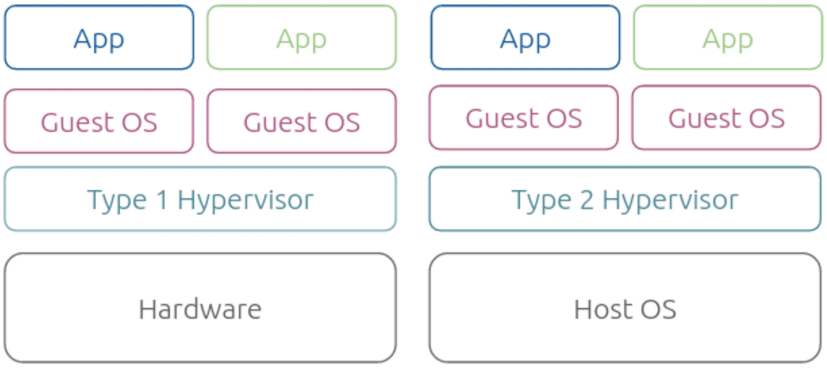
- Hosted hypervisor. Εκτελείται ως εφαρμογή πάνω στο κύριο λειτουργικό σύστημα του hypervisor. Χαρακτηριστικά παραδείγματα είναι το Oracle VirtualBox [61] και το VMware Workstation [62]. Αν και ευκολότερος στην εγκατάσταση, εισάγει πρόσθετη επιβάρυνση στους πόρους του συστήματος λόγω του ενδιάμεσου λειτουργικού συστήματος.

Η Εικόνα 2.1 απεικονίζει τη διαφορά μεταξύ των δύο τύπων. Αριστερά, ο bare-metal hypervisor (Τύπος 1) εκτελείται απευθείας πάνω στο φυσικό εξοπλισμό και φιλοξενεί τα λειτουργικά συστήματα χωρίς ενδιάμεσο στρώμα. Δεξιά, ο hosted hypervisor (Τύπος 2) εκτελείται ως εφαρμογή πάνω σε υπάρχον λειτουργικό σύστημα, με αποτέλεσμα τα εικονικά περιβάλλοντα να βρίσκονται σε ψηλότερο επίπεδο αφαίρεσης.

Ανεξαρτήτως τύπου, ο hypervisor παρέχει κρίσιμες λειτουργίες, όπως τον διαμοιρασμό μνήμης (memory ballooning), τον χρονοπρογραμματισμό εικονικών επεξεργαστών και τη μεταφορά VMs σε πραγματικό χρόνο (live migration). Οι λειτουργίες αυτές αποτελούν τη βάση πάνω στην οποία κτίστηκαν οι υπηρεσίες cloud [57].

2.3 Περιορισμοί του virtualization για σύγχρονα συστήματα microservices

Παρά τα πλεονεκτήματά του, το virtualization μέσω VMs παρουσιάζει ουσιαστικούς περιορισμούς όταν χρησιμοποιείται σε αρχιτεκτονικές microservices. Στις αρχιτεκτονικές αυτές, μία εφαρμογή αποσυντίθεται σε μικρότερες ανεξάρτητες υπηρεσίες, καθεμία με τον δικό



Εικόνα 2.1: *Σύγκριση bare-metal (Τύπος 1) και hosted (Τύπος 2) hypervisors. Πηγή: Ubuntu Blog [63].*

της κύκλο ανάπτυξης και κλιμάκωσης. Η εκτέλεση κάθε υπηρεσίας σε ξεχωριστό VM εισάγει σημαντικά ζητήματα:

- **Αυξημένη κατανάλωση πόρων**. Κάθε VM εκτελεί ολόκληρο λειτουργικό σύστημα, καταναλώνοντας αρκετή μνήμη και αποθηκευτικό χώρο μόνο για τον πυρήνα και τις βασικές υπηρεσίες [64]. Στη βιβλιογραφία [65] μετρήθηκε ποσοτικά αυτή η επιβάρυνση σε περιβάλλον υψηλών επιδόσεων (HPC), αποδεικνύοντας ότι ο hypervisor Xen εισάγει επιβάρυνση 5 έως 15% στην επεξεργαστική ισχύ και στην πρόσβαση μνήμης σε σχέση με εγγενή εκτέλεση. Σε ένα σύστημα με δεκάδες μικροϋπηρεσίες, αυτό μεταφράζεται σε σημαντική σπατάλη πόρων.
- **Χρόνος εκκίνησης**. Η εκκίνηση ενός VM απαιτεί τυπικά δεκάδες δευτερόλεπτα έως λεπτά, λόγω της αρχικοποίησης του λειτουργικού συστήματος. Αυτό καθιστά αργή την ελαστική κλιμάκωση και την αποκατάσταση μετά από αστοχία.
- **Χαμηλή πυκνότητα φόρτου**. Ο αριθμός VMs που μπορούν να φιλοξενηθούν σε έναν φυσικό εξυπηρετητή είναι περιορισμένος, σε αντίθεση με τα containers όπου ο ίδιος εξυπηρετητής μπορεί να φιλοξενήσει εκατοντάδες containers [66].
- **Πολυπλοκότητα διαχείρισης**. Η ενημέρωση (patching) και η ασφάλεια (security) πολλαπλών λειτουργικών συστημάτων εισάγει σημαντική λειτουργική επιβάρυνση.

Οι περιορισμοί αυτοί οδήγησαν σταδιακά στην αναζήτηση εναλλακτικών τεχνολογιών, ελαφρύτερων και ταχύτερων, που μοιράζονται τον πυρήνα του λειτουργικού συστήματος χωρίς να εισάγουν πλήρη virtualization, δηλαδή τα containers [67].

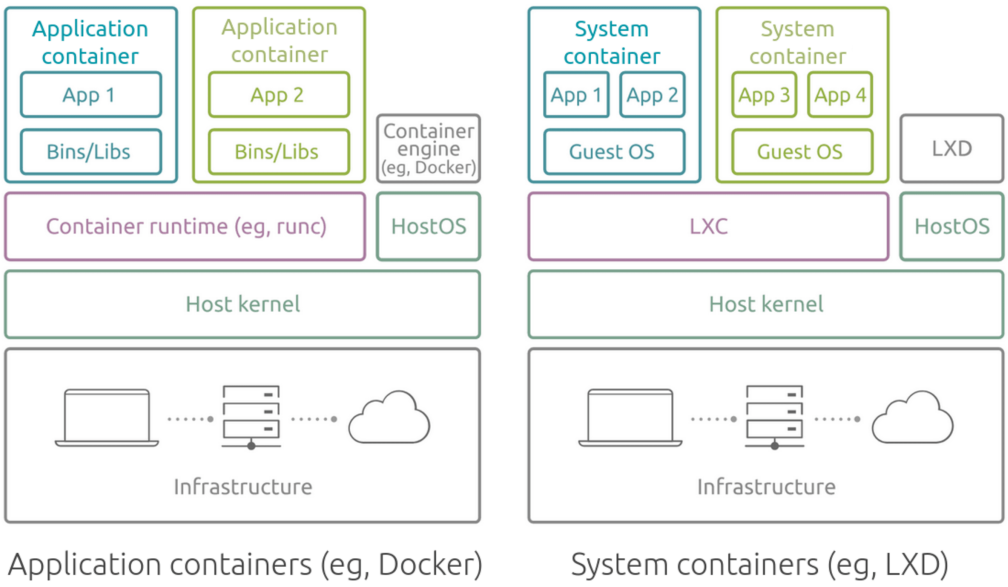
2.4 Containers και διαφοροποίηση από τα VMs

Τα containers αποτελούν μία ελαφριά μονάδα εκτέλεσης εφαρμογών, η οποία μοιράζεται τον πυρήνα (kernel) του λειτουργικού συστήματος του ξενιστή, αντί να εκτελεί δικό της λειτουργικό σύστημα. Κάθε container περιέχει τον κώδικα της εφαρμογής, τις βιβλιοθήκες, τα αρχεία ρύθμισης και τις εξαρτήσεις, πακεταρισμένα με τρόπο που εξασφαλίζει ομοιόμορφη εκτέλεση ανεξαρτήτως περιβάλλοντος [68].

Η θεμελιώδης διαφορά από τα VMs έγκειται στο επίπεδο αφαίρεσης:

- Τα VMs εικονικοποιούν και τον εξοπλισμό (hardware virtualization), εκτελώντας πλήρες λειτουργικό σύστημα πάνω στον host.
- Τα containers εικονικοποιούν μόνο στο επίπεδο λειτουργικού συστήματος (OS-level virtualization), χρησιμοποιώντας μηχανισμούς του πυρήνα (kernel) για απομόνωση.

Τα containers δεν αποτελούν ομοιογενή κατηγορία. Όπως φαίνεται στην Εικόνα 2.2, διακρίνονται σε application containers όπως το Docker [68], όπου κάθε container εκτελεί μία μεμονωμένη εφαρμογή μαζί με τις εξαρτήσεις της, και σε system containers όπως τα Linux Containers (LXC) και την υπηρεσία διαχείρισής τους, το Linux Container Daemon (LXD) [69], τα οποία προσομοιώνουν ολόκληρο λειτουργικό σύστημα και μπορούν να φιλοξενήσουν πολλαπλές διεργασίες εντός ενός container. Στην παρούσα εργασία αξιοποιούνται αποκλειστικά application containers, καθώς κάθε υπηρεσία (Kafka, Flink, Trino) εκτελείται σε ξεχωριστό container με τις δικές της εξαρτήσεις.



Εικόνα 2.2: *Αρχιτεκτονική application containers (Docker) και system containers (LXC/LXD). Πηγή: Ubuntu Blog [63].*

Αυτή η προσέγγιση, ελαφρύτερη ως προς την κατανάλωση υπολογιστικών πόρων σε σύγκριση με τα VMs, επιτυγχάνει χρόνους εκκίνησης της τάξης των χιλιοστών του δευτερολέπτου, κατανάλωση μνήμης μόλις λίγων Megabytes (MB) ανά container και δυνατότητα φιλοξενίας πολλαπλών container σε έναν μόνο node [66]. Η πειραματική αξιολόγηση στη βιβλιογραφία [65] επιβεβαιώνει ότι τα containers επιτυγχάνουν σχεδόν εγγενή απόδοση σε υπολογιστικά εντατικούς φόρτους, με επιβάρυνση μόλις 1 έως 3% στη χρήση της κεντρικής μονάδας επεξεργασίας, Central Processing Unit (CPU) [70], έναντι εκτέλεσης απευθείας στον εξοπλισμό (bare metal), ενώ η είσοδος/έξοδος δίσκου και δικτύου (I/O) παρουσιάζει μεγαλύτερη αλλά ακόμα αισθητά μικρότερη επιβάρυνση σε σύγκριση με VMs. Τα πλεονεκτήματα αυτά καθιστούν τα containers ιδανική τεχνολογία για αρχιτεκτονικές microservices, όπου κάθε υπηρεσία, π.χ. στην παρούσα εργασία ένας Kafka broker, ένας Flink Task Manager ή

ένας Trino worker, εκτελείται σε ξεχωριστό container που μπορεί να κλιμακώνεται ανεξάρτητα.

Οι βασικοί στόχοι που πετυχαίνονται μέσω των containers είναι [69]:

1. **Μεταφερσιμότητα** (portability). Μία εφαρμογή πακεταρισμένη ως container εκτελείται ομοιόμορφα σε τοπικό περιβάλλον ανάπτυξης, σε Continuous Integration / Continuous Delivery (CI/CD) pipeline [71] και σε παραγωγικό cloud.

2. **Αναπαραγωγιμότητα** (reproducibility). Η εικόνα (image) του container καθορίζει πλήρως το περιβάλλον εκτέλεσης, εξαλείφοντας αποκλίσεις μεταξύ περιβαλλόντων ("works on my machine" πρόβλημα) [72].

3. **Αποδοτική αξιοποίηση πόρων**. Πολλαπλά containers μοιράζονται τον ίδιο πυρήνα, εξαλείφοντας την επιβάρυνση πολλαπλών λειτουργικών συστημάτων.

4. **Ταχεία κλιμάκωση**. Η στιγμιαία εκκίνηση επιτρέπει ελαστική κλιμάκωση ως απόκριση σε μεταβολές φόρτου [73].

2.5 Container images, container registries και ο κύκλος ζωής τους

Ένα container image αποτελεί ένα αμετάβλητο (immutable) πακέτο που περιλαμβάνει όλα τα αρχεία, τις βιβλιοθήκες και τις ρυθμίσεις που χρειάζεται μία εφαρμογή για να εκτελεστεί [68, 72]. Τα images οργανώνονται σε στρώματα (layers), όπου κάθε στρώμα αντιστοιχεί σε ένα βήμα κατασκευής. Η στρωματοποιημένη δομή επιτρέπει τη μερική κοινή χρήση στρωμάτων μεταξύ images, μειώνοντας τον απαιτούμενο αποθηκευτικό χώρο και επιταχύνοντας τη μεταφορά [68].

Η κατασκευή images πραγματοποιείται μέσω αρχείων ρυθμίσεων και παραμέτρων (Dockerfiles) που ορίζουν δηλωτικά τα βήματα όπως το αρχικό image, την εγκατάσταση εξαρτήσεων, την αντιγραφή κώδικα και την εντολή εκκίνησης. Η δηλωτική αυτή κατασκευή εξασφαλίζει συνέπεια κατασκευής.

Τα αποθετήρια images (container registries) αποτελούν κεντρικά σημεία αποθήκευσης και διανομής images [74]. Μεταξύ των πιο διαδεδομένων περιλαμβάνονται το Docker Hub [75], το δημόσιο αποθετήριο αναφοράς με χιλιάδες δημόσια images, το Azure Container Registry (ACR) [76], ιδιωτικό αποθετήριο στο Azure που αξιοποιείται στην παρούσα εργασία για τη φιλοξενία images των προσομοιωτών δεδομένων IoT, του Flink και του Trino, καθώς και το Google Container Registry (GCR) [77], το Amazon Elastic Container Registry (ECR) [78] και αυτοδιαχειριζόμενα αποθετήρια.

Ο κύκλος ζωής ενός image ακολουθεί τυπικά τα στάδια που απεικονίζονται στην Εικόνα 2.3. Η αυτοματοποίηση αυτής της ροής μέσω CI/CD αγωγών (pipelines) εξασφαλίζει ταχεία και αξιόπιστη παράδοση λογισμικού [79].



Εικόνα 2.3: *Κύκλος ζωής image container, από την κατασκευή έως την εκτέλεση.*

2.6 Περιβάλλον εκτέλεσης container και βασικά ζητήματα ασφάλειας

Το περιβάλλον εκτέλεσης container (container runtime) είναι το λογισμικό που αναλαμβάνει τη δημιουργία, εκκίνηση και διαχείριση containers σε επίπεδο λειτουργικού συστήματος. Η αρχιτεκτονική του οικοσυστήματος container runtime ακολουθεί πλέον πρότυπα που ορίζονται από το Open Container Initiative (OCI) [80]:

- **Χαμηλού επιπέδου runtimes** (π.χ. runc). Υλοποιούν τη δημιουργία και εκτέλεση containers αξιοποιώντας απευθείας τα namespaces [81] και cgroups [82] του πυρήνα (kernel) [83].
- **Υψηλού επιπέδου runtimes** (π.χ. containerd, CRI-O). Παρέχουν διεπαφή διαχείρισης images, δικτύωσης και κύκλου ζωής containers. Το Kubernetes επικοινωνεί με τα runtimes αυτά μέσω της διεπαφής Container Runtime Interface (CRI) [84].

Δεδομένου ότι τα containers μοιράζονται τον πυρήνα του host, εισάγουν ζητήματα ασφάλειας που αντιμετωπίζονται μέσω πρακτικών όπως η εκτέλεση χωρίς δικαιώματα διαχειριστή (root), η χρήση ελαφριών εικόνων βάσης, η σάρωση ευπαθειών και η εφαρμογή πολιτικών δικτύου και φιλτραρίσματος κλήσεων συστήματος [64]. Οι πρακτικές αυτές αποκτούν ιδιαίτερη σημασία σε εφαρμογές IoT, όπου η μαζική επεξεργασία τηλεμετρικών δεδομένων διευρύνει την επιφάνεια επίθεσης [36].

Κεφάλαιο 3

Συστήματα Ορχήστρωσης Containers και Kubernetes

Στο κεφάλαιο αυτό παρουσιάζονται τα συστήματα ορχήστρωσης containers και τα βασικά δομικά στοιχεία του Kubernetes. Ενώ τα containers παρέχουν την απομόνωση (isolation) και τη μεταφερσιμότητα σε επίπεδο μεμονωμένης εφαρμογής, η εκτέλεση πολλαπλών containers σε κατανεμημένο περιβάλλον απαιτεί ένα ανώτερο στρώμα αυτοματοποίησης, το σύστημα ορχήστρωσης [7]. Το Kubernetes, αποτελώντας τη de facto πλατφόρμα ορχήστρωσης, αξιοποιείται στην παρούσα εργασία για τη φιλοξενία ολόκληρης της λύσης.

3.1 Ανάγκη συστημάτων ορχήστρωσης containers και βασικές λειτουργίες

Η ανάγκη για ορχήστρωση containers γίνεται εμφανής μόλις μία εφαρμογή ξεπεράσει τη φάση ανάπτυξης και πρέπει να εκτελεστεί σε παραγωγικό περιβάλλον πολλαπλών κόμβων (nodes). Σε ένα σύστημα microservices, κάθε υπηρεσία εκτελείται σε ένα ή περισσότερα containers τα οποία πρέπει να κατανέμονται σε nodes, να επιβιώνουν αστοχιών και να κλιμακώνονται ανάλογα με τον φόρτο. Η χειροκίνητη διαχείριση αυτών των ενεργειών σε πολλαπλά nodes είναι αδύνατη στην πράξη [85].

Ένα σύστημα ορχήστρωσης πρέπει να παρέχει τουλάχιστον τις ακόλουθες λειτουργίες:

- **Χρονοπρογραμματισμός** (scheduling). Αυτόματη τοποθέτηση containers σε κατάλληλο node, λαμβάνοντας υπόψη τους διαθέσιμους πόρους (CPU, μνήμη), τις ανάγκες των δεδομένων, καθώς και περιορισμούς τοποθέτησης. Αυτοί περιλαμβάνουν τους κανόνες συγγένειας (affinity) μεταξύ container και node, καθώς και τον μηχανισμό αποκλεισμού nodes (taints), ο οποίος αποτρέπει την εγκατάσταση ενός container εκτός αν το ίδιο διαθέτει την αντίστοιχη ανοχή (toleration).

- **Αυτοΐαση** (self-healing). Αυτόματη επανεκκίνηση containers που αποτυγχάνουν, αντικατάσταση containers σε nodes που παρουσιάζουν αστοχία και επαναδρομολόγηση δικτυακής κίνησης και αιτημάτων μόνο σε υγιή στιγμιότυπα.

- **Κλιμάκωση** (scaling). Οριζόντια κλιμάκωση (horizontal scaling), δηλαδή μεταβολή του πλήθους αντιγράφων (replicas) μίας υπηρεσίας, ή κατακόρυφη κλιμάκωση (vertical scaling), δηλαδή προσαρμογή των εκχωρημένων υπολογιστικών πόρων (CPU,

μνήμη) ανά container. Η κλιμάκωση μπορεί να είναι χειροκίνητη ή αυτοματοποιημένη βάσει μετρικών, μέσω μηχανισμών όπως ο Horizontal Pod Autoscaler (HPA) [86] και ο Vertical Pod Autoscaler (VPA) [87].

- **Δηλωτική ανάπτυξη** (declarative deployment). Ο χρήστης ορίζει την επιθυμητή κατάσταση (desired state) του συστήματος, και ο ενορχηστρωτής μεριμνά για τη σύγκλιση της τρέχουσας κατάστασης.

- **Ανακάλυψη υπηρεσιών και εξισορρόπηση φόρτου** (service discovery & load balancing). Αυτόματη ανάθεση ονομάτων μέσω Domain Name System (DNS), ιεραρχικού συστήματος ονοματοδοσίας για δίκτυα υπολογιστών [88], και δρομολόγηση αιτημάτων μεταξύ αντιγράφων μίας υπηρεσίας.

- **Κυλιόμενες ενημερώσεις** (rolling updates) και **αναστροφή** (rollback). Σταδιακή αντικατάσταση παλιών εκδόσεων με νέες, χωρίς διακοπή υπηρεσίας.

Πριν την επικράτηση του Kubernetes, πλατφόρμες όπως Docker Swarm [89], Apache Mesos [90]/Marathon [91], οι οποίες αμφότερες πλέον έχουν εγκαταλειφθεί [92] και τα εσωτερικά εργαλεία της Google όπως το Borg [85] και το Omega [7] επιχείρησαν να λύσουν τα ίδια προβλήματα. Το Kubernetes, επηρεασμένο σε μεγάλο βαθμό από τη δεκαπενταετή εμπειρία της Google με το Borg, αξιοποίησε τη δηλωτική προσέγγιση, τη μοντελοποίηση μέσω αντικειμένων Application Programming Interface (API) [93] και την επεκτασιμότητα μέσω Custom Resource Definitions [94], ώστε να καθιερωθεί ως το κυρίαρχο ανοιχτού κώδικα σύστημα ορχήστρωσης [7].

3.2 Kubernetes βασικές έννοιες και αρχιτεκτονική

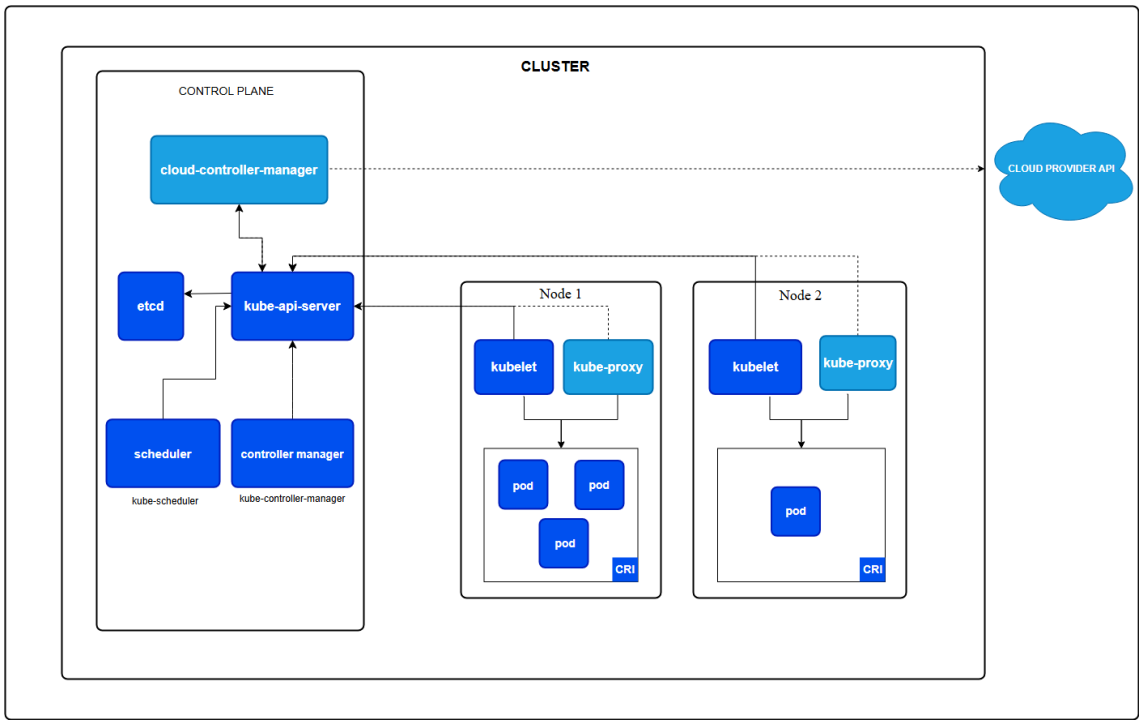
Το Kubernetes είναι ένα σύστημα ορχήστρωσης containers ανοιχτού κώδικα, που αυτοματοποιεί την ανάπτυξη, την κλιμάκωση και τη διαχείριση πακεταρισμένων (containerized) εφαρμογών [95, 96]. Σχεδιάστηκε αρχικά από μηχανικούς της Google και δωρήθηκε στο Cloud Native Computing Foundation (CNCF) [97] το 2014.

Η θεμελιώδης αρχιτεκτονική στηρίζεται σε ένα cluster που αποτελείται από δύο λογικά επίπεδα:

- Το **επίπεδο ελέγχου** (control plane), που είναι υπεύθυνο για τη λήψη αποφάσεων όπως ο χρονοπρογραμματισμός εργασιών, η αυτοΐαση και η κλιμάκωση της υποδομής καθώς και τη διατήρηση της επιθυμητής κατάστασης.

- Το **επίπεδο δεδομένων** (data plane ή worker nodes), όπου εκτελούνται τα containers των εφαρμογών.

Η Εικόνα 3.1 παρουσιάζει την αρχιτεκτονική ενός Kubernetes cluster [95]. Στο αριστερό τμήμα φαίνεται το επίπεδο ελέγχου με τα βασικά συστατικά (kube-apiserver, etcd, scheduler, controller-manager), ενώ στο δεξί τμήμα απεικονίζονται οι κόμβοι-εργάτες (worker-nodes) που εκτελούν τα containers των εφαρμογών μέσω kubelet, kube-proxy και



Εικόνα 3.1: *Αρχιτεκτονική cluster Kubernetes. Επίπεδο ελέγχου (αριστερά) και worker-nodes (δεξιά). Πηγή: Kubernetes Documentation [95].*

container runtime. Η σχέση μεταξύ των δύο επιπέδων γίνεται μέσω του kube-apiserver, ο οποίος αποτελεί το μοναδικό σημείο επικοινωνίας.

Η βασική μονάδα ανάπτυξης στο Kubernetes είναι το pod, το οποίο είναι ένα σύνολο ενός ή περισσότερων containers που μοιράζονται δικτυακό χώρο (network namespace) και, προαιρετικά, αποθηκευτικό χώρο (volumes). Κάθε pod λαμβάνει μοναδική Internet Protocol (IP) διεύθυνση εντός του cluster, επιτρέποντας απευθείας επικοινωνία μεταξύ οποιωνδήποτε pods χωρίς μετάφραση διευθύνσεων δικτύου (Network Address Translation, NAT) [98], δηλαδή NAT-free networking [99].

3.2.1 Συστατικά του Kubernetes

Τα βασικά συστατικά του επιπέδου ελέγχου συνοψίζονται ακολούθως:

- **kube-apiserver**. Ο κεντρικός εξυπηρετητής API, σημείο εισόδου για κάθε αλληλεπίδραση (χρήστες, εργαλεία Command Line Interface (CLI), εσωτερικά στοιχεία). Παρέχει ένα RESTful API και επικυρώνει τα αντικείμενα πριν τα αποθηκεύσει.

- **etcd**. Κατανεμημένη βάση κλειδιού-τιμής (key-value store) που αποθηκεύει ολόκληρη την κατάσταση του cluster. Χρησιμοποιεί τον αλγόριθμο Raft [100] για ομοφωνία (consensus).

- **kube-scheduler**. Ο χρονοπρογραμματιστής, υπεύθυνος για την ανάθεση pods σε nodes. Αξιολογεί βάσει φίλτρων (filtering) και βαθμολόγησης (scoring), λαμβάνοντας υπόψη πόρους, περιορισμούς, τοπικότητα δεδομένων και κανόνες συγγένειας.

- **kube-controller-manager**. Εκτελεί βρόχους ελέγχου (control loops) που ελέγχουν τη σύγκλιση της πραγματικής κατάστασης με την επιθυμητή. Περιλαμβάνει ελεγκτές για τους τύπους εργασιών που θα αναλύσουμε παρακάτω όπως Deployments, ReplicaSets, Nodes, Endpoints, κ.ά.
- **cloud-controller-manager**. Επεκτείνει τους ελεγκτές με λειτουργίες ειδικές για τον πάροχο public cloud, όπως η δημιουργία load balancers ή δίσκων στο Azure.

Σε κάθε worker node εκτελούνται:

- **kubelet**. Ο πράκτορας (agent) δέχεται οδηγίες από τον apiserver και φροντίζει για τη δημιουργία και εποπτεία containers μέσω του container runtime.
- **kube-proxy**. Υλοποιεί τους κανόνες δικτύωσης (iptables ή IPVS) για την πρόσβαση στις υπηρεσίες (Services).
- **Container runtime**. Το λογισμικό εκτέλεσης containers (π.χ. containerd), συμβατό με τη διεπαφή CRI.

3.2.2 Πρόσθετα δικτύωσης cluster

Η δικτύωση στο Kubernetes ακολουθεί ένα επίπεδο μοντέλο όπου κάθε pod αποκτά δική του IP διεύθυνση, και κάθε pod μπορεί να επικοινωνήσει με οποιοδήποτε άλλο pod χωρίς NAT [98]. Η υλοποίηση αυτού του μοντέλου εξαρτάται από τα πρόσθετα Container Network Interface (CNI) [99], τα οποία αναλαμβάνουν την εκχώρηση IP διευθύνσεων σε pods, τη δημιουργία εικονικών δικτυακών διεπαφών (veth pairs), τη δρομολόγηση κυκλοφορίας εντός και μεταξύ nodes, και προαιρετικά, την κρυπτογράφηση κυκλοφορίας (WireGuard, IPsec) και την επιβολή πολιτικών δικτύου (NetworkPolicies).

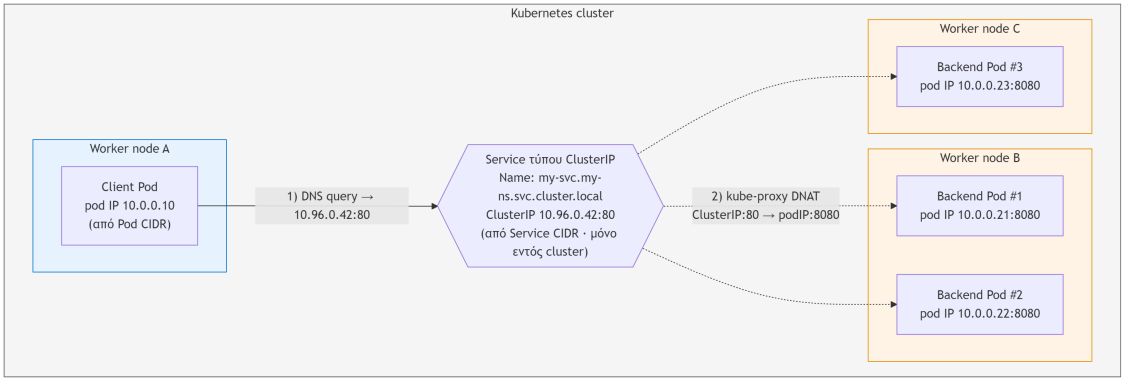
Δημοφιλή CNI plugins περιλαμβάνουν τα Calico [101], Cilium [102], Flannel [103] και Azure CNI [104]. Στην παρούσα εργασία, το cluster AKS χρησιμοποιεί Azure CNI, το οποίο εκχωρεί IP διευθύνσεις από το υποδίκτυο του VNet, επιτρέποντας απευθείας δικτυακή επικοινωνία μεταξύ pods και Azure υπηρεσιών όπως το ADLS χωρίς επιπρόσθετη δρομολόγηση [104].

3.2.3 Υπηρεσίες (services) και δημοσίευση εφαρμογών

Τα pods στο Kubernetes είναι εφήμερα, δηλαδή μπορεί να τερματιστούν και να αναδημιουργηθούν σε διαφορετικό node, λαμβάνοντας νέα IP. Η αφαίρεση (abstraction) service παρέχει ένα σταθερό σημείο πρόσβασης (stable endpoint) σε ένα σύνολο pods, δρομολογώντας αυτόματα την κυκλοφορία στα υγιή στιγμιότυπα [99].

Οι κύριοι τύποι υπηρεσιών είναι:

- **ClusterIP**. Εσωτερική IP εντός του cluster (βλ. Εικόνα 3.2). Ο τύπος αυτός χρησιμοποιείται π.χ. για τη σύνδεση Flink → Kafka ή Trino → Paimon.
- **NodePort**. Αντιστοιχίζει την υπηρεσία σε μία θύρα κάθε node, επιτρέποντας εξωτερική πρόσβαση δηλαδή από εκτός του cluster.



Εικόνα 3.2: *Η αφαίρεση Service με παράδειγμα τύπου ClusterIP. Στο Service εκχωρείται μια εσωτερική IP (η ClusterIP, εδώ 10.96.0.42) από ξεχωριστό εύρος διευθύνσεων (Service CIDR), διακριτό από το Pod CIDR (10.0.0.0/24) από όπου παίρνουν IP τα pods. Ο Client Pod (1) επιλύει μέσω DNS το όνομα του Service στη ClusterIP, και ο kube-proxy (2) εκτελεί μετάφραση διευθύνσεων (DNAT) από τη ClusterIP προς την IP ενός υγιούς backend pod, ανεξάρτητα από τον worker node στον οποίο εκτελείται. Η ClusterIP είναι προσβάσιμη μόνο εντός του cluster.*

- **LoadBalancer**. Δημιουργεί αυτόματα ένα load balancer στον εκάστοτε πάροχο cloud (π.χ. Azure Load Balancer), δημοσιεύοντας μία εξωτερική IP. Στην παρούσα εργασία, αξιοποιείται για την πρόσβαση στο Flink UI, στο Grafana UI και στο Trino UI.

- **Headless Service**. Υπηρεσία χωρίς ClusterIP, που επιστρέφει απευθείας τις IP διευθύνσεις των pods μέσω DNS. Χρησιμοποιείται από StatefulSets (π.χ. Kafka brokers) όπου κάθε pod χρειάζεται σταθερή ταυτότητα.

3.2.4 Πόροι φόρτου εργασίας (workloads)

Στον πυρήνα του Kubernetes βρίσκεται το πρότυπο των ελεγκτών (controllers) οι οποίοι είναι βρόχοι ελέγχου (control loops) που παρακολουθούν συνεχώς την πραγματική κατάσταση του cluster και τη συγκλίνουν προς την επιθυμητή κατάσταση που έχει δηλώσει ο χρήστης [105]. Πάνω σε αυτή την αρχή, παρέχονται εξειδικευμένοι ελεγκτές φόρτου εργασίας (workload controllers) για τη δηλωτική διαχείριση των pods, καθένας με διαφορετική σημασιολογία κύκλου ζωής [106]. Στην παρούσα εργασία αξιοποιούνται οι εξής τύποι workload:

- **Deployment**. Ο πιο συνηθισμένος πόρος για εφαρμογές χωρίς κατάσταση (stateless). Παρέχει δηλωτικές ενημερώσεις για pods και τα αντίγραφά τους, καθώς ο χρήστης περιγράφει την επιθυμητή κατάσταση και ο ελεγκτής μεταβάλλει την πραγματική κατάσταση με ελεγχόμενο ρυθμό. Υποστηρίζει κυλιόμενες ενημερώσεις (rolling update), αναστροφή σε προηγούμενη αναθεώρηση (rollback) και κλιμάκωση πλήθους αντιγράφων (replicas) [107]. Στην παρούσα εργασία, χρησιμοποιείται για τους προσομοιωτές IoT και τον Flink JobManager.

- **StatefulSet**. Σχεδιασμένο για εφαρμογές με κατάσταση (stateful). Σε αντίθεση με ένα Deployment, διατηρεί σταθερή (sticky) ταυτότητα για κάθε pod, παρέχοντας σταθερό όνομα, σταθερό αποθηκευτικό χώρο μέσω Persistent Volume Claims (PVCs) [108]

και ελεγχόμενη σειρά εκκίνησης και τερματισμού, δηλαδή τα pods δημιουργούνται διαδοχικά και τερματίζονται με αντίστροφη σειρά [109]. Αξιοποιείται στην παρούσα εργασία για τους Kafka brokers και τους Trino coordinator και workers.

- **Job**. Πόρος μίας εκτέλεσης που δημιουργεί pods και παρακολουθεί τις επιτυχημένες ολοκληρώσεις τους. Όταν επιτευχθεί η σειρά βημάτων που του έχει ανατεθεί, η εργασία θεωρείται ολοκληρωμένη. Υποστηρίζει τόσο μη παράλληλη εκτέλεση όσο και παράλληλη, με σταθερό πλήθος ολοκληρώσεων ή ουρά εργασίας (work queue) [110]. Στην εργασία, χρησιμοποιείται για την αρχικοποίηση θεμάτων Kafka (init jobs).

- **CronJob**. Δημιουργεί Jobs σε επαναλαμβανόμενο χρονοδιάγραμμα γραμμένο σε μορφοποίηση Cron, αντίστοιχα με το crontab σε συστήματα Unix. Υποστηρίζει πολιτική ταυτοχρονισμού (Allow, Forbid, Replace) και όρια ιστορικού επιτυχημένων ή αποτυχημένων εκτελέσεων [111]. Στην παρούσα εργασία, αξιοποιείται για χρονοπρογραμματισμένες εργασίες αθροίσεων των δεδομένων.

- **DaemonSet**. Εξασφαλίζει ότι σε όλους (ή σε επιλεγμένους) nodes εκτελείται ένα αντίγραφο ενός pod. Καθώς nodes προστίθενται ή αφαιρούνται, τα αντίστοιχα pods δημιουργούνται ή συλλέγονται αυτόματα. Τυπική χρήση αποτελεί η εκτέλεση δαιμόνων αποθήκευσης, συλλογής αρχείων καταγραφής ή παρακολούθησης nodes [112]. Στην παρούσα εργασία, χρησιμοποιείται έμμεσα μέσω του εργαλείου παρακολούθησης Prometheus.

3.3 Κατανεμημένα συστήματα

Η ροή δεδομένων IoT που υλοποιείται στην παρούσα εργασία αποτελεί στην ουσία ένα κατανεμημένο σύστημα στο οποίο πολλαπλοί nodes συνεργάζονται για την επεξεργασία και αποθήκευση ροών δεδομένων, μοιραζόμενοι κατάσταση και συντονίζοντας ενέργειες μέσω δικτύου. Η κατανόηση των θεμελιωδών αρχών κατανεμημένων συστημάτων είναι απαραίτητη για τη σχεδίαση αξιόπιστων λύσεων [93].

3.3.1 Θεμελιώδεις αρχές

Ένα κατανεμημένο σύστημα ορίζεται ως σύνολο αυτόνομων υπολογιστικών οντοτήτων που συνεργάζονται μέσω ανταλλαγής μηνυμάτων, παρουσιαζόμενες στον χρήστη ως ενιαίο σύστημα [93]. Οι βασικές αρχές περιλαμβάνουν τη διαφάνεια (transparency), όπου ο χρήστης δεν χρειάζεται να γνωρίζει τη φυσική τοποθεσία των δεδομένων ή τον τρόπο αναπαραγωγής τους, την ανεκτικότητα σφαλμάτων (fault tolerance), όπου το σύστημα συνεχίζει τη λειτουργία του ακόμη και αν αποτύχουν μεμονωμένοι nodes ή δικτυακές συνδέσεις, την κλιμακωσιμότητα (scalability), δηλαδή την ικανότητα αύξησης χωρητικότητας μέσω προσθήκης nodes χωρίς δομικές αλλαγές, και τη συνέπεια (consistency), όπου όλοι οι nodes βλέπουν τα ίδια δεδομένα σε οποιαδήποτε χρονική στιγμή ή τουλάχιστον σε κάποιο χρονικό πλαίσιο.

3.3.2 Προκλήσεις συντονισμού και συνέπειας

Η σχεδίαση κατανεμημένων συστημάτων οφείλει να αντιμετωπίσει θεμελιώδεις αδυναμίες, όπως ορίζονται από το θεώρημα Συνέπειας (Consistency), Διαθεσιμότητας (Availability) και Ανοχής σε κατατμήσεις (partitions) δικτύου (Partition tolerance), γνωστό ως θεώρημα CAP [113], σύμφωνα με το οποίο ένα κατανεμημένο σύστημα μπορεί να εγγυηθεί ταυτόχρονα μόνο δύο από τις τρεις αυτές ιδιότητες [28].

Στο πλαίσιο της παρούσας εργασίας, οι επιλογές σχεδίασης αντανακλούν ρητά αυτή τη θεωρητική βάση. Το Kafka ταξινομείται ως AP καθώς επιλέγει διαθεσιμότητα και ανοχή partitions, με ρυθμιζόμενη συνέπεια μέσω του μηχανισμού αναπαραγωγής συγχρονισμένων αντιγράφων (In-Sync Replicas, ISR) και επιβεβαιώσεων εγγραφής (acks) [114]. Στο Flink διατηρείται ακριβώς μία (exactly-once) σημασιολογία μέσω σημείων ελέγχου (checkpoints), αξιοποιώντας αλγόριθμους κατανεμημένου στιγμιότυπου (Chandy-Lamport) [115]. Το etcd [116] του Kubernetes εγγυάται ισχυρή συνέπεια μέσω ομοφωνίας Raft [100], θυσιάζοντας μερικώς τη διαθεσιμότητα κατά τη διάρκεια partitions δικτύου.

3.3.3 Ο ρόλος της ορχήστρωσης στη διαχείριση κατανεμημένων πόρων

Το Kubernetes λειτουργεί ως στρώμα αφαίρεσης πάνω από την κατανεμημένη υποδομή, αναλαμβάνοντας τον αυτοματοποιημένο χειρισμό πόρων πολλαπλών nodes [7]. Δυνατότητες όπως ο αυτόματος χρονοπρογραμματισμός, η επανεκκίνηση αποτυχημένων pods, η κατανομή αποθηκευτικού χώρου μέσω Persistent Volumes [108] και η δυναμική κλιμάκωση μέσω Horizontal Pod Autoscaler (HPA) αντιμετωπίζουν πρακτικά πολλές από τις προκλήσεις που αναφέρθηκαν. Ωστόσο, η ορχήστρωση δεν εξαλείφει τα θεμελιώδη ζητήματα κατανεμημένων συστημάτων αλλά πρακτικά τα μεταφέρει σε ψηλότερο επίπεδο αφαίρεσης, ώστε οι μηχανικοί να εστιάσουν στη σχεδίαση δεδομένων και στη λογική εφαρμογής, αντί στη χαμηλού επιπέδου διαχείριση nodes [117, 118].

Κεφάλαιο 4

Ροές Δεδομένων IoT και Αρχιτεκτονική 'Data Lakehouse'

Στο κεφάλαιο αυτό παρουσιάζεται το υπόβαθρο των ροών δεδομένων IoT και η αρχιτεκτονική data lakehouse που αξιοποιείται στην εργασία. Αφού στα προηγούμενα κεφάλαια εξετάστηκαν τα θέματα virtualization και container orchestration, εδώ εστιάζουμε στο αντικείμενο εφαρμογής της λύσης, δηλαδή τη διαχείριση τηλεμετρικών ροών υψηλής κλιμάκωσης, αξιοποιώντας τεχνολογίες ανοιχτού κώδικα για τη μεταφορά, επεξεργασία, αποθήκευση και ανάλυση δεδομένων.

4.1 IoT και χαρακτηριστικά τηλεμετρίας

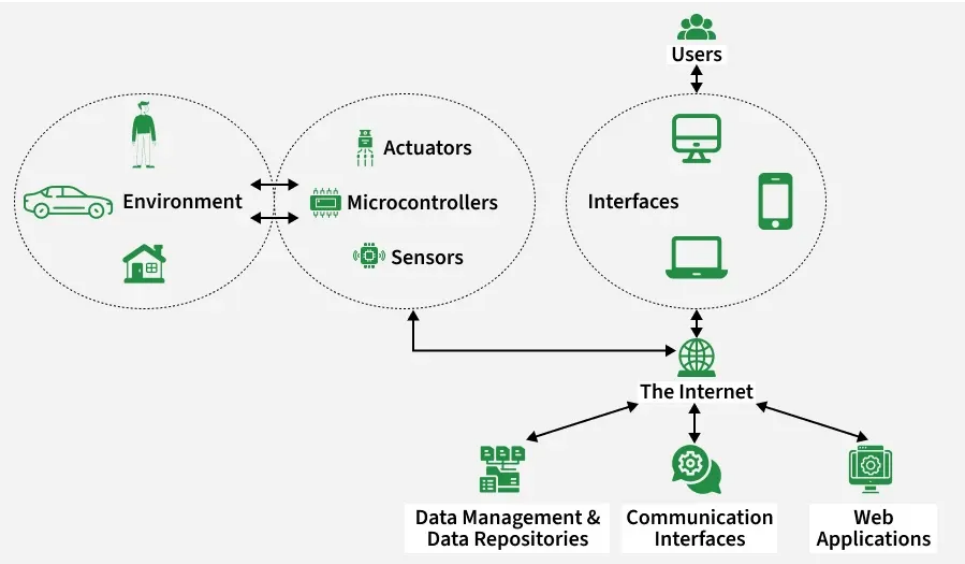
Το Διαδίκτυο των Πραγμάτων (Internet of Things, IoT) αναφέρεται σε ένα δίκτυο φυσικών αντικειμένων, αισθητήρων, εκκινητών, ενσωματωμένων συστημάτων και έξυπνων συσκευών, τα οποία συλλέγουν, επεξεργάζονται και ανταλλάσσουν δεδομένα μέσω δικτυακών υποδομών χωρίς ανθρώπινη παρέμβαση [16]. Η ιδέα αναφέρθηκε για πρώτη φορά από τον Kevin Ashton το 1999 [119], στο πλαίσιο της αλυσίδας εφοδιασμού και της τεχνολογίας Radio-Frequency Identification (RFID), αλλά εξελίχθηκε ταχέως ώστε να καλύψει ένα ευρύτατο φάσμα εφαρμογών, από βιομηχανικό αυτοματισμό και έξυπνες πόλεις μέχρι υγεία, γεωργία και ενέργεια [120, 121]. Το IoT ορίζεται [16] ως η σύγκλιση τριών οπτικών. Η οπτική 'things-oriented' αποδίδει σε κάθε φυσικό αντικείμενο ψηφιακή ταυτότητα, η οπτική 'internet-oriented' επεκτείνει τα πρωτόκολλα IP σε ενσωματωμένες συσκευές, ενώ η οπτική 'semantic-oriented' προσδίδει στα δεδομένα νοηματική δομή κατάλληλη για αυτόματη επεξεργασία.

Το αρχιτεκτονικό μοντέλο [1] τεσσάρων επιπέδων που έχει επικρατήσει ευρέως στη βιβλιογραφία περιλαμβάνει το επίπεδο αίσθησης (sensing), όπου φυσικοί αισθητήρες μετατρέπουν αναλογικά μεγέθη του περιβάλλοντος, θερμοκρασία, υγρασία, τάση, ρεύμα, σε ψηφιακά σήματα. Στο επίπεδο δικτύωσης (networking), πρωτόκολλα όπως το Message Queuing Telemetry Transport (MQTT) [122], το Constrained Application Protocol (CoAP) [123], το Hypertext Transfer Protocol (HTTP) [124] και το Advanced Message Queuing Protocol (AMQP) [125] αναλαμβάνουν τη μεταφορά δεδομένων από τις περιφερειακές συσκευές προς κεντρικούς κόμβους [121]. Επιπλέον, τεκμηριώνεται [121] ότι η επιλογή πρωτοκόλλου εξαρτάται από τους περιορισμούς εύρους ζώνης, κατανάλωσης ενέργειας και αξιοπιστίας.

Έτσι, το MQTT υπερέχει σε περιβάλλοντα χαμηλού εύρους ζώνης χάρη στο ελαφρύ μοντέλο δημοσίευσης-εγγραφής (publish-subscribe), ενώ το CoAP προσφέρει σημασιολογία αιτήματος-απόκρισης (request-response) αντίστοιχη του HTTP με ελάχιστη επιβάρυνση. Στο επίπεδο επεξεργασίας (processing), τα δεδομένα αποθηκεύονται, φιλτράρονται, συγκεντρώνονται και αναλύονται, ενώ στο επίπεδο εφαρμογής (application), οι τελικοί χρήστες αξιοποιούν τα αποτελέσματα μέσω πινάκων ελέγχου (dashboards), ειδοποιήσεων (alerts) και αναφορών (reports). Η αρχιτεκτονική αυτή αποτελεί τη βάση πάνω στην οποία σχεδιάστηκε και η ροή δεδομένων της παρούσας εργασίας.

Η ραγδαία εξάπλωση του IoT αποτυπώνεται με σαφήνεια στις προβλέψεις της περιόδου 2020-2030. Σύμφωνα με τη βιβλιογραφία [126], η παγκόσμια αγορά λύσεων IoT αναμένεται να αυξηθεί από 245 δισ. δολάρια USD το 2020 σε περίπου 8,1 τρισ. δολάρια το 2030, με σύνθετο ετήσιο ρυθμό ανάπτυξης (Compound Annual Growth Rate, CAGR) [127] 39%. Παράλληλα, ο αριθμός των συνδεδεμένων συσκευών παγκοσμίως προβλέπεται να εκτοξευθεί από 30,7 δισεκατομμύρια το 2020 σε 195,7 δισεκατομμύρια το 2030 (21%), ενώ ο μέσος αριθμός συσκευών ανά άτομο θα αυξηθεί από 6,6 σε 31,7 (16%) [126]. Ιδιαίτερο ενδιαφέρον παρουσιάζει η τομεακή ανάλυση δαπανών. Ο κλάδος ενέργειας και κοινής ωφέλειας εμφανίζει τον υψηλότερο ρυθμό αύξησης (CAGR 62%), ακολουθούμενος από τα ηλεκτρονικά καταναλωτικά προϊόντα (19%) και την υγεία (17%), ενώ ο τομέας βιομηχανίας και παραγωγής εκτιμάται σε σταθερότερη αύξηση (CAGR 6,5%). Συνολικά, η εγκατεστημένη βάση IoT στον βιομηχανικό τομέα αναπτύσσεται κατά πολύ ταχύτερα από τον καταναλωτικό (42% έναντι 14%), γεγονός που αναδεικνύει τον μετασχηματιστικό ρόλο του IIoT και εντείνει τις απαιτήσεις για κλιμακούμενες υποδομές αποθήκευσης και επεξεργασίας τηλεμετρικών ροών [126, 128]. Στον τομέα αναλυτικής IoT, τα έσοδα από κέντρα δεδομένων και αναλύσεις αναμένεται να αυξηθούν με 64%, ενώ η ενοποίηση (integration) συστημάτων με 40%, υπογραμμίζοντας την ανάγκη για ολοκληρωμένες πλατφόρμες δεδομένων [126].

Η Εικόνα 4.1 απεικονίζει τη γενική αρχιτεκτονική ενός συστήματος IoT.



Εικόνα 4.1: *Γενική αρχιτεκτονική του IoT, από τα φυσικά αντικείμενα και τους αισθητήρες, μέσω δικτυακής μεταφοράς, ως την επεξεργασία στο cloud και τις εφαρμογές. Πηγή: GeeksforGeeks [129].*

Τα δεδομένα τηλεμετρίας IoT παρουσιάζουν θεμελιώδεις διαφορές σε σχέση με τα παραδοσιακά δεδομένα επιχειρηματικών εφαρμογών. Η πρώτη και πλέον χαρακτηριστική ιδιότητα είναι η υψηλή συχνότητα παραγωγής. Εκατομμύρια αισθητήρες εκπέμπουν μετρήσεις σε διαστήματα δευτερολέπτων ή ακόμη και χιλιοστών του δευτερολέπτου, με αποτέλεσμα ο ρυθμός εισερχόμενων μηνυμάτων σε ένα κεντρικό σύστημα να φθάνει τα εκατομμύρια ανά ώρα [36]. Σε κλίμακα δεκάδων χιλιάδων εγκαταστάσεων, όπως στα πειραματικά σενάρια της παρούσας εργασίας (π.χ. 10.000 εγκαταστάσεις $\times$ 5 μηνύματα/δευτερόλεπτο $\approx$ 180 εκατ. μηνύματα/ώρα), η ανάγκη για κλιμακούμενη αρχιτεκτονική καθίσταται αναπόφευκτη.

Η δεύτερη ιδιότητα είναι η εγγενής δομή χρονοσειράς (time-series). Κάθε μέτρηση φέρει χρονοσφραγίδα (timestamp) και αναγνωριστικό Universally Unique Identifier (UUID) αισθητήρα, σχηματίζοντας μία διατεταγμένη ακολουθία σημείων στον χρόνο. Η χρονική σειρά αποτελεί τον βασικό τρόπο πρόσβασης, ανάλυσης και οπτικοποίησης τηλεμετρικών δεδομένων, ενώ αλγόριθμοι ανίχνευσης ανωμαλιών (anomaly detection) και πρόβλεψης (forecasting) εξαρτώνται άμεσα από τη χρονική διάταξη [128].

Η τρίτη ιδιότητα αφορά τον τεράστιο όγκο δεδομένων που σωρεύεται ακόμη και από μία μεμονωμένη πηγή. Ο Stankovic [128] επισημαίνει ότι η αποθήκευση ακατέργαστων τηλεμετρικών δεδομένων σε κλίμακα IoT απαιτεί αρχιτεκτονικές χαμηλού κόστους ανά gigabyte (GB), ικανές να υποστηρίζουν δεκάδες terabyte (TB) ημερησίως σε μεγάλες εγκαταστάσεις. Η ανάγκη αυτή συνδέεται άμεσα με την επιλογή αποθηκευτικών λύσεων τύπου data lake ή data lakehouse, που εξετάζονται στην ενότητα 4.5.

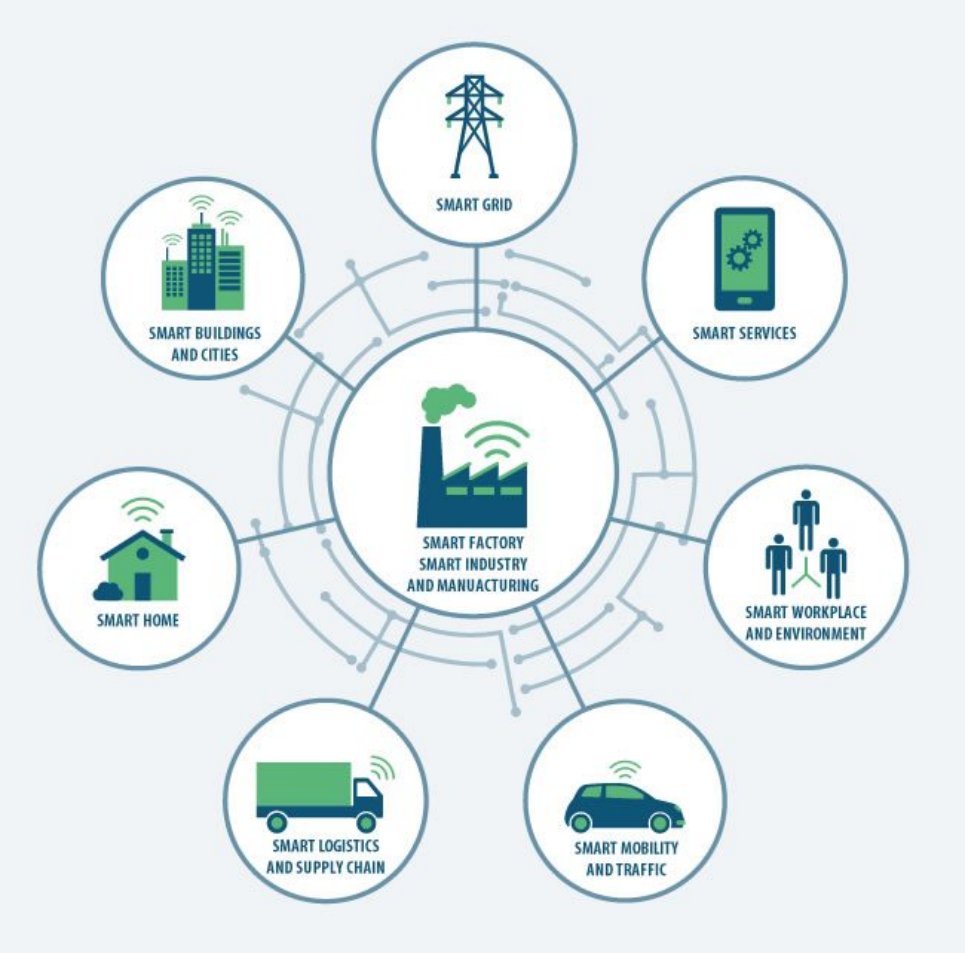
Η τέταρτη ιδιότητα σχετίζεται με την ανομοιογένεια. Διαφορετικοί τύποι αισθητήρων παράγουν μετρήσεις με ετερογενή δομή, μονάδα μέτρησης και συχνότητα δειγματοληψίας. Η ενοποίηση αυτών των δεδομένων σε κοινό σχήμα αποτελεί σημαντική σχεδιαστική πρόκληση, ιδιαίτερα σε περιβάλλοντα όπου νέοι τύποι αισθητήρων προστίθενται δυναμικά. Οι Sethi και Sarangi [130] παρατηρούν ότι η ετερογένεια δεν περιορίζεται μόνο στο επίπεδο δεδομένων αλλά εκτείνεται και στα πρωτόκολλα επικοινωνίας, τα λειτουργικά συστήματα και τους υπολογιστικούς πόρους των συσκευών, γεγονός που απαιτεί ενδιάμεσα στρώματα (middleware) ικανά για αφαίρεση υλικού (hardware abstraction) [131].

Τέλος, η πέμπτη ιδιότητα αφορά τις απώλειες και καθυστερήσεις μετάδοσης. Λόγω αστάθειας ασύρματου δικτύου, προσωρινής αστοχίας αισθητήρα ή δικτυακής συμφόρησης, τα μηνύματα ενδέχεται να χαθούν ολοσχερώς ή να φθάσουν εκτός χρονικής σειράς (out-of-order). Στη βιβλιογραφία [27] τεκμηριώνεται ότι η αξιόπιστη μεταφορά δεδομένων IoT απαιτεί πρωτόκολλα με δυνατότητα επανάληψης (retry) και αποθήκευσης και προώθησης (store-and-forward), ενώ το σύστημα επεξεργασίας οφείλει να διαχειρίζεται καθυστερημένα γεγονότα (late events) χωρίς απώλεια ορθότητας.

Πέραν της βιομηχανίας και των έξυπνων πόλεων, το IoT βρίσκει εφαρμογή σε πληθώρα τομέων. Στην ευφυή γεωργία, αισθητήρες εδάφους και κλίματος σε συνδυασμό με αλγορίθμους τεχνητής νοημοσύνης βελτιστοποιούν την κατανάλωση νερού και λιπασμάτων [132]. Στον κλάδο της υγείας, φορητοί αισθητήρες παρακολουθούν ζωτικές ενδείξεις ασθενών σε πραγματικό χρόνο, επιτρέποντας απομακρυσμένη διάγνωση [130]. Στον τομέα των μεταφορών, τα συνδεδεμένα οχήματα ανταλλάσσουν δεδομένα θέσης και ταχύτητας για τη βελτιστοποίηση της κυκλοφορίας [133]. Εκτιμάται ότι η οικονομική αξία που δημιουργεί το IoT θα υπερβεί τα 11 τρισεκατομμύρια δολάρια ετησίως μέσα στην επόμενη δεκαετία, καθιστώντας

το μία από τις πλέον επιδραστικές τεχνολογίες της εποχής [17].

Στο ευρύτερο πλαίσιο των βιομηχανικών εφαρμογών IoT, η έννοια της έξυπνης βιομηχανίας (smart industry) και της έξυπνης παραγωγής (smart manufacturing) αποτελεί βασικό πυλώνα του μετασχηματισμού που επιφέρει το Industry 4.0. Η έξυπνη παραγωγή ορίζεται ως πλήρως ενοποιημένα, συνεργατικά συστήματα παραγωγής που ανταποκρίνονται σε πραγματικό χρόνο στις μεταβαλλόμενες απαιτήσεις και συνθήκες του έξυπνου εργοστασίου (smart factory), του δικτύου εφοδιασμού και των αναγκών των πελατών [134]. Ο μετασχηματισμός αυτός οφείλεται κυρίως στην αυξανόμενη αυτοματοποίηση, τον ψηφιακό μετασχηματισμό, τη γεφύρωση ψηφιακού και φυσικού περιβάλλοντος μέσω του IoT, τις εξελισσόμενες τεχνολογίες παραγωγής και την εντατική αξιοποίηση των δεδομένων, με σύγκλιση τεχνολογιών πληροφορικής (Information Technology, IT) και επιχειρησιακής τεχνολογίας (Operational Technology, OT) γνωστή ως IT/OT convergence [135, 134]. Η Εικόνα 4.2 απεικονίζει τη σχέση μεταξύ έξυπνης βιομηχανίας και έξυπνης παραγωγής στο πλαίσιο του Industry 4.0.



Εικόνα 4.2: *'Εξυπνη βιομηχανία και έξυπνη παραγωγή. Η σχέση μεταξύ Industry 4.0, IoT, αυτοματοποίησης και ψηφιακού μετασχηματισμού στου βιομηχανικό τομέα. Πηγή: i-SCOOP [134].*

Στο πεδίο της ενεργειακής διαχείρισης, που αποτελεί το σενάριο εφαρμογής αυτής της εργασίας, οι αισθητήρες μετρούν μεγέθη κρίσιμα για την παρακολούθηση και βελτιστοποίηση κατανάλωσης, όπως η τάση (V), το ρεύμα (A), η ενεργός ισχύς (W, kW), η σωρευτική

κατανάλωση (kWh), ο συντελεστής ισχύος (power factor) και η συχνότητα δικτύου (Hz) [133]. Τα δεδομένα αυτά χρησιμοποιούνται τόσο σε πραγματικό χρόνο, για την ανίχνευση υπερφόρτωσης, βλαβών ή κλοπής ενέργειας, όσο και ιστορικά, για αναλύσεις τάσεων (trend analysis), πρόβλεψη φορτίου (load forecasting) και ενεργειακό σχεδιασμό [121]. Η ανάγκη ταυτόχρονης υποστήριξης και των δύο αυτών μοντέλων πρόσβασης, πραγματικού χρόνου και ιστορικού, καθιστά αναγκαία μία αρχιτεκτονική που ενοποιεί ροές γεγονότων με μόνιμη αποθήκευση, ακριβώς όπως η αρχιτεκτονική data lakehouse που υιοθετείται στην παρούσα εργασία.

4.2 Συλλογή δεδομένων και ροές γεγονότων (event streaming)

Η παραδοσιακή προσέγγιση για τη μεταφορά δεδομένων από την πηγή όπου παράγονται έως την αποθήκευσή τους βασιζόταν σε μοντέλα στα οποία αυτές οι ενέργειες γίνονται κατά παρτίδες (batch), όπου τα δεδομένα συγκεντρώνονταν σε τακτά χρονικά διαστήματα όπως κάθε ώρα ή ημέρα και μεταφέρονταν ως σύνολο. Αν και απλό στην υλοποίηση, το μοντέλο αυτό εισάγει καθυστέρηση που κυμαίνεται από λεπτά έως ώρες, σε αντίθεση με τις απαιτήσεις πραγματικού χρόνου πολλών εφαρμογών IoT [28].

Η εναλλακτική προσέγγιση βασίζεται στις ροές γεγονότων (event streaming), όπου κάθε μέτρηση αισθητήρα μοντελοποιείται ως ένα αμετάβλητο γεγονός (event) με timestamp, και δημοσιεύεται σε ένα σύστημα μηνυμάτων αμέσως μετά τη δημιουργία του. Οι καταναλωτές (consumers) μπορούν να διαβάσουν τα γεγονότα σε πραγματικό χρόνο ή σε ιστορική αναδρομή, ανάλογα με τις ανάγκες τους.

Τα πλεονεκτήματα του μοντέλου event streaming περιλαμβάνουν [21]:

1. **Χαμηλή καθυστέρηση**. Τα γεγονότα γίνονται διαθέσιμα σε χιλιοστά δευτερολέπτου.

2. **Αποσύζευξη** (decoupling). Οι παραγωγοί δεν γνωρίζουν τους καταναλωτές, επιτρέποντας ανεξάρτητη εξέλιξη κάθε υπηρεσίας.

3. **Δυνατότητα επανάληψης** (replayability). Τα γεγονότα διατηρούνται για ρυθμιζόμενο χρονικό διάστημα, ώστε νέοι καταναλωτές να μπορούν να επαναδιαβάσουν ιστορικά δεδομένα.

4. **Κλιμάκωση**. Η κατανομή δεδομένων σε partitions επιτρέπει πολλαπλούς καταναλωτές να λειτουργούν παράλληλα.

4.3 Apache Kafka, βασικές έννοιες (topics, partitions, brokers, replication)

Το Apache Kafka [21] είναι μία κατανεμημένη πλατφόρμα αποθήκευσης (distributed event store) και επεξεργασίας ροών γεγονότων (stream-processing platform). Αναπτύχθηκε αρχικά το 2010 στο LinkedIn από τους Jay Kreps, Neha Narkhede και Jun Rao, με σκοπό την αξιόπιστη διαχείριση μεγάλου όγκου δεδομένων δραστηριότητας σε πραγματικό χρόνο [136], και δόθηκε ως λογισμικό ανοικτού κώδικα μέσω του Apache Software

Foundation το 2011 [137]. Σήμερα αποτελεί de facto πρότυπο για αρχιτεκτονικές βασισμένες σε γεγονότα (event-driven architectures), με τυπικές περιπτώσεις χρήσης που περιλαμβάνουν συστήματα ανταλλαγής μηνυμάτων (messaging), παρακολούθηση δραστηριότητας ιστοτόπων (website activity tracking), συγκέντρωση μετρικών και αρχείων καταγραφής (metrics and log aggregation), επεξεργασία ροών (stream processing), καθώς και καταγραφή κατάστασης βάσει γεγονότων (event sourcing) σε κατανεμημένα συστήματα [138].

Τα θεμελιώδη συστατικά του Kafka είναι:

- **Θέμα** (**topic**). Λογική οργανωτική μονάδα στην οποία δημοσιεύονται τα γεγονότα, ανάλογη ενός φακέλου σε σύστημα αρχείων [139]. Ένα topic υποστηρίζει πολλαπλούς παραγωγούς (producers) και καταναλωτές (consumers) ταυτόχρονα, ενώ ο χρόνος διατήρησης των γεγονότων είναι ρυθμιζόμενος. Στην παρούσα εργασία ορίζονται δύο topics, ένα για μετρήσεις κεντρικών μετρητών ανά εγκατάσταση και ένα για μετρήσεις επιμέρους συσκευών (πρίζες, μηχανήματα κ.λπ.).

- **Partitions**. Κάθε topic κατανέμεται σε ένα ή περισσότερα partitions, τα οποία βρίσκονται σε διαφορετικούς brokers του Kafka cluster. Τα γεγονότα με το ίδιο κλειδί (key) αποθηκεύονται στο ίδιο partition, και οι καταναλωτές τα διαβάζουν ακριβώς με τη σειρά εγγραφής τους [139]. Τα partitions επιτρέπουν παράλληλη ανάγνωση, καθώς κάθε partition μπορεί να εξυπηρετείται από διαφορετικό καταναλωτή [140].

- **Αναπαραγωγή** (**replication**). Τα δεδομένα κάθε partition αναπαράγονται σε πολλαπλούς brokers (βλ. παρακάτω) για ανοχή σφαλμάτων και υψηλή διαθεσιμότητα, με προαρμοζόμενη τιμή πλήθους αντιγράφων (replication factor) [139]. Ο ηγέτης (leader) εξυπηρετεί τις αναγνώσεις και εγγραφές, ενώ οι ακόλουθοι (followers) διατηρούν αντίγραφα μέσω του μηχανισμού ISR [114].

- **Broker**. Εξυπηρετητής που αποτελεί το επίπεδο αποθήκευσης (storage layer) του Kafka cluster. Οι brokers μπορούν να εκτείνονται σε πολλαπλά κέντρα δεδομένων (datacenters), παρέχοντας ανοχή σφαλμάτων σε περίπτωση αστοχίας οποιουδήποτε εξυπηρετητή [139].

- **Εγγυήσεις παράδοσης**. Το Kafka παρέχει εγγυήσεις παράδοσης μηνυμάτων σε τρία επίπεδα [137, 139]:

  - **At-most-once** (το πολύ μία φορά). Κάθε μήνυμα παραδίδεται είτε μία φορά είτε καθόλου. Σε περίπτωση αστοχίας, ενδέχεται να χαθούν μηνύματα, αλλά δεν υπάρχουν διπλότυπα.

  - **At-least-once** (τουλάχιστον μία φορά). Κάθε μήνυμα παραδίδεται μία ή περισσότερες φορές. Δεν χάνονται μηνύματα, αλλά σε περίπτωση επανάληψης μετά από αστοχία ενδέχεται να εμφανιστούν διπλότυπα.

  - **Exactly-once** (ακριβώς μία φορά). Κάθε μήνυμα παραδίδεται ακριβώς μία φορά, χωρίς απώλειες ή διπλότυπα. Επιτυγχάνεται μέσω ταυτοδύναμων παραγωγών (idempotent producers) και συναλλαγών (transactions), με υψηλότερο υπολογιστικό κόστος.

Το Kafka λειτουργεί ως κατανεμημένο αρχείο καταγραφής (distributed commit log), στο οποίο τα μηνύματα αποθηκεύονται σε δίσκο με σειριακή πρόσβαση, αξιοποιώντας τους μηχανισμούς zero-copy, δηλαδή απευθείας μεταφορά δεδομένων από δίσκο στο δίκτυο χωρίς ενδιάμεση αντιγραφή στη μνήμη χρήστη, και page cache, δηλαδή χρήση της κύριας μνήμης ως κρυφή μνήμη δίσκου από το λειτουργικό σύστημα, ώστε οι αναγνώσεις να εξυπηρετούνται από μνήμη αντί δίσκου, για να επιτύχει εξαιρετικά υψηλή απόδοση, της τάξης εκατοντάδων χιλιάδων μηνυμάτων ανά δευτερόλεπτο ανά broker [21, 141].

Στο περιβάλλον Kubernetes, το Kafka αναπτύσσεται τυπικά ως StatefulSet, ώστε κάθε broker να διατηρεί σταθερή ταυτότητα δικτύου και μόνιμο αποθηκευτικό χώρο. Στην παρούσα εργασία, η ανάπτυξη γίνεται μέσω Kubernetes manifests.

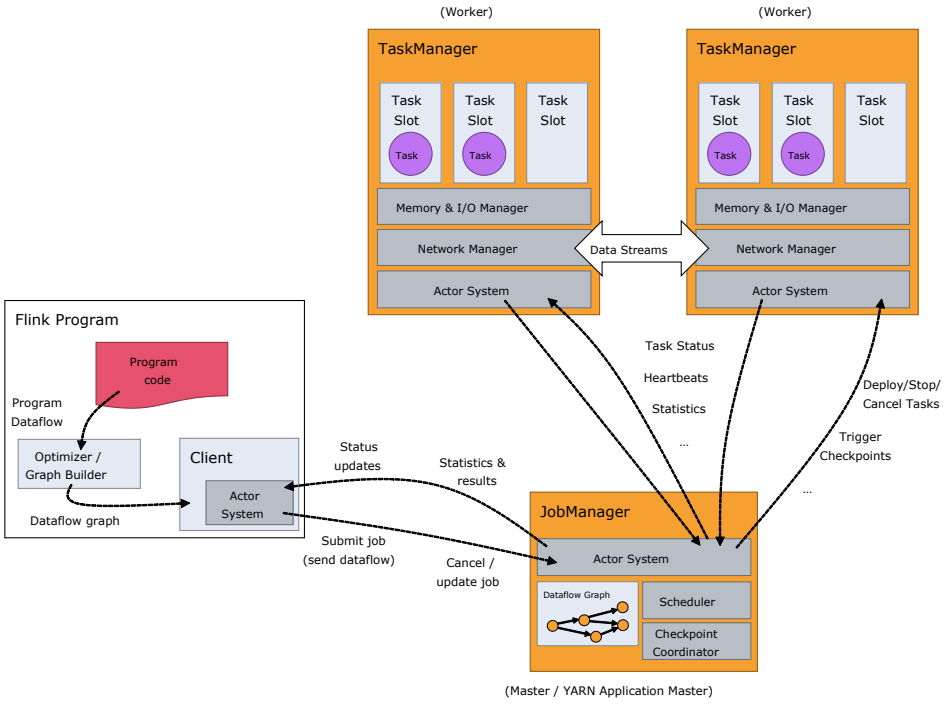
4.4 Apache Flink, επεξεργασία ροών δεδομένων

Το Apache Flink αποτελεί κατανεμημένο σύστημα που απαιτεί αποτελεσματική εκχώρηση και διαχείριση υπολογιστικών πόρων για την εκτέλεση εφαρμογών επεξεργασίας ροών [142]. Σχεδιάστηκε ώστε να εκτελείται σε όλα τα περιβάλλοντα τύπου clusters, να πραγματοποιεί υπολογισμούς σε ταχύτητα μνήμης (in-memory speed) και σε οποιαδήποτε κλίμακα [22]. Το Flink αντιμετωπίζει τα δεδομένα ως συνεχή ροή γεγονότων, ενώ την επεξεργασία κατά παρτίδες ορίζει ως ειδική περίπτωση περιορισμένης ροής. Οι απεριόριστες ροές δεν τερματίζουν και απαιτούν συνεχή επεξεργασία, ενώ οι περιορισμένες ροές έχουν καθορισμένη αρχή και τέλος και μπορούν να επεξεργαστούν ως σύνολα [22]. Αναδείχθηκε [143] ότι οι σύγχρονες μηχανές επεξεργασίας ροών, συμπεριλαμβανομένου του Flink, υπολειτουργούν σε σχέση με τις δυνατότητες του σύγχρονου υλικού. Συγκεκριμένα, η υψηλή συχνότητα αστοχιών κρυφής μνήμης (cache misses) λόγω δομών δεδομένων βασισμένων σε δείκτες (pointer-heavy data structures) και η επεξεργασία ανά πλειάδα (tuple-at-a-time) αντί κατά παρτίδες (vectorized processing) αποτελούν τους κύριους περιοριστικούς παράγοντες, ενώ η υιοθέτηση διατάξεων δεδομένων σε στήλες (columnar layouts) και η τοποθέτηση νημάτων με επίγνωση αρχιτεκτονικής μη ομοιόμορφης πρόσβασης μνήμης (Non-Uniform Memory Access, NUMA) [144] μπορούν να βελτιώσουν τη διεκπεραιωτική ικανότητα (throughput) έως και δεκαπλάσια.

4.4.1 Αρχιτεκτονική

Η αρχιτεκτονική του βασίζεται σε δύο τύπους διεργασιών, τον JobManager και έναν ή περισσότερους TaskManagers. Ο πελάτης (Client) δεν αποτελεί μέρος του χρόνου εκτέλεσης (runtime), αλλά χρησιμοποιείται για την προετοιμασία και αποστολή ενός γράφου ροής δεδομένων (dataflow graph) στον JobManager, μετά από την οποία μπορεί είτε να αποσυνδεθεί (detached mode) είτε να παραμείνει συνδεδεμένος για τη λήψη αναφορών προόδου (attached mode) [142]. Η Εικόνα 4.3 απεικονίζει τη δομή ενός Flink cluster.

Ο JobManager αναλαμβάνει τον συντονισμό της κατανεμημένης εκτέλεσης καθώς αποφασίζει πότε θα χρονοπρογραμματιστεί η επόμενη εργασία, αντιδρά σε ολοκληρωμένες εργασίες ή αστοχίες εκτέλεσης, συντονίζει τα σημεία ελέγχου (checkpoints) και ενορχηστρώνει την ανάκτηση μετά από σφάλματα [142]. Εσωτερικά αποτελείται από τρία συστατικά. Ο

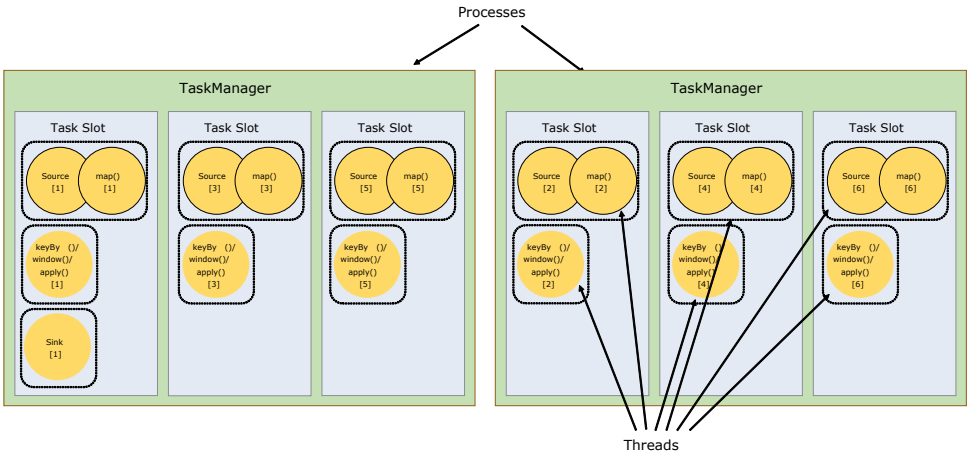


Εικόνα 4.3: *Δομή συστοιχίας Apache Flink. Ο JobManager συντονίζει την εκτέλεση, ενώ οι TaskManagers εκτελούν τις εργασίες της εφαρμογής. Ο πελάτης (Client) υποβάλλει τον γράφο εκτέλεσης. Πηγή: Εγχειρίδιο Apache Flink [142].*

Resource Manager διαχειρίζεται την εκχώρηση και αποδέσμευση πόρων μέσω υποδοχών εργασιών (task slots), οι οποίες αποτελούν τη μονάδα χρονοπρογραμματισμού πόρων, ενώ παράλληλα παρέχει διεπαφές προσαρμογής σε διαφορετικά περιβάλλοντα ενορχήστρωσης, όπως ο διαχειριστής πόρων Yet Another Resource Negotiator (YARN) [145], το Kubernetes, καθώς και αυτόνομες (standalone) εγκαταστάσεις. Ο Dispatcher παρέχει διεπαφή REST για την υποβολή εφαρμογών και εκκινεί νέο Job Master για κάθε υποβαλλόμενη εργασία, ενώ φιλοξενεί και τη διεπαφή ιστού (Flink Web UI). Ο Job Master είναι υπεύθυνος για τη διαχείριση της εκτέλεσης ενός μεμονωμένου γράφου εργασίας (Job Graph), καθώς λαμβάνει τη λογική εκτέλεση, τη μετατρέπει σε φυσική (Execution Graph) και χρονοπρογραμματίζει τις εργασίες. Πολλαπλές εργασίες μπορούν να εκτελούνται ταυτόχρονα στο ίδιο cluster, κάθε μία με τον δικό της Job Master. Σε ρύθμιση υψηλής διαθεσιμότητας, πολλαπλοί Job Managers λειτουργούν ταυτόχρονα, με έναν ως ηγέτη (leader) και τους υπόλοιπους σε αναμονή (standby) [142].

Οι TaskManagers (εργάτες, workers) εκτελούν τις εργασίες (tasks) του γράφου ροής δεδομένων, αποθηκεύουν προσωρινά (buffer) και ανταλλάσσουν τις ροές δεδομένων μεταξύ τους [142]. Η μικρότερη μονάδα πόρων σε έναν TaskManager είναι η υποδοχή εργασίας (task slot). Κάθε υποδοχή αντιπροσωπεύει ένα σταθερό υποσύνολο πόρων όπως για παράδειγμα, ένας TaskManager με τρεις υποδοχές αφιερώνει το 1/3 της διαχειριζόμενης μνήμης σε κάθε μία. Η κατανομή πόρων σε υποδοχές εξασφαλίζει ότι οι εργασίες δεν ανταγωνίζονται μεταξύ τους για μνήμη, αν και δεν πραγματοποιείται απομόνωση επεξεργαστή (CPU isolation). Το Flink επιτρέπει εξ ορισμού τον διαμοιρασμό υποδοχών (slot sharing) μεταξύ εργασιών διαφορετικών τελεστών της ίδιας εργασίας, ώστε μία μόνο υποδοχή να φι-

λοξενεί ολόκληρο το pipeline επεξεργασίας. Αυτό σημαίνει ότι το cluster χρειάζεται τόσες υποδοχές εργασίας όσες ο μέγιστος βαθμός παραλληλίας, χωρίς να απαιτείται υπολογισμός του συνολικού αριθμού εργασιών [142]. Η Εικόνα 4.4 απεικονίζει τον μηχανισμό υποδοχών εργασίας και τον διαμοιρασμό τους.



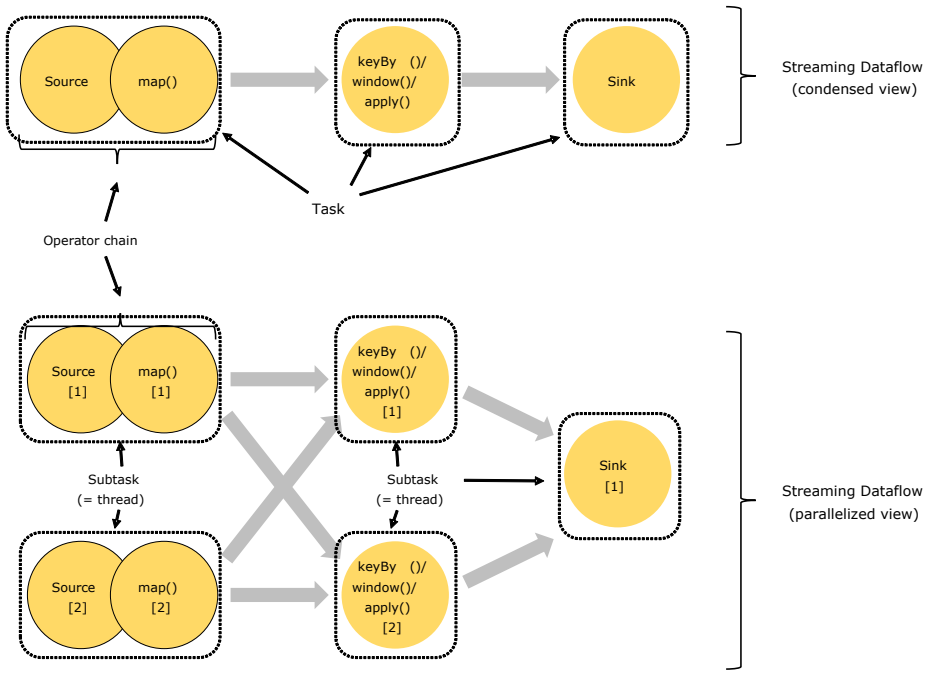
Εικόνα 4.4: *Μηχανισμός διαμοιρασμού υποδοχών εργασίας (slot sharing) στον Apache Flink. Εργασίες διαφορετικών τελεστών μοιράζονται την ίδια υποδοχή, μεγιστοποιώντας τη χρήση πόρων. Πηγή: Εγχειρίδιο Apache Flink [142].*

Το Flink βελτιστοποιεί περαιτέρω την εκτέλεση μέσω αλυσιδωτής σύνδεσης τελεστών (operator chaining), όπου υποεργασίες (subtasks) τελεστών συνδέονται σε ενιαίες εργασίες που εκτελούνται από ένα νήμα (thread), μειώνοντας την επιβάρυνση μεταφοράς δεδομένων μεταξύ νημάτων και ενδιάμεσης αποθήκευσης (buffering), αυξάνοντας τη συνολική διεκπεραιωτική ικανότητα και μειώνοντας την καθυστέρηση [142]. Η Εικόνα 4.5 απεικονίζει τον μηχανισμό αλυσιδωτής σύνδεσης τελεστών.

Ως προς τους τρόπους εκτέλεσης εφαρμογών, το Flink υποστηρίζει δύο κύρια μοντέλα ανάπτυξης [142]. Στη συστοιχία συνεδρίας (session cluster), ο πελάτης συνδέεται σε μία προϋπάρχουσα, μακρόβια συστοιχία που δέχεται πολλαπλές υποβολές εργασιών. Η διάρκεια ζωής της δεν εξαρτάται από τη διάρκεια κάποιας εργασίας, ο χρόνος εκκίνησης μειώνεται αλλά παρέχεται μειωμένη απομόνωση πόρων μεταξύ εργασιών. Στη συστοιχία εφαρμογής (application cluster), μία αποκλειστική συστοιχία δημιουργείται για κάθε εφαρμογή. Η τελευταία, συσκευάζεται σε εκτελέσιμο αρχείο Java ARchive (JAR) [146], η μέθοδος main() εκτελείται στη συστοιχία και η διάρκεια ζωής συνδέεται με εκείνη της εφαρμογής, προσφέροντας πλήρη απομόνωση πόρων. Στην παρούσα εργασία, η ανάπτυξη του Flink στο Kubernetes ακολουθεί τη λειτουργία συνεδρίας.

4.4.2 Διαχείριση κατάστασης (state)

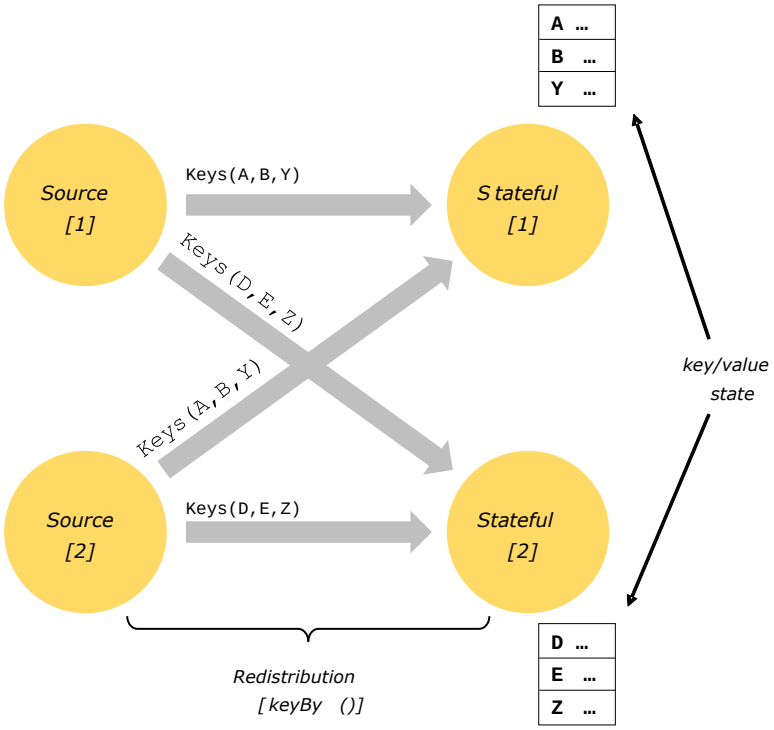
Η καρδιά του Flink βρίσκεται στη δυνατότητα διατήρησης κατάστασης (stateful processing) κατά την οποία κάθε τελεστής μπορεί να διατηρεί τοπική κατάσταση, π.χ. μετρητές, πίνακες κατακερματισμού, αθροιστές. Πολλές λειτουργίες σε μία ροή δεδομένων εξετάζουν ένα μεμονωμένο γεγονός κάθε φορά, ενώ κάποιες αποθηκεύουν πληροφορίες από πολλαπλά γεγονότα. Η αθροιστική λειτουργία ενός παραθύρου και η αναζήτηση μοτίβων γεγονότων αποτελούν χαρακτηριστικά παραδείγματα τελεστών με κατάσταση [147].



Εικόνα 4.5: *Αλυσιδωτή σύνδεση τελεστών (operator chaining) στον Apache Flink. Υποεργασίες τελεστών συγχωνεύονται σε ενιαίες εργασίες, μειώνοντας την επιβάρυνση επικοινωνίας. Πηγή: Εγχειρίδιο Apache Flink [142].*

Κατάσταση με κλειδί (keyed state). Η κατάσταση με κλειδί διατηρείται σε ένα ενσωματωμένο αποθηκευτικό χώρο κλειδιού-τιμής (key/value store). Η κατάσταση κατατέμνεται (partitioned) και κατανέμεται μαζί με τις ροές που γίνονται ανάγνωση από τους τελεστές, ώστε η πρόσβαση στην κατάσταση να γίνεται μόνο σε ροές με κλειδί (keyed streams), δηλαδή μετά από ανταλλαγή δεδομένων βάσει κλειδιού (keyBy), και να περιορίζεται στις τιμές που σχετίζονται με το κλειδί του τρέχοντος γεγονότος [147]. Η ευθυγράμμιση κλειδιών ροής και κατάστασης εγγυάται ότι όλες οι ενημερώσεις είναι τοπικές λειτουργίες, εξασφαλίζοντας συνέπεια χωρίς επιβάρυνση συναλλαγών, ενώ επιτρέπει στον Flink να αναδιανείμει την κατάσταση και να προσαρμόσει τα partitions ροής με διαφάνεια [147]. Η Εικόνα 4.6 απεικονίζει το partition κατάστασης.

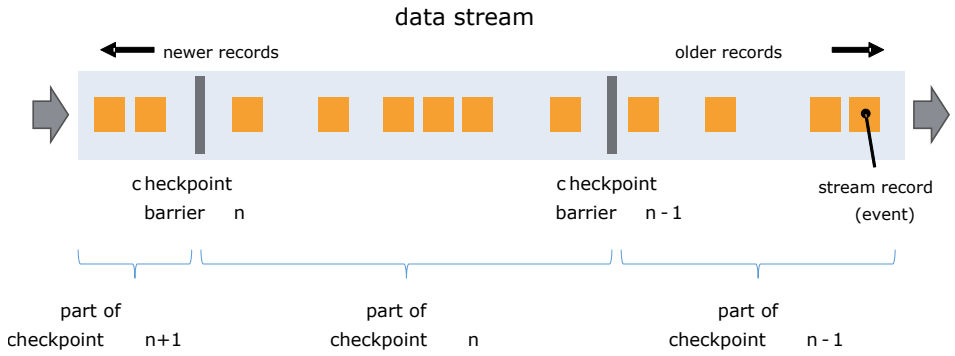
Το Flink παρέχει πολλαπλά πρωτόγονα κατάστασης (state primitives) για διαφορετικές δομές δεδομένων (ατομικές τιμές, λίστες και πίνακες κατακερματισμού), ώστε οι προγραμματιστές να επιλέγουν το αποδοτικότερο πρωτόγονο ανάλογα με το μοτίβο πρόσβασης [147]. Η κατάσταση αποθηκεύεται σε ένα από τα διαθέσιμα state backends, είτε στη μνήμη μέσω πίνακα κατακερματισμού (in-memory hash map) είτε στο RocksDB [148], ένα ενσωματωμένο σύστημα αποθήκευσης κλειδιού-τιμής σε δίσκο για μεγαλύτερες καταστάσεις [147, 149]. Τα state backends υλοποιούν και τη λογική λήψης στιγμιότυπου (snapshot) της κατάστασης ως μέρος ενός σημείου ελέγχου, ενώ η αλλαγή backend δεν απαιτεί τροποποίηση της λογικής εφαρμογής. Στην παρούσα εργασία χρησιμοποιείται το προεπιλεγμένο backend στη μνήμη (HashMapStateBackend), καθώς το μέγεθος κατάστασης στα ερωτήματα SQL είναι διαχειρίσιμο και η απόδοση ενδο-μνημιακής πρόσβασης υπερέχει [147].



Εικόνα 4.6: *Partition κατάστασης στον Apache Flink. Η κατάσταση κατανέμεται μαζί με τα κλειδιά της ροής, εξασφαλίζοντας τοπική πρόσβαση χωρίς επιβάρυνση συναλλαγών. Πηγή: Εγχειρίδιο Apache Flink [147].*

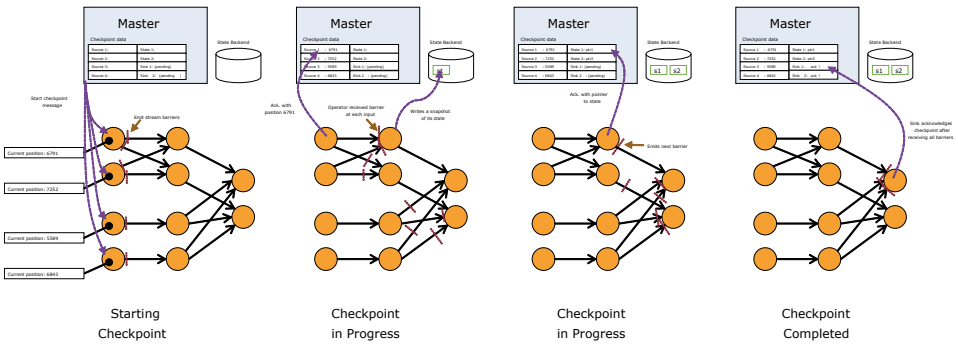
Checkpoints. Το Flink υλοποιεί ανοχή σφαλμάτων (fault tolerance) μέσω συνδυασμού αναπαραγωγής ροών (stream replay) και σημείων ελέγχου (checkpoints). Ένα checkpoint σημαδεύει ένα συγκεκριμένο σημείο σε κάθε ροή εισόδου μαζί με την αντίστοιχη κατάσταση κάθε τελεστή, ώστε η ροή δεδομένων να μπορεί να ανακτηθεί διατηρώντας σημασιολογία exactly-once [147]. Ο μηχανισμός βασίζεται στον αλγόριθμο κατανεμημένου στιγμιότυπου Chandy-Lamport [115] και λειτουργεί μέσω φραγμάτων (barriers). Ειδικά μηνύματα που εισάγονται στη ροή δεδομένων στις πηγές και ρέουν μαζί με τις εγγραφές χωρίς να τις υπερβαίνουν [147, 149]. Τα φράγματα χωρίζουν τις εγγραφές σε δύο σύνολα, αυτές που ανήκουν στο τρέχον snapshot και αυτές που ανήκουν στο επόμενο. Κάθε φράγμα φέρει τον αναγνωριστικό αριθμό του snapshot, και πολλαπλά φράγματα διαφορετικών snapshots μπορούν να υπάρχουν ταυτόχρονα στη ροή, επιτρέποντας ταυτόχρονη λήψη snapshots [147]. Η Εικόνα 4.7 απεικονίζει τα φράγματα σε μία ροή δεδομένων.

Οι τελεστές με πολλαπλές εισόδους πρέπει να ευθυγραμμίσουν τα φράγματα (barrier alignment). Όταν ένας τελεστής λάβει φράγμα snapshot n από μία ροή εισόδου, δεν μπορεί να επεξεργαστεί εγγραφές αυτής της ροής μέχρι να λάβει το αντίστοιχο φράγμα από όλες τις υπόλοιπες εισόδους. Μόλις φθάσει το τελευταίο φράγμα, ο τελεστής εκπέμπει όλες τις εκκρεμείς εγγραφές εξόδου, εκπέμπει το φράγμα snapshot n και αποθηκεύει ασύγχρονα την κατάστασή του στο state backend [147]. Το αποτέλεσμα κάθε snapshot περιλαμβάνει, για κάθε παράλληλη πηγή, τη θέση (offset) στη ροή κατά την έναρξη του snapshot, και για κάθε τελεστή, έναν δείκτη στην αποθηκευμένη κατάσταση. Κατά την ανάκτηση (recovery), το Flink επιλέγει το τελευταίο ολοκληρωμένο snapshot k, επαναδιατάσσει ολόκληρο τον κατανεμημένο γράφο ροής δεδομένων, αποδίδει σε κάθε τελεστή την κατάσταση του snapshot k



Εικόνα 4.7: *Φράγματα σημείων ελέγχου (checkpoint barriers) σε ροή δεδομένων Flink. Τα φράγματα εισάγονται στις πηγές και ρέουν μαζί με τις εγγραφές, χωρίζοντας τα δεδομένα σε snapshots. Πηγή: Εγχειρίδιο Apache Flink [147].*

και θέτει τις πηγές να ξεκινήσουν ανάγνωση από τη θέση S_k [147]. Η Εικόνα 4.8 απεικονίζει τον μηχανισμό σημείων ελέγχου.



Εικόνα 4.8: *Μηχανισμός σημείων ελέγχου στον Apache Flink. Κατά την ανάκτηση ο Flink επαναφέρει την κατάσταση και τις θέσεις (offsets) των πηγών από το τελευταίο ολοκληρωμένο snapshot. Πηγή: Εγχειρίδιο Apache Flink [147].*

Το Flink υποστηρίζει, επιπλέον, μη ευθυγραμμισμένα (unaligned checkpoints), στα οποία τα φράγματα μπορούν να προηγηθούν δεδομένων εν πτήσει (in-flight data), μειώνοντας τον χρόνο ολοκλήρωσης σε εφαρμογές με αργές διαδρομές δεδομένων, αλλά αυξάνοντας την πίεση I/O [147]. Στην παρούσα εργασία, ο μηχανισμός αυτός είναι απενεργοποιημένος (unaligned=false), καθώς τα ευθυγραμμισμένα checkpoints επαρκούν για τον φόρτο εργασίας. Αντίστοιχα, το Flink παρέχει και σημεία αποθήκευσης (savepoints), δηλαδή χειροκίνητα ενεργοποιούμενα snapshots κατάστασης για αναβάθμιση ή μετάβαση σε νέα συστοιχία [147], τα οποία ωστόσο δεν αξιοποιούνται στην τρέχουσα υλοποίηση.

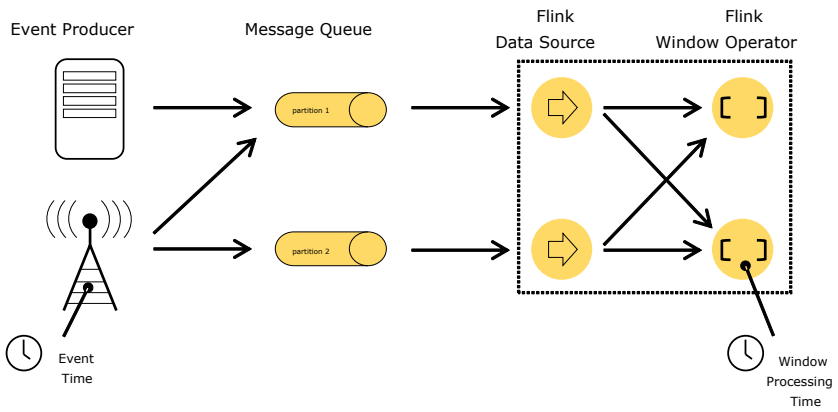
Σημασιολογία exactly-once έναντι at-least-once. Το barrier alignment εισάγει μικρή πρόσθετη καθυστέρηση, τυπικά της τάξης χιλιοστών δευτερολέπτου, αλλά εγγυάται σημασιολογία exactly-once, στην οποία κάθε εγγραφή επεξεργάζεται ακριβώς μία φορά, χωρίς απώλειες και χωρίς διπλότυπα. Εναλλακτικά, το Flink παρέχει τη δυνατότητα παράλειψης της ευθυγράμμισης για σημασιολογία at-least-once, στην οποία κάθε εγγραφή επεξεργάζεται τουλάχιστον μία φορά, χωρίς απώλειες, αλλά με πιθανά διπλότυπα, μειώνοντας την καθυστέρηση αλλά επιτρέποντας αναπαραγωγή ορισμένων εγγραφών κατά την ανάκτηση [147].

Στην παρούσα εργασία, η ροή εισαγωγής δεδομένων από Kafka σε Paimon εκτελείται σε λειτουργία EXACTLY_ONCE με διάστημα checkpoints ίσο με 60 δευτερόλεπτα, εξασφαλίζοντας ακρίβεια χωρίς διπλότυπα ακόμη και σε περίπτωση αστοχίας.

4.4.3 Χρονικές σημασιολογίες

Η χρονικά ευαίσθητη επεξεργασία ροών (timely stream processing) αποτελεί επέκταση της επεξεργασίας ροών με κατάσταση, στην οποία ο χρόνος διαδραματίζει ρόλο στον υπολογισμό. Αναλύσεις χρονοσειρών (time-series analysis), συγκεντρώσεις βάσει χρονικών περιόδων ή παράθυρα (windowed aggregations) και επεξεργασία γεγονότων βάσει χρόνου εμφάνισης (event-time processing) αποτελούν χαρακτηριστικά παραδείγματα [150].

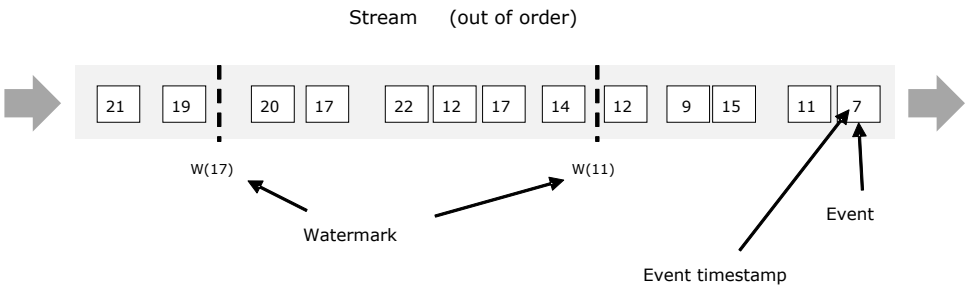
Το Flink υποστηρίζει δύο βασικές αντιλήψεις χρόνου [151, 150]. Ο χρόνος επεξεργασίας (processing time) αναφέρεται στο ρολόι συστήματος του μηχανήματος που εκτελεί τον αντίστοιχο τελεστή. Αποτελεί την απλούστερη αντίληψη χρόνου, δεν απαιτεί συντονισμό μεταξύ ροών και μηχανημάτων και παρέχει την καλύτερη απόδοση και τη χαμηλότερη καθυστέρηση. Ωστόσο, σε κατανεμημένα και ασύγχρονα περιβάλλοντα δεν παρέχει ντετερμινισμό, καθώς εξαρτάται από την ταχύτητα άφιξης εγγραφών στο σύστημα, τη ροή μεταξύ τελεστών και τυχόν διακοπές λειτουργίας [150]. Ο χρόνος γεγονότος (event time) αναφέρεται στον χρόνο κατά τον οποίο δημιουργήθηκε κάθε γεγονός στη συσκευή παραγωγής. Το timestamp ενσωματώνεται στην εγγραφή πριν αυτή εισέλθει στο Flink, και η πρόοδος του χρόνου εξαρτάται αποκλειστικά από τα δεδομένα. Σε ιδανικές συνθήκες, η επεξεργασία βάσει χρόνου γεγονότος παράγει πλήρως συνεπή και ντετερμινιστικά αποτελέσματα, ανεξάρτητα από τη σειρά άφιξης ή τον χρόνο επεξεργασίας, ακόμη και κατά την επανεπεξεργασία ιστορικών δεδομένων [150]. Η Εικόνα 4.9 απεικονίζει τη σχέση μεταξύ χρόνου γεγονότος και χρόνου επεξεργασίας.



Εικόνα 4.9: *Σχέση μεταξύ χρόνου γεγονότος (event time) και χρόνου επεξεργασίας (processing time) στο Apache Flink. Ο χρόνος γεγονότος πηγάζει από τα δεδομένα, ενώ ο χρόνος επεξεργασίας από το ρολόι του node. Πηγή: Εγχειρίδιο Apache Flink [150].*

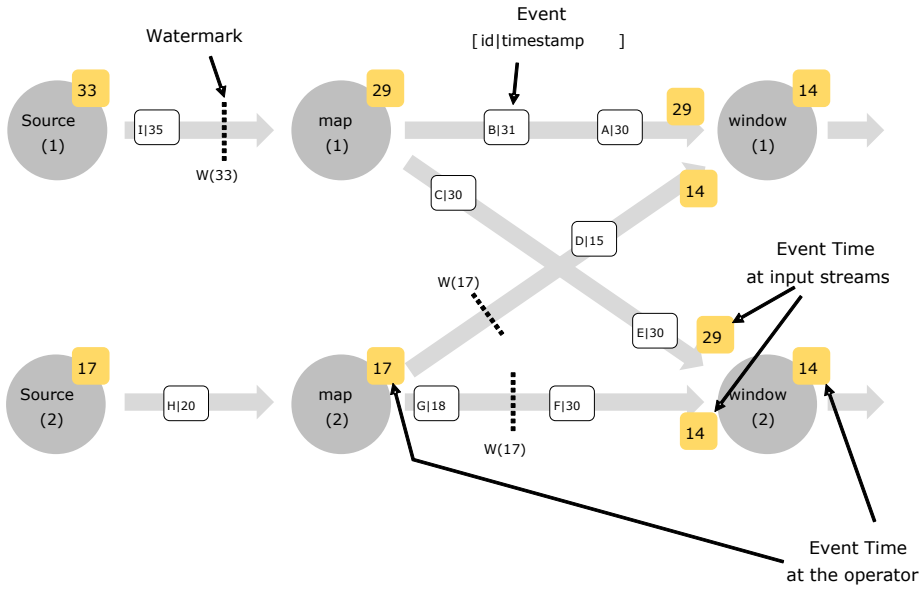
Υδατόσημα. Ένας επεξεργαστής ροών που υποστηρίζει χρόνο γεγονότος χρειάζεται μηχανισμό μέτρησης της προόδου αυτού του χρόνου. Ο μηχανισμός αυτός στο Flink είναι τα υδατόσημα (watermarks) τα οποία είναι ειδικά μηνύματα που ρέουν ως μέρος της ροής δεδομένων και φέρουν ένα timestamp t [150]. Ένα watermark $W(t)$ δηλώνει ότι ο χρόνος

γεγονότος στη ροή έχει φθάσει στο t, δηλαδή δεν αναμένονται πλέον γεγονότα με timestamp t' μικρότερο ή ίσο του t ($t' \leq t$). Τα watermarks είναι κρίσιμα για τη διαχείριση ροών εκτός σειράς (out-of-order streams), μόλις ένα watermark φθάσει σε έναν τελεστή, αυτός προχωρά το εσωτερικό ρολόι χρόνου γεγονότος στην τιμή του watermark [150]. Η Εικόνα 4.10 απεικονίζει τα watermarks σε ροή εκτός σειράς.



Εικόνα 4.10: *Watermarks σε ροή δεδομένων εκτός σειράς (out-of-order) στο Apache Flink. Τα watermarks δηλώνουν ότι όλα τα γεγονότα μέχρι ένα timestamp θα έπρεπε να έχουν φθάσει. Πηγή: Εγχειρίδιο Apache Flink [150].*

Σε παράλληλες ροές, τα watermarks παράγονται ανεξάρτητα σε κάθε παράλληλη εκτέλεση μίας συνάρτησης πηγής. Καθώς ρέουν στο πρόγραμμα, προωθούν τον χρόνο γεγονότος σε κάθε τελεστή. Τελεστές με πολλαπλές εισόδους χρησιμοποιούν ως τρέχοντα χρόνο γεγονότος το ελάχιστο (minimum) μεταξύ των χρόνων γεγονότος των ροών εισόδου τους [150]. Η Εικόνα 4.11 απεικονίζει τη ροή watermarks σε παράλληλες ροές.



Εικόνα 4.11: *Watermarks σε παράλληλες ροές δεδομένων Flink. Κάθε πηγή παράγει ανεξάρτητα watermarks, ενώ τελεστές με πολλαπλές εισόδους χρησιμοποιούν το ελάχιστο μεταξύ τους. Πηγή: Εγχειρίδιο Apache Flink [150].*

Καθυστερημένα γεγονότα (lateness). Ορισμένα γεγονότα ενδέχεται να παραβιάσουν τη συνθήκη του watermark, φθάνοντας μετά το watermark $W(t)$ ενώ φέρουν timestamp $t' \leq t$. Σε πολλά πρακτικά σενάρια, ιδιαίτερα σε περιβάλλοντα IoT με ασύρματη επικοινωνία, ορι-

σμένα γεγονότα μπορούν να καθυστερήσουν αυθαίρετα. Η υπερβολική καθυστέρηση των watermarks δεν αποτελεί λύση, καθώς αυξάνει αντίστοιχα την καθυστέρηση αποτίμησης παραθύρων [150]. Το Flink αντιμετωπίζει αυτή την πρόκληση μέσω ρυθμιζόμενης ανεκτικότητας (allowed lateness), ενώ γεγονότα που υπερβαίνουν αυτό το όριο ανακατευθύνονται σε πλευρικές εξόδους (side outputs) ή ενεργοποιούν ενημέρωση προηγουμένως ολοκληρωμένων αποτελεσμάτων [150, 152].

Στις εφαρμογές IoT της παρούσας εργασίας, η χρήση χρόνου γεγονότος σε συνδυασμό με watermark εξασφαλίζει ορθά αποτελέσματα ακόμη και σε συνθήκες καθυστέρησης δικτύου ή εκτός σειράς άφιξης μετρήσεων.

4.4.4 Παράθυρα

Τα παράθυρα (windows) αποτελούν τον θεμέλιο λίθο της επεξεργασίας απεριόριστων ροών. Χωρίζουν τη ροή σε 'κουβάδες' (buckets) πεπερασμένου μεγέθους, πάνω στους οποίους εφαρμόζονται υπολογισμοί όπως μετρήσεις, αθροίσματα και μέσοι όροι [152]. Κάθε window δημιουργείται μόλις φθάσει το πρώτο γεγονός που ανήκει σε αυτό, και αφαιρείται πλήρως όταν ο χρόνος (γεγονότος ή επεξεργασίας) υπερβεί το timestamp τέλους του.

Το Flink παρέχει τέσσερις προκαθορισμένους τύπους ανάθεσης παραθύρων (window assigners) [152] οι οποίοι είναι τα χρονικά παράθυρα ολίσθησης (tumbling windows), τα παράθυρα κύλισης (sliding windows), τα παράθυρα συνεδρίας (session windows) και τα καθολικά παράθυρα (global windows). Στην παρούσα εργασία αξιοποιούνται αποκλειστικά τα tumbling windows τα οποία είναι μη επικαλυπτόμενα windows σταθερού μήκους, στα οποία κάθε γεγονός ανήκει σε ακριβώς ένα window. Ο τύπος αυτός είναι ιδανικός για περιοδικές συγκεντρώσεις τηλεμετρικών δεδομένων, καθώς παράγει μία μοναδική τιμή ανά χρονικό διάστημα χωρίς επικαλύψεις. Οι υπόλοιποι τύποι οι οποίοι είναι sliding windows με επικαλυπτόμενα διαστήματα, session windows βάσει αδράνειας και τα global windows με προσαρμοσμένο ενεργοποιητή δεν αξιοποιούνται, καθώς η φύση των ενεργειακών μετρήσεων απαιτεί σταθερά, αμοιβαία αποκλειστικά χρονικά διαστήματα.

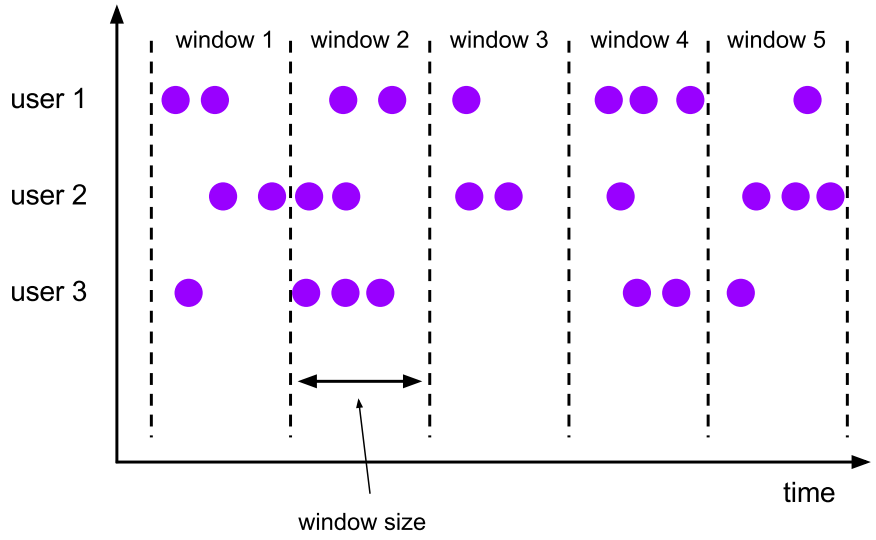
Η Εικόνα 4.12 απεικονίζει τα tumbling windows.

Συγκεκριμένα, στη λύση χρησιμοποιούνται δύο tumbling windows βάσει χρόνου γεγονότος. Το Flink παρέχει επιπλέον μηχανισμό ανεκτικότητας καθυστέρησης (allowed lateness) και πλευρικές εξόδους (side outputs) για καθυστερημένα γεγονότα [152], οι οποίοι ωστόσο δεν ενεργοποιούνται στην τρέχουσα υλοποίηση.

4.4.5 Στρώματα API

Το Flink προσφέρει τρία στρώματα API, κάθε ένα από τα οποία παρέχει διαφορετική ισορροπία μεταξύ συντομίας και εκφραστικότητας [142, 152]:

1. **ProcessFunctions**. Οι διεπαφές με τη μεγαλύτερη ευελιξία, παρέχουν λεπτομερή έλεγχο χρόνου και κατάστασης. Κάθε ProcessFunction μπορεί να τροποποιεί αυθαίρετα την κατάστασή της και να καταχωρεί χρονόμετρα (timers) για μελλοντικές κλήσεις, υλοποιώντας σύνθετη λογική ανά γεγονός.



Εικόνα 4.12: *Tumbling windows στο Apache Flink, μη επικαλυπτόμενα, σταθερού μήκους. Πηγή: Εγχειρίδιο Apache Flink [152].*

2. **DataStream API**. Παρέχει βασικά δομικά στοιχεία (primitives) για κοινές λειτουργίες επεξεργασίας ροών, όπως παραθύρωση, μετασχηματισμούς ανά εγγραφή (record-at-a-time transformations) και εμπλουτισμό γεγονότων μέσω εξωτερικών πηγών. Υποστηρίζει συναρτήσεις map(), reduce() και aggregate() σε Java, εμπνευσμένες από το προγραμματιστικό μοντέλο MapReduce [153].

3. **SQL & Table API**. Ενοποιημένα σχεσιακά API για streams και batches, αξιοποιώντας τον Apache Calcite για ανάλυση, επικύρωση και βελτιστοποίηση ερωτημάτων [154]. Τα ερωτήματα εκτελούνται με την ίδια σημασιολογία τόσο σε απεριόριστες ροές πραγματικού χρόνου όσο και σε περιορισμένα σύνολα δεδομένων, παράγοντας τα ίδια αποτελέσματα.

Στην παρούσα εργασία αξιοποιείται κυρίως η SQL & Table API, καθώς ο ορισμός πινάκων Paimon, η σύνδεση με Kafka topics και η επεξεργασία δεδομένων γίνονται αποκλειστικά μέσω δηλωτικών ερωτημάτων SQL.

4.4.6 Κλιμακωσιμότητα και απόδοση

Το Flink σχεδιάστηκε ώστε να εκτελεί εφαρμογές με κατάσταση stateful σε οποιαδήποτε κλίμακα. Οι εφαρμογές παραλληλοποιούνται σε ενδεχομένως χιλιάδες εργασίες (tasks) που κατανέμονται και εκτελούνται ταυτόχρονα σε μία συστοιχία, αξιοποιώντας πρακτικά απεριόριστους πόρους επεξεργαστών, μνήμης, δίσκου και δικτύου [142]. Χρήστες αναφέρουν εφαρμογές σε παραγωγικά περιβάλλοντα που επεξεργάζονται πολλαπλά τρισεκατομμύρια γεγονότων ημερησίως, διατηρούν κατάσταση πολλών terabytes και εκτελούνται σε χιλιάδες πυρήνες [13, 142].

Ως προς την απόδοση, το Flink σχεδιάστηκε ώστε η κατάσταση κάθε εργασίας να παραμένει τοπικά στον κόμβο εκτέλεσής της (local state access). Διατηρείται στη μνήμη ή, όταν

το μέγεθός της υπερβαίνει τη διαθέσιμη μνήμη, σε αποδοτικές δομές δεδομένων στον δίσκο. Έτσι, οι υπολογισμοί εκτελούνται χωρίς απομακρυσμένες κλήσεις δικτύου, επιτυγχάνοντας πολύ χαμηλή καθυστέρηση επεξεργασίας [147].

4.5 Δεδομένα τύπου 'data lakehouse'

4.5.1 Από τα data warehouses στις data lakes και την αρχιτεκτονική data lakehouse

Η εξέλιξη των αρχιτεκτονικών αποθήκευσης αναλυτικών δεδομένων ακολούθησε τρία διαδοχικά στάδια. Κατά τη δεκαετία του 1990, οι αποθήκες δεδομένων (data warehouses) κυριάρχησαν ως η βασική λύση. Δομημένα δεδομένα εισάγονταν μέσω διαδικασιών ETL σε σχεσιακά σχήματα βελτιστοποιημένα για αναλυτικά ερωτήματα [28]. Αν και αυτή η προσέγγιση εξασφάλιζε υψηλή απόδοση ερωτημάτων και εγγυήσεις ACID δηλαδή ατομικότητας (atomicity), συνέπειας (consistency), απομόνωσης (isolation) και μονιμότητας (durability) [155], παρουσίαζε σημαντικούς περιορισμούς όπως υψηλό κόστος αποθήκευσης, αδυναμία διαχείρισης ημι-δομημένων (semi-structured) ή μη δομημένων (unstructured) δεδομένων, και δυσκαμψία στην εξέλιξη σχημάτων [6].

Η έννοια του data lake εμφανίστηκε ως εναλλακτική στις αρχές της δεκαετίας του 2010, με στόχο την αποθήκευση τεράστιων όγκων δεδομένων, δομημένων, ημι-δομημένων και μη δομημένων, σε ακατέργαστη μορφή, χωρίς προκαθορισμένο σχήμα, σε χαμηλού κόστους αποθηκευτικό χώρο [156]. Οι Khine και Wang [157] ορίζουν το data lake ως ένα κεντρικό αποθετήριο που επιτρέπει την αποθήκευση δεδομένων σε οποιαδήποτε κλίμακα και μορφή, εφαρμόζοντας τη λογική 'schema-on-read', σύμφωνα με την οποία το μοντέλο-σχήμα καθορίζεται κατά την ανάγνωση αντί κατά την εγγραφή. Στη βιβλιογραφία [158] επισημαίνεται ότι η φιλοσοφία 'αποθήκευσε τα πάντα, ανάλυσέ τα αργότερα' προσέλκυσε οργανισμούς που επιθυμούσαν να διατηρήσουν ακατέργαστα δεδομένα χωρίς να δεσμεύονται εκ των προτέρων σε συγκεκριμένη χρήση.

Ωστόσο, οι λίμνες δεδομένων παρουσίασαν σημαντικά μειονεκτήματα. Στη βιβλιογραφία [10] εντοπίζονται τρεις κεντρικές προκλήσεις οι οποίες είναι η ανακάλυψη δεδομένων (data discovery), η ενοποίηση (integration) και η εξασφάλιση ποιότητας. Χωρίς μεταδεδομένα (metadata), μοντέλο (schema) και εγγυήσεις ακεραιότητας, τα data lakes εκφυλίζονται εύκολα σε 'βάλτους δεδομένων' (data swamps) [159]. Οι Sawadogo και Darmont [160] παρατηρούν ότι η σημαντικότερη αδυναμία αφορά τη διαχείριση μεταδεδομένων. Σε αντίθεση με τα data warehouses, τα data lakes δεν παρέχουν κεντρικό κατάλογο σχημάτων, γενεαλογία δεδομένων (data lineage) ή αυτόματο έλεγχο ποιότητας. Επιπλέον, η αδυναμία εφαρμογής ACID συναλλαγών οδηγεί σε ασυνέπειες κατά τις ταυτόχρονες εγγραφές, ενώ η απόδοση ερωτημάτων υστερεί σημαντικά σε σύγκριση με τις εξειδικευμένες αποθήκες δεδομένων [161].

Η αρχιτεκτονική data lakehouse [6] εισήχθη ακριβώς για να αντιμετωπίσει αυτά τα ζητήματα. Βασικός μηχανισμός της είναι ένα ανοιχτό στρώμα μεταδεδομένων (open metadata layer) που εδράζεται πάνω σε αρχεία αποθηκευμένα ως αντικείμενα σε χαμηλού κόστους χώρο object storage (π.χ. ADLS [162], Amazon S3 [163]) και προσδίδει σε αυτά εγγυήσεις αντίστοιχες μίας αποθήκης δεδομένων όπως ACID συναλλαγές, επιβολή και εξέλιξη

σχήματος (schema evolution), ευρετηρίαση (indexing), πρόσβαση σε παλαιότερες εκδόσεις των δεδομένων (time travel) και υψηλή απόδοση ερωτημάτων [6]. Με τον τρόπο αυτό, η αρχιτεκτονική μειώνει δραστικά τα αντίγραφα δεδομένων, απαλείφοντας τη δαπανηρή αντιγραφή που επιβάλλει η παραδοσιακή αρχιτεκτονική λίμνης–αποθήκης.

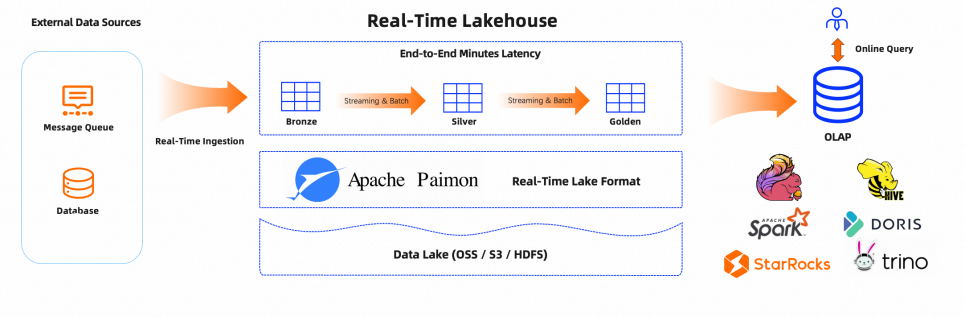
Στην πράξη, η αρχιτεκτονική data lakehouse υλοποιείται μέσω μορφοτύπων πινάκων (open table formats) που προσθέτουν τη δομή μεταδεδομένων πάνω σε αρχεία σε στήλες (columnar files), όπως το Parquet [164] ή το Optimized Row Columnar (ORC) [165]. Οι πλέον γνωστοί μορφότυποι format είναι ο Apache Iceberg [166], ο Delta Lake [167] και ο Apache Hudi [168]. Η συγκριτική ανάλυση στη βιβλιογραφία [169] αποτυπώνει τις αρχιτεκτονικές διαφορές μεταξύ αυτών των συστημάτων. Το Iceberg χρησιμοποιεί δενδροειδή δομή αρχείων δήλωσης (manifest files) με στατιστικά ανά στήλη που επιτρέπουν αποδοτικότερο κλάδεμα (pruning) partitions σε πολύ μεγάλους πίνακες, ο Delta Lake βασίζεται σε αρχείο συναλλαγών (transaction log) και παρέχει αυστηρή σειριοποιήσιμη απομόνωση (serializable isolation), ενώ το Hudi υποστηρίζει εγγενώς τη στρατηγική συγχώνευσης κατά την ανάγνωση (merge-on-read) που ευνοεί φόρτους εντατικής εγγραφής. Σημειώνεται ότι κανένα σύστημα δεν υπερέχει σε όλες τις διαστάσεις και ότι τα τρία συστήματα συγκλίνουν σταδιακά σε χαρακτηριστικά. Ωστόσο, κανένα από αυτά δεν σχεδιάστηκε εξαρχής για ροές δεδομένων (streaming-first design). Υποστηρίζουν κυρίως εγγραφές κατά παρτίδες και οι δυνατότητες συνεχούς εισαγωγής (streaming ingestion) προστέθηκαν εκ των υστέρων [169]. Η παρούσα εργασία αξιοποιεί το Apache Paimon [23, 170], το οποίο αναλύεται στην επόμενη ενότητα.

4.5.2 Apache Paimon, αρχιτεκτονική και ενοποιημένη αποθήκευση

Το Apache Paimon [23, 170] αποτελεί format πίνακα τύπου data lakehouse που αναπτύχθηκε αρχικά ως μέρος του οικοσυστήματος Apache Flink υπό την ονομασία Flink Table Store και στη συνέχεια εξελίχθηκε σε ανεξάρτητο έργο του Apache Software Foundation [171]. Σε αντίθεση με τους μορφοτύπους Delta Lake, Apache Iceberg και Apache Hudi, οι οποίοι σχεδιάστηκαν πρωτίστως για φόρτους κατά παρτίδες και απέκτησαν εκ των υστέρων δυνατότητες εισαγωγής ροών (streaming), το Paimon υιοθέτησε εξαρχής τη σχεδίαση ροόκεντρικού μορφοτύπου πίνακα (streaming-first lakehouse table store), ενοποιώντας εγγενώς τη σημασιολογία ροής και παρτίδας [170] σε ένα ενιαίο αποθηκευτικό χώρο.

Η εσωτερική αρχιτεκτονική του Paimon βασίζεται σε τρεις θεμελιώδεις αρχές [170]. Πρώτον, τα δεδομένα αποθηκεύονται ως αρχεία σε στήλες (columnar files) σε σύστημα αρχείων ή αποθηκευτικό χώρο που αποθηκεύει τα δεδομένα ως αντικείμενα (object store), όπως HDFS, Amazon S3 ή ADLS. Δεύτερον, τα μεταδεδομένα (metadata) διατηρούνται σε αρχεία δηλώσεων (manifest files), τα οποία παρέχουν πληροφορίες στατιστικών (min/max, null count) ανά στήλη και επιτρέπουν την παράλειψη δεδομένων (data skipping) κατά την εκτέλεση ερωτημάτων σε μεγάλη κλίμακα. Τρίτον, για πίνακες με πρωτεύον κλειδί, χρησιμοποιείται δομή τύπου Log-Structured Merge-tree (LSM tree) που υποστηρίζει μεγάλο όγκο ενημερώσεων και ερωτήματα υψηλής απόδοσης [170, 172]. Η Εικόνα 4.13 απεικονίζει τη συνολική αρχιτεκτονική του Apache Paimon.

Το Paimon υποστηρίζει δύο θεμελιώδεις τύπους πινάκων [172, 173]. Ο πίνακας με πρω-



Εικόνα 4.13: *Αρχιτεκτονική του Apache Paimon. Streaming και κατά παρτίδες ανάγνωση/εγγραφή, αποθήκευση αρχείων σε στήλες σε σύστημα αρχείων, μεταδεδομένα σε αρχεία δηλώσεων και δομή LSM tree για πίνακες με πρωτεύον κλειδί. Πηγή: Εγχειρίδιο Apache Paimon [170].*

τεύον κλειδί (primary key table) ορίζει ένα σύνολο στηλών που αποτελούν μοναδικό αναγνωριστικό κάθε εγγραφής. Το Paimon επιβάλλει ταξινόμηση βάσει πρωτεύοντος κλειδιού εντός κάθε κάδου (bucket), γεγονός που επιτρέπει υψηλής απόδοσης αναζητήσεις σημείου (point queries) και ενημερώσεις (upserts). Αντιθέτως, ο πίνακας προσθήκης (append table) δεν ορίζει πρωτεύον κλειδί και δέχεται μόνο νέες εγγραφές χωρίς δυνατότητα ενημέρωσης των υπάρχοντων. Ο πίνακας προσθήκης συμπεριφέρεται ως κλασικός πίνακας αποθήκευσης, παρέχοντας αυτόματη συγχώνευση μικρών αρχείων, ιστορικότητα, δυνατότητα διαγραφής ή ενημέρωσης χαμηλού κόστους και streaming ανάγνωση ή εγγραφή [173].

Ενοποιημένη αποθήκευση (unified storage). Το Paimon παρέχει ενοποιημένη αποθήκευση που γεφυρώνει τα μοντέλα ροής και παρτίδας [170]. Σε λειτουργία batch, ένας πίνακας Paimon συμπεριφέρεται ως κλασικός πίνακας τύπου Hive [174] δηλαδή τα ερωτήματα SQL επιστρέφουν τα δεδομένα του πιο πρόσφατου snapshot. Σε λειτουργία streaming, ο ίδιος πίνακας συμπεριφέρεται ως αρχείο καταγραφής αλλαγών (changelog) αντίστοιχο ουράς μηνυμάτων, όπου τα ιστορικά δεδομένα δεν λήγουν ποτέ. Η ιδιότητα αυτή είναι ιδιαίτερα σημαντική για εφαρμογές IoT, όπου η ίδια πηγή δεδομένων πρέπει να εξυπηρετεί ταυτόχρονα παρακολούθηση πραγματικού χρόνου και ιστορική ανάλυση.

Ως προς τη λειτουργία ανάγνωσης, το Paimon υποστηρίζει τρεις τρόπους οι οποίοι είναι η ανάγνωση ιστορικών snapshots (batch), η ανάγνωση από τον τελευταίο offset (streaming) και η υβριδική ανάγνωση σταδιακών (incremental) snapshots. Ως προς τη λειτουργία εγγραφής, υποστηρίζεται ο συγχρονισμός ροών μεταβολών από βάσεις δεδομένων μέσω της τεχνικής καταγραφής αλλαγών Change Data Capture (CDC) [28], η οποία παρακολουθεί το αρχείο καταγραφής συναλλαγών (transaction log) μιας βάσης και εξάγει τις μεταβολές ως ροή συμβάντων, καθώς και η μαζική εισαγωγή ή αντικατάσταση δεδομένων (batch insert/overwrite) [170].

Μηχανές συγχώνευσης (merge engines). Όταν το Paimon λαμβάνει δύο ή περισσότερες εγγραφές με τα ίδια πρωτεύοντα κλειδιά, τις συγχωνεύει σε μία ενιαία εγγραφή σύμφωνα με τη ρυθμιζόμενη μηχανή συγχώνευσης [175]. Η προεπιλεγμένη μηχανή είναι η deduplicate, η οποία υλοποιεί τη σημασιολογία της επικράτησης τελευταίας εγγραφής (last-write-wins),

διατηρώντας από κάθε ομάδα εγγραφών με κοινό πρωτεύον κλειδί μόνο την πιο πρόσφατη, με τη χρονική προτεραιότητα να καθορίζεται από έναν αύξοντα αριθμό ακολουθίας (sequence number) που εκχωρείται με τη σειρά εισόδου ή από ρητά ορισμένο πεδίο της εγγραφής [28, 175]. Εναλλακτικά, το Paimon παρέχει τις μηχανές partial-update, η οποία εφαρμόζει μερικές ενημερώσεις σε επιμέρους στήλες, aggregation, η οποία εφαρμόζει συναρτήσεις συγκέντρωσης κατά τη συγχώνευση, και first-row, η οποία διατηρεί την πρώτη εμφάνιση κάθε κλειδιού [175]. Στην παρούσα εργασία χρησιμοποιείται η προεπιλεγμένη μηχανή deduplicate, η οποία είναι κατάλληλη για τη φύση των τηλεμετρικών δεδομένων IoT όπου κάθε μέτρηση αντικαθιστά πλήρως την προηγούμενη.

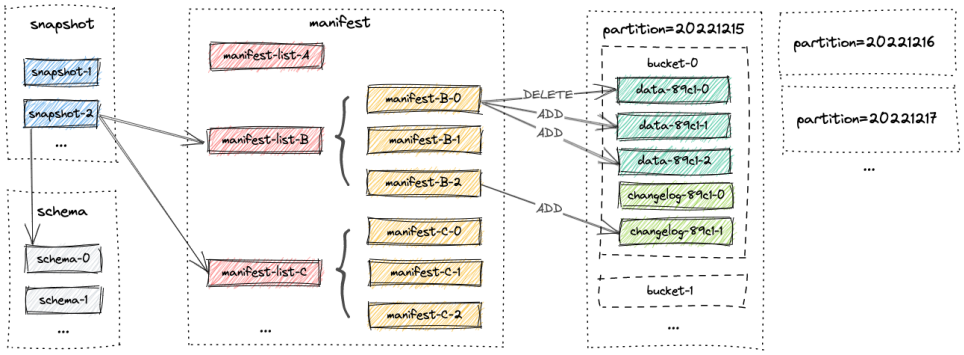
Παραγωγοί αρχείου καταγραφής αλλαγών (changelog producers). Η streaming εγγραφή στο Paimon παράγει συνεχώς τις πιο πρόσφατες αλλαγές για streaming ανάγνωση [176]. Το Paimon προσφέρει τέσσερις τρόπους παραγωγής αρχείου καταγραφής αλλαγών μέσω της ιδιότητας πίνακα changelog-producer. Στην προεπιλεγμένη τιμή (none), η πηγή Paimon παρέχει μόνο τις συγχωνευμένες αλλαγές μεταξύ snapshots, χωρίς τις παλιές τιμές, γεγονός κατάλληλο για καταναλωτές τύπου βάσης δεδομένων. Η ρύθμιση input αποθηκεύει τις πλήρεις εγγραφές εισόδου σε ξεχωριστά αρχεία αρχείου καταγραφής αλλαγών, ιδανική όταν η είσοδος προέρχεται από CDC ή από υπολογισμούς Flink με κατάσταση. Η ρύθμιση lookup παράγει πλήρες αρχείο καταγραφής αλλαγών μέσω αναζήτησης παλαιών τιμών σε τοπική κρυφή μνήμη πριν τη δέσμευση, ενώ η ρύθμιση full-compaction συγκρίνει τα αποτελέσματα μεταξύ πλήρων συμπιέσεων, κατάλληλη για σενάρια με μεγαλύτερη ανεκτικότητα στην καθυστέρηση [176].

Ως προς τη διαλειτουργικότητα, πέρα από το Apache Flink, το Paimon υποστηρίζει ανάγνωση από πλήθος μηχανών ερωτημάτων, μεταξύ των οποίων τα Apache Spark [45], StarRocks [177], Apache Doris [178], Apache Hive και Trino [170]. Στην παρούσα εργασία, η ανάγνωση δεδομένων από τον Trino αξιοποιεί αυτή τη δυνατότητα για την εξυπηρέτηση των αναλυτικών ερωτημάτων των Business Intelligence dashboards του Grafana.

4.5.3 Βασικές έννοιες, αρχεία, snapshots και partitions

Τα αρχεία ενός πίνακα Paimon οργανώνονται σε ιεραρχική δομή κάτω από έναν βασικό κατάλογο (catalog) [179]. Ξεκινώντας από ένα αρχείο snapshot, οι αναγνώστες μπορούν αναδρομικά να προσπελάσουν όλες τις εγγραφές του πίνακα. Η οργάνωση αυτή βασίζεται σε τρεις κεντρικές έννοιες οι οποίες είναι τα snapshots, τα αρχεία δηλώσεων και τα αρχεία δεδομένων. Η Εικόνα 4.14 απεικονίζει τη δομή αρχείων ενός πίνακα Paimon.

Snapshots. Τα snapshots αποτελούν αρχεία τύπου JavaScript Object Notation (JSON) [180] που αποθηκεύονται στον κατάλογο snapshots του πίνακα. Κάθε snapshot καταγράφει πληροφορίες μεταδεδομένων, μεταξύ των οποίων τον αναγνωριστικό αριθμό του σχήματος (schemaId) και δύο λίστες δηλώσεων που περιλαμβάνουν τη βασική λίστα (baseManifestList) η οποία αντιπροσωπεύει τη θεμελιώδη κατάσταση, και τη λίστα αλλαγών (deltaManifestList) η οποία περιγράφει τις αλλαγές σε σχέση με τη βασική [179, 181]. Κάθε snapshot καταγράφει επίσης τον τύπο δέσμευσης (commitKind), ο οποίος μπορεί να είναι



Εικόνα 4.14: *Ιεραρχική δομή αρχείων ενός πίνακα Apache Paimon. Τα snapshots αναφέρονται σε λίστες δηλώσεων, οι οποίες με τη σειρά τους περιγράφουν τα αρχεία δεδομένων σε κάθε partition και κάδο. Πηγή: Εγχειρίδιο Apache Paimon [179].*

APPEND για νέες εγγραφές ή COMPACT για λειτουργίες συμπίεσης, καθώς και στατιστικά στοιχεία όπως ο συνολικός αριθμός εγγραφών και το timestamp του watermark. Μέσω των snapshots, οι χρήστες μπορούν να προσπελάσουν τόσο την πιο πρόσφατη κατάσταση ενός πίνακα, μέσω του αρχείου LATEST, όσο και παλαιότερες εκδόσεις μέσω time travel [179].

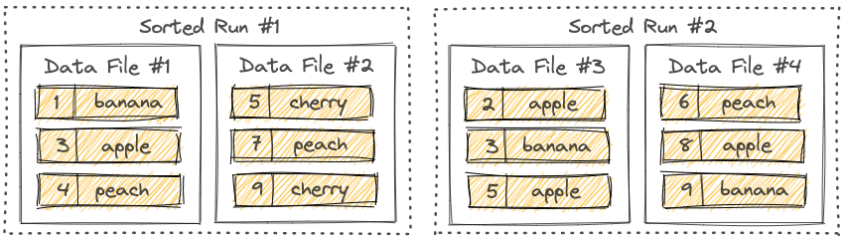
Αρχεία δηλώσεων (manifest files). Τα αρχεία δηλώσεων αποτελούν τον κεντρικό μηχανισμό διαχείρισης μεταδεδομένων και αποθηκεύονται στον κατάλογο manifest [179]. Μία λίστα δηλώσεων (manifest list) περιέχει τα ονόματα των αρχείων δηλώσεων, καθένα από τα οποία περιγράφει τις αλλαγές σε αρχεία δεδομένων LSM και αρχεία αρχείου καταγραφής αλλαγών (changelog files). Κάθε εγγραφή εντός ενός αρχείου δηλώσεων περιέχει μία λειτουργία τύπου ADD ή DELETE, υποδεικνύοντας αν το αντίστοιχο αρχείο δεδομένων δημιουργήθηκε ή καταργήθηκε λογικά στο αντίστοιχο snapshot [181]. Τα αρχεία δηλώσεων επιτρέπουν αποδοτική παράλειψη δεδομένων κατά τα ερωτήματα, διότι περιλαμβάνουν στατιστικά στοιχεία (min/max, null count) ανά στήλη για κάθε αρχείο δεδομένων [170].

Partitions, κάδοι και αρχεία δεδομένων. Το Paimon υιοθετεί την ίδια φιλοσοφία partitioning με το Apache Hive. Τα δεδομένα χωρίζονται σε υποσύνολα βάσει τιμών συγκεκριμένων στηλών, ώστε τα ερωτήματα να προσπελαύνουν μόνο τα σχετικά partitions [179]. Το partitioning αποτελεί προαιρετικό μηχανισμό, αλλά σε εφαρμογές χρονοσειρών IoT, όπως στην παρούσα εργασία, το partitioning ανά ημερομηνία μειώνει δραστικά τον όγκο δεδομένων που σαρώνονται κατά τα αναλυτικά ερωτήματα.

Κάτω από κάθε partition ή απευθείας στον πίνακα εάν αυτός δεν είναι κατατμημένος, τα δεδομένα υποδιαιρούνται περαιτέρω σε κάδους (buckets) [172]. Κάθε bucket αντιστοιχεί σε έναν κατάλογο που περιέχει ένα δέντρο LSM και τα αντίστοιχα αρχεία καταγραφής αλλαγών (changelog files). Η ανάθεση εγγραφών σε buckets γίνεται βάσει του κατακερματισμού (hash) μίας ή περισσοτέρων στηλών, ενώ ο αριθμός των buckets καθορίζει τον μέγιστο βαθμό παραλληλισμού ανάγνωσης και εγγραφής. Σημειώνεται ότι ο συνιστώμενος όγκος δεδομένων ανά bucket κυμαίνεται μεταξύ 200MB και 1GB [172]. Τα αρχεία δεδομένων αποθηκεύονται σε μορφή στηλών (columnar format), με τον τύπο Parquet [164] ως προεπιλογή, ενώ υποστηρίζονται επίσης τα format τύπου ORC και Avro [179].

Δομή LSM tree και ταξινομημένες ακολουθίες. Για πίνακες με πρωτεύον κλειδί, το Paimon αξιοποιεί τη δομή LSM tree [172]. Αυτό οργανώνει τα αρχεία σε πολλαπλές ταξινομημένες ακολουθίες (sorted runs), κάθε μία από τις οποίες αποτελείται από ένα ή περισσότερα αρχεία δεδομένων. Εντός μιας ταξινομημένης ακολουθίας, οι εγγραφές είναι ταξινομημένες κατά πρωτεύον κλειδί και τα εύρη κλειδιών μεταξύ αρχείων δεν επικαλύπτονται. Ωστόσο, διαφορετικές ταξινομημένες ακολουθίες μπορούν να περιέχουν επικαλυπτόμενα εύρη ή ακόμη και τα ίδια κλειδιά, γεγονός που σημαίνει ότι κατά την ανάγνωση όλες οι ακολουθίες πρέπει να συνδυαστούν και οι εγγραφές με τα ίδια κλειδιά πρέπει να συγχωνευθούν σύμφωνα με τη μηχανή συγχώνευσης [172, 175].

Οι νέες εγγραφές αποθηκεύονται αρχικά σε ένα χώρο προσωρινής μνήμης (write buffer) στο σωρό (heap) [182]. Όταν η μνήμη εξαντληθεί, όλες οι εγγραφές ταξινομούνται και εκκαθαρίζονται (flush) στον δίσκο, δημιουργώντας μια νέα ταξινομημένη ακολουθία [181]. Περιοδικά, μία διαδικασία συμπίεσης (compaction) συγχωνεύει τις ταξινομημένες ακολουθίες, μειώνοντας τον αριθμό αρχείων και εφαρμόζοντας ενημερώσεις και διαγραφές, ώστε να διατηρείται η απόδοση ανάγνωσης. Αυτή η δομή είναι ιδανική για φόρτους εργασίας με υψηλή ένταση εγγραφής, όπως η συνεχής εισαγωγή τηλεμετρικών δεδομένων IoT [170, 172]. Η Εικόνα 4.15 απεικονίζει τη δομή LSM tree με ταξινομημένες ακολουθίες.



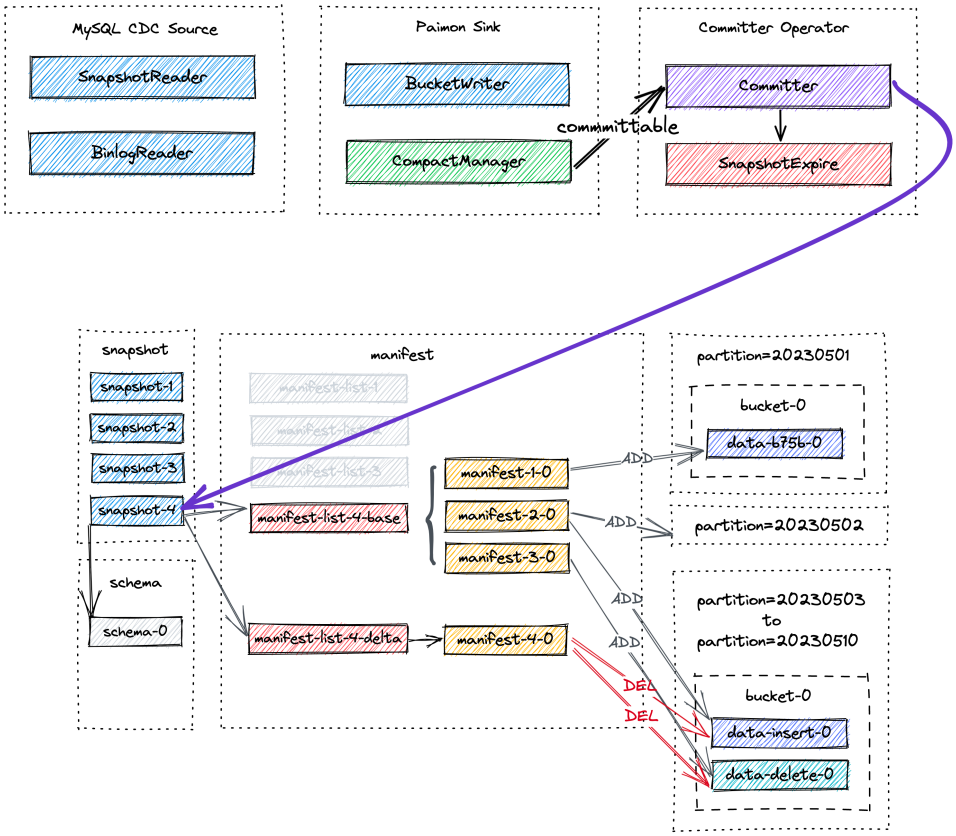
Εικόνα 4.15: *Δομή LSM tree στον Apache Paimon. Οι εγγραφές ταξινομούνται σε πολλαπλά sorted runs εντός κάθε bucket, ενώ η compaction τα συγχωνεύει περιοδικά. Πηγή: Εγχειρίδιο Apache Paimon [172].*

4.5.4 Λειτουργίες αρχείων και διαχείριση μικρών αρχείων

Η κατανόηση των λειτουργιών που επηρεάζουν τα αρχεία ενός πίνακα Paimon είναι σημαντική για τον σχεδιασμό αποδοτικών pipelines δεδομένων [181].

Streaming εγγραφή μέσω Flink. Κατά τη streaming εγγραφή μέσω CDC ή απευθείας από Kafka topics, η ροή δεδομένων ακολουθεί τρία στάδια [181]. Αρχικά, το Paimon Sink αποθηκεύει τις νέες εγγραφές σε μία δομή LSM tree βασισμένη στο heap και τις εκκαθαρίζει στον δίσκο όταν η μνήμη εξαντληθεί. Κάθε αρχείο δεδομένων που εγγράφεται αποτελεί μια ταξινομημένη ακολουθία (sorted run). Σε αυτό το στάδιο δεν δημιουργείται snapshot. Αμέσως πριν τη λήψη σημείου ελέγχου (checkpoint) του Flink, το Paimon Sink εκκαθαρίζει όλες τις εκκρεμείς εγγραφές και αποστέλλει μήνυμα δέσμευσης στον τελεστή Committer Operator. Ο τελεστής αυτός, κατά τη διάρκεια του σημείου ελέγχου, δημιουργεί ένα νέο snapshot, το συσχετίζει με τις αντίστοιχες λίστες δηλώσεων και ελέγχει αν πρέπει να ενεργοποιηθεί λήξη παλαιότερων snapshots [181]. Ο διαχειριστής συμπίεσης (CompactManager)

εντός του Paimon Sink μπορεί να ενεργοποιήσει ασύγχρονα ένα compaction, η οποία παράγει πρόσθετο snapshot τύπου COMPACT. Η Εικόνα 4.16 απεικονίζει τη ροή δεδομένων κατά τη streaming εγγραφή μέσω Flink.



Εικόνα 4.16: *Streaming εγγραφή Apache Paimon μέσω Flink. Το Paimon Sink αποθηκεύει δεδομένα σε LSM tree στο heap, ο CompactManager ενεργοποιεί ασύγχρονα το compaction και ο Committer Operator δημιουργεί snapshots κατά τα checkpoints του Flink. Πηγή: Εγχειρίδιο Apache Paimon [181].*

Compaction. Καθώς οι διαδοχικές δεσμεύσεις αυξάνουν τον αριθμό μικρών αρχείων, η πλήρης συμπίεση (full compaction) είναι απαραίτητη για τη βελτίωση της απόδοσης ανάγνωσης [181]. Κατά το compaction, τα αρχεία πολλαπλών ταξινομημένων ακολουθιών συγχωνεύονται σε λιγότερα, μεγαλύτερα αρχεία. Το snapshot που παράγεται κατά το compaction έχει τύπο COMPACT και περιλαμβάνει εγγραφές δηλώσεων τύπου DELETE για τα παλαιά αρχεία και ADD για τα νέα συγχωνευμένα. Η ρύθμιση full-compaction.delta-commits επιτρέπει τη ρύθμιση της συχνότητας πλήρους compaction σε σχέση με τον αριθμό δεσμεύσεων [181].

Λήξη (snapshots). Το Paimon διατηρεί πολλαπλές εκδόσεις αρχείων, καθώς η διαγραφή αρχείων κατά το compaction είναι λογική και δεν αφαιρεί φυσικά τα αρχεία από τον δίσκο [181]. Τα αρχεία διαγράφονται φυσικά μόνο κατά τη λήξη (expiration) των snapshots, εφόσον κανένας καταναλωτής δεν εξαρτάται πλέον από αυτά. Η διαδικασία λήξης αρχικά καθορίζει το εύρος snapshots προς λήξη, στη συνέχεια διαγράφει τα σημαδεμένα αρχεία

δεδομένων, τα αρχεία καταγραφής αλλαγών (changelog files) και τις σχετικές δηλώσεις, και τέλος αφαιρεί τα ίδια τα snapshots. Τυχόν κενοί κατάλογοι αφαιρούνται επίσης [181]. Ο Flink Committer Operator ενεργοποιεί αυτόματα τη λήξη snapshots.

Διαχείριση μικρών αρχείων. Η συσσώρευση μικρών αρχείων αποτελεί συχνή ανησυχία σε συστήματα data lakehouse, διότι μπορεί να προκαλέσει υπερφόρτωση του κόμβου ονομάτων (NameNode) σε Hadoop Distributed File System (HDFS) [183], αύξηση κόστους λόγω ελάχιστου μεγέθους μπλοκ, και μείωση της απόδοσης ερωτημάτων [181]. Το Paimon αντιμετωπίζει αυτό το πρόβλημα μέσω τεσσάρων στρατηγικών. Πρώτον, η αύξηση του διαστήματος (interval) των checkpoints στο Flink μειώνει τα snapshots και κατ' επέκταση τα παραγόμενα αρχεία ανά μονάδα χρόνου. Δεύτερον, η αύξηση του μεγέθους του χώρου προσωρινής εγγραφής (write-buffer-size) ή η ενεργοποίηση της ρύθμισης write-buffer-spillable επιτρέπει τη δημιουργία μεγαλύτερων αρχείων στον κατανεμημένο αποθηκευτικό χώρο. Τρίτον, ο κατάλληλος αριθμός κάδων περιορίζει τον πολλαπλασιασμό αρχείων αφού εάν ο συνολικός όγκος δεδομένων είναι μικρός, λίγα buckets αποτρέπουν τη δημιουργία πληθώρας μικρών αρχείων. Τέταρτον, η τακτική πλήρους compaction, ιδιαίτερα σε partitions που δεν είναι πλέον ενεργά, εξασφαλίζει ελάχιστο αριθμό αρχείων ανά bucket [181].

4.5.5 Εγγυήσεις συνέπειας και έλεγχος ταυτοχρονισμού

Το Paimon παρέχει εγγυήσεις ACID μέσω πρωτοκόλλου δέσμευσης δύο φάσεων (two-phase commit) [184]. Κάθε δέσμευση παράγει το πολύ δύο snapshots, ανάλογα με τη στρατηγική σταδιακής εγγραφής και compaction. Εάν εκτελείται μόνο σταδιακή εγγραφή χωρίς ενεργοποίηση compaction, δημιουργείται μόνο ένα σταδιακό (incremental) snapshot. Εάν ενεργοποιηθεί και το compaction, τότε δημιουργείται πρόσθετο snapshot τύπου COMPACT [179, 181].

Ως προς τον ταυτοχρονισμό, το Paimon υιοθετεί αισιόδοξο μοντέλο (optimistic concurrency), δηλαδή πολλαπλές εργασίες εγγραφής μπορούν να λειτουργούν ανεξάρτητα, παράγοντας νέα snapshots βάσει του τρέχοντος [184]. Για δύο εγγραφείς που τροποποιούν διαφορετικά partitions, οι δεσμεύσεις εκτελούνται παράλληλα χωρίς σύγκρουση. Εάν τροποποιούν το ίδιο partition, εξασφαλίζεται απομόνωση (isolation) snapshot, δηλαδή η τελική κατάσταση του πίνακα μπορεί να αποτελεί μείγμα των δύο δεσμεύσεων, αλλά δεν χάνονται αλλαγές. Ο συνιστώμενος τρόπος λειτουργίας σε σενάρια πολλαπλών ταυτόχρονων εγγραφέων είναι η χρήση αποκλειστικής εργασίας συμπίεσης (dedicated compaction job). Ως εκ τούτου οι εργασίες εγγραφής λειτουργούν σε κατάσταση write-only χωρίς να εκτελούν compaction, ενώ μία αυτοτελής εργασία αναλαμβάνει αποκλειστικά το compaction [184].

Το Paimon αξιοποιεί τον μηχανισμό ατομικής μετονομασίας (atomic rename) του συστήματος αρχείων για τη δέσμευση snapshots. Σε κατανεμημένα συστήματα αρχείων όπως το HDFS, η μετονομασία είναι ατομική και εγγυάται τη συνέπεια. Για αποθηκευτικό χώρο τύπου object storage όπως τα Amazon S3 ή Azure Blob Storage, όπου η μετονομασία δεν είναι ατομική, το Paimon συνιστά τη χρήση εξωτερικού αποθετήριου μεταδεδομένων (Hive ή Java Database Connectivity (JDBC) [185] metastore) σε συνδυασμό με μηχανισμό κλειδώματος (lock.enabled) για τη διασφάλιση ακεραιότητας [184].

4.5.6 Catalog

Το Paimon παρέχει μία αφαίρεση καταλόγου (catalog) για τη διαχείριση πινάκων και μεταδεδομένων, η οποία διευκολύνει την ενσωμάτωση με μηχανές υπολογισμού [186]. Το catalog αποτελεί το σημείο εισόδου για τη δημιουργία, διαγραφή και αναζήτηση πινάκων, και υποστηρίζει τρεις βασικούς τύπους metastore [186].

Ο κατάλογος συστήματος αρχείων (filesystem catalog) αποτελεί την προεπιλογή και αποθηκεύει τόσο τα μεταδεδομένα (snapshots, σχήματα, δηλώσεις) όσο και τα αρχεία δεδομένων απευθείας στο σύστημα αρχείων ή το χώρο αποθήκευσης των δεδομένων ως αντικείμενα. Δεν απαιτεί εξωτερική υποδομή μεταδεδομένων, αλλά δεν παρέχει μηχανισμό κλειδώματος για ταυτόχρονες εγγραφές σε object storage [186].

Το Hive catalog αποθηκεύει πρόσθετα μεταδεδομένα στον Hive Metastore, ώστε οι πίνακες Paimon να είναι προσβάσιμοι απευθείας μέσω Hive, Spark ή άλλων μηχανών που τον χρησιμοποιούν. Οι πίνακες μπορούν να εγγραφούν αυτόματα ως κατατμημένοι πίνακες σε αυτόν μέσω της ρύθμισης metastore.partitioned-table [186].

Το JDBC catalog αποθηκεύει μεταδεδομένα σε σχεσιακή βάση δεδομένων όπως MySQL [187] ή PostgreSQL [50], παρέχοντας μηχανισμό κλειδώματος για αξιόπιστο ταυτοχρονισμό. Πέρα από αυτούς τους τρεις τύπους, το Paimon υποστηρίζει και REST catalog για ενσωμάτωση με εξωτερικά συστήματα μεταδεδομένων μέσω REST API [186]. Στην παρούσα εργασία χρησιμοποιείται ο κατάλογος συστήματος αρχείων, με βασικό catalog αποθηκευμένο στο Azure ADLS.

4.5.7 Οργάνωση δεδομένων σε στρώματα

Η αρχιτεκτονική medallion οργανώνει τα δεδομένα σε τρία στρώματα σταδιακά αυξανόμενης ποιότητας [6]. Το πρώτο, το ακατέργαστο στρώμα (bronze/raw), αποθηκεύει τα δεδομένα ακριβώς όπως ελήφθησαν, χωρίς καθαρισμό ή μετασχηματισμό, εξασφαλίζοντας πλήρη ιστορικότητα. Το δεύτερο, το επεξεργασμένο στρώμα (silver/curated), περιλαμβάνει δεδομένα που έχουν φιλτραριστεί, καθαριστεί και εμπλουτιστεί, ενώ εφαρμόζονται μετατροπές, αποδιπλασιασμός (deduplication) και κανονικοποίηση μονάδων. Το τρίτο, το συγκεντρωτικό στρώμα (gold/aggregated), περιέχει υπολογισμένους ωριαίους ή ημερήσιους δείκτες (Key Performance Indicators, KPIs), έτοιμους για κατανάλωση από dashboards ή αναφορές (reports).

Στην παρούσα εργασία, κάθε πηγή δεδομένων όπως ο κεντρικός μετρητής εγκατάστασης ή ο επιμέρους αισθητήρας συσκευής, αποκτά τα τρία αυτά στρώματα ως πίνακες στο Paimon, αποθηκευμένους σε μορφή στηλών (columnar). Η εγγραφή ροής (streaming write) του Flink τροφοδοτεί απευθείας το ακατέργαστο στρώμα μέσω ερωτημάτων INSERT INTO, ενώ οι μετασχηματισμοί σε στρώματα ανώτερης ποιότητας εκτελούνται μέσω ερωτημάτων ροής SQL που παράγουν ωριαίες και δεκάλεπτες συγκεντρώσεις.

4.5.8 Ιστορικότητα και χρονική μεταβολή

Το Paimon υποστηρίζει παλαιότερες εκδόσεις των δεδομένων μέσω time travel, τη δυνατότητα ανάγνωσης παλαιότερων εκδόσεων ενός πίνακα μέσω αναφοράς σε συγκεκριμένο

snapshot [179]. Κάθε snapshot αντιστοιχεί σε μία ατομική δέσμευση, και ο αναγνώστης βλέπει πάντα μία συνεπή εικόνα χωρίς κίνδυνο ολισθηρών αναγνώσεων (dirty reads). Η δυνατότητα αυτή είναι χρήσιμη τόσο για ελεγκτικούς σκοπούς (auditing) όσο και για επαναφορά μετά από λανθασμένη εγγραφή, και αντιστοιχεί στο μοντέλο ACID transactional data lake που περιγράφεται στη βιβλιογραφία [6].

Η διαχείριση ιστορικότητας στο Paimon συνδέεται άμεσα με τη λήξη snapshot [181]. Ο αριθμός των snapshots που διατηρούνται καθορίζει το βάθος χρόνου στο οποίο μπορεί να εκτελεστεί επαναφορά παλαιότερων εκδόσεων των δεδομένων μέσω time travel. Σε συνδυασμό με τη σταδιακή ανάγνωση (incremental read), οι καταναλωτές μπορούν να ανιχνεύσουν μόνο τις αλλαγές μεταξύ δύο snapshots χωρίς να σαρώσουν ολόκληρο τον πίνακα, ιδιότητα κρίσιμη για τη δημιουργία αποδοτικών σωλήνων επεξεργασίας ροής [170]. Σε εφαρμογές IoT, όπως στην παρούσα εργασία, η ορθότητα ιστορικών αναφορών είναι εξίσου σημαντική με την ταχύτητα εισαγωγής, καθιστώντας τη διαχείριση snapshots θεμελιώδες χαρακτηριστικό της αρχιτεκτονικής.

Τα δεδομένα που αποθηκεύονται στο Paimon χρειάζονται στη συνέχεια να αναλυθούν μέσω ερωτημάτων SQL και να οπτικοποιηθούν σε dashboards. Τα εργαλεία ανάλυσης, παρακολούθησης και οπτικοποίησης (Trino, Prometheus, Grafana) παρουσιάζονται αναλυτικά στο επόμενο κεφάλαιο.

Κεφάλαιο 5

Παρακολούθηση, Παρατηρησιμότητα και Ανάλυση Ερωτημάτων

Στο κεφάλαιο αυτό παρουσιάζονται οι αρχές παρακολούθησης (monitoring) και παρατηρησιμότητας (observability) κατανεμημένων συστημάτων, μαζί με τα εργαλεία ανάλυσης ερωτημάτων (query analysis) και οπτικοποίησης (visualization) δεδομένων που χρησιμοποιούνται στην παρούσα εργασία. Ένα σύστημα data lakehouse που εκτελείται σε Kubernetes χρειάζεται συνεχή εποπτεία τόσο στο επίπεδο της υποδομής (nodes, pods, κλπ.) όσο και στο επίπεδο της ροής δεδομένων (ρυθμοί εισαγωγής, καθυστέρηση, σφάλματα). Ταυτόχρονα, η δυνατότητα πρόσβασης στα αποθηκευμένα δεδομένα μέσω κατανεμημένων μηχανών ερωτημάτων και η οπτικοποίηση αποτελεσμάτων αποτελούν αναπόσπαστα μέρη του συνολικού συστήματος.

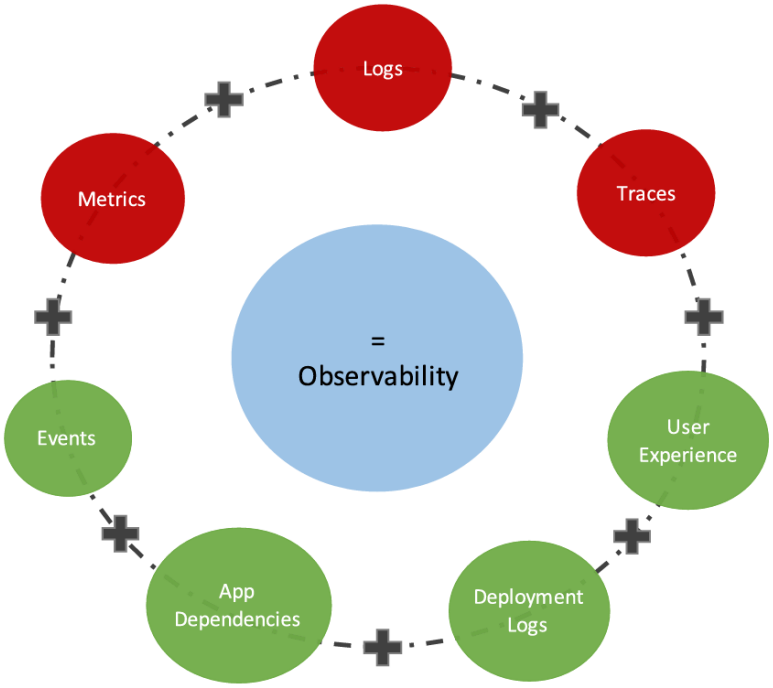
5.1 Βασικές αρχές παρακολούθησης και παρατηρησιμότητας

Η παρακολούθηση (monitoring) αφορά τη συλλογή, αποθήκευση και οπτικοποίηση μετρικών (metrics) από ένα σύστημα, ώστε να εντοπίζονται αποκλίσεις από τη σταθερή λειτουργία και να ενεργοποιούνται ειδοποιήσεις (alerts) όταν κάποια τιμή ξεπερνά ένα καθορισμένο κατώφλι [31]. Η παρατηρησιμότητα (observability) είναι μια ευρύτερη έννοια και αναφέρεται στην ικανότητα κατανόησης της εσωτερικής κατάστασης ενός συστήματος εξετάζοντας μόνο τις εξωτερικές εξόδους του [188]. Η διάκριση αυτή επιβεβαιώνεται και στην ακαδημαϊκή βιβλιογραφία καθώς οι Usman κ.ά. [189] ταξινομούν συστηματικά τις λύσεις παρατηρησιμότητας για περιβάλλοντα μικροϋπηρεσιών σε containers, ενώ οι Niedermaier κ.ά. [190], μέσω συνεντεύξεων με μηχανικούς κατανεμημένων συστημάτων, αναδεικνύουν τις πρακτικές προκλήσεις κατά τη μετάβαση από την παραδοσιακή παρακολούθηση στην πλήρη παρατηρησιμότητα.

Το πλαίσιο Site Reliability Engineering (SRE), που εισήγαγε η Google [31], αποτελεί μια προσέγγιση η οποία εφαρμόζει αρχές μηχανικής λογισμικού στη διαχείριση και λειτουργία μεγάλης κλίμακας παραγωγικών συστημάτων. Στόχος του είναι η ισορροπία μεταξύ ταχείας ανάπτυξης νέων χαρακτηριστικών και διατήρησης της αξιοπιστίας, μέσω της αυτοματοποίησης επαναλαμβανόμενων χειριστικών εργασιών και της ποσοτικοποίησης της ποιότητας υπηρεσίας με τη χρήση δεικτών εξυπηρέτησης (Service Level Indicators, SLI) [31, 32], στόχων εξυπηρέτησης (SLO) [31, 32] και προϋπολογισμών σφαλμάτων (error budgets) [191]. Σε

αυτό το πλαίσιο, η παρατηρησιμότητα αποτελεί σημαντική υποδομή, καθώς χωρίς αξιόπιστη μέτρηση των δεικτών αυτών δεν είναι δυνατή ούτε η αξιολόγηση των στόχων αξιοπιστίας ούτε ο έγκαιρος εντοπισμός παρεκκλίσεων στη συμπεριφορά μιας υπηρεσίας.

Σύμφωνα με το ίδιο πλαίσιο [31], η παρατηρησιμότητα στηρίζεται σε τρία θεμελιώδη στοιχεία (pillars). Το πρώτο είναι τα metrics, δηλαδή αριθμητικές τιμές χρονοσειράς, όπως η χρήση CPU, ο ρυθμός μηνυμάτων Kafka ή ο χρόνος απόκρισης ερωτημάτων Trino, που επιτρέπουν στατιστική ανάλυση και ειδοποιήσεις. Το δεύτερο είναι τα αρχεία καταγραφής (logs), δομημένες ή αδόμητες εγγραφές γεγονότων που χρησιμεύουν κυρίως στην αναδρομική ανάλυση αστοχιών (post-mortem analysis). Το τρίτο είναι τα ίχνη (traces), που αναπαριστούν τη διαδρομή ενός αιτήματος μέσα από πολλαπλές υπηρεσίες και βοηθούν στον εντοπισμό σημείων συμφόρησης (latency bottlenecks). Η αρχή της κατανεμημένης ιχνηλάτησης τεκμηριώθηκε συστηματικά στο σύστημα Dapper της Google [192], το οποίο αποτέλεσε τη βάση για ανοιχτά εργαλεία όπως τα Zipkin [193], Jaeger [194] και OpenTelemetry [195].



Εικόνα 5.1: *Τα τρία θεμελιώδη στοιχεία της παρατηρησιμότητας (κόκκινο) και επιπλέον δεδομένα που θεωρούνται ζωτικά από διάφορους παρόχους (πράσινο). Πηγή: Usman κ.ά. [189].*

5.1.1 Four Golden Signals

Στο πλαίσιο SRE, ορίζονται τέσσερα κρίσιμα σήματα για την παρακολούθηση κάθε υπηρεσίας [31]:

1. **Καθυστέρηση** (latency). Μετρά τον χρόνο εξυπηρέτησης ενός αιτήματος, και διακρίνεται μεταξύ επιτυχών και αποτυχημένων αιτημάτων.

2. **Κυκλοφορία** (traffic). Αντανακλά τον όγκο ζήτησης, π.χ. αιτήματα ή μηνύματα ανά δευτερόλεπτο στο Kafka.

3. **Σφάλματα** (errors). Καταγράφουν τον ρυθμό αιτημάτων που αποτυγχάνουν, είτε ρητά είτε λόγω χρονικού ορίου (timeout).

4. **Κορεσμός** (saturation). Εκφράζει τον βαθμό χρησιμοποίησης πόρων, όπως το ποσοστό χρήσης CPU, μνήμης ή δίσκου.

Στην παρούσα εργασία, τα σήματα αυτά εφαρμόζονται σε πολλαπλά επίπεδα. Στα Kubernetes nodes παρακολουθείται η χρήση πόρων, στους Kafka brokers ο ρυθμός μηνυμάτων και η καθυστέρηση καταναλωτή, στο Flink ο ρυθμός εισαγωγής και τα checkpoints, στο Trino ο χρόνος εκτέλεσης ερωτημάτων και οι αποτυχίες, ενώ στη ροή δεδομένων ως σύνολο μετράται η συνολική καθυστέρηση από άκρο σε άκρο (end-to-end latency).

5.1.2 Μοντέλα παρακολούθησης τύπου push και pull

Η συλλογή metrics σε κατανεμημένα συστήματα ακολουθεί δύο βασικά μοντέλα [196]. Στο μοντέλο έλξης (pull-based), ο διακομιστής (server) παρακολούθησης ζητά περιοδικά metrics από τα endpoints κάθε υπηρεσίας, π.χ. μέσω αιτήματος HTTP στον ακροδέκτη (endpoint) metrics. Το Prometheus αποτελεί χαρακτηριστικό παράδειγμα αυτής της προσέγγισης. Αντίθετα, στο μοντέλο ώθησης (push-based), κάθε υπηρεσία αποστέλλει ενεργά τα metrics της σε εξειδικευμένα εργαλεία, όπως το συλλέκτη StatsD [197] ή τη βάση δεδομένων χρονοσειρών InfluxDB [48].

Σε περιβάλλοντα Kubernetes, το μοντέλο pull προτιμάται χάρη στην εύκολη ανακάλυψη υπηρεσιών (service discovery). Το Prometheus μπορεί να ανακαλύπτει αυτόματα νέα pods μέσω του Kubernetes API και να αρχίζει τη συλλογή metrics χωρίς κανενός είδους χειροκίνητη ρύθμιση [198].

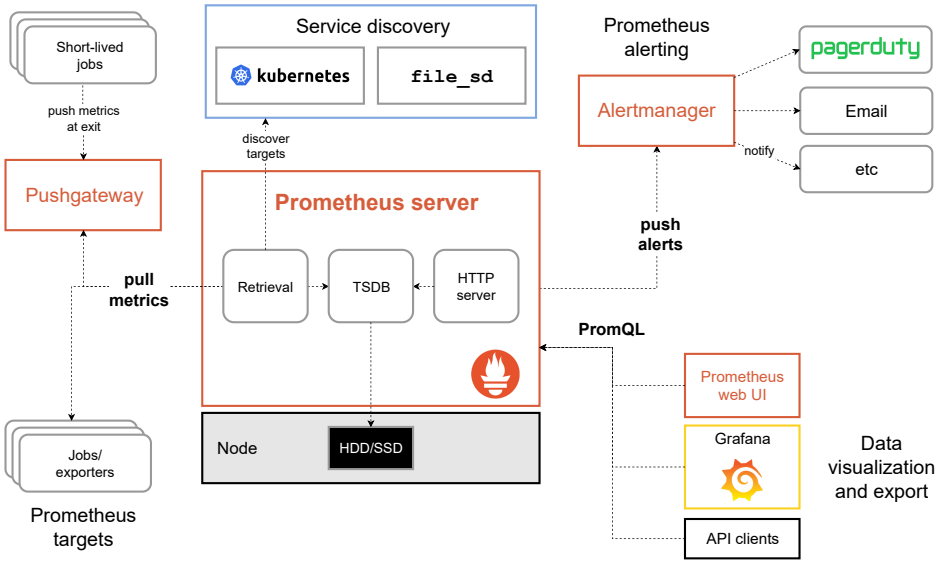
5.2 Prometheus, συλλογή metrics χρονοσειράς

Το Prometheus [198, 199] είναι ένα σύστημα παρακολούθησης και ειδοποιήσεων ανοιχτού κώδικα, το οποίο αναπτύχθηκε αρχικά στο SoundCloud και σήμερα αποτελεί έργο του Cloud Native Computing Foundation (CNCF). Θεωρείται η de facto λύση παρακολούθησης για περιβάλλοντα Kubernetes [31].

5.2.1 Αρχιτεκτονική Prometheus

Η αρχιτεκτονική του Prometheus απαρτίζεται από τέσσερα βασικά συστατικά [198]. Ο Prometheus Server αποτελεί τον κεντρικό server, ο οποίος εκτελεί τη συλλογή (scraping) metrics, τα αποθηκεύει σε ενσωματωμένη βάση χρονοσειρών (TSDB), η οποία αξιοποιεί τεχνικές συμπίεσης εμπνευσμένες από τη βάση Gorilla της Facebook [200], και αξιολογεί τους κανόνες ειδοποίησης. Οι exporters είναι εξειδικευμένες εφαρμογές που εκθέτουν metrics τρίτων υπηρεσιών σε μορφή κατάλληλη για τον Prometheus, π.χ. ο node-exporter για metrics λειτουργικού συστήματος και ο kube-state-metrics για metrics του Kubernetes. Ο Alertmanager λαμβάνει ειδοποιήσεις από τον Prometheus Server και εφαρμόζει ομαδοποίηση (grouping), καταστολή (silencing) και δρομολόγηση σε κανάλια, όπως email, Slack,

Teams ή PagerDuty. Τέλος, ο Pushgateway λειτουργεί ως μεσολαβητής για βραχύβιες εργασίες (batch jobs) που δεν μπορούν να εξυπηρετηθούν μέσω pull μεθόδου, επιτρέποντάς τους να ωθήσουν τα metrics τους πριν τερματιστούν. Η συνολική αρχιτεκτονική αποτυπώνεται στην Εικόνα 5.2.



Εικόνα 5.2: *Αρχιτεκτονική του Prometheus. Ο κεντρικός server συλλέγει metrics από εξωτερικές εργασίες και exporters, αποθηκεύει χρονοσειρές και ενεργοποιεί ειδοποιήσεις μέσω του Alertmanager. Πηγή: Prometheus Authors [199].*

5.2.2 Μοντέλο δεδομένων και τύποι metrics

Κάθε metric στο Prometheus αποθηκεύεται ως χρονοσειρά (time series) και αναγνωρίζεται μοναδικά από το όνομά της σε συνδυασμό με ένα σύνολο ετικετών (labels). Ένα τυπικό παράδειγμα είναι:

```
container_cpu_usage_seconds_total{
  pod="kafka-0", namespace="data-ingest"
}
```

Το Prometheus ορίζει τέσσερις τύπους metrics [198, 201]. Το metric counter είναι ένας μονοτονικά αυξανόμενος μετρητής, κατάλληλος π.χ. για την καταμέτρηση του συνόλου μηνυμάτων Kafka. Το metric gauge αντιπροσωπεύει μια τιμή που μπορεί να αυξομειώνεται, όπως η χρήση μνήμης ή ο αριθμός ενεργών pods. Το metric histogram κατανέμει τιμές σε ρυθμιζόμενα buckets και χρησιμοποιείται, για παράδειγμα, στη μέτρηση χρόνων εκτέλεσης ερωτημάτων. Το metric summary μοιάζει με το histogram, αλλά υπολογίζει ποσοστημόρια (quantiles) στην πλευρά του πελάτη.

5.2.3 Γλώσσα ερωτημάτων PromQL

Η Prometheus Query Language (PromQL) [202] επιτρέπει την αναζήτηση, το φιλτράρισμα, τη συνάθροιση και τον μετασχηματισμό χρονοσειρών [198]. Στην παρούσα εργασία,

ενδεικτικά ερωτήματα PromQL χρησιμοποιούνται για τον υπολογισμό του ρυθμού χρήσης CPU ανά pod σε κυλιόμενο παράθυρο πέντε λεπτών:

```
rate(container_cpu_usage_seconds_total{namespace="data-ingest"}[5m])
```

για την παρακολούθηση χρήσης μνήμης ενός Kafka broker:

```
container_memory_working_set_bytes{pod=~"kafka-.*"}
```

και για τον ρυθμό εισερχομένων μηνυμάτων στο Flink σε κυλιόμενο παράθυρο ενός λεπτού:

```
rate(flink_taskmanager_job_task_numRecordsIn[1m])
```

Η PromQL αποτελεί τη βάση τόσο για τη δημιουργία κανόνων ειδοποίησης (alerting rules) όσο και για τα ερωτήματα που εκτελεί το Grafana στα πλαίσια (panels) παρακολούθησης.

5.2.4 Ανακάλυψη υπηρεσιών στο Kubernetes

Στο Kubernetes, η ανακάλυψη υπηρεσιών στηρίζεται σε δηλωτικές σημειώσεις (annotations), τις οποίες φέρει κάθε pod ή service που πρόκειται να παρακολουθείται. Οι σημειώσεις αυτές δηλώνουν αν και με ποιον τρόπο πρέπει να συλλεχθούν τα metrics του, για παράδειγμα [95]:

```
annotations:
  prometheus.io/scrape: "true"
  prometheus.io/port: "9249"
```

Το πρώτο annotation ενεργοποιεί τη συλλογή, ενώ το δεύτερο ορίζει τη θύρα στην οποία η υπηρεσία εκθέτει τα metrics της. Το Prometheus διαβάζει τα annotations αυτά και προσαρμόζει δυναμικά το σύνολο των στόχων συλλογής (targets), καθώς pods δημιουργούνται ή τερματίζονται στο cluster.

5.2.5 kube-prometheus-stack, ολοκληρωμένη λύση παρακολούθησης cluster

Στην πράξη, η εγκατάσταση του Prometheus σε Kubernetes γίνεται συνήθως μέσω του kube-prometheus-stack, ενός Helm chart [203] που παρέχει ως ενιαίο πακέτο τον Prometheus Operator, τον Prometheus Server με προρυθμισμένους στόχους συλλογής, τον Alertmanager για ειδοποιήσεις, το Grafana στο οποίο περιλαμβάνονται προεγκατεστημένα dashboards, ειδικά για Kubernetes, καθώς και τον node-exporter και τον kube-state-metrics για metrics nodes και αντικειμένων Kubernetes [198].

Στην παρούσα εργασία, ο kube-prometheus-stack χρησιμεύει ως βάση τόσο για την παρακολούθηση υποδομής (χρήση CPU, μνήμης και δίσκου ανά node και pod) όσο και για τη συλλογή metrics εφαρμογής, εν προκειμένω Flink metrics μέσω Prometheus reporter.

5.3 Metrics των Flink, Kafka και Kubernetes

5.3.1 Flink metrics

Το Flink [13, 22] εκθέτει ένα εκτεταμένο σύνολο metrics μέσω του ενσωματωμένου συστήματος metrics reporters. Μεταξύ των πιο κρίσιμων metrics, ο ρυθμός εισαγωγής (`numRecordsInPerSecond`) και ο ρυθμός εξαγωγής (`numRecordsOutPerSecond`) δείχνουν πόσες εγγραφές εισέρχονται και εξέρχονται από κάθε τελεστή, αντίστοιχα. Τα metrics σημείων ελέγχου (`checkpoint duration`, `checkpoint size`) καταγράφουν τη διάρκεια και το μέγεθος κάθε σημείου ελέγχου, παράμετροι που σχετίζονται άμεσα με τον μηχανισμό διαχείρισης κατάστασης του Flink, ο οποίος περιγράφεται αναλυτικά στην βιβλιογραφία [149]. Η καθυστέρηση του καταναλωτή Kafka (`consumer lag`) μετρά τη διαφορά μεταξύ του τελευταίου μηνύματος στο θέμα και του τελευταίου αναγνωσμένου, ενώ η χρήση μνήμης (`managed memory`, `heap`) είναι κρίσιμη για τον εντοπισμό σφαλμάτων (`OutOfMemoryError`).

5.3.2 Kafka metrics

Οι Kafka brokers [21] δημοσιεύουν metrics μέσω Java Management Extensions (JMX) [204], οι οποίες μετατρέπονται σε μορφή Prometheus μέσω JMX exporter. Ο Πίνακας 5.1 συνοψίζει τις βασικότερες από αυτές.

Πίνακας 5.1: *Βασικά Kafka metrics που συλλέγονται μέσω JMX exporter.*

Metrics	**Περιγραφή**
`MessagesInPerSec`	Ρυθμός εισερχομένων μηνυμάτων ανά topic
`BytesInPerSec / BytesOutPerSec`	Όγκος δεδομένων εισόδου και εξόδου
`UnderReplicatedPartitions`	Partitions χωρίς πλήρη αντιγραφή
`Consumer group lag`	Μηνύματα σε αναμονή ανάγνωσης από το Flink

5.3.3 Kubernetes metrics

Στο επίπεδο υποδομής, ο kube-state-metrics και ο node-exporter παρέχουν metrics κρίσιμης σημασίας [95]. Η χρήση CPU (`container_cpu_usage_seconds_total`) και μνήμης (`container_memory_working_set_bytes`) μετρούνται ανά container, ενώ η κατάσταση κάθε pod (`kube_pod_status_phase`) δείχνει αν βρίσκεται σε `Running`, `Pending` ή `CrashLoopBackOff`. Η διαθέσιμη χωρητικότητα δίσκου (`kubelet_volume_stats_available_bytes`) είναι ιδιαίτερα σημαντική για τα PVC του Kafka και του Prometheus, ενώ η κυκλοφορία δικτύου (`container_network_receive_bytes_total`, `container_network_transmit_bytes_total`) καταγράφει τον όγκο δεδομένων που ανταλλάσσονται μεταξύ containers.

Το σύνολο αυτών των metrics αποτελεί τη βάση των πειραμάτων αξιολόγησης, καθώς παρέχει ποσοτικά δεδομένα για τη χρήση πόρων κάθε υποσυστήματος υπό διαφορετικά σενάρια φόρτου.

5.4 Trino, μηχανή ερωτημάτων SQL

Το Trino [24, 205], γνωστό παλαιότερα ως PrestoSQL, είναι μια μηχανή κατανεμημένων ερωτημάτων SQL σχεδιασμένη για διαδραστικές αναλύσεις πάνω σε μεγάλα σύνολα δεδομένων. Η αρχιτεκτονική του ακολουθεί μοντέλο συντονιστή (coordinator) και εργάτη (workers). Ο coordinator λαμβάνει τα ερωτήματα SQL, δημιουργεί το σχέδιο εκτέλεσης (query plan), κατανέμει τα τμήματα (splits) στους workers και συντονίζει τη συγκέντρωση αποτελεσμάτων. Οι workers, με τη σειρά τους, εκτελούν τα τμήματα αυτά, διαβάζοντας δεδομένα από τους connectors όπως ο Paimon connector και επεξεργάζοντάς τα στη μνήμη τους.

5.4.1 Κατάλογοι δεδομένων

Ένα κεντρικό χαρακτηριστικό του Trino είναι η δυνατότητα πρόσβασης σε πολλαπλές πηγές δεδομένων μέσω του μηχανισμού catalogs. Ένα μόνο ερώτημα μπορεί να συνδυάσει δεδομένα από Paimon, PostgreSQL, S3 ή Kafka χωρίς να χρειαστεί μετακίνηση δεδομένων [205]. Η δυνατότητα αυτή, γνωστή ως ομοσπονδία δεδομένων (data federation), επιτρέπει στο Trino να λειτουργεί ως ενιαίο σημείο πρόσβασης SQL σε ετερογενείς αποθήκες δεδομένων.

Στην εργασία αυτή, το Paimon catalog χρησιμοποιείται ως κύρια πηγή και εκθέτει τα στρώματα δεδομένων όπως τα ακατέργαστα δεδομένα (raw), ή τις ωριαίες συγκεντρώσεις ως πίνακες SQL σε αντίστοιχα σχήματα (schemas).

5.4.2 Μοντέλο εκτέλεσης

Το Trino λειτουργεί αποκλειστικά στη μνήμη (in-memory), χωρίς να αποθηκεύει ενδιάμεσα αποτελέσματα σε δίσκο [205]. Κάθε ερώτημα αναλύεται σε σχέδιο εκτέλεσης από τον coordinator, ο οποίος κατανέμει τμήματα εργασίας (splits) στους workers. Τα τμήματα εκτελούνται παράλληλα σε pipeline, χωρίς υλοποίηση ενδιάμεσων αποτελεσμάτων στο δίσκο. Αυτή η αρχιτεκτονική εξηγεί τόσο τις υψηλές απαιτήσεις μνήμης όσο και τους χαμηλούς χρόνους απόκρισης σε ερωτήματα που χωρούν στη μνήμη.

5.4.3 Βελτιστοποίηση ερωτημάτων

Κατά τη δημιουργία του σχεδίου εκτέλεσης, το Trino εφαρμόζει σειρά βελτιστοποιήσεων [205]. Με τη τεχνική προώθησης κατηγορημάτων (predicate pushdown) [206], τα φίλτρα WHERE μεταφέρονται στον connector, έτσι ώστε ο Paimon να διαβάζει μόνο τα σχετικά αρχεία, π.χ. φιλτράροντας κατά partition χρονοσφραγίδας (dt). Μέσω κλαδέματος στηλών (column pruning), αναγνωρίζονται μόνο οι στήλες που χρειάζονται πραγματικά, αξιοποιώντας τη μορφή Parquet. Η βελτιστοποίηση βάσει κόστους χρησιμοποιεί στατιστικά πινάκων για την επιλογή των βέλτιστων σχεδίων ένωσης (join).

5.5 Οπτικοποίηση και Grafana dashboards

Το Grafana [20] είναι μια πλατφόρμα ανοιχτού κώδικα για τη δημιουργία διαδραστικών dashboards. Σχεδιάστηκε αρχικά για δεδομένα παρακολούθησης, αλλά πλέον χρησιμοποιείται ευρέως και για οπτικοποίηση δεδομένων εφαρμογών. Στη βιβλιογραφία [207] ορίζεται ο χώρος σχεδίασης πινάκων ελέγχου και αναγνωρίζεται η λειτουργική παρακολούθηση (operational monitoring) ως μία από τις κεντρικές κατηγορίες χρήσης τους, γεγονός που αντανακλά ακριβώς τον ρόλο του Grafana στην παρούσα εργασία.

Στον πυρήνα του, το Grafana λειτουργεί ως στρώμα οπτικοποίησης πάνω σε εξωτερικές πηγές δεδομένων. Μέσω data source plugins όπως για παράδειγμα Prometheus, InfluxDB, αλλά και SQL βάσεις μέσω JDBC [185] ή Open Database Connectivity (ODBC) [208], το Grafana εκτελεί ερωτήματα, λαμβάνει τα αποτελέσματα και τα αποδίδει σε διαδραστικά γραφήματα χρονοσειράς, πίνακες, μετρητές και χάρτες.

5.5.1 Βασικά χαρακτηριστικά

Στην παρούσα εργασία αξιοποιούνται αρκετά χαρακτηριστικά του Grafana. Κάθε dashboard αποτελείται από πλαίσια (panels), καθένα από τα οποία εκτελεί ένα ερώτημα σε μία πηγή δεδομένων, εν προκειμένω το Prometheus ή το Trino, και αποδίδει τα αποτελέσματα γραφικά. Οι μεταβλητές (template variables) λειτουργούν ως δυναμικά φίλτρα, π.χ. για την επιλογή εγκατάστασης ή μετρητή ή χρονικού διαστήματος, και παραμετροποιούν τα ερωτήματα. Ο ρυθμός ανανέωσης (refresh rate) καθορίζει την περιοδικότητα αυτόματων ερωτημάτων (π.χ. κάθε 30 δευτερόλεπτα) και επηρεάζει τόσο την εμπειρία χρήστη όσο και τον φόρτο στο Trino. Επιπλέον, τα dashboards μπορούν να οριστούν ως αρχεία JSON και να φορτωθούν αυτόματα κατά την εκκίνηση (provisioning as code), εξαλείφοντας τη χειροκίνητη ρύθμιση. Τέλος, το Grafana υποστηρίζει ενσωματωμένους κανόνες ειδοποίησης βάσει κατωφλίων σε metrics, ως εναλλακτική ή συμπλήρωμα του Alertmanager.

5.5.2 Παρακολούθηση επίδοσης υποδομής και ανάλυση δεδομένων εφαρμογής

Στην παρούσα εργασία, το Grafana εξυπηρετεί δύο ρόλους. Ο πρώτος αφορά την παρακολούθηση υποδομής. Συγκεκριμένα μέσω της πηγής δεδομένων Prometheus, εμφανίζονται metrics χρήσης CPU, μνήμης και δίσκου ανά node και pod, κατάσταση pods και ρυθμοί δικτύου. Τα προεγκατεστημένα dashboards του kube-prometheus-stack παρέχουν αυτή τη λειτουργικότητα χωρίς πρόσθετη ρύθμιση. Ο δεύτερος ρόλος αφορά την ανάλυση δεδομένων IoT αφού μέσω της πηγής δεδομένων Trino, εγκατεστημένης ως plugin, εμφανίζονται δείκτες ενέργειας (KPIs), ωριαίες τάσεις κατανάλωσης, ποιότητα ισχύος και ζωντανές ακατέργαστες μετρήσεις.

Ο διπλός αυτός ρόλος ενοποιεί σε μία μόνο πλατφόρμα τη λειτουργική παρακολούθηση που απαντά στο ερώτημα «λειτουργεί σωστά η υποδομή;» και τη λειτουργική ανάλυση γύρω από το «τι δείχνουν τα δεδομένα IoT;», αποφεύγοντας τη χρήση πολλαπλών εργαλείων οπτικοποίησης.

5.6 Ολοκληρωμένη λύση, από τη ροή δεδομένων στην οπτικοποίηση

Με βάση την αρχιτεκτονική που περιγράφηκε στα προηγούμενα κεφάλαια, η ροή δεδομένων σχηματίζει έναν πλήρη κύκλο. Τα δεδομένα IoT παράγονται, μεταφέρονται μέσω Kafka, επεξεργάζονται από το Flink, αποθηκεύονται στο Paimon σε Azure ADLS Gen2, αναλύονται μέσω Trino και οπτικοποιούνται σε Grafana dashboards. Παράλληλα, το Prometheus συλλέγει metrics από κάθε υποσύστημα, τροφοδοτώντας τα dashboards υποδομής και τους κανόνες ειδοποίησης.

Αυτή η διπλή ροή, δεδομένα εφαρμογής αφενός και metrics παρακολούθησης αφετέρου, συνιστά θεμελιώδες χαρακτηριστικό μίας ολοκληρωμένης αρχιτεκτονικής data lakehouse πάνω σε Kubernetes. Εξασφαλίζει ταυτόχρονα τη λειτουργική αξιοπιστία του συστήματος και τη δυνατότητα εξαγωγής χρήσιμης πληροφορίας από τα δεδομένα.

Μέρος II

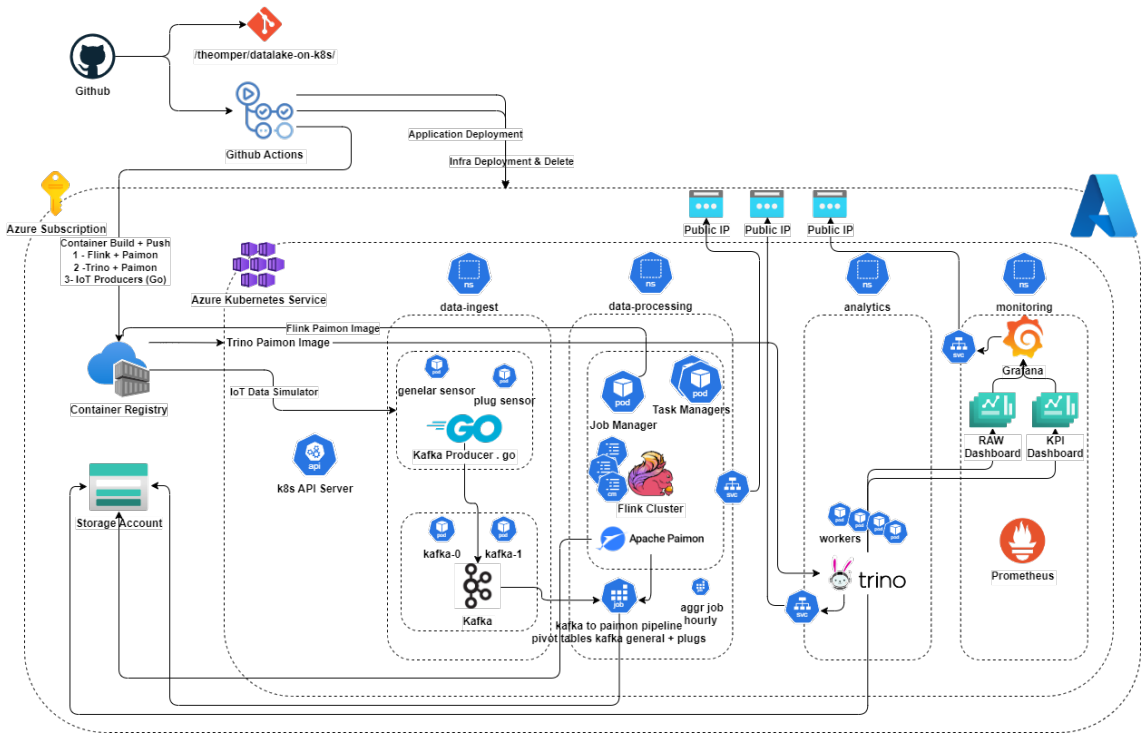
Πρακτικό Μέρος

Κεφάλαιο 6

Αρχιτεκτονικός Σχεδιασμός

Στο κεφάλαιο αυτό παρουσιάζεται ο αρχιτεκτονικός σχεδιασμός τόσο της υποδομής στο Azure όσο και της λύσης στο Kubernetes. Ο σχεδιασμός ακολουθεί τη φιλοσοφία του Infrastructure as Code [25, 26], ώστε κάθε πόρος να είναι αναπαράξιμος, ελεγχόμενος μέσω έλεγχου πηγαίου κώδικα (Git) [209] και αυτοματοποιημένος. Η αρχιτεκτονική χωρίζεται σε δύο αλληλένδετα επίπεδα τα οποία είναι το επίπεδο υποδομής cloud στο Azure, και το επίπεδο εφαρμογής, που αφορά τις υπηρεσίες δεδομένων εντός του Kubernetes cluster.

Η Εικόνα 6.1 απεικονίζει τη συνολική αρχιτεκτονική του συστήματος, συμπεριλαμβανομένων των πόρων υποδομής στο Azure και της οργάνωσης των λογικών χώρων ονομάτων (namespaces) εντός του Kubernetes cluster.



Εικόνα 6.1: *Αρχιτεκτονικός σχεδιασμός υποδομής Azure και Kubernetes*

6.1 Σχεδιασμός Υποδομής στο Azure

Οι πόροι υποδομής οργανώνονται σε ένα ενιαίο Resource Group στο Azure, διαχειριζόμενοι μέσω Terraform μονάδων (modules). Η χρήση ενοτήτων (modules) εξασφαλίζει διαχωρισμό αρμοδιοτήτων (separation of concerns) μεταξύ υπολογιστικών πόρων, αποθήκευσης και διαχείρισης images [14]. Η αρχιτεκτονική αναφοράς βασίζεται στις βέλτιστες πρακτικές που περιγράφονται στο εγχειρίδιο Microsoft Learn [210] για clusters τύπου Azure Kubernetes Service (AKS) [211].

6.1.1 Tenant

Κάθε οργανισμός ή ιδιώτης που χρησιμοποιεί υπηρεσίες στο Azure λειτουργεί εντός ενός ενοικιαστή (tenant), ο οποίος αντιστοιχεί σε μια αποκλειστική παρουσία του καταλόγου Microsoft Entra ID (πρώην Azure Active Directory) [212]. Ο tenant αποτελεί το ανώτατο επίπεδο ταυτοτήτων και πρόσβασης καθώς περιέχει τους χρήστες, τις ομάδες, και τις πολιτικές ασφαλείας που ελέγχουν ποιος μπορεί να δημιουργεί ή να διαχειρίζεται πόρους. Στο πλαίσιο της παρούσας εργασίας χρησιμοποιείται ένας μοναδικός προσωπικός tenant με έναν μόνο χρήστη-διαχειριστή.

6.1.2 Subscription

Κάτω από τον tenant, η συνδρομή (subscription) αποτελεί το βασικό όριο χρέωσης και διαχείρισης πόρων [213]. Κάθε πόρος στο Azure ανήκει σε ακριβώς μία συνδρομή, η οποία καθορίζει τα ανώτατα όρια (quotas) υπηρεσιών, τις πολιτικές πρόσβασης μέσω ελέγχου πρόσβασης βάσει ρόλου, Role-Based Access Control (RBAC), και το μοντέλο τιμολόγησης. Στην παρούσα εργασία χρησιμοποιείται ένα subscription τύπου Pay-as-you-go δηλαδή χρέωση ανά χρήση, χωρίς κάποια δέσμευση, εντός της οποίας δημιουργούνται όλοι οι πόροι υποδομής που περιγράφονται στις επόμενες ενότητες.

6.1.3 Ομαδοποίηση πόρων και περιοχές φιλοξενίας (Resource Group / Region)

Όλοι οι πόροι ανήκουν σε ένα μοναδικό Resource Group το οποίο ονομάστηκε rg-datalake-on-k8s-we-dev-01 στην περιοχή φιλοξενίας West Europe. Η επιλογή μίας μόνο ομάδας πόρων απλοποιεί τη διαχείριση κύκλου ζωής αφού η διαγραφή ή αναδημιουργία ολόκληρου του περιβάλλοντος επιτυγχάνεται με μία μόνο εντολή Terraform. Η γεωγραφική περιοχή West Europe (Ολλανδία) επιλέχθηκε λόγω χαμηλής καθυστέρησης δικτύου σε επικοινωνία με την Ελλάδα και διαθεσιμότητας των απαιτούμενων μεγεθών Stock Keeping Unit (SKUs) των VMs.

6.1.4 Azure Kubernetes Service και node pools

Το cluster AKS (aks-datalake-on-k8s-we-dev-01) αποτελεί τον πυρήνα της υπολογιστικής υποδομής [95, 214]. Ο σχεδιασμός προβλέπει δύο node pools, ακολουθώντας τη σύσταση του Microsoft Learn για διαχωρισμό system και user node pools [215]:

- **System Pool**. Ένα node μεγέθους Standard_E4s_v6 με 4 εικονικούς επεξεργαστές, virtual CPUs ή vCPUs [53], και 32GB μνήμης τυχαίας προσπέλασης, Random Access Memory ή RAM [70]. Το node αυτό είναι αφιερωμένο στα συστατικά του Kubernetes, ήτοι στα kube-system pods, στο Prometheus, στο Grafana και γενικότερα σε pods υποδομής. Σημειώνεται ότι, για τις ανάγκες της παρούσας εργασίας, στο pool αυτό τοποθετείται επιπλέον και ο ελαφρύς Flink JobManager (βλ. Παράρτημα Γ'.2), ενώ οι απαιτητικοί TaskManagers παραμένουν στο user pool. Η αυτόματη κλιμάκωση κόμβων (autoscaling) ορίζεται από ένα εύρος της μορφής $[N_{\min}, N_{\max}]$, όπου $N_{\min}$ και $N_{\max}$ είναι ο ελάχιστος και ο μέγιστος επιτρεπόμενος αριθμός nodes στο pool αντίστοιχα. Για τη συγκεκριμένη ομάδα το εύρος ορίζεται ως $[1, 1]$, οπότε ο αριθμός nodes παραμένει σταθερά στον έναν.

- **User Pool**. Δύο nodes μεγέθους Standard_E8s_v6 με 8 vCPUs και 64GB RAM ανά node, οι οποίοι φιλοξενούν τους φόρτους εργασίας επεξεργασίας των δεδομένων, ήτοι Kafka, Flink, Trino και τους παραγωγούς δεδομένων IoT. Με την ίδια σύμβαση, το εύρος κλιμάκωσης ορίζεται ως $[2, 2]$ για τα βασικά πειράματα, δηλαδή σταθερά δύο nodes, ενώ μπορεί να αυξηθεί σε σενάρια υψηλού φόρτου.

Ο διαχωρισμός σε δύο ομάδες εξασφαλίζει ότι οι φόρτοι δεδομένων δεν ανταγωνίζονται τα συστατικά ελέγχου για πόρους, αποφεύγοντας κρίσιμες αστοχίες λόγω έλλειψης μνήμης γνωστές ως OutOfMemory (OOM) kills σε pods υποδομής.

6.1.5 Azure Virtual Network, υποδίκτυα και πρόσβαση στο cluster

Το AKS cluster αναπτύσσεται εντός ενός εικονικού δικτύου (Azure Virtual Network, VNet) με ξεχωριστό υποδίκτυο (subnet) για τους nodes. Το δικτυακό μοντέλο χρησιμοποιεί τον Azure CNI [99, 104], το οποίο αναθέτει IP διευθύνσεις του VNet απευθείας στα pods. Αυτό απλοποιεί τη δρομολόγηση μεταξύ pods και υπηρεσιών Azure (ADLS, ACR), καθώς δεν απαιτείται μετάφραση διευθύνσεων δικτύου (NAT) [98].

Η πρόσβαση στο cluster γίνεται μέσω του εργαλείου γραμμής εντολών kubectl με διαπιστευτήρια που παρέχονται από το Azure CLI (εντολή az aks get-credentials), ενώ ο εσωτερικός DNS (CoreDNS [216]) επιλύει αυτόματα τα ονόματα υπηρεσιών (π.χ. kafka-0.kafka-svc.data-ingest.svc.cluster.local) που φιλοξενούνται σε αυτό.

6.1.6 Έκθεση υπηρεσιών μέσω Azure Load Balancer

Ορισμένες υπηρεσίες εκτίθενται εξωτερικά μέσω Kubernetes Services τύπου LoadBalancer, τα οποία δημιουργούν αυτόματα έναν Azure Load Balancer στο δημόσιο Internet. Συγκεκριμένα, εκτίθενται:

- Το Grafana, μέσω της θύρας 80, για πρόσβαση στα dashboards.

- Το Trino Web UI, μέσω της θύρας 8080, για παρακολούθηση και εκτέλεση ερωτημάτων SQL.

- Το Flink Web UI, μέσω της θύρας 8081, για την παρακολούθηση ενεργών εργασιών.

Σε παραγωγικό περιβάλλον, η πρόσβαση αυτή θα περιοριζόταν μέσω Network Security Groups (NSGs) ή Azure Application Gateway με την παρουσία πιστοποιητικού κρυπτογράφησης, αλλά στο πλαίσιο αυτής της εργασίας παραμένει ανοιχτή για λόγους ευκολίας στις δοκιμές.

6.1.7 Αποθήκευση δεδομένων τύπου object, Azure Data Lake Storage Gen2 και σύστημα αρχείων τύπου ABFS

Η μόνιμη αποθήκευση δεδομένων data lakehouse γίνεται σε Azure Storage Account τύπου Data Lake Storage Gen2 που ονομάστηκε stdatalakeonk8swe01 και ο οποίος παρέχει ιεράρχηση ονομάτων (hierarchical namespace) [6, 217]. Σύμφωνα με το εγχειρίδιο Microsoft Learn [162], η ιεράρχηση ονομάτων επιτρέπει ατομικές λειτουργίες μετονομασίας και διαγραφής καταλόγων, βελτιώνοντας δραστικά την απόδοση αναλυτικών φόρτων εργασίας και των commit λειτουργιών του Paimon.

Το Azure Storage υποστηρίζει πολλαπλά επίπεδα πρόσβασης (access tiers) που επιτρέπουν βελτιστοποίηση κόστους ανάλογα με τη συχνότητα πρόσβασης στα δεδομένα [218]. Το επίπεδο Hot προορίζεται για δεδομένα που προσπελαύνονται συχνά και προσφέρει το χαμηλότερο κόστος ανάγνωσης και εγγραφής ανά λειτουργία, αλλά υψηλότερο κόστος αποθήκευσης ανά Gigabyte (GB) [218]. Το επίπεδο Cool απευθύνεται σε δεδομένα που αποθηκεύονται για τουλάχιστον 30 ημέρες και σπάνια προσπελαύνονται, επιτυγχάνοντας χαμηλότερο κόστος αποθήκευσης σε αντάλλαγμα υψηλότερου κόστους ανά λειτουργία ανάγνωσης [218]. Το επίπεδο Cold στοχεύει σε δεδομένα με ελάχιστη διάρκεια αποθήκευσης 90 ημερών, επιτυγχάνοντας ακόμα μεγαλύτερη μείωση κόστους αποθήκευσης κατάλληλη για αρχεία που σπάνια ή ποτέ δεν προσπελαύνονται [218]. Τέλος, το επίπεδο Archive προσφέρει το χαμηλότερο κόστος αποθήκευσης για δεδομένα με ελάχιστη διάρκεια αποθήκευσης 180 ημερών, ωστόσο αποτελεί offline επίπεδο με χρόνο ανάκτησης της τάξης των ωρών [218]. Στην παρούσα εργασία, ο λογαριασμός αποθήκευσης λειτουργεί αποκλειστικά σε επίπεδο Hot [218], καθώς τόσο τα ακατέργαστα δεδομένα όσο και οι συγκεντρώσεις προσπελαύνονται τακτικά από το Flink και το Trino. Ωστόσο, σε παραγωγικό περιβάλλον, η εφαρμογή πολιτικών κύκλου ζωής (lifecycle management policies) θα μπορούσε να μεταφέρει αυτόματα παλαιότερα δεδομένα συγκεντρώσεων σε επίπεδο Cool, μειώνοντας το κόστος αποθήκευσης κατά περίπου 50%.

Η πρόσβαση από τα pods γίνεται μέσω Azure Blob File System (ABFS):

```
abfs://warehouse@stdatalakeonk8swe01.dfs.core.windows.net/paimon
```

Η αυθεντικοποίηση (authentication) πραγματοποιείται μέσω κλειδιού (Storage Account Key), το οποίο αποθηκεύεται ως Kubernetes Secret και εισάγεται στα pods μέσω μεταβλητών περιβάλλοντος (environment variables).

6.1.8 Azure Container Registry

Τα εξειδικευμένα images της παρούσας εργασίας είναι αυτό του Flink με connectors Paimon και Kafka, του Trino με Paimon connector, και των παραγωγών IoT γραμμένων σε Go, τα οποία αποθηκεύονται σε ιδιωτικό αποθετήριο ACR (με ονομασία

acrdatalakeonk8swe01) [79]. Το ACR συνδέεται με το AKS cluster μέσω της ταυτότητας kubelet (kubelet managed identity), ώστε τα pods να μπορούν να φορτώνουν images χωρίς πρόσθετη ρύθμιση κλειδιών (imagePullSecrets) [76]. Η ρύθμιση αυτή αυτοματοποιείται μέσω του Terraform module acr, το οποίο αναθέτει τον ρόλο AcrPull στην ταυτότητα kubelet, ακολουθώντας το μοντέλο συσχετισμού ACR-AKS που συνιστάται από τη Microsoft [76].

6.1.9 Αποθήκευση τύπου block, Azure Managed Disks για Persistent Volume Claims

Τα stateful υποσυστήματα Kafka και Prometheus χρησιμοποιούν PVCs που αντιστοιχούν σε Azure Managed Disks μέσω του CSI driver (managed-csi StorageClass) [219]. Κάθε Kafka pod αιτείται 128Gi (gibibytes) αποθηκευτικού χώρου τύπου Premium SSD για τη μόνιμη αποθήκευση logs και partitions, ενώ το Prometheus αιτείται 50Gi για τη διατήρηση χρονοσειράς metrics διάρκειας 15 ημερών.

6.1.10 Εκτίμηση κόστους σε παρόχους Public Cloud

Η εκτίμηση κόστους παρουσιάζεται σε τέσσερα μέρη. Πρώτα, η αναλυτική κοστολόγηση για τη μέγιστη διαμόρφωση στο Azure, η οποία χρησιμοποιήθηκε στα πειράματα. Στη συνέχεια, μια συγκριτική ανάλυση κόστους στο Azure ανά ζεύγος βαθμίδων, βασισμένη στους ελάχιστους πόρους που απαιτεί κάθε επίπεδο φόρτου. Τέλος, δύο υποενότητες που παρέχουν προσεγγιστική συγκριτική εκτίμηση έναντι των παρόχων Amazon Web Services (AWS) [220] και Google Cloud Platform (GCP) [221], ώστε να αναδειχθεί η σχετική τιμολόγηση ισοδύναμων υπηρεσιών στους τρεις μεγαλύτερους παρόχους public cloud.

6.1.10.1 Αναλυτική κοστολόγηση μέγιστης διαμόρφωσης στο Azure

Πριν από την έναρξη των πειραμάτων, καταρτίστηκε εκτίμηση μηνιαίου κόστους με τις ακόλουθες παραδοχές, λειτουργία 24×7 (730 ώρες ανά μήνα), περιοχή West Europe, τιμολόγηση Pay-as-you-go, νόμισμα EUR (€). Ο Πίνακας 6.1 συνοψίζει την ανάλυση κόστους ανά πόρο, όπως δημιουργήθηκε και εξήχθη από τον Azure Pricing Calculator [222][1].

Αναλυτικές παραδοχές ανά πόρο

- **Υπολογιστικοί πόροι (AKS)**. Η υπηρεσία AKS δεν χρεώνεται ξεχωριστά δηλαδή το control plane παρέχεται δωρεάν. Το κόστος αφορά αποκλειστικά τα VMs των node pools. Το node pool συστήματος (systempool) περιλαμβάνει ένα node τύπου Standard_E4s_v6 το οποίο κοστίζει €249,90 ανά μήνα, ενώ το node pool χρήστη (userpool) περιλαμβάνει δύο nodes τύπου Standard_E8s_v6 τα οποία κοστίζουν €789,29 ανά μήνα. Το συνολικό κόστος των υπολογιστικών πόρων είναι €1.039,19.

- **Δίσκοι (Managed Disks)**. Κάθε Kafka broker χρησιμοποιεί δίσκο Premium SSD τύπου P10 (128GiB, 500 IOPS) για τα log τμήματα (2×€18,37 = €36,74). Το Prometheus χρησιμοποιεί δίσκο Standard SSD τύπου E10 (128GiB) με 10.000 IOPS

[1] Η πλήρης εκτίμηση είναι διαθέσιμη στη διεύθυνση https://azure.com/e/d8397f43f0564d89a15e790f54715ef4

Πίνακας 6.1: *Εκτίμηση μηνιαίου κόστους Azure μέσω Azure Pricing Calculator (περιοχή West Europe, Pay-as-you-go, τιμές σε €).*

Κατηγορία	**Υπηρεσία**	**Ρύθμιση**	**€/μήνα**
Compute	AKS	1×E4s_v6 (4 vCPUs, 32GB), 730h, Linux	249,90
Compute	Virtual Machines	2×E8s_v6 (8 vCPUs, 64GB), 730h, Linux	789,29
Storage	Managed Disks	Premium SSD P10, LRS, 2 δίσκοι (Kafka PVCs)	36,74
Storage	Managed Disks	Standard SSD E10, LRS, 1 δίσκος (Prometheus PVC)	25,08
Storage	Managed Disks	Standard SSD E6, LRS, 1 δίσκος (Grafana PVC)	5,76
Storage	ADLS	Standard LRS, Hot, HNS, 1.000GB χωρητικότητα	108,10
Containers	ACR	Basic tier, 10GB αποθήκευση	5,08
Networking	Load Balancer	Standard, 3 κανόνες, 50GB επεξεργ.	15,68
Networking	IP Addresses	Basic (Classic), 3 δυναμικές IP	4,95
Networking	Bandwidth	10GB εξερχόμενη κίνηση (Microsoft Global Network)	0,00
Support	–	Free tier	0,00
Σύνολο (εκτιμώμενο μηνιαίο κόστος)			**1.240,58**

(€25,08). Το Grafana χρησιμοποιεί δίσκο τύπου Standard SSD E6 (32GiB) με 1.000 μέγιστο αριθμό IOPS (€5,76). Το συνολικό κόστος δίσκων είναι €67,58 ανά μήνα.

- **Αποθήκευση (ADLS)**. Azure Storage Account τύπου Standard LRS με ενεργοποιημένη ιεράρχηση ονομάτων (HNS) και επίπεδο πρόσβασης τύπου Hot. Εκτιμώμενη χωρητικότητα 1.000GB. Το κόστος €108,10 ανά μήνα αναλύεται στον Πίνακα 6.2. Τα επίπεδα Hot δεν χρεώνουν ανάκτηση ή εγγραφή δεδομένων ανά GB, επομένως τα πεδία Data Retrieval και Data Write τίθενται σε 0. Ο αποθηκευτικός χώρος μεταδεδομένων (snapshot + manifest JSON) του Paimon εκτιμάται σε περίπου 1GB.

Πίνακας 6.2: *Ανάλυση κόστους ADLS ανά κατηγορία λειτουργιών (μηνιαία βάση, Hot tier).*

Κατηγορία λειτουργίας	**Ποσότητα**	**Τιμή μονάδας**	**€/μήνα**
Χωρητικότητα (Hot)	1.000 GB	€0,02/GB	20,00
Εγγραφές (Write ops, 4MB)	5.000.000	€0,0595/10K	29,74
Αναγνώσεις (Read ops, 4MB)	10.000.000	€0,0047/10K	4,75
Επαναλ. αναγνώσεις (Iterative read)	5.000.000	€0,059/10K	29,50
Επαναλ. εγγραφές (Iterative write)	45.000	€0,059/100	26,55
Λοιπές λειτουργίες (Other ops)	1.000.000	€0,005/10K	0,50
Metadata αποθήκευση	1 GB	€0,024/GB	0,02
Σύνολο ADLS			**108,10**

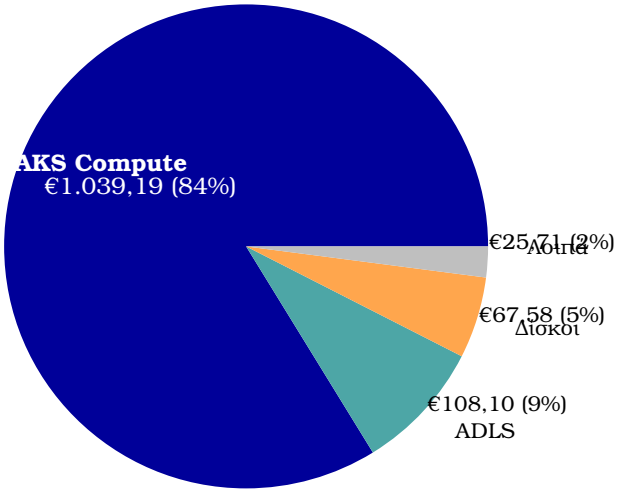
- **ACR**. Είναι επιπέδου Basic (10GB αποθηκευτικός χώρος) και κοστίζει €5,08 ανά μήνα. Επαρκεί για τα εξειδικευμένα images της εργασίας (flink-paimon-kafka, trino-paimon, go-iot-sim-producer).

- **Load Balancer**. Είναι τύπου Standard SKU, με δυνατότητα δημοσίευσης στο Ίντερ-

νετ, με τρεις κανόνες (Grafana θύρα 80, Trino θύρα 8080 και Flink Web UI θύρα 8081) και 50GB εκτιμώμενη δικτυακή κίνηση (€15,68 ανά μήνα). Ο Load Balancer δημιουργείται αυτόματα από το AKS κατά τον ορισμό services τύπου LoadBalancer.

- **Δημόσιες IP**. Τρεις δυναμικές τύπου Basic (Classic) διευθύνσεις (€4,95 ανά μήνα).
- **Εξερχόμενη κίνηση (egress traffic)**. Περίπου 10GB ανά μήνα μέσω Microsoft Global Network όμως το κόστος είναι €0,00 αφού τα πρώτα GB είναι δωρεάν. Αφορά κυρίως πρόσβαση των χρηστών στα Grafana dashboards από εξωτερικό δίκτυο.

Η κύρια πηγή κόστους, δηλαδή περίπου το 84% του μηνιαίου συνόλου, είναι οι υπολογιστικοί πόροι, δηλαδή τα node pools του AKS (€1.039,19). Το ADLS αποτελεί τη δεύτερη μεγαλύτερη δαπάνη (€108,10, δηλαδή περίπου το 9%), ακολουθούμενη από τους δίσκους Managed Disks οι οποίοι κοστίζουν €67,58, δηλαδή περίπου το 5%. Οι υπόλοιπες υπηρεσίες (ACR, Load Balancer, IP, Bandwidth) αντιστοιχούν αθροιστικά σε περίπου 2%, που μεταφράζεται σε €25,71. Η μετάβαση σε δέσμευση για τους υπολογιστικούς πόρους τύπου Reserved Instances ενός έτους μειώνει το κόστος κατά περίπου 37%, ενώ τριών ετών κατά περίπου 56% σύμφωνα με την εκπτωτική πολιτική της Microsoft. Η μετακίνηση ωριαίων δεδομένων παλαιότερων του ενός μήνα σε επίπεδο Cool θα μειώσει περαιτέρω το κόστος αποθήκευσης κατά περίπου 50%.



Σχήμα 6.1: *Κατανομή εκτιμώμενου μηνιαίου κόστους Azure ανά κατηγορία πόρων (σύνολο €1.240,58).*

6.1.10.2 Συγκριτική εκτίμηση κόστους ανά ζεύγος βαθμίδων στο Azure

Η αναλυτική εκτίμηση της Ενότητας 6.1.10.1 αντιστοιχεί στη χειρότερη περίπτωση κόστους, καθώς το cluster διαστασιολογείται ώστε να εξυπηρετεί τις μέγιστες βαθμίδες των πειραμάτων T9-T10 (έως 87.500 RPS, βλ. Πίνακα 8.4). Ωστόσο, για χαμηλότερες βαθμίδες φόρτου, το user pool μπορεί να περιοριστεί σε μικρότερα μεγέθη VMs, και η χωρητικότητα ADLS μπορεί να μειωθεί ανάλογα με τον όγκο δεδομένων. Τα σταθερά στοιχεία κόστους, system pool, δίσκοι, ACR, Load Balancer και δημόσιες IP, παραμένουν αμετάβλητα σε όλες τις διαμορφώσεις, αθροίζοντας €343,19 ανά μήνα.

Η διαστασιολόγηση του user pool βασίζεται στα metrics εκτεταμένης χρήσης πόρων (CPU, μνήμη) που παρουσιάζονται αναλυτικά στο Κεφάλαιο 8. Για κάθε ζεύγος βαθμίδων, η μέγιστη βαθμίδα του ζεύγους καθορίζει το ελάχιστο μέγεθος των nodes:

- **T1-T2** (≤2.500 RPS, 1 παραγωγός Go). Μέγιστη χρήση Kafka ≤0,23 CPU/broker, Flink TM ≤0,19 CPU, ελάχιστος φόρτος. Επαρκούν 2×E2s_v6 (4 vCPUs, 32GB συνολικά).

- **T3-T4** (≤17.500 RPS, 1 παραγωγός Go). Μέγιστη χρήση Kafka ≤1,5 CPU/broker, Flink TM ≤1,1 CPU, μέτριος φόρτος. Απαιτούνται 2×E4s_v6 (8 vCPUs, 64GB).

- **T5-T6** (≤37.500 RPS, 2–3 παραγωγοί Go). Μέγιστη χρήση Kafka ≤2,3 CPU/broker, Flink TM ≤1,7 CPU. Απαιτούνται 1×E4s_v6 + 1×E8s_v6 (12 vCPUs, 96GB).

- **T7-T8** (≤62.500 RPS, 3–4 παραγωγοί Go). Μέγιστη χρήση Kafka ≤2,4 CPU/broker, μνήμη ≤11GiB/broker, πλήρης εκμετάλλευση. Απαιτούνται 2×E8s_v6 (16 vCPUs, 128GB).

- **T9-T10** (≤87.500 RPS, 5 παραγωγοί Go). Ίδια διαμόρφωση με T7-T8, η τρέχουσα μέγιστη υποδομή (βλ. Ενότητα 6.1.10.1).

Η εκτιμώμενη χωρητικότητα του ADLS προσαρμόζεται ανάλογα με τον αποθηκευτικό αποτύπωμα κάθε ζεύγους βαθμίδων, λαμβάνοντας υπόψη τη λήξη partitions 7 ημερών στο ακατέργαστο στρώμα και τη σταδιακή συσσώρευση ωριαίων και δεκάλεπτων συγκεντρώσεων. Ο Πίνακας 6.3 συνοψίζει την εκτιμώμενη μηνιαία δαπάνη ανά ζεύγος βαθμίδων.

Πίνακας 6.3: *Εκτίμηση μηνιαίου κόστους Azure ανά ζεύγος βαθμίδων (Pay-as-you-go, West Europe, τιμές σε €). Σταθερά κόστη ίσα με €343,19 ανά μήνα (system pool, δίσκοι, ACR, δικτύωση).*

Βαθμίδες	**user pool SKUs**	**user pool (€)**	**ADLS (GB)**	**ADLS (€)**	**Σύνολο (€)**
T1-T2	2×E2s_v6 (4 vCPU, 32GB)	197	50	50	~**590**
T3-T4	2×E4s_v6 (8 vCPU, 64GB)	395	200	65	~**803**
T5-T6	1×E4s_v6 + 1×E8s_v6 (12 vCPU, 96GB)	592	400	80	~**1.015**
T7-T8	2×E8s_v6 (16 vCPU, 128GB)	789	700	95	~**1.227**
T9-T10	2×E8s_v6 (16 vCPU, 128GB)	789	1.000	108	~**1.241**

Η κύρια παρατήρηση είναι ότι η μείωση κόστους είναι σημαντική στις χαμηλότερες βαθμίδες. Συγκεκριμένα, η μετάβαση από τη μέγιστη διαμόρφωση (T9-T10) στην ελάχιστη (T1-T2) εξοικονομεί κατά προσέγγιση €650 ανά μήνα, δηλαδή το 52% του συνολικού κόστους, κυρίως λόγω μικρότερων VMs στο user pool. Αντίθετα, οι βαθμίδες T7-T8 απαιτούν ήδη τη μέγιστη υπολογιστική υποδομή, με μικρή διαφορά μόνο στο κόστος αποθήκευσης ADLS. Αυτό επιβεβαιώνει ότι ο κυρίαρχος παράγοντας κόστους παραμένουν οι υπολογιστικοί πόροι, ακολουθώντας το μοτίβο του Σχήματος 6.1.

6.1.10.3 Συγκριτική εκτίμηση μέγιστης διαμόρφωσης σε AWS και GCP

Για να εκτιμηθεί η σχετική τιμολόγηση της λύσης στους άλλους δύο μεγάλους παρόχους public cloud, η μέγιστη διαμόρφωση της Ενότητας 6.1.10.1 μεταφράζεται σε ισοδύναμες υπηρεσίες των AWS και GCP. Η αντιστοίχιση γίνεται σε επίπεδο κατηγορίας πόρου και ισοδύναμης διαστασιολόγησης (vCPUs, RAM, χωρητικότητα, λειτουργίες I/O), χωρίς εξαντλητική σύγκριση των ιδιαιτεροτήτων κάθε υπηρεσίας. Επιλέχθηκαν οι περιοχές eu-west-1 (Ιρλανδία) για το AWS και europe-west4 (Ολλανδία) για το GCP, ως γεωγραφικά ισοδύναμες της West Europe (Ολλανδία) του Azure. Όλες οι τιμές βασίζονται σε τιμολόγηση Pay-as-you-go και λειτουργία 24×7 (730 ώρες/μήνα), όπως εξήχθησαν από τους αντίστοιχους υπολογιστές κόστους AWS Pricing Calculator [223] και GCP Pricing Calculator [224], με προσεγγιστική μετατροπή 1 USD ≈ 0,93 €.

Η αντιστοίχιση των υπηρεσιών μεταξύ των τριών παρόχων συνοψίζεται στον Πίνακα 6.4. Σε ό,τι αφορά τα VMs, επιλέχθηκαν οι σειρές που δίνουν έμφαση στην μνήμη, ώστε να διατηρηθεί η ίδια αναλογία μνήμης ανά vCPU (8GB/vCPU) με τη σειρά E_v6 του Azure, αφού οι φόρτοι Kafka, Flink και Trino απαιτούν μεγάλη ποσότητα μνήμης για να λειτουργήσουν αποδοτικά.

Πίνακας 6.4: *Αντιστοίχιση υπηρεσιών μεταξύ Azure, AWS και GCP για την αρχιτεκτονική της εργασίας.*

Κατηγορία	**Azure**	**AWS**	**GCP**
Υπηρεσία Kubernetes	AKS	EKS	GKE
VMs	E{4,8}s_v{5,6}	r7i.{xl,2xl}arge	n2-highmem-{4,8}
Δίσκος τύπου block	Managed Disk	EBS	Persistent Disk
Object Storage	ADLS	S3	Cloud Storage
Image Registry	ACR	ECR	Artifact Registry
Load Balancer	Azure LB	NLB	Cloud LB
Δημόσιες IP	Public IP	Elastic IP	External IP

Σημεία μη πλήρους ισοδυναμίας. Δύο διαφορές χρήζουν προσοχής χωρίς να επηρεάζουν ουσιωδώς την εκτίμηση κόστους. Πρώτον, η υπηρεσία object storage του AWS δεν διαθέτει εγγενή hierarchical namespace όπως το ADLS. Το Paimon συνεργάζεται και με τους τρεις παρόχους μέσω αντίστοιχων file system connectors. Δεύτερον, οι αντίστοιχες υπηρεσίες Kubernetes σε AWS και GCP χρεώνουν τέλος διαχείρισης cluster ίσο με 0,10USD ανά ώρα (≈€68 ανά μήνα), σε αντίθεση με το AKS όπου το control plane προσφέρεται δωρεάν.

Ο Πίνακας 6.5 συνοψίζει την εκτιμώμενη μηνιαία δαπάνη ανά κατηγορία πόρων στους τρεις παρόχους.

Συγκριτική παρατήρηση. Για την ίδια αρχιτεκτονική και ισοδύναμη διαστασιολόγηση πόρων, το GCP είναι κατά προσέγγιση 14% οικονομικότερο από το Azure, με εξοικονόμηση περίπου €175 ανά μήνα, κυρίως λόγω χαμηλότερης ωριαίας τιμολόγησης των VMs μεγάλης μνήμης και της οικονομικότερης υπηρεσίας object storage. Το AWS τοποθετείται ενδιάμεσα, με προσέγγιση 5% χαμηλότερο κόστος έναντι του Azure, καθώς η εξοικονόμηση στους δίσκους και στο object storage αντισταθμίζεται από το τέλος διαχείρισης της υπηρεσίας

Πίνακας 6.5: *Συγκριτική εκτίμηση μηνιαίου κόστους μέγιστης διαμόρφωσης σε Azure (West Europe), AWS (eu-west-1) και GCP (europe-west4). Τιμολόγηση Pay-as-you-go, τιμές σε € ανά μήνα.*

Κατηγορία	**Υπηρεσία / Ρύθμιση**	**Azure**	**AWS**	**GCP**
Compute	Υπηρεσία Kubernetes, control plane	0,00	67,89	67,89
Compute	1× systempool (4 vCPU, 32GB)	249,90	194,00	170,00
Compute	2× userpool (8 vCPU, 64GB)	789,29	776,00	680,00
Storage	Δίσκοι (3 PVCs, 288GB)	67,58	32,00	59,00
Storage	Object Storage (1.000GB + λειτουργίες)	108,10	76,00	56,00
Containers	Image Registry (10GB)	5,08	0,93	0,93
Networking	Load Balancer + 3 δημόσιες IP + 10GB egress	20,63	34,70	31,20
Σύνολο (εκτιμώμενο μηνιαίο κόστος)		**1.240,58**	**~1.181**	**~1.065**

Kubernetes και το υψηλότερο κόστος των δικτυακών πόρων. Σημειώνεται ότι όλα τα ποσά αφορούν Pay-as-you-go τιμολόγηση. Με δεσμεύσεις τύπου Savings Plans, Reserved Instances (AWS) ή Committed Use Discounts (GCP) διάρκειας ενός έτους, οι δύο πάροχοι προσφέρουν εκπτώσεις της τάξης του 30-40% στο υπολογιστικό κόστος, συγκρίσιμες με την αντίστοιχη πολιτική του Azure, διατηρώντας σταθερή τη σχετική κατάταξη.

6.1.10.4 Συγκριτική εκτίμηση κόστους ανά ζεύγος βαθμίδων σε AWS και GCP

Ακολουθώντας τη μεθοδολογία της Ενότητας 6.1.10.2, η διαστασιολόγηση του user pool προσαρμόζεται ανά ζεύγος βαθμίδων ώστε να εξυπηρετήσει τους φόρτους εργασίας T1-T10. Η ισοδύναμη αντιστοίχιση VMs στους τρεις παρόχους public cloud συνοψίζεται στον Πίνακα 6.6.

Πίνακας 6.6: *Αντιστοίχιση user pool VMs ανά ζεύγος βαθμίδων στους τρεις παρόχους.*

Βαθμίδες	**Azure**	**AWS**	**GCP**
T1-T2	2×E2s_v6	2×r7i.large	2×n2-highmem-2
T3-T4	2×E4s_v6	2×r7i.xlarge	2×n2-highmem-4
T5-T6	1×E4s_v6 + 1×E8s_v6	1×r7i.xlarge + 1×r7i.2xlarge	1×n2-highmem-4 + 1×n2-highmem-8
T7-T8	2×E8s_v6	2×r7i.2xlarge	2×n2-highmem-8
T9-T10	2×E8s_v6	2×r7i.2xlarge	2×n2-highmem-8

Τα σταθερά στοιχεία κόστους, όπως αναφέρθηκαν και παραπάνω, αθροίζονται σε περίπου €343 για Azure, €329 για AWS και €328 για GCP. Ο Πίνακας 6.7 συνοψίζει την εκτιμώμενη συνολική μηνιαία δαπάνη ανά ζεύγος βαθμίδων στους τρεις παρόχους.

Η σχετική κατάταξη παραμένει σταθερή σε όλες τις βαθμίδες, με το GCP ως πιο οικονομική επιλογή (περίπου 11-14% μείωση έναντι Azure, ανάλογα με τη βαθμίδα) και το AWS ενδιάμεσα (περίπου 5% μείωση). Η μεγαλύτερη απόλυτη εξοικονόμηση παρατηρείται στις υψηλότερες βαθμίδες, καθώς το υπολογιστικό κόστος είναι ο κυρίαρχος παράγοντας. Σε όλους τους παρόχους το ποσοστό του υπολογιστικού κόστους έναντι του συνολικού παραμένει στην περιοχή του 80-85% (βλ. Σχήμα 6.1), επιβεβαιώνοντας ότι η επιλογή τύπου SKU

Πίνακας 6.7: *Συγκριτική εκτίμηση μηνιαίου κόστους ανά ζεύγος βαθμίδων σε Azure, AWS και GCP (Pay-as-you-go, ισοδύναμες ευρωπαϊκές περιοχές, τιμές σε € ανά μήνα). Στήλη User: κόστος user pool μόνο. Στήλη* ***Σύν.****: συνολική μηνιαία δαπάνη της διαμόρφωσης (user pool + σταθερά κόστη + αποθήκευση object).*

	Azure		**AWS**		**GCP**	
Βαθμ.	User	**Σύν.**	User	**Σύν.**	User	**Σύν.**
T1-T2	197	~**590**	194	~**558**	170	~**526**
T3-T4	395	~**803**	388	~**765**	340	~**706**
T5-T6	592	~**1.015**	582	~**969**	510	~**883**
T7-T8	789	~**1.227**	776	~**1.173**	680	~**1.060**
T9-T10	789	~**1.241**	776	~**1.181**	680	~**1.064**

των VMs είναι ο πλέον επιδραστικός μοχλός κόστους ανεξαρτήτως παρόχου public cloud.

6.2 Σχεδιασμός Εφαρμογής

Το επίπεδο εφαρμογής οργανώνεται σε τέσσερα namespaces τα οποία είναι ομαδοποιήσεις πόρων εντός του Kubernetes cluster, αντανακλώντας τα λογικά στάδια της ροής δεδομένων. Κάθε namespace απομονώνει τους πόρους και τις πολιτικές του αντίστοιχου υποσυστήματος [7]. Τα τέσσερα namespaces είναι:

Namespace	**Εφαρμογή**
data-ingest	Kafka, προσομοιωτές IoT δεδομένων
data-processing	Flink
analytics	Trino
monitoring	Prometheus, Grafana

Πίνακας 6.8: *Namespaces του Kubernetes cluster και οι αντίστοιχες εφαρμογές που φιλοξενούν.*

6.2.1 Ροή δεδομένων

Η συνολική ροή δεδομένων ακολουθεί ένα μοντέλο συνεχών γεγονότων (event-driven architecture) [28], διατρέχοντας τα στάδια Προσομοιωτής IoT → Kafka → Flink → Paimon → Trino → Grafana.

Αναλυτικά:

1. **Παραγωγή**. Οι προσομοιωτές IoT, γραμμένοι στη γλώσσα προγραμματισμού Go, δημιουργούν τηλεμετρικά γεγονότα σε μορφή JSON και τα δημοσιεύουν στο Kafka.

2. **Εισαγωγή** (ingestion). Το Kafka δέχεται τα γεγονότα σε δύο θέματα (topics) και τα αποθηκεύει σε partitions.

3. **Επεξεργασία**. Το Flink διαβάζει τα γεγονότα σε πραγματικό χρόνο, εκτελεί μετασχηματισμούς και τα γράφει στο Paimon (raw layer). Παράλληλα, μια χρονοπρογραμματισμένη εργασία τύπου CronJob εκτελεί ωριαίες συγκεντρώσεις σε batch λειτουργία. Αντίστοιχες υπάρχουν και για συγκεντρώσεις τριάντα και δέκα λεπτών.

4. **Ανάλυση**. Το Trino παρέχει πρόσβαση ερωτημάτων SQL στα τρία στρώματα δεδομένων.

5. **Οπτικοποίηση**. Το Grafana εκτελεί ερωτήματα μέσω του Trino και αποδίδει τα αποτελέσματα αυτών των ερωτημάτων σε dashboards.

6.2.2 Σχεδιασμός Kafka

Το Kafka αναπτύσσεται ως StatefulSet με δύο brokers, εντός του namespace data-ingest [21]. Ορίζονται δύο θέματα (topics):

- **iot-sensor-general**. Δέχεται μετρήσεις κεντρικών μετρητών ενέργειας ανά εγκατάσταση (κατοικία, εργοστάσιο, βιομηχανική μονάδα). Ρυθμίζεται με 4 partitions και συντελεστή αντιγράφων (replication factor) 2.

- **iot-sensor-plug**. Δέχεται μετρήσεις επιμέρους συσκευών (πρίζες, μηχανήματα, εξοπλισμός παραγωγής). Ρυθμίζεται με 6 partitions, αντανακλώντας τον μεγαλύτερο όγκο μηνυμάτων (κάθε εγκατάσταση παράγει μετρήσεις για πολλαπλές συσκευές, π.χ. 4 μηνύματα ανά δευτερόλεπτο έναντι 1 από τον κεντρικό μετρητή).

Η επιλογή replication factor ίση με 2 εξασφαλίζει ότι η αστοχία ενός broker δεν οδηγεί σε απώλεια δεδομένων, ενώ ο ελάχιστος αριθμός αντιγράφων in-sync replicas (min.insync.replicas ίσο με 1) επιτρέπει τη συνέχιση εγγραφών ακόμη και με μία αντιγραφή ενεργή [140]. Τα δεδομένα αποθηκεύονται σε PersistentVolumes (128Gi ανά broker), ώστε η επανεκκίνηση ενός pod να μην απαιτεί επανεισαγωγή δεδομένων.

6.2.3 Σχεδιασμός ροών επεξεργασίας στο Flink

Το Flink αναπτύσσεται ως standalone session, εντός του namespace data-processing, αποτελούμενο από έναν JobManager και δύο TaskManagers [13]. Κάθε Task Manager διαθέτει 4 task slots και 8Gi μνήμη σε επίπεδο διεργασίας, επιτυγχάνοντας εσωτερικό παραλληλισμό ίσο με 4, αριθμός ο οποίος αντιστοιχεί στο πλήθος των Kafka partitions. Το state αποθηκεύεται τοπικά, ενώ τα checkpoints εκτελούνται κάθε 60 δευτερόλεπτα σε λειτουργία `EXACTLY_ONCE` (βλ. §4.4.2) [149].

Λειτουργία εφαρμογής (application mode), εισαγωγή ακατέργαστων δεδομένων (streaming ingest) . Η κύρια ροή εισαγωγής εκτελείται ως μόνιμη εργασία (long-running streaming job). Ορίζονται δύο Kafka source πίνακες οι `sensor_data_kafka_home` και `sensor_data_kafka_plug` που ενοποιούνται σε ένα view τύπου `UNION ALL` το οποίο είναι η ένωση δύο ροών σε μία, χωρίς αφαίρεση διπλοτύπων. Κάθε εγγραφή μετασχηματίζεται κατά τις ενέργειες, πρώτον, της εξαγωγής της ημερομηνίας partition (`dt`) από το timestamp του γεγονότος, δεύτερον, της εφαρμογής `COALESCE` η οποία επιστρέφει την πρώτη μη-NULL τιμή από μια λίστα ορισμάτων στο `plug_id` (η κεντρική μέτρηση λαμβάνει τιμή "`general`"), και τέλος της εγγραφής των δεδομένων στον πίνακα `sensor_data_raw` του Paimon σε πραγματικό χρόνο.

Λειτουργία συνεδρίας (session mode), ωριαίες συγκεντρώσεις (batch aggregation) Οι ωριαίες συγκεντρώσεις εκτελούνται μέσω εργασίας Kubernetes CronJob στο δέκατο λεπτό κάθε ώρας. Η εργασία λειτουργεί σε batch mode και εφαρμόζει tumbling-window μίας ώρας στα δεδομένα των τελευταίων δύο ημερών[2], υπολογίζοντας μέση τάση, ρεύμα, συντελεστή ισχύος, συνολική ενέργεια (kWh), αριθμό δειγμάτων και μέγιστη ισχύ, ανά εγκατάσταση και συσκευή. Τα αποτελέσματα εγγράφονται στον πίνακα `sensor_data_hourly`. Κατ' αναλογία, μία δεύτερη εργασία CronJob (`tenmin-aggregation`) παράγει δεκάλεπτες συγκεντρώσεις στον πίνακα `sensor_data_10min` [151].

6.2.4 Σχεδιασμός Paimon και οργάνωση πινάκων σε 3 στρώματα partition δεδομένων ανά πηγή

Το Paimon catalog ορίζεται στη βάση δεδομένων IoT με warehouse path στο ADLS. Η οργάνωση σε στρώματα (medallion architecture) [52] υλοποιείται ως εξής:

- **Ακατέργαστο στρώμα (raw)**. Πίνακας `sensor_data_raw`, κατατμημένος ανά ημερομηνία (`dt`). Ως πρωτεύον κλειδί ορίζεται η πλειάδα `(dt, home_id, plug_id, ts)`. Η λήξη partitions (`partition.expiration-time`) ορίζεται σε 7 ημέρες, ώστε τα παλαιότερα δεδομένα να διαγράφονται αυτόματα.
- **Συγκεντρωτικό στρώμα (hourly)**. Πίνακας `sensor_data_hourly`, κατατμημένος ανά ημερομηνία. Δεν εφαρμόζεται λήξη partition και τα δεδομένα διατηρούνται επ' αόριστον, ενώ η ιστορία στιγμιοτύπων (`snapshots`) περιορίζεται σε 30 ημέρες.
- **Συγκεντρωτικό στρώμα (30-min)**. Πίνακας `sensor_data_30min`, με ίδια δομή partition και πολιτική διατήρησης με τον hourly.
- **Συγκεντρωτικό στρώμα (10-min)**. Πίνακας `sensor_data_10min`, με ίδια δομή partition και πολιτική διατήρησης με τον hourly.

Ο αριθμός buckets ο οποίος ρυθμίζεται για τον ακατέργαστο (raw) πίνακα, ο οποίος διαθέτει υψηλό ρυθμό εγγραφών, είναι 6 και 1 για τους hourly, 30-min και 10-min οι οποίοι έχουν χαμηλό όγκο δεδομένων ανά partition. Η χρήση στηλοτύπου (columnar) σε μορφή Parquet εξασφαλίζει χαμηλό μέγεθος αποθήκευσης και ταχεία αναζήτηση στηλών [23].

6.2.5 Σχεδιασμός Trino (κατάλογοι, σχήματα, βασικά ερωτήματα KPIs)

Το Trino [24] εγκαθίσταται μέσω Helm chart στο namespace analytics, με έναν συντονιστή (coordinator) και δύο εργάτες (workers). Ο coordinator ρυθμίζεται ώστε να συμμετέχει και ως εργάτης (`includeCoordinator: true`), κάτι χρήσιμο σε μικρά clusters. Κάθε node εκχωρεί 2GB μέγιστη μνήμη Java Virtual Machine (JVM) [225] heap.

Ο κύριος κατάλογος (catalog) ονομάζεται paimon και συνδέεται απευθείας στο warehouse path μέσω του Paimon connector. Η αυθεντικοποίηση στο ADLS γίνεται μέσω `ACCESS_KEY` που εισάγεται από Kubernetes Secret. Τα ερωτήματα KPIs εκτελούνται κατά

[2]Στα πειράματα του Κεφαλαίου 8, το ερώτημα περιορίζει τη σάρωση στις τελευταίες τρεις (3) ώρες μέσω πρόσθετου φίλτρου χρονοσφραγίδας (ts), ώστε να παραμένει σταθερό το κόστος ανάγνωσης (βλ. Παράρτημα Γ.7).

κύριο λόγο πάνω στον hourly πίνακα για ιστορικές αναλύσεις ενέργειας, αιχμής ζήτησης, μέσης τάσης και δευτερευόντως πάνω στον raw πίνακα για μετρήσεις σε πραγματικό χρόνο.

6.2.6 Σχεδιασμός Grafana dashboards και ρυθμός ανανέωσής τους

Το Grafana [20] εμπεριέχεται εντός του kube-prometheus-stack (Helm chart) και εγκαθίσταται στο namespace monitoring. Η σύνδεση με το Trino πραγματοποιείται μέσω του trino-datasource plugin, το οποίο εγκαθίσταται αυτόματα κατά την εκκίνησή του. Ο σχεδιασμός περιλαμβάνει τέσσερα dashboards:

1. **IoT Energy KPIs**. Ωριαίοι δείκτες ενέργειας, αιχμή ζήτησης, ενεργές εγκαταστάσεις, ανάλυση ανά εγκατάσταση ή συσκευή, μηνιαία τάση κατανάλωσης.
2. **IoT Live (RAW)**. Τελευταίες 100 ακατέργαστες μετρήσεις σε πραγματικό χρόνο, για επαλήθευση της ζωντανής ροής.
3. **IoT Energy Overview (Hourly)**. Επισκόπηση ωριαίων μεγεθών (τάση, ρεύμα, συντελεστής ισχύος, κατανάλωση).
4. **IoT Power Quality & Voltage (Hourly)**. Ποιότητα ισχύος, εκτός εύρους τάσης (%), μέσος συντελεστής ισχύος.

Τα dashboards ορίζονται ως αρχεία JSON και φορτώνονται αυτόματα. Ο ρυθμός ανανέωσης ρυθμίζεται δυναμικά ανά dashboard και η επίδρασή του στον φόρτο του Trino αναλύεται στο Κεφάλαιο 9.

Κεφάλαιο 7

Υλοποίηση

Στο κεφάλαιο αυτό παρουσιάζονται αναλυτικά τα βήματα υλοποίησης, από τη δημιουργία των πόρων στο cloud μέχρι την εγκατάσταση και την παραμετροποίηση κάθε υπηρεσίας εντός του Kubernetes cluster. Κάθε ενότητα αντιστοιχεί σε ένα στρώμα της αρχιτεκτονικής που περιγράφηκε στο Κεφάλαιο 6 και τεκμηριώνεται με αποσπάσματα κώδικα ή δηλωτικά αρχεία ρυθμίσεων (manifests).

7.1 Υλοποίηση υποδομής στο Azure μέσω Terraform

Η υποδομή ορίζεται δηλωτικά μέσω Terraform [26] σε τρεις ενότητες (modules), οι οποίες είναι σχετικές με το `aks`, το `adls` και το `acr`. Ο κεντρικός φάκελος `infra/` περιέχει τα αρχεία `main.tf`, `variables.tf`, `terraform.tfvars` και `backend.tf`.

7.1.1 Terraform modules

Η υποδομή οργανώνεται σε τρεις ενότητες (modules) της Terraform, καθεμία υπεύθυνη για έναν διακριτό πόρο Azure:

- `module.aks`. Δέχεται ως παραμέτρους τις ρυθμίσεις τόσο των `system` όσο και των `user` node pools. Κάθε node pool ορίζεται ως αντικείμενο Terraform με τα ακόλουθα πεδία:
  (α) ονόματος,
  (β) αρχικού αριθμού nodes,
  (γ) μεγέθους εικονικής μηχανής (`vm_size`),
  (δ) ελάχιστου/μέγιστου ορίου κλιμάκωσης (δηλαδή του πλήθους των nodes) και
  (ε) ενεργοποίησης ή όχι της αυτόματης κλιμάκωσης.
- `module.adls`. Δημιουργεί τον αποθηκευτικό χώρο τύπου ADLS με ιεράρχηση ονομάτων Hierarchical Namespace (HNS) [217].
- `module.acr`. Δημιουργεί το ACR και αναθέτει τα απαραίτητα δικαιώματα με κατάλληλο RBAC τύπου `AcrPull` στο `AKS` ώστε το τελευταίο να μπορεί να αντλεί images από το ACR.

7.1.2 Κεντρικό αρχείο Terraform και αρχείο παραμέτρων

Στο αρχείο `main.tf` ορίζεται ο provider του Azure (`azurerm`), αναφέρεται η υπάρχουσα ομάδα πόρων (data source) και καλούνται τα τρία παραπάνω modules. Οι τιμές `tfvars` στο παραγωγικό περιβάλλον ορίζονται ως εξής:

```
resource_group_name  = "rg-datalake-on-k8s-we-dev-01"
location             = "westeurope"
cluster_name         = "aks-datalake-on-k8s-we-dev-01"
system_vm_size       = "Standard_E4s_v6"
user_vm_size         = "Standard_E8s_v6"
system_node_count    = 1
user_node_count      = 2
storage_account_name = "stdatalakeonk8swe01"
acr_name             = "acrdatalakeonk8swe01"
```

Η αρχικοποίηση γίνεται με `terraform init` η οποία εντολή είναι υπεύθυνη για τη φόρτωση `providers` και `backend`. Στη συνέχεια γίνεται ο σχεδιασμός με την εντολή `terraform plan` η οποία εκτελεί την επαλήθευση των αλλαγών και τέλος η εφαρμογή της νέας κατάστασης γίνεται με την εντολή `terraform apply`. Η κατάσταση (`state`) σε παραγωγικά περιβάλλοντα αποθηκεύεται απομακρυσμένα σε Azure Blob Storage, εξασφαλίζοντας συνεργατική πρόσβαση και κλείδωμα (`locking`) [14, 226], ωστόσο για την παρούσα εργασία χρησιμοποιείται τοπική κατάσταση για απλότητα.

7.2 Υλοποίηση Kubernetes cluster

Μετά τη δημιουργία του AKS cluster, η σύνδεση πραγματοποιείται, αφότου εκτελεστεί η εντολή az login, με την εντολή:

```
az aks get-credentials --resource-group rg-datalake-on-k8s-we-dev-01 \
  --name aks-datalake-on-k8s-we-dev-01
```

7.2.1 Namespaces

Κάθε αρχείο manifest (π.χ. `data-ingest.yaml`, `data-processing.yaml`) ξεκινά με τη δήλωση Namespace, ώστε η εφαρμογή ενός μόνο αρχείου να είναι αυτάρκης:

```
apiVersion: v1
kind: Namespace
metadata:
  name: data-ingest
```

Τα namespaces (`data-ingest`, `data-processing`, `analytics`, `monitoring`) δημιουργούνται αυτόματα κατά την εκτέλεση της εντολής `kubectl apply -f` [95] με την οποία εφαρμόζονται τα αρχεία manifest, π.χ.:

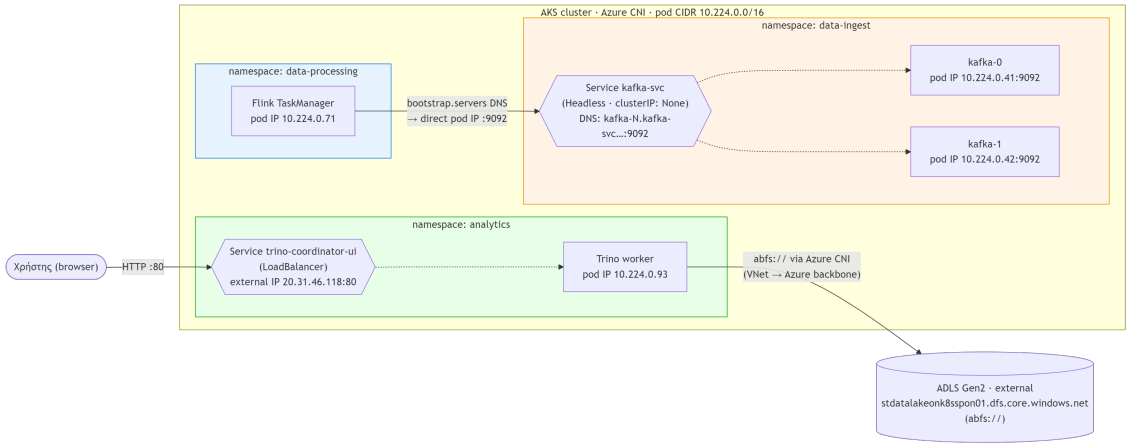
```
kubectl apply -f data-processing.yaml
```

7.2.2 Αποθήκευση κλειδιών (Secrets) και ρύθμιση μόνιμου αποθηκευτικού χώρου (PersistentVolumes)

Το κλειδί πρόσβασης στο ADLS αποθηκεύεται σε Kubernetes Secret (`azure-storage-secret`) στα namespaces που το χρειάζονται (`data-processing`, `analytics`). Τα StatefulSets δηλώνουν πρότυπα αιτήσεων μόνιμου αποθηκευτικού χώρου (`volumeClaimTemplates`), ενώ το StorageClass αντιστοιχεί σε Azure Managed Disks μέσω του CSI driver ο οποίος είναι οδηγός διεπαφής αποθηκευτικού χώρου. Τα αναλυτικά manifests που περιγράφουν αυτά παρουσιάζονται στο Παράρτημα Γ'.

7.2.3 Δίκτυο και επικοινωνία υπηρεσιών

Το Azure CNI [104] εκχωρεί σε κάθε pod διεύθυνση IP απευθείας από το υποδίκτυο του VNet, χωρίς NAT [98]. Η εσωτερική επικοινωνία μεταξύ υπηρεσιών στηρίζεται στο DNS του Kubernetes, το οποίο υλοποιείται από τον CoreDNS. Κάθε Service αντιστοιχεί σε ένα όνομα (domain name) της μορφής `<service>.<namespace>.svc.cluster.local`, το οποίο επιλύεται αυτόματα στη διεύθυνση IP της υπηρεσίας. Για παράδειγμα, ο ένας εκ των δύο Kafka broker προσεγγίζεται μέσω του ονόματος `kafka-0.kafka-svc.data-ingest.svc.cluster.local`. Τα headless Services (`clusterIP: None`) ενεργοποιούν τη δημιουργία ατομικών εγγραφών DNS ανά pod του StatefulSet, ώστε κάθε αντίγραφο να είναι προσβάσιμο με σταθερό όνομα τομέα [99]. Η Εικόνα 7.1 συνοψίζει τη συνολική εικόνα διασύνδεσης των υπηρεσιών στο cluster της παρούσας υλοποίησης.



Εικόνα 7.1: *Διασύνδεση υπηρεσιών στο AKS cluster της εργασίας. Τα pods λαμβάνουν IP απευθείας από το VNet (Azure CNI, ενδεικτικό CIDR 10.224.0.0/16). Στη ροή Flink → Kafka, το Service kafka-svc είναι Headless (`clusterIP: None`) και η ανάλυση DNS επιστρέφει απευθείας τις IP των kafka-N pods. Η σύνδεση Trino → Paimon δεν περνά από Service, καθώς το Trino χρησιμοποιεί τον native Azure filesystem connector και προσπελαύνει απευθείας το ADLS Gen2 μέσω abfs://, αξιοποιώντας τη δυνατότητα του Azure CNI για άμεση δρομολόγηση προς υπηρεσίες Azure. Η εξωτερική πρόσβαση στο Trino UI γίνεται μέσω LoadBalancer Service με δημόσια IP του Azure Load Balancer.*

7.3 Εγκατάσταση Kafka και αρχικοποίηση topics

Το Kafka ορίζεται ως StatefulSet με δύο αντίγραφα (replicas) στο αρχείο `data-ingest.yaml`. Κάθε pod χρησιμοποιεί το image `doughgle/kafka-kraft:latest` [21], το οποίο ενσωματώνει τον KRaft controller.

Ρυθμίσεις περιβάλλοντος. Οι μεταβλητές περιβάλλοντος ρυθμίζουν τη λειτουργία του Kafka:

- `REPLICAS=2`. Ο αριθμός brokers.
- `SERVICE=kafka-svc`. Το headless Service για εσωτερική, δηλαδή εντός του cluster, ονοματοδοσία.
- `CLUSTER_ID`. Στατικό αναγνωριστικό, ώστε τα pods να ενώνονται στο ίδιο cluster μετά από επανεκκίνηση.
- `DEFAULT_REPLICATION_FACTOR=2` και `DEFAULT_MIN_INSYNC_REPLICAS=1`. Εξισορρόπηση ανθεκτικότητας και διαθεσιμότητας εγγραφών [140].

Μόνιμη αποθήκευση. Κάθε broker δηλώνει σταθερό αποθηκευτικό χώρο μέσω `volumeClaimTemplate` ίσο με 128Gi όπου το StorageClass είναι τύπου Managed Disk. Τα δεδομένα αποθηκεύονται στην τοποθεσία `/var/run/kafka-data` εντός του container, αντιστοιχισμένη σε Azure Premium SSD δίσκο. Η δοκιμή ετοιμότητας (`readinessProbe`) του pod ελέγχει τη θύρα 9092 με αρχική καθυστέρηση 10 δευτερολέπτων, εξασφαλίζοντας ότι η κυκλοφορία DNS κατευθύνεται μόνο σε brokers που έχουν ολοκληρώσει την επανεκκίνησή τους.

Αρχικοποίηση topics. Τα δύο topics, το `iot-sensor-general`, το οποίο διαθέτει 4 partitions και συντελεστή αντιγραφής RF ίσο με 2, καθώς και το `iot-sensor-plug` με 6 partitions και συντελεστή αντιγραφής RF επίσης ίσο με 2, δημιουργούνται μέσω εργασίας Kubernetes Job. Αυτό το job εκτελεί το script αρχικοποίησης `kafka-topics.sh -create` εντός του Kafka pod. Τα 6 partitions του `iot-sensor-plug` αντανακλούν το γεγονός ότι κάθε εγκατάσταση παράγει 4 μηνύματα συσκευών έναντι 1 κεντρικού. Η εκτέλεση με εργασία τύπου Job εξασφαλίζει ότι η δημιουργία topics γίνεται μία φορά και δεν επαναλαμβάνεται σε κάθε επανεκκίνηση.

7.4 Εγκατάσταση Flink, jobs εισαγωγής και συγκεντρώσεων

7.4.1 Εξειδικευμένο Docker image

Το image `flink-paimon-kafka` βασίζεται στο `apache/flink:1.20.2-scala_2.12` και ενσωματώνει τα αναγκαία JARs εκτέλεσης τα οποία φαίνονται στον Πίνακα 7.1. Η αναλυτική περιγραφή της κατασκευής του, μαζί με τα υπόλοιπα εξειδικευμένα images του συστήματος, παρουσιάζεται στην Ενότητα 7.10.

Πίνακας 7.1: *JARs εκτέλεσης που ενσωματώνονται στο image Flink.*

JAR	**Περιγραφή**
flink-sql-connector-kafka	σύνδεσμος connector Kafka για την ανάγνωση και εγγραφή topics
paimon-flink	σύνδεσμος Paimon για τη δημιουργία καταλόγων, πινάκων, καθώς και εγγραφή και ανάγνωση αυτών
paimon-azure	υποστήριξη ABFS για Paimon.
flink-azure-fs-hadoop	plugin σύνδεσης Azure στο σύστημα αρχείων του Flink.
flink-shaded-hadoop-2-uber	πακέτο βιβλιοθηκών Hadoop για την υποστήριξη του πρωτοκόλλου ABFS.

7.4.2 Αρχείο ρυθμίσεων Flink

Το αρχείο `flink-conf-configmap.yaml` ορίζει τη δομή `config.yaml` που τοποθετείται στο `/opt/flink/conf/` κάθε Flink pod. Ο Πίνακας 7.2 συνοψίζει τις βασικές ρυθμίσεις.

Πίνακας 7.2: *Βασικές ρυθμίσεις ConfigMap flink-config.*

Παράμετρος	**Τιμή**	**Περιγραφή**
jobmanager.memory.process.size	1600m	Μνήμη JobManager.
taskmanager.memory.process.size	8192m	Μνήμη TaskManager (συμπ. 512m off-heap εργασίας και 256m off-heap framework).
taskmanager.numberOfTaskSlots	4	Υποδοχείς εργασίας ανά TM.
parallelism.default	4	Εισαγωγή ροής, $2 \times 4 = 8$ διαθέσιμοι υποδοχείς (4 για εισαγωγή ροής, 4 ελεύθεροι για περιοδική συγκέντρωση).
execution.checkpointing.mode	EXACTLY_ONCE	Σημασιολογία παράδοσης.
execution.checkpointing.interval	60s	Περίοδος σημείων ελέγχου, χρονικό όριο 600s, ελάχιστη παύση 30s [149].
execution.checkpointing.dir	`abfs://checkpoints@stdatalakeonk8swe01.dfs.core.windows.net/flink-checkpoints`	Αποθήκευση στο ADLS, επιτρέπει ανάκαμψη σε περίπτωση αποτυχίας node ή pod.
metrics.reporter.prom.port	9249–9259	Prometheus metrics reporter [198] για συλλογή Flink metrics.

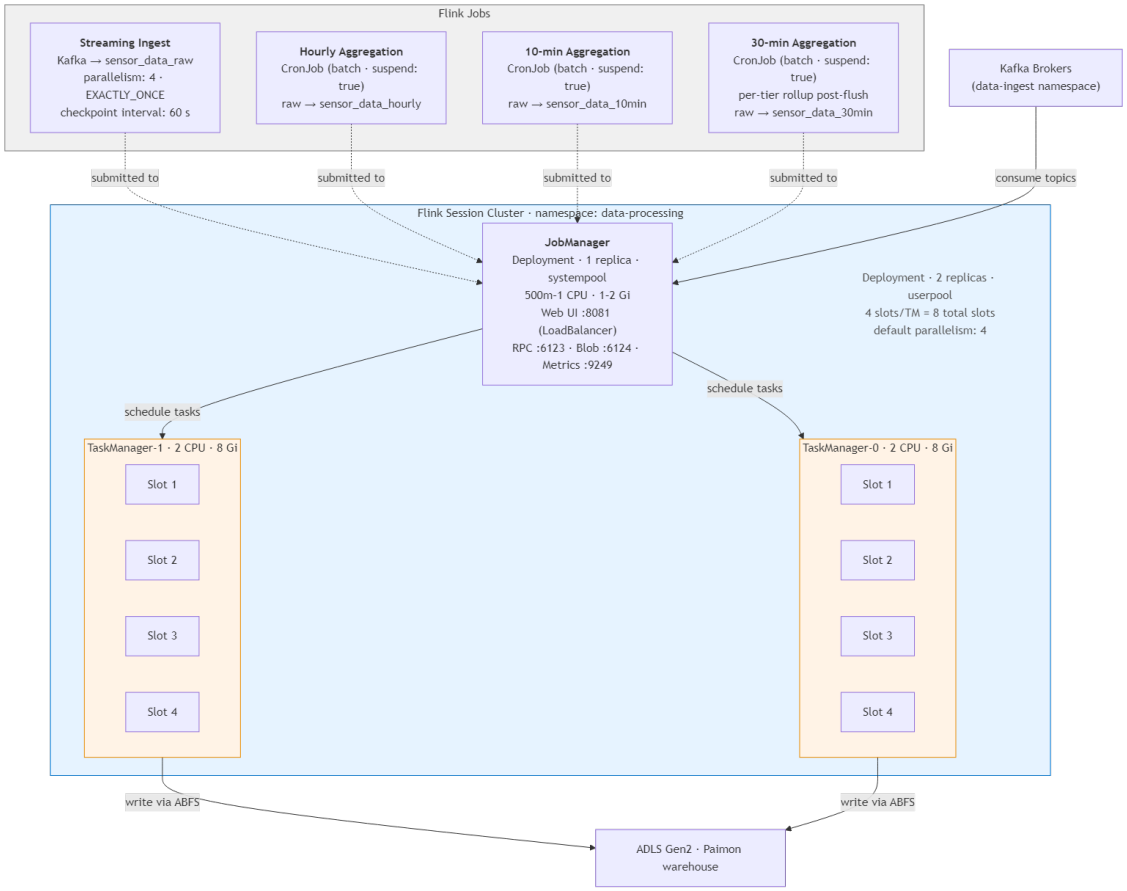
7.4.3 Διαμόρφωση των JobManager και TaskManagers

Ο JobManager αναπτύσσεται ως Deployment με ένα μόνο replica και εκτίθεται μέσω δύο Services. Το πρώτο service είναι τύπου ClusterIP για κλήσεις Remote Procedure Call (RPC), στη θύρα 6123, καθώς και τη μεταφορά μεγάλων δυαδικών αντικειμένων τύπου blob στη θύρα 6124. Το δεύτερο service είναι τύπου LoadBalancer και προορίζεται για τη δικτυακή διεπαφή χρήστη (Web User Interface, UI) στη θύρα 8081. Όσον αφορά τους πόρους του συστήματος, το κάτω όριο αιτημάτων (requests) από το cluster ορίζεται σε 500 millicores (m) CPU [227] και 1Gi μνήμη, ενώ το άνω όριο (limits) σε 1 CPU και 2Gi μνήμης.

Οι TaskManagers ρυθμίστηκαν ως Deployment με 2 replicas. Η αίτηση πόρων ορίζε-

ται σε 2 CPU και 8Gi μνήμης, χωρίς ρητό ανώτατο όριο, δηλαδή αφήνεται ελεύθερη η burstable (στιγμιαία υπέρβαση) χρήση πόρων του node. Κάθε pod ορίζει `nodeSelector` με τιμή το `agentpool = userpool` ώστε να φιλοξενείται αποκλειστικά στο userpool [7] το οποίο και διαθέτει μεγάλη συνολική μνήμη (`128Gi`).

Η Εικόνα 7.2 απεικονίζει τη συνολική τοπολογία του Flink cluster. Συγκεκριμένα παρουσιάζει τον JobManager που δρομολογεί tasks, τους δύο TaskManagers με τέσσερις υποδοχείς (slots) έκαστος, δηλαδή σύνολο 8 υποδοχείς, καθώς και τα τρία tasks που υποβάλλονται στο Flink cluster.

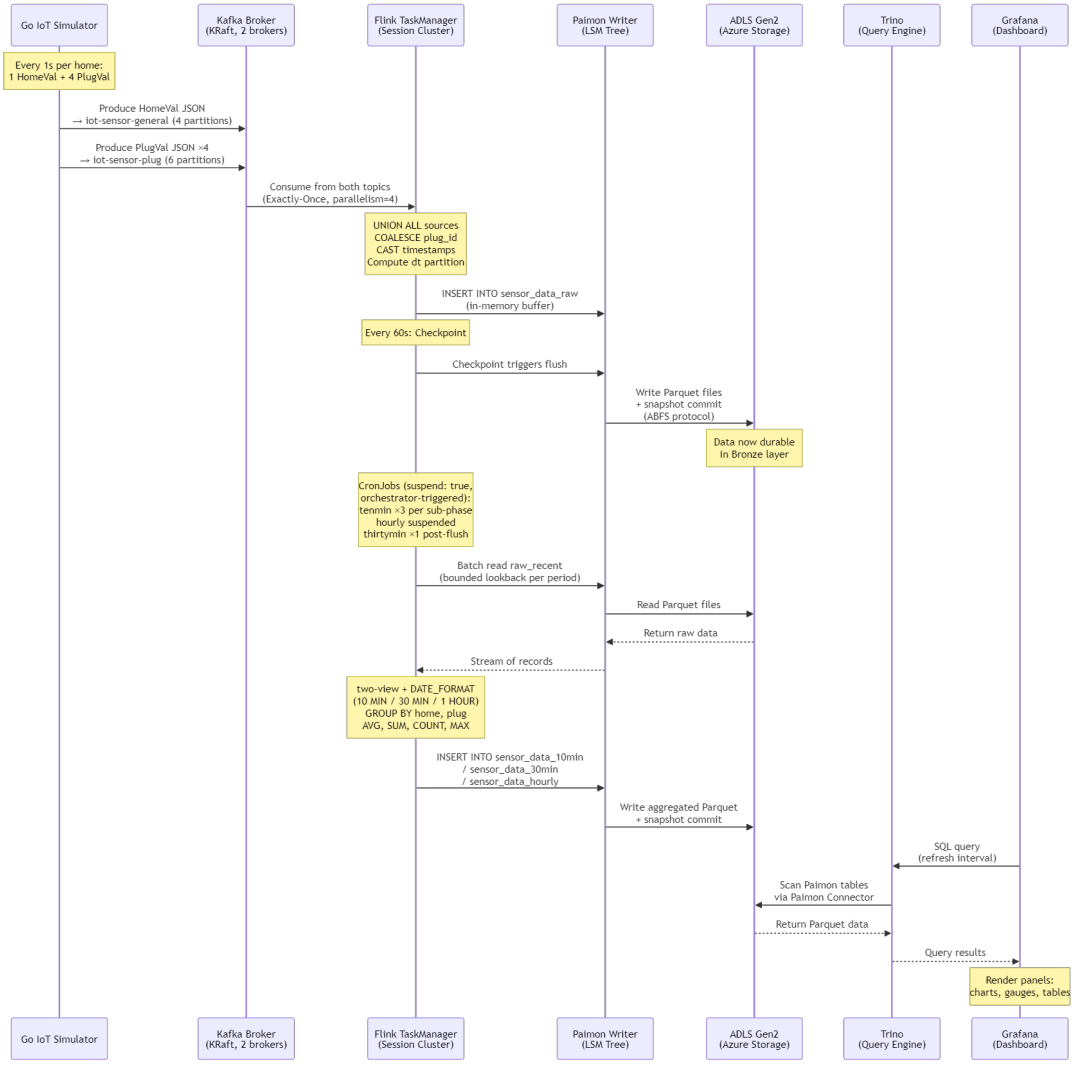


Εικόνα 7.2: *Τοπολογία Flink cluster. Οι JobManager και TaskManagers με tasks slots και υποβαλλόμενα tasks εισαγωγής ή επεξεργασίας δεδομένων.*

7.4.4 Job εισαγωγής streaming δεδομένων

Η Εικόνα 7.3 απεικονίζει τη λεπτομερή ακολουθία μηνυμάτων μεταξύ των συστατικών στοιχείων (εφαρμογών) του pipeline, από την παραγωγή μηνυμάτων τύπου JSON στον προσομοιωτή IoT, μέσω της κατανάλωσης και επεξεργασίας από τον Flink, έως την εγγραφή αρχείων τύπου Parquet στο ADLS μέσω Paimon και την εξυπηρέτηση αναλυτικών ερωτημάτων από το Trino προς το Grafana.

Ένα Kubernetes Job (`flink-submit-kafka-to-paimon`) εκτελεί τον Flink SQL client. Το αρχείο αρχικοποίησης (`init.sql`) δημιουργεί τον κατάλογο Paimon με τοποθεσία warehouse στο ADLS και επιλέγει τη βάση δεδομένων `iot`.



Εικόνα 7.3: *Διάγραμμα ακολουθίας μηνυμάτων. Αλληλεπίδραση συστατικών από την παραγωγή τηλεμετρίας έως την οπτικοποίησή τους, συμπεριλαμβανομένων των σημείων ελέγχου και της ωριαίας συγκέντρωσης.*

Το αρχείο εισαγωγής (`ingest.sql`) ορίζει δύο TEMPORARY TABLE, οι οποίοι είναι οι `sensor_data_kafka_home` και `sensor_data_kafka_plug` που αντιστοιχούν στα Kafka topics, τα ενοποιεί σε πίνακα τύπου VIEW μέσω UNION ALL και εκτελεί `INSERT INTO sensor_data_raw` με εξαγωγή `dt` από τη χρονοσφραγίδα και `COALESCE(plug_id, 'general')` (βλ. Ενότητα Γ΄.6).

Οι ρυθμίσεις χρόνου εκτέλεσης του Flink job καθορίζονται μέσω των παραμέτρων `-D`. Ο βαθμός παραλληλισμού ορίζεται σε τέσσερα (`parallelism=4`) και η σημασιολογία επεξεργασίας σε ακριβώς μία φορά (`EXACTLY_ONCE`), ενώ η δημιουργία σημείων ελέγχου (checkpointing) πραγματοποιείται ανά εξήντα δευτερόλεπτα. Το κλειδί αποθήκευσης του Azure μεταβιβάζεται με ασφάλεια μέσω μεταβλητής περιβάλλοντος (environment variable). Τέλος, η παράμετρος `backoffLimit=3` ορίζει ανώτατο όριο τριών επαναπροσπαθειών, εξασφαλίζοντας την αυτόματη επανεκκίνηση της εργασίας σε περίπτωση αστοχίας [13].

7.4.5 Ωριαία συγκέντρωση δεδομένων

Το Job ωριαίας συγκέντρωσης υλοποιείται ως CronJob με πρόγραμμα "`10 * * * *`" που αντιστοιχεί σε εκτέλεση στο 10ο λεπτό κάθε ώρας. Η πολιτική `concurrencyPolicy = Forbid` αποτρέπει παράλληλες εκτελέσεις.

Η διαδικασία επεξεργασίας μέσω SQL περιλαμβάνει τα εξής στάδια:

1. Ορίζεται προσωρινή προβολή πίνακα (TEMPORARY VIEW) `raw_recent` που φιλτράρει τα δεδομένα από τον κεντρικό πίνακα ακατέργαστων δεδομένων `sensor_data_raw` στις τελευταίες δύο ημέρες (`dt >= CURRENT_DATE - 1 day`). Στα πειράματα (Κεφάλαιο 8), η σάρωση περιορίζεται περαιτέρω στις τελευταίες τρεις (3) ώρες μέσω πρόσθετου φίλτρου χρονοσφραγίδας (`ts`), ώστε να παραμένει σταθερό το κόστος ανάγνωσης (βλ. Παράρτημα Γ'.7).

2. Εφαρμόζεται η συνάρτηση παραθύρου TUMBLE (Table-Valued Function) μίας ώρας.

3. Υπολογίζονται μέση τάση `AVG(voltage)`, μέση ένταση ρεύματος `AVG(current_amp)`, μέσος συντελεστής ισχύος `AVG(power_factor)`, πλήθος δειγμάτων `COUNT(*)`, συνολική κατανάλωση ενέργειας:
$$\frac{\sum energy_wh}{1000} \quad \text{(kWh)}$$
και μέγιστη αιχμή ισχύος:
$$\max(V \times I \times \cos\varphi) \quad \text{(W)}$$

4. Τα αποτελέσματα εγγράφονται στον πίνακα `sensor_data_hourly` [151] (βλ. Ενότητα Γ'.7).

Η λειτουργία σε batch mode (-Dexecution.runtime-mode=batch) εξασφαλίζει ότι το job τερματίζεται μετά την επεξεργασία όλων των δεδομένων, απελευθερώνοντας task slots για τη ροή εισαγωγής δεδομένων.

7.4.6 Δεκάλεπτη συγκέντρωση δεδομένων

Εκτός από την ωριαία συγκέντρωση, ορίζεται και μια δεκαλεπτή (10-minute) περιοδική εργασία (CronJob) με πρόγραμμα "`*/10 * * * *`" (εκτέλεση κάθε 10 λεπτά). Ο σκοπός της είναι η παραγωγή συγκεντρωτικών metrics υψηλότερης χρονικής ανάλυσης, κατάλληλων για dashboards σχεδόν πραγματικού χρόνου.

Η SQL λογική είναι ανάλογη της ωριαίας:

1. Ορίζεται προσωρινή προβολή πίνακα (TEMPORARY VIEW) `raw_recent_10m` που φιλτράρει τα δεδομένα από τον κεντρικό πίνακα ακατέργαστων δεδομένων `sensor_data_raw` στην τελευταία ημέρα.

2. Εφαρμόζεται η συνάρτηση παραθύρου TUMBLE με διάστημα 10 λεπτών:

```
INSERT INTO sensor_data_10min
SELECT
  DATE_FORMAT(window_start, 'yyyy-MM-dd') AS dt,
  home_id, plug_id, window_start, window_end,
```

```
  AVG(voltage)                          AS avg_voltage,
  AVG(current_amp)                      AS avg_current,
  AVG(power_factor)                     AS avg_pf,
  SUM(energy_wh) / 1000.0              AS energy_kwh,
  COUNT(*)                              AS samples,
  MAX(voltage * current_amp * power_factor) AS peak_power_w
FROM TABLE(
  TUMBLE(TABLE raw_recent_10m, DESCRIPTOR(ts),
         INTERVAL '10' MINUTE)
)
GROUP BY home_id, plug_id, window_start, window_end;
```

Η ρύθμιση `concurrencyPolicy = Forbid` αποτρέπει αλληλοεπικάλυψη tasks σε περίπτωση που μια εκτέλεση διαρκέσει περισσότερο από 10 λεπτά, ενώ η ρύθμιση `activeDeadlineSeconds = 540` (9 λεπτά) εξασφαλίζει τερματισμό πριν την επόμενη δρομολόγηση του ίδιου task. Όπως και η ωριαία εργασία, εκτελείται σε batch mode [151].

7.5 Ρύθμιση Paimon και αποθήκευση σε ADLS

Η αρχικοποίηση πινάκων γίνεται μέσω Kubernetes Job, το `paimon-bootstrap`, που εκτελεί στον Flink SQL client δύο αρχεία τύπου SQL τα οποία είναι τοποθετημένα σε ConfigMaps. Το πλήρες περιεχόμενο και των δύο αρχείων παρατίθεται στο Παράρτημα Γ΄.4 και Γ΄.5 αντίστοιχα.

7.5.1 Αρχείο αρχικοποίησης Paimon αντικειμένων

Δημιουργεί τον κατάλογο Paimon μέσω της εντολής:

```
CREATE CATALOG IF NOT EXISTS paimon
```

με προορισμό το container με όνομα warehouse στο ADLS, δηλαδή:

```
abfs://warehouse@stdatalakeonk8swe01.dfs.core.windows.net/paimon
```

Στη συνέχεια δημιουργεί τη βάση δεδομένων `iot` και την επιλέγει μέσω των εντολών:

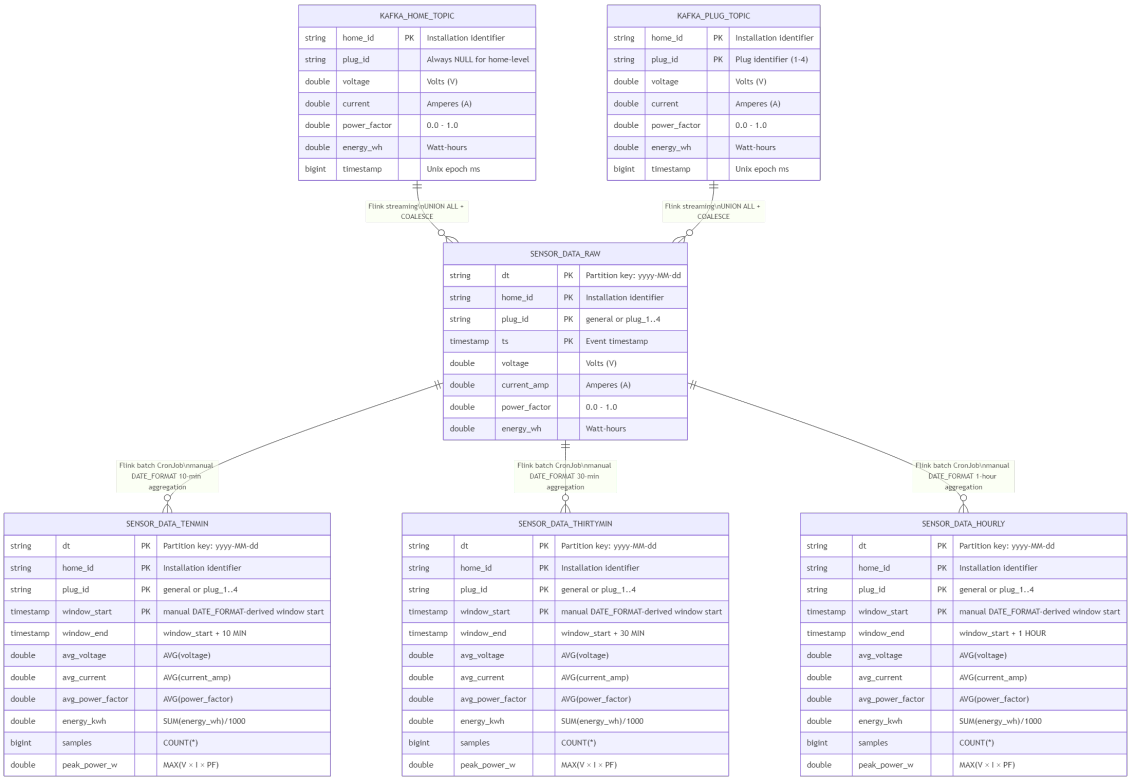
```
CREATE DATABASE IF NOT EXISTS iot;
USE iot;
```

Για το πλήρες αρχείο βλ. Παράρτημα Γ΄.4.

Η Εικόνα 7.4 παρουσιάζει το μοντέλο δεδομένων του συστήματος σε μορφή διαγράμματος οντοτήτων-συσχετίσεων. Απεικονίζονται τα δύο Kafka topics (iot-sensor-general, iot-sensor-plug), ο πίνακας ακατέργαστων δεδομένων sensor_data_raw, ο πίνακας ωριαίων συγκεντρώσεων sensor_data_hourly και ο πίνακας δεκάλεπτων συγκεντρώσεων sensor_data_10min, μαζί με τους μετασχηματισμούς που τα συνδέουν.

7.5.2 Αρχείο εκκίνησης πινάκων Paimon

Δημιουργεί τους τέσσερις πίνακες (βλ. Παράρτημα Γ΄.5):



Εικόνα 7.4: *Μοντέλο δεδομένων. Από τα Kafka topics στους πίνακες Paimon μέσω ενοποίησης των δεδομένων και ωριαίας και δεκάλεπτης συγκέντρωσής τους.*

- Πίνακας `sensor_data_raw`:
  - Partition κατά `dt` για βελτιστοποίηση ερωτημάτων που φιλτράρουν κατά ημερομηνία.
  - 6 buckets (κάδοι κατακερματισμού) για ομοιόμορφη κατανομή δεδομένων και παράλληλη επεξεργασία.
  - λήξη διατήρησης δεδομένων ίση με 7 ημέρες, ώστε να περιορίζεται ο αποθηκευτικός χώρος και να διατηρείται η απόδοση.
  - 7 πεδία δεδομένων
  - πρωτεύον κλειδί το οποίο είναι η τούπλα `(dt, home_id, plug_id, ts)`
- Πίνακας `sensor_data_hourly`:
  - 11 πεδία επεξεργασμένων δεδομένων, συμπεριλαμβανομένων των `window_start` και `window_end` που ορίζουν το χρονικό παράθυρο της ωριαίας συγκέντρωσης.
  - 1 bucket (κάδος κατακερματισμού), καθώς τα δεδομένα είναι ήδη συγκεντρωμένα και λιγότερο ευαίσθητα σε ανισορροπία κατανομής.
  - λήξη διατήρησης δεδομένων ίση με 30 ημέρες, καθώς οι συγκεντρωτικές εγγραφές είναι λιγότερο συχνές και καταλαμβάνουν λιγότερο χώρο.
  - πρωτεύον κλειδί η τούπλα `(dt, home_id, plug_id, window_start)`

- Πίνακας `sensor_data_30min`. Ίδια δομή με τον ωριαίο πίνακα, αλλά με παράθυρο 30 λεπτών. 1 bucket, διατήρηση στιγμιοτύπων 30 ημερών.
- Πίνακας `sensor_data_10min`. Ίδια δομή με τον ωριαίο πίνακα, αλλά με παράθυρο 10 λεπτών. 1 bucket, διατήρηση στιγμιοτύπων 30 ημερών. Εξυπηρετεί dashboards σχεδόν πραγματικού χρόνου [23].

Η χρήση `DROP TABLE IF EXISTS` πριν από τη δημιουργία των πινάκων εξασφαλίζει καθαρή κατάσταση κατά την αρχικοποίηση κάθε φάσης του πειράματος, αν και διαγράφει υπάρχοντα δεδομένα για αυτό είναι κατάλληλο μόνο για περιβάλλον έρευνας και ανάπτυξης και όχι παραγωγικό.

7.6 Διαμόρφωση Trino και σύνδεση με Paimon

Το Trino [24] εγκαθίσταται μέσω του επίσημου Helm chart [203] στο namespace analytics. Το εξειδικευμένο image `trino-paimon`, του οποίου η κατασκευή παρουσιάζεται στην Ενότητα 7.10, περιέχει το σύνδεσμο με το Paimon εγκατεστημένο στον φάκελο `/usr/lib/trino/plugin/paimon/`.

7.6.1 Αρχείο ρυθμίσεων Trino

Ως προς τις βασικές ρυθμίσεις, το μέγεθος του heap του JVM ορίζεται στα 2 GB και εφαρμογή συλλέκτη απορριμμάτων (garbage collector) G1. Η εγκατάσταση περιλαμβάνει δύο workers, ενώ ο coordinator ρυθμίζεται ώστε να λειτουργεί και ως worker, αποδίδοντας συνολικά τρεις κόμβους εκτέλεσης ερωτημάτων. Η εξωτερική πρόσβαση στη θύρα 8080 παρέχεται μέσω υπηρεσίας τύπου LoadBalancer. Τέλος, οι προεπιλεγμένοι κατάλογοι του Trino απενεργοποιούνται και ορίζεται ρητά ο κατάλογος `paimon`:

```
connector.name=paimon
warehouse=abfs://warehouse@stdatalakeonk8swe01.dfs.core.windows.net/paimon
fs.native-azure.enabled=true
azure.auth-type=ACCESS_KEY
azure.access-key=${ENV:AZURE_STORAGE_ACCOUNT_KEY}
```

Η μεταβλητή `AZURE_STORAGE_ACCOUNT_KEY` εισάγεται από το Kubernetes Secret. Η χρήση του έμφυτου Azure filesystem (`fs.native-azure.enabled=true`) αντί Hadoop βελτιώνει τις επιδόσεις ανάγνωσης, καθώς αποφεύγεται η εκκίνηση Hadoop FileSystem [6].

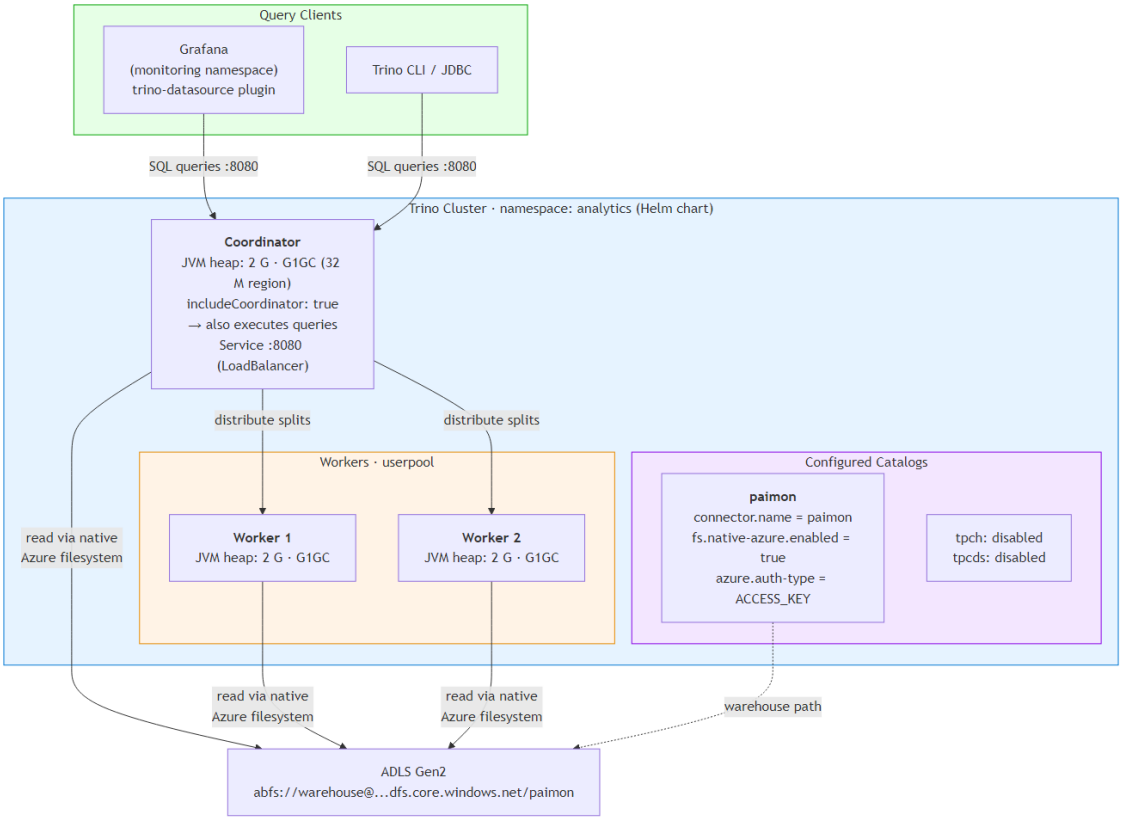
7.6.2 Εγκατάσταση Trino

Η εγκατάσταση πραγματοποιείται μέσω Helm με την εντολή:

```
helm upgrade --install trino-standalone trino/trino \
  -n analytics -f k8s/helm/trino-values.yaml
```

Η Εικόνα 7.5 απεικονίζει τη συνολική τοπολογία του Trino cluster δηλαδή τον coordinator που δέχεται ερωτήματα SQL και κατανέμει τμήματα (splits) στους τρεις,

συμπεριλαμβανομένου και του coordinator, workers, τον ρυθμισμένο κατάλογο Paimon και τις πηγές ερωτημάτων οι οποίες είναι το Grafana και το Trino CLI.



Εικόνα 7.5: *Τοπολογία Trino cluster. Ο coordinator, ο οποίος λειτουργεί και ως worker, δύο αποκλειστικοί workers και σύνδεση με το Paimon catalog στο ADLS.*

7.7 Διαμόρφωση Grafana

Το Grafana [20] εγκαθίσταται ως μέρος του `kube-prometheus-stack` μέσω Helm chart στο `namespace monitoring`. Οι βασικές ρυθμίσεις, όπως η εισαγωγή προκαθορισμένων dashboards, ορίζονται στο αρχείο `kube-prometheus-stack-values.yaml`.

7.7.1 Εγκατάσταση datasource και εξωτερική πρόσβαση

Το plugin πηγής δεδομένων trino-datasource εγκαθίσταται αυτόματα μέσω της ρύθμισης:

```
grafana:
  plugins:
    - trino-datasource
  service:
    type: LoadBalancer
    port: 80
```

Ο τύπος υπηρεσίας LoadBalancer διαφημίζει το Grafana εξωτερικά στη θύρα 80.

7.7.2 Αυτόματη εισαγωγή πηγών δεδομένων μέσω βοηθητικού (sidecar) container

Ένα ConfigMap στο `namespace monitoring` φέρει ετικέτα (label) `grafana_datasource: "1"` και ορίζει τη σύνδεση Trino:

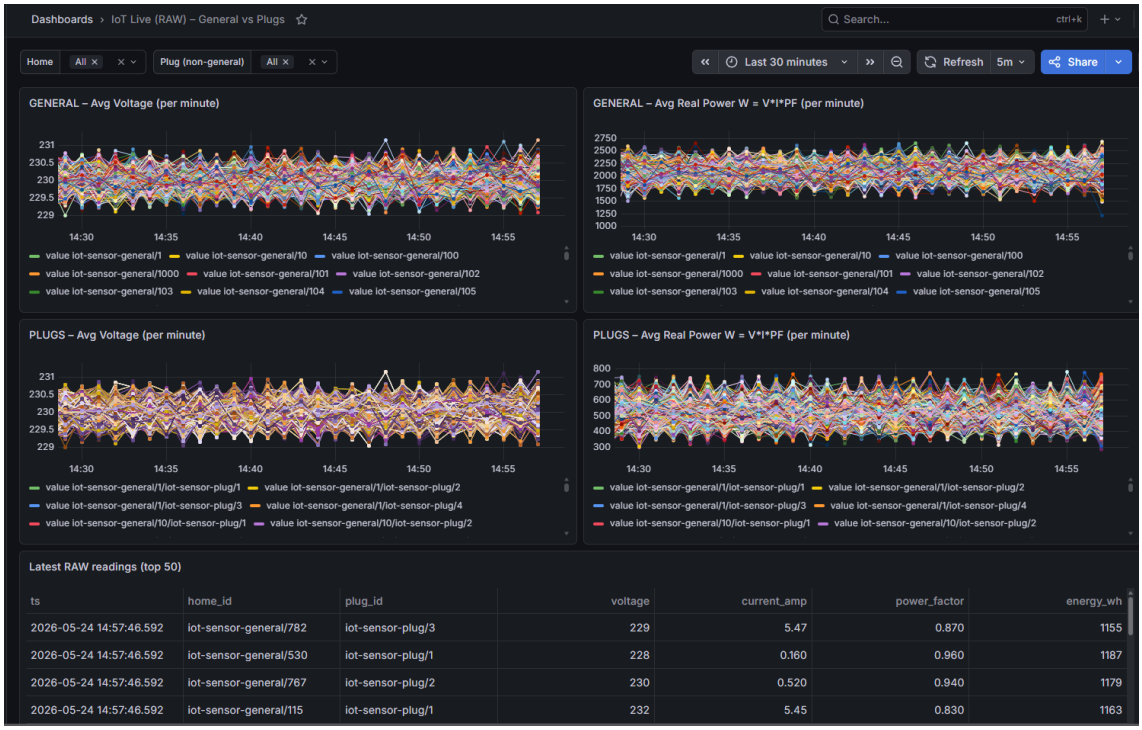
```
datasources:
  - name: Trino
    type: trino-datasource
    url: http://trino-standalone-trino.analytics.svc.cluster.local:8080
    jsonData:
      catalog: paimon
      schema: iot
```

Το βοηθητικό sidecar container του Grafana σαρώνει περιοδικά όλα τα ConfigMaps με αυτή την ετικέτα (`searchNamespace: ALL`) και τα φορτώνει ως πηγές δεδομένων.

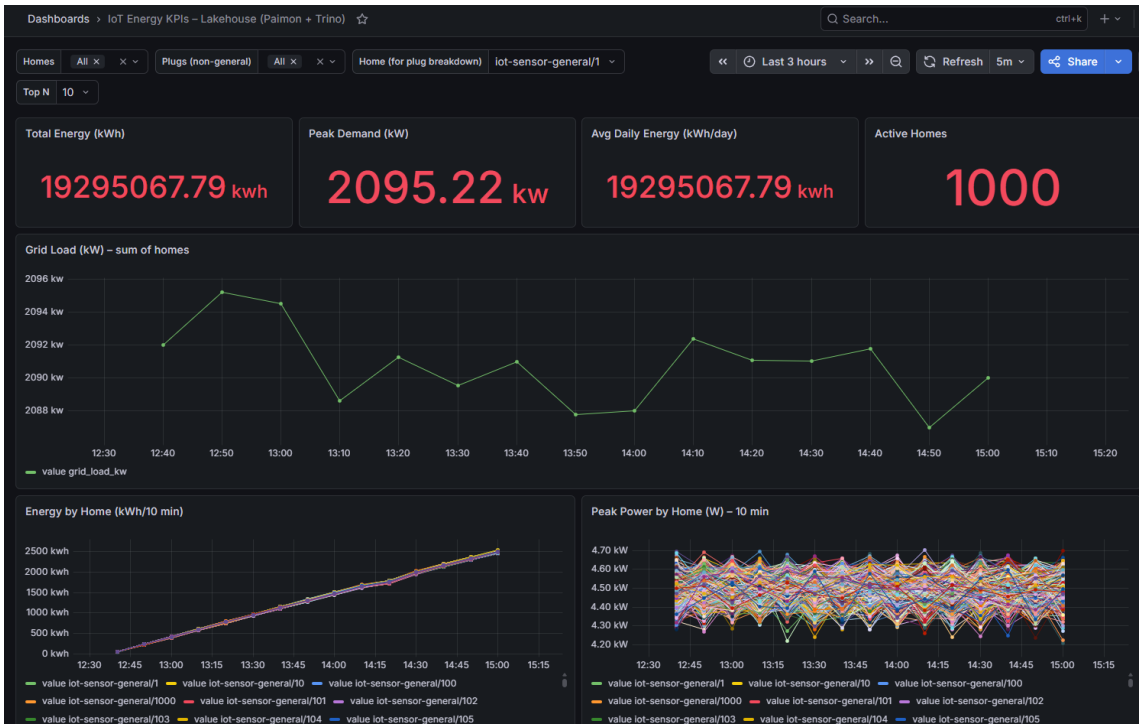
7.7.3 Αυτόματη εισαγωγή πινάκων ελέγχου

Με τον ίδιο μηχανισμό, κάθε ConfigMap με ετικέτα `grafana_dashboard: "1"` φορτώνεται αυτόματα. Τα αρχεία JSON των πινάκων ελέγχου (π.χ. `iot-energy-kpis.json`, `iot-live-raw.json`) μετατρέπονται σε ConfigMaps και εφαρμόζονται μαζί με τα υπόλοιπα manifests.

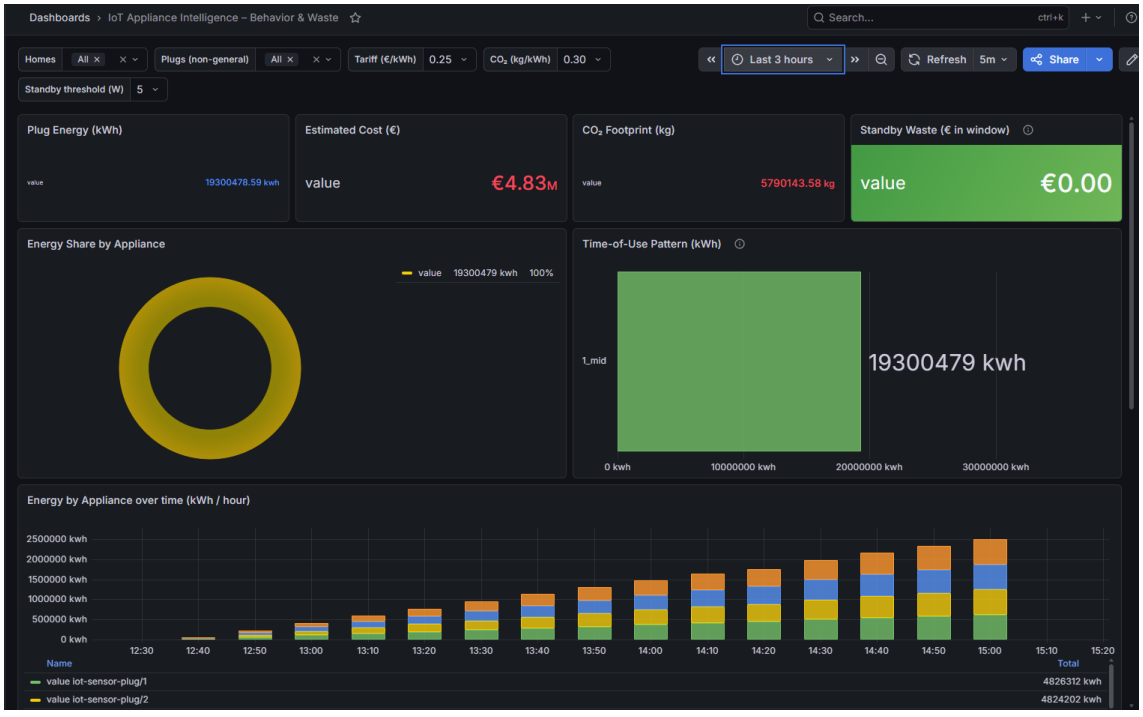
Οι Εικόνες 7.6, 7.7 και 7.8 παρουσιάζουν ενδεικτικά στιγμιότυπα τέτοιων πινάκων ελέγχου, όπως φορτώνονται αυτόματα στο Grafana.



Εικόνα 7.6: *Παράδειγμα IoT dashboard στο Grafana το οποίο παρουσιάζει τη ροή ακατέργαστων μετρήσεων.*



Εικόνα 7.7: *Παράδειγμα IoT dashboard στο Grafana το οποίο παρουσιάζει τους δείκτες κατανάλωσης ενέργειας.*



Εικόνα 7.8: *Παράδειγμα IoT dashboard στο Grafana το οποίο παρουσιάζει την ανάλυση συμπεριφοράς συσκευών.*

7.8 Ρυθμίσεις Prometheus

Το Prometheus αποθηκεύει δεδομένα για 15 ημέρες, μέσω της ρύθμισης `retention: 15d`, σε μόνιμο χώρο αποθήκευσης PVC 50Gi. Ο χώρος αυτός κρίνεται επαρκής για τις ανάγκες

του παρόντος πειράματος, το οποίο διαρκεί μερικές ώρες και παράγει metrics από περιορισμένο αριθμό nodes και υπηρεσιών. Σε περιβάλλον παραγωγής, το μέγεθος αποθήκευσης του Prometheus απαιτεί ειδικό υπολογισμό βάσει του αριθμού των χρονοσειρών, του ρυθμού συλλογής και του επιθυμητού παραθύρου διατήρησης. Οι στόχοι συλλογής metrics (scrape targets) (`kubelet`, `cAdvisor`) ενεργοποιούνται, καθώς παρέχουν metrics χρήσης CPU, μνήμης, δικτύου και αποθήκευσης σε επίπεδο node και container, οι οποίες είναι άμεσα σχετικές με τους στόχους του πειράματος. Τα συστατικά control plane που διαχειρίζεται το AKS (`kubeControllerManager`, `kubeScheduler`, `kubeEtcd`) απενεργοποιούνται καθώς τα endpoints τους δεν εκτίθενται από το AKS ως managed service του Azure και, επιπλέον, η συμπεριφορά του control plane δεν αποτελεί αντικείμενο μέτρησης στο παρόν πείραμα, γεγονός που καθιστά τη συλλογή τους μη χρήσιμη.

7.9 Προσομοιωτής παραγωγής IoT δεδομένων

Ο προσομοιωτής τηλεμετρικών δεδομένων υλοποιείται στη γλώσσα προγραμματισμού Go [228] και παράγει τεχνητά γεγονότα κατανάλωσης ενέργειας. Κάθε pod προσομοιώνει ένα εύρος εγκαταστάσεων (`home_id`), π.χ. κατοικίες, εργοστάσια ή βιομηχανικές μονάδες, με παραμετροποιήσιμο αριθμό συσκευών ανά εγκατάσταση (`SIM_PLUGS_PER_HOME=4`), όπου κάθε 'συσκευή' μπορεί να αντιπροσωπεύει οικιακή πρίζα, μηχάνημα ή γραμμή παραγωγής.

Παραμετροποίηση Η συνολική συμπεριφορά και οι ρυθμίσεις του προσομοιωτή καθορίζονται και ελέγχονται μέσω συγκεκριμένων μεταβλητών περιβάλλοντος, οι κυριότερες από τις οποίες είναι οι ακόλουθες:

- `KFK_BOOTSTRAP_SERVERS`. Καθορίζει τη δικτυακή διεύθυνση των Kafka brokers για τη σύνδεση του προσομοιωτή.
- `KFK_TOPIC_HOME` και `KFK_TOPIC_PLUG`. Ορίζουν τα αντίστοιχα ονόματα των Kafka topics στα οποία διοχετεύονται τα δεδομένα.
- `SIM_HOME_ID_START` και `SIM_HOME_ID_END`. Προσδιορίζουν το εύρος και το συνολικό πλήθος των προσομοιωμένων εγκαταστάσεων.
- `SIM_SLEEP_SEC`. Ρυθμίζει την περίοδο αναμονής ανάμεσα στους διαδοχικούς κύκλους αποστολής δεδομένων, με προεπιλεγμένη τιμή το ένα δευτερόλεπτο (1.0s).

Δομή δεδομένων Κατά τη διάρκεια κάθε κύκλου αποστολής, ο προσομοιωτής παράγει δύο διακριτούς τύπους μηνυμάτων σε μορφή JSON, οι οποίοι εξυπηρετούν διαφορετικά επίπεδα πληροφορίας:

- `HomeVal`. Αντιπροσωπεύει την κεντρική μέτρηση ανά εγκατάσταση και περιλαμβάνει παραμέτρους όπως η τάση, η ένταση του ρεύματος, ο συντελεστής ισχύος και η αθροιστική ενέργεια σε Wh.
- `PlugVal`. Αφορά τη μέτρηση σε επίπεδο μεμονωμένης συσκευής, όπως μια πρίζα ή ένα μηχάνημα, και χαρακτηρίζεται από το πρόσθετο αναγνωριστικό πεδίο `plug_id`.

Ενδεικτικά, ένα μήνυμα τύπου `HomeVal` που δημοσιεύεται στο Kafka topic κεντρικών μετρητών `iot-sensor-general` έχει τη μορφή:

```
{
  "home_id":      "iot-sensor-general/7",
  "plug_id":      null,
  "voltage":      231.4,
  "current":      12.57,
  "power_factor": 0.91,
  "power_w":      2645.83,
  "energy_wh":    18.654,
  "timestamp":    "2026-04-07T14:23:01.512Z"
}
```

Αντίστοιχα, ένα μήνυμα `PlugVal` στο Kafka topic συσκευών (`iot-sensor-plug`) περιλαμβάνει επιπλέον το πεδίο `plug_id` με τιμή διαφορετική του `null`:

```
{
  "home_id":      "iot-sensor-general/7",
  "plug_id":      "iot-sensor-plug/3",
  "voltage":      231.4,
  "current":      3.14,
  "power_factor": 0.91,
  "power_w":      661.46,
  "energy_wh":    4.663,
  "timestamp":    "2026-04-07T14:23:01.512Z"
}
```

Τα πεδία `voltage`, `current` και `power_factor` αντιστοιχούν στις ηλεκτρικές παραμέτρους της μέτρησης, ενώ τα `power_w` και `energy_wh` υπολογίζονται από τον τύπο της ισχύος:

$$P = V \times I \times \cos \varphi$$

και αθροιστική ενέργεια αντίστοιχα. Στην περίπτωση των `PlugVal`, το άθροισμα των επιμέρους ισχύων όλων των συσκευών ισούται ακριβώς με τη συνολική ισχύ της αντίστοιχης κεντρικής εγγραφής `HomeVal`, εξασφαλίζοντας συνέπεια δεδομένων.

Κάθε εγκατάσταση παράγει $1+n$ μηνύματα ανά κύκλο, δηλαδή μία κεντρική μέτρηση και n μετρήσεις συσκευών. Στη συγκεκριμένη υλοποίηση, παράγονται $1 + 4 = 5$ μηνύματα ανά δευτερόλεπτο ανά εγκατάσταση. Αυτό σημαίνει ότι, για παράδειγμα, 10.000 εγκαταστάσεις παράγουν 50.000 μηνύματα το δευτερόλεπτο (msg/s) [1].

7.10 Κατασκευή εξειδικευμένων Docker images

Ο φάκελος `docker/` του κώδικα της παρούσας εργασίας περιέχει τα αρχεία Dockerfile για τα εξειδικευμένα images που απαιτεί το σύστημα. Κάθε image επεκτείνει ένα επίσημο βασικό image (base image) με τις εξαρτήσεις εκτέλεσης που αντιστοιχούν στον ρόλο της στην αρχιτεκτονική, και ωθείται (push) στο ιδιωτικό Azure Container Registry (ACR). Ο

Πίνακας 7.3 συνοψίζει τα images. Όλα τα images ωθούνται (push) στο ACR με ετικέτα (label) `latest`. Τα Kubernetes Deployments και Jobs ορίζουν πολιτική λήψης των images ως `imagePullPolicy: Always`, ώστε κάθε (επαν)εκκίνηση pod να αντλεί πάντα το πιο πρόσφατο image.

Πίνακας 7.3: *Εξειδικευμένα images Docker του συστήματος.*

(Image) δημοσιευμένο στο ACR	**(Base image)**	**Ρόλος**
flink-paimon-kafka	flink:1.20.2-scala_-2.12	Εκτέλεση Flink tasks (εισαγωγή ροής, συγκεντρώσεις, αρχικοποίηση Paimon).
trino-paimon	trinodb/trino:440	Αναλυτικά ερωτήματα SQL στους πίνακες Paimon μέσω ABFS.
go_iot_sim_data_producer	golang:1.23-alpine	Προσομοιωτής τηλεμετρίας IoT σε Go.
go_iot_sim_data_producer_-batched	golang:1.23-alpine	Παραλλαγή ομαδικής αποστολής (batched) με παραμετροποιήσιμο μέγεθος δεσμίδας.

7.10.1 Flink image

Η κατασκευή του image `flink-paimon-kafka` πραγματοποιείται σε ένα στάδιο. Τα πέντε JARs εκτέλεσης (Πίνακας 7.1) ανακτώνται (download) κατά τη φάση κατασκευής του CI/CD pipeline (Ενότητα 7.11.2) και αντιγράφονται στον φάκελο `/opt/flink/lib/`. Κρίσιμη λεπτομέρεια αποτελεί η τοποθέτηση του plugin `flink-azure-fs-hadoop` στο φάκελο `/opt/flink/plugins/flink-azure-fs-hadoop/` (αντί του `lib/`), ώστε το Flink να το ανακαλύψει μέσω του μηχανισμού plugin loading κατά την εκκίνησή του [79]. Το image χρησιμοποιείται τόσο από τον JobManager και τους TaskManagers (ως Deployments) καθώς και από τα Jobs/CronJobs αρχικοποίησης και συγκεντρώσεων.

7.10.2 Trino image

Η κατασκευή του image `trino-paimon` πραγματοποιείται σε δύο στάδια (multi-stage build). Στο πρώτο στάδιο (builder), γίνεται download ο σύνδεσμος Paimon για την έκδοση Trino 440 [229] από το αποθετήριο (repository) στιγμιοτύπων (snapshots) της Apache, προστίθενται οι εξαρτήσεις (dependencies) Guava και failureaccess, και επανακατασκευάζεται ο `trino-filesystem-manager` με αφαίρεση του ελέγχου κλάσης HDFS (βλ. Ενότητα 7.12). Στο δεύτερο στάδιο, το τελικό image `trinodb/trino:440` λαμβάνει το φάκελο `plugin/paimon/` που έχει ως περιεχόμενο τα αρχεία JAR οργανωμένα στον υποφάκελο `hdfs/`.

7.10.3 Image προσομοιωτή παραγωγής IoT δεδομένων

Οι δύο παραλλαγές του προσομοιωτή βασίζονται στο base image `golang:1.23-alpine`, δηλαδή ένα ελαφρύ image που περιέχει μόνο τα απαραίτητα εργαλεία για την εκτέλεση εφαρμογών γραμμένων στη γλώσσα Go. Η batched παραλλαγή ομαδοποιεί μηνύματα σε δεσμίδες (`BATCH_SIZE_HOME=500`, `BATCH_SIZE_PLUG=1000`, `BATCH_TIMEOUT_MS=50`) πριν τα αποστείλει στο Kafka, μειώνοντας τον αριθμό δικτυακών κλήσεων σε σενάρια υψηλού φόρτου. Κάθε

image μεταγλωττίζει τον πηγαίο κώδικα Go εντός του container (`go build`) και ορίζει τις μεταβλητές περιβάλλοντος ως προεπιλογές (ENV).

7.11 Pipelines αυτοματοποίησης CI/CD με GitHub Actions

Η αυτοματοποίηση κατασκευής, ανάπτυξης και αποδόμησης του συστήματος βασίζεται σε πέντε ροές εργασίας GitHub Actions [96], οι οποίες καλύπτουν ολόκληρο τον κύκλο ζωής της υποδομής και των εφαρμογών. Όλες ενεργοποιούνται χειροκίνητα μέσω της ρύθμισης πυροδότησης `workflow_dispatch` και υποστηρίζουν επιλογή περιβάλλοντος, ώστε η ίδια βάση κώδικα να εξυπηρετεί πολλαπλά Azure Subscriptions με ACRs, λογαριασμούς αποθήκευσης και αρχεία ρυθμίσεων. Η επιλογή περιβάλλοντος καθορίζει δυναμικά τις τιμές μέσω εκφράσεων `fromJSON()` στις μεταβλητές περιβάλλοντος του workflow.

Η Εικόνα 7.9 απεικονίζει τις σχέσεις εξάρτησης μεταξύ των ροών εργασίας και τη σειρά εκτέλεσής τους.



Εικόνα 7.9: *Ροές εργασίας GitHub Actions. Σχέσεις εξάρτησης και σειρά εκτέλεσης από τη δημιουργία υποδομής (infra-ci) έως την αποδόμηση (infra-destroy)*

.

Κοινά βήματα Οι ροές εργασίας που αλληλεπιδρούν με πόρους Azure ακολουθούν κοινό μοτίβο αρχικοποίησης, η οποία εκτελείται σε διαδοχικά στάδια.

Αρχικά, πραγματοποιείται η πιστοποίηση του χρήστη μέσω της ενέργειας `azure/login@v2`, με τη χρήση διαπιστευτηρίων τύπου Service Principal τα οποία είναι αποθηκευμένα με ασφάλεια στα κλειδιά GitHub Secrets του εκάστοτε περιβάλλοντος. Στη συνέχεια, όπου απαιτείται πρόσβαση στο AKS cluster, εγκαθίσταται η διεπαφή γραμμής εντολών kubectl μέσω της ενέργειας `azure/setup-kubectl@v3`. Αμέσως μετά, εκτελείται η λήψη των απαραίτητων διαπιστευτηρίων του AKS με χρήση της εντολής `az aks get-credentials`. Τέλος, στις περιπτώσεις που είναι απαραίτητη image push, υλοποιείται η πιστοποίηση στο αποθετήριο ACR μέσω της εντολής `az acr login`.

Η συγκεκριμένη ακολουθία ενεργειών αποτελεί τη βάση για όλες τις επιμέρους διαδικασίες που περιγράφονται στις επόμενες υποενότητες.

7.11.1 Εγκατάσταση υποδομής μέσω Terraform

Η πρώτη ροή εργασίας που εκτελείται σε νέο περιβάλλον δημιουργεί την υποδομή Azure μέσω Terraform [26]. Τα βήματά της περιλαμβάνουν τη δημιουργία ομάδας πόρων (`az group create`), την αρχικοποίηση Terraform (`terraform init`), τον έλεγχο μορφοποίησης του Terraform κώδικα (`fmt -check`) και εγκυρότητας (`validate`), το σχέδιο εκτέλεσης (`plan`) και την εφαρμογή του (`apply`). Η παραμετροποίηση γίνεται μέσω αρχείου tfvars ανά περιβάλλον (`terraform.tfvars` ή `terraform.secondary.tfvars`).

7.11.2 Ανάπτυξη εφαρμογών

Αποτελεί την κεντρική και πιο εκτενή ροή εργασίας, η οποία ενορχηστρώνει σε μία ενιαία εκτέλεση την κατασκευή του Flink image καθώς και την ανάπτυξη ολόκληρου του συστήματος στο AKS cluster. Η διαδικασία αυτή υλοποιείται μέσα από μια σειρά διαδοχικών και τυποποιημένων σταδίων, τα οποία εξασφαλίζουν την ορθή παραμετροποίηση του περιβάλλοντος:

1. **Δημιουργία namespaces**. Πραγματοποιείται η δημιουργία ή η επιβεβαίωση της ύπαρξης των τεσσάρων απαραίτητων namespaces, και συγκεκριμένα των `data-ingest`, `data-processing`, `analytics` και `monitoring`, μέσω της εντολής:

```
kubectl create --dry-run=client | kubectl apply
```

2. **Λήψη εξαρτήσεων τύπου JAR**. Μεταφορτώνονται από το κεντρικό αποθετήριο Maven Central τα πέντε αρχεία JARs εκτέλεσης που παρουσιάζονται στον Πίνακα 7.1. Τα αρχεία αυτά αποθηκεύονται στον φάκελο `docker/lib/`, με τις εκδόσεις τους να καθορίζονται δυναμικά ως μεταβλητές περιβάλλοντος του workflow.

3. **Build και push Flink image**. Κατασκευάζεται το `flink-paimon-kafka` image με βάση τις οδηγίες του αρχείου `flink-paimon-kafka.Dockerfile` και στη συνέχεια ωθείται στο ACR.

4. **Δημιουργία κλειδιών Kubernetes Secrets**. Ανακτάται το κλειδί πρόσβασης του Azure Storage Account με την εκτέλεση της εντολής:

```
az storage account keys list
```

με σκοπό τη δημιουργία του secret στα namespaces `data-processing` και `analytics`.

5. **Εφαρμογή δηλωτικών αρχείων**. Εφαρμόζονται τα απαραίτητα αρχεία manifests για το Kafka μέσω του αρχείου `data-ingest.yaml`, για τις εργασίες του Flink μέσω του `data-processing.yaml`, καθώς και για τα αντίστοιχα ConfigMaps ρυθμίσεων, τα Deployments και τους προσομοιωτές IoT δεδομένων. Παράλληλα, τα dashboards του Grafana εισάγονται με τη μετατροπή των αρχείων JSON σε ConfigMaps που φέρουν την ετικέτα `grafana_dashboard=1` και εντάσσονται στον φάκελο `grafana_folder="IoT"`.

6. **Ανάπτυξη Helm charts**. Ολοκληρώνεται η εγκατάσταση του Trino με τη χρήση του `trino` base image στο namespace `analytics`, καθώς και του πακέτου `kube-prometheus-stack` για τα εργαλεία Prometheus και Grafana στο namespace `monitoring`, αξιοποιώντας τα αρχεία ρυθμίσεων που αντιστοιχούν στο εκάστοτε περιβάλλον.

7.11.3 Κατασκευή images IoT προσομοιωτή

Η ροή αυτή κατασκευάζει και ωθεί στο ACR τα images του προσομοιωτή τηλεμετρικών δεδομένων. Τον βασικό προσομοιωτή Go (`go_iot_sim_data_producer`) και την batched παραλλαγή (`go_iot_sim_data_producer_batched`).

7.11.4 Κατασκευή image Trino-Paimon

Η ροή κατασκευάζει το εξειδικευμένο image `trino-paimon` (Ενότητα 7.10) και το ωθεί στο ACR με ετικέτες (labels) αριθμού έκδοσης 440 [229] και `latest`. Η κατασκευή εκτελείται ως ανεξάρτητη ροή εργασίας λόγω της διάρκειάς της. Το πρώτο στάδιο κλωνοποιεί και μεταγλωττίζει μέρος του πηγαίου κώδικα του Trino, διαδικασία που απαιτεί αρκετά λεπτά ακόμα και σε εκτέλεση μέσω GitHub Actions.

7.11.5 Διαγραφή υποδομής

Η ροή αυτή διαγράφει (decommission) ολόκληρη την υποδομή στο Azure, καταργώντας το resource group μέσω της εντολής `az group delete -yes -no-wait`, αφαιρώντας έτσι το AKS cluster, το ACR, το ADLS και κάθε συσχετισμένο πόρο. Η παράμετρος `-no-wait` επιτρέπει μη σύγχρονη εκτέλεση, ενώ η διαγραφή σε επίπεδο resource group εξασφαλίζει ότι δεν απομένουν ορφανοί πόροι που θα μπορούσαν να επιβαρύνουν το κόστος.

7.12 Προκλήσεις υλοποίησης

Κατά τη διάρκεια της υλοποίησης αντιμετωπίστηκαν δύο σημαντικές τεχνικές προκλήσεις, οι οποίες σχετίζονται με τη συμβατότητα βιβλιοθηκών μεταξύ των στοιχείων ανοικτού κώδικα του συστήματος.

7.12.1 Εντοπισμός ελάχιστου συνόλου βιβλιοθηκών Flink για σύνδεση Paimon-ADLS

Η πρώτη πρόκληση αφορούσε τον εντοπισμό του ελάχιστου συνδυασμού JARs που επιτρέπει στο Flink να γράφει και να αναγνωρίζει πίνακες Paimon αποθηκευμένους σε ADLS μέσω του πρωτοκόλλου ABFS. Το εγχειρίδιο κάθε στοιχείου (Flink, Paimon, Hadoop Azure) παρέχει πληροφορίες μόνο για τη δική του πλευρά, χωρίς να περιγράφει τον πλήρη συνδυασμό τους. Μετά από εκτεταμένη δοκιμή, ο ελάχιστος λειτουργικός συνδυασμός αποτελείται από τα πέντε JARs του Πίνακα 7.1. Ιδιαίτερα κρίσιμη ήταν η ανάγκη τοποθέτησης του `flink-azure-fs-hadoop` στο φάκελο `plugins/` αντί στο `lib/`, καθώς χωρίς αυτή τη μετακίνηση ο Flink αδυνατούσε να αναγνωρίσει το σχήμα `abfs://`. Ομοίως, η απουσία του `flink-shaded-hadoop-2-uber` προκαλούσε το σφάλμα ClassNotFoundException κατά τη φάση εκκίνησης του Paimon καταλόγου (catalog).

7.12.2 Συνδυασμός Paimon και Trino

Η δεύτερη πρόκληση αφορούσε τη δημιουργία ενός εξειδικευμένου Trino image το οποίο να υποστηρίζει το Paimon. Ο συνδυασμός `paimon-trino` δεν διατίθεται σε σταθερή έκδοση ώστε να υποστηρίζει και το Azure Storage Account, με αποτέλεσμα η εγκατάστασή του να βασίζεται σε δοκιμαστικές εκδόσεις, δηλαδή σε ενδιάμεσες, μη επίσημες εκδόσεις του πηγαίου κώδικα του Apache repository. Η εγκατάσταση αυτή προκαλούσε τρία σωρευτικά σφάλματα. Αρχικά εντοπίστηκε η απουσία των εξαρτήσεων Guava [230] και failureaccess, δηλαδή των βασικών βιβλιοθηκών γενικής χρήσης της Google σε γλώσσα Java (συλλογές δεδομένων, διαχείριση null τιμών, βοηθητικές δομές για ταυτόχρονο προγραμματισμό), τις οποίες το Paimon αξιοποιεί κατά την αρχικοποίηση του catalog. Στη συνέχεια εμφανίστηκε το σφάλμα “HDFS should not be on the plugin classpath” από τον εκκινητή συστήματος αρχείων `HdfsFileSystemLoader` του Trino, το οποίο επιβάλλει αυστηρό διαχωρισμό μεταξύ των αρχείων JAR [146] του πυρήνα του συστήματος (core classpath) και των αρχείων JAR κάθε πρόσθετου (plugin classpath), προκειμένου να αποτρέπονται συγκρούσεις εκδόσεων μεταξύ κοινών βιβλιοθηκών. Τέλος, διαπιστώθηκε εσφαλμένη οργάνωση των αρχείων JAR του Hadoop [183] εντός του φακέλου plugin, με αποτέλεσμα το Trino να μην τα εντοπίζει στη δομή που αναμένει.

Η λύση βασίστηκε σε σχετική αναφορά της κοινότητας [231] και περιλαμβάνει τρία αλληλένδετα βήματα. Αυτά είναι η προσθήκη των αρχείων JARs του Guava και του failureaccess που έλειπαν προς κάλυψη των εξαρτήσεων χρόνου εκτέλεσης, η επανακατασκευή του υποσυστήματος `trino-filesystem-manager`, το οποίο είναι υπεύθυνο για την εγγραφή και τη φόρτωση των διαθέσιμων συστημάτων αρχείων στο Trino, αφού προηγουμένως απενεργοποιηθεί η ρουτίνα ελέγχου `verify()` μέσω σχολιασμού (commenting out) στον κώδικα του αρχείου `HdfsFileSystemLoader.java`, καθώς ακριβώς αυτή προκαλούσε την απόρριψη φόρτωσης του HDFS, και η αναδιοργάνωση του φακέλου `plugin/paimon/` με τη δημιουργία του υποφακέλου `hdfs/`, στον οποίο τοποθετούνται τα σχετικά αρχεία JAR του Hadoop ώστε το Trino να τα εντοπίζει στη δομή που αναμένει. Η πλήρης αυτή διαδικασία αυτοματοποιείται στο αρχείο `trino-paimon.Dockerfile` αξιοποιώντας την τεχνική των πολλαπλών σταδίων κατασκευής (multi-stage build) [79], η οποία διαχωρίζει το στάδιο μεταγλώττισης και προ-

ετοιμασίας των εξαρτήσεων από το τελικό image εκτέλεσης, μειώνοντας έτσι το συνολικό μέγεθος και αποκλείοντας περιττές εξαρτήσεις χρόνου κατασκευής.

Κεφάλαιο 8

Δοκιμές και Αξιολόγηση

Στο κεφάλαιο αυτό παρουσιάζονται τα πειράματα κλιμάκωσης που διενεργήθηκαν προκειμένου να αξιολογηθεί η συμπεριφορά του συστήματος υπό αυξανόμενο φόρτο [118, 232, 30, 233]. Τα πειράματα σχεδιάστηκαν ώστε να απαντήσουν στα ερευνητικά ερωτήματα της Εισαγωγής, εστιάζοντας στα όρια κλιμάκωσης, τους ρυθμούς μηνυμάτων, τους χρόνους ερωτημάτων και την κατανάλωση πόρων. Χρησιμοποιείται μεθοδολογία που εκτελεί δύο υπο-φάσεις ανά βαθμίδα, δηλαδή μη ομαδοποιημένους (nonbatched) και ομαδοποιημένους (batched) παραγωγούς. Η διάκριση αυτή αντικατοπτρίζει πραγματικές αρχιτεκτονικές IoT, στις οποίες ενδιάμεσοι κόμβοι (edge gateways) συσσωρεύουν μετρήσεις πολλαπλών αισθητήρων και τις προωθούν ομαδικά στο κεντρικό σύστημα [121]. Χαρακτηριστικά παραδείγματα του μοτίβου αυτού αποτελούν ο τοπικός MQTT broker που γεφυρώνει μηνύματα προς τον Kafka και ο CoAP reverse proxy που ενοποιεί αιτήματα συσκευών περιορισμένων πόρων πριν τα αποστείλει στο cloud [27]. Πέρα από την κατανάλωση πόρων, αξιολογείται ένα σύνολο ερωτημάτων Trino (Q1-Q5) που δοκιμάζει αναλυτικές λειτουργίες κλιμάκωσης.

8.1 Πειραματικό περιβάλλον και παραδοχές

8.1.1 Υλικό (hardware)

Στον Πίνακα 8.1 συνοψίζεται η υλική υποδομή του AKS cluster επί της οποίας εκτελέστηκαν τα πειράματα.

Πίνακας 8.1: *Υλικό υποδομής υπολογιστικού AKS cluster.*

Συνιστώσα	**Διαμόρφωση**
System pool	1 × Standard_E4s_v6 (4 vCPUs, 32GB)
User pool	2 × Standard_E8s_v6 (8 vCPUs, 64GB/node, σύνολο 16 vCPUs, 128GB)
Αποθήκευση (Kafka)	Azure Premium SSD, 128Gi ανά broker
Αποθήκευση (Prometheus)	Azure Premium SSD, 50Gi
Δικτύωση	Azure CNI εντός VNet, ADLS στην ίδια περιοχή (West Europe)

8.1.2 Λογισμικό

Στον Πίνακα 8.2 παρατίθενται οι βασικές εφαρμογές της πλατφόρμας και η διαμόρφωσή τους.

Πίνακας 8.2: *Λογισμικές συνιστώσες της πλατφόρμας.*

Συνιστώσα	**Διαμόρφωση**
Kafka	2 brokers, KRaft mode, iot-sensor-general (4 part.), iot-sensor-plug (6 part.)
Flink	v1.20.2, 1 JobManager, 2 TaskManagers, $4 \times 2 = 8$ slots, parallelism=4
Paimon	Κατάλογος στο ADLS, raw = 6 buckets, 7d TTL, 10min/30min/hourly = 1 bucket
Trino	Coordinator + 2 workers, 2GB heap, G1GC
Go IoT προσομοιωτής	4 πρίζες/κατοικία, 1 κύκλος/s/pod, χωρητικότητα 3.500 κατοικιών/pod

Με TTL (Time To Live) συμβολίζεται ο χρόνος ζωής των δεδομένων στον πίνακα, μετά τον οποίο διαγράφονται αυτόματα.

8.1.3 Παραδοχές

Στην ενότητα που ακολουθεί καταγράφονται οι βασικές παραδοχές που διέπουν το πειραματικό περιβάλλον και πρέπει να λαμβάνονται υπόψη κατά την ερμηνεία των αποτελεσμάτων.

1. Η εγκατάσταση που προσομοιώνεται αντιστοιχεί σε μία κατοικία και οι τελικοί αισθητήρες είναι οικιακές πρίζες ή συσκευές.

2. Ο ρυθμός παραγωγής ανά κατοικία είναι σταθερός και ίσος με 1 + 4 = 5 μηνύματα ανά δευτερόλεπτο (1 κεντρικός μετρητής + 4 πρίζες).

3. Τα δεδομένα είναι τεχνητά (προσομοίωση), επομένως η στατιστική κατανομή δεν αντανακλά πραγματική τηλεμετρία.

4. Το Flink εκτελεί μόνο μία εργασία εισαγωγής (streaming ingest). Η δεκάλεπτη συγκέντρωση εκτελείται σε batch mode τρεις φορές κατά τη διάρκεια του παραθύρου (στα +10/+20/+30 min) με χειροκίνητη ενεργοποίηση. Η τριαντάλεπτη συγκέντρωση εκτελείται μία φορά μετά τη φάση εκκαθάρισης (flush cooldown) και παράγει μία γραμμή rollup ανά (κατοικία, πρίζα, παράθυρο). Η αυτόματη CronJob ωριαίας συγκέντρωσης παραμένει σε κατάσταση αναστολής (suspended) στη διάρκεια των πειραμάτων ώστε να αποφεύγονται ταυτόχρονες εργασίες batch επί των ίδιων ροών Kafka/Paimon.

5. Η αυτόματη κλιμάκωση (οριζόντια ή κάθετη) δεν ενεργοποιείται και ο αριθμός nodes παραμένει σταθερός κατά τη διάρκεια κάθε πειράματος.

6. Η μέτρηση δικτύου (εισερχόμενη και εξερχόμενη κίνηση) γίνεται μέσω cAdvisor metrics στον Prometheus [117].

7. Κάθε βαθμίδα εκτελείται σε δύο υπο-φάσεις. Συγκεκριμένα, πρώτα με μη ομαδοποιημένους (nonbatched) και στη συνέχεια με ομαδοποιημένους (batched) παραγωγούς Kafka, ώστε να συγκριθούν οι δύο στρατηγικές εντός του ίδιου πειραματικού κύκλου [137].

8.2 Σενάρια φόρτου

8.2.1 Ροή δεδομένων υπό αξιολόγηση

Τα πειράματα αξιολογούν ολόκληρη την αλυσίδα επεξεργασίας που σχεδιάστηκε στο Κεφάλαιο 7. Ο προσομοιωτής IoT παράγει τηλεμετρικά δεδομένα ενέργειας και τα αποστέλλει στα topics του Kafka, το Flink τα καταναλώνει σε πραγματικό χρόνο και τα γράφει ακατέργαστα στο Paimon (επίπεδο bronze), οι batch εργασίες συγκέντρωσης (aggregation), οι οποίες επίσης εκτελούνται στο Flink, παράγουν τα επίπεδα silver και gold, και τέλος το Trino εξυπηρετεί ad-hoc ερωτήματα, τα οποία απεικονίζονται στο Grafana.

$$\text{IoT παραγωγός} \longrightarrow \text{Kafka} \longrightarrow \text{Flink} \longrightarrow \text{Paimon} \longrightarrow \text{Trino} \longrightarrow \text{Grafana}$$

Κάθε σενάριο φόρτου ασκεί πίεση στη ροή αυτή ρυθμίζοντας τον αριθμό των εικονικών κατοικιών (και άρα των παραγόμενων μηνυμάτων), ενώ η υπόλοιπη σωλήνωση (pipeline) παραμένει σταθερή.

8.2.2 Μεθοδολογία δημιουργίας σεναρίων

Η δημιουργία σεναρίων φόρτου αυτοματοποιείται μέσω σεναρίου γραμμένου σε PowerShell (`run-experiment-combined-improved.ps1`, Παράρτημα ΣΤ) που εκτελεί σειριακά κάθε βαθμίδα (tier). Για την αξιοπιστία του πειράματος έχουν ληφθεί τα παρακάτω μέτρα:

Επικύρωση πριν την εκτέλεση (pre-run validation). Πριν από κάθε πείραμα, ελέγχονται η διαθεσιμότητα του Flink REST API, η ύπαρξη topics Kafka, η σύνδεση καταλόγου Trino και η προσβασιμότητα του Azure Storage Account. Σε περίπτωση αποτυχίας, η εκτέλεση ακυρώνεται πριν σπαταληθούν πόροι.

Πύλη υγείας Flink (health gate). Μετά την υποβολή της εργασίας εισαγωγής, το σενάριο επαληθεύει ότι η εργασία βρίσκεται σε κατάσταση RUNNING μέσα από τρεις διαδοχικούς ελέγχους (με μεσοδιάστημα 10 δευτερολέπτων), αποτρέποντας τη συλλογή δεδομένων από αποτυχημένες εκτελέσεις [13].

Περίοδος εκκαθάρισης (flush cooldown). Μετά τη διακοπή των παραγωγών, υπάρχει αναμονή εκατόν είκοσι (120) δευτερολέπτων (flush cooldown) ώστε το Flink να ολοκληρώσει δύο κύκλους checkpoint (60s διάστημα) και να γράψει τα εναπομείναντα δεδομένα στο Paimon.

Χειροκίνητη ενεργοποίηση συγκεντρώσεων. Οι εργασίες δεκάλεπτης συγκέντρωσης ενεργοποιούνται χειροκίνητα τρεις φορές κατά τη διάρκεια του παραθύρου μέτρησης (στα run-elapsed +10/+20/+30 min), με ενσωματωμένη προστασία ταυτόχρονων slots ώστε να αποφεύγεται η συσσώρευση εργασιών στις ίδιες θέσεις Flink. Η εργασία τριαντάλεπτης συγκέντρωσης ενεργοποιείται μία φορά μετά τη φάση εκκαθάρισης και παράγει το rollup ολόκληρου του παραθύρου στον πίνακα sensor_data_30min.

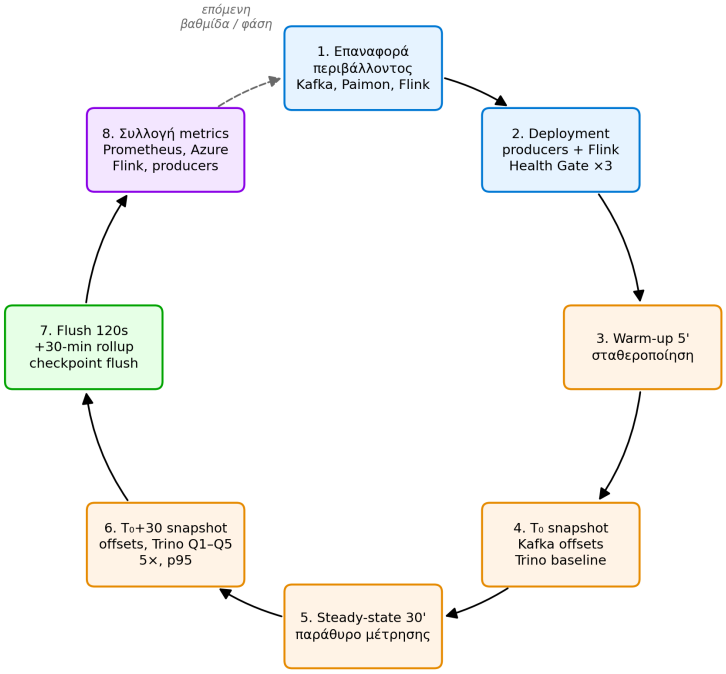
Ερώτημα προθέρμανσης Trino. Πριν από τα ερωτήματα αναφοράς, εκτελείται μια εντολή τύπου `SELECT 1` ως ερώτημα προθέρμανσης (warm-up), αποφεύγοντας την επιβάρυνση ψυχρής εκκίνησης στο πρώτο ερώτημα [24].

Για κάθε βαθμίδα, η ροή εκτέλεσης περιλαμβάνει:

1. Πλήρη επαναφορά περιβάλλοντος με διαγραφή παραγωγών, ακύρωση εργασιών Flink, διαγραφή blobs Paimon στο ADLS, επαναδημιουργία πινάκων.
2. Εκτέλεση παραγωγών Go, όπου κάθε pod χειρίζεται έως 3.500 κατοικίες.
3. Επανεκκίνηση ροής εισαγωγής Flink και πύλη υγείας (3 διαδοχικοί έλεγχοι).
4. Περίοδος προθέρμανσης πέντε (5′) λεπτών.
5. Καταγραφή offsets Kafka στο χρονικό σημείο T_0, εκτέλεση ερωτημάτων Trino αναφοράς.
6. Τριάντα (30′) λεπτά σταθερής λειτουργίας.
7. Καταγραφή offsets στο χρονικό σημείο T_0+30 λεπτά, εκτέλεση των ερωτημάτων αναφοράς Trino (5 εκτελέσεις ανά ερώτημα, αναφορά 95th percentile (p95), η τιμή δηλαδή κάτω από την οποία εμπίπτει το 95% των δεδομένων).
8. Περίοδος εκκαθάρισης εκατόν είκοσι (120′) δευτερολέπτων, ενεργοποίηση τριαντάλεπτης συγκέντρωσης για την παραγωγή του rollup του παραθύρου.
9. Συλλογή metrics τύπου Prometheus, Azure Storage, Flink checkpoints και στατιστικά παραγωγών.

Η συνολική κυκλική ροή του πειραματικού πρωτοκόλλου ανά βαθμίδα και υπο-φάση αποτυπώνεται εποπτικά στο Σχήμα 8.1, όπου κάθε ολοκληρωμένος κύκλος ενεργοποιεί τον επόμενο μέσω της διακεκομμένης μετάβασης. Υποφάση αποτελεί η ομαδοποίηση ή μη των μηνυμάτων των παραγωγών δεδομένων.

Το όριο των 3.500 κατοικιών ανά pod προέκυψε από δοκιμαστικές εκτελέσεις βαθμονόμησης (calibration runs). Οι παραγωγοί εκτελέστηκαν με σταδιακά αυξανόμενο φόρτο ανά pod, από 50 έως 5.000 κατοικίες ανά βήμα 250, ενώ παρακολουθούνταν η καθυστέρηση παραγωγής και ο ρυθμός CPU του container. Κριτήριο αποδοχής ορίστηκε η ολοκλήρωση της παραγωγής όλων των μηνυμάτων ενός κύκλου εντός ενός δευτερολέπτου, ώστε ο παραγωγός να μην συσσωρεύει καθυστέρηση και να προσομοιώνει συνθήκες πραγματικού κόσμου.



Σχήμα 8.1: *Κυκλική σύνοψη του πειραματικού πρωτοκόλλου ανά βαθμίδα. Οι οκτώ διαδοχικές φάσεις (επαναφοράς, deployment, προθέρμανσης (warm-up), T_0 snapshot, σταθερής λειτουργίας (steady-state), T_0+30 snapshot, flush με τριαντάλεπτο rollup, συλλογή metrics) εκτελούνται κυκλικά και η διακεκομμένη μετάβαση από τη φάση 8 πίσω στη φάση 1 σηματοδοτεί την έναρξη της επόμενης υπο-φάσης ή της επόμενης βαθμίδας.*

Πάνω από τις 3.500 κατοικίες παρατηρήθηκε σταθερή υπέρβαση αυτής της καθυστέρησης-στόχου, υποδεικνύοντας κορεσμό. Κάτω από αυτό το όριο η κατανάλωση πόρων παρέμενε χαμηλή με σταθερή απόδοση. Ο δείκτης ρυθμού μηνυμάτων ανά δευτερόλεπτο, στο εξής RPS, ορίζεται αναλυτικά στην ενότητα 8.4. Τα αναλυτικά αποτελέσματα παρουσιάζονται στον Πίνακα 8.3.

Πίνακας 8.3: *Αποτελέσματα βαθμονόμησης παραγωγού IoT δεδομένων γραμμένου σε γλώσσα προγραμματισμού Go (5 μετρήσεις/κατοικία ανά κύκλο).*

Κατοικίες	RPS	Μέσος κύκλος	Μέγ. κύκλος	Διάρκεια κύκλου	Κορεσμός
50	250	6ms	9ms	1,0s	Όχι
100	501	12ms	17ms	1,0s	Όχι
250	1.116	31ms	50ms	1,0s	Όχι
500	2.501	46ms	66ms	1,0s	Όχι
750	3.750	70ms	97ms	1,0s	Όχι
1.000	4.999	106ms	146ms	1,0s	Όχι
1.250	6.251	146ms	206ms	1,0s	Όχι
1.500	7.498	182ms	252ms	1,0s	Όχι
1.750	8.750	180ms	256ms	1,0s	Όχι
2.000	10.001	225ms	307ms	1,0s	Όχι
2.250	11.250	273ms	370ms	1,0s	Όχι
2.500	12.501	300ms	418ms	1,0s	Όχι
2.750	13.753	288ms	402ms	1,0s	Όχι
3.000	14.994	349ms	477ms	1,0s	Όχι
3.250	16.255	387ms	529ms	1,0s	Όχι
3.500	**17.504**	**411ms**	**561ms**	**1,0s**	**Όχι**
3.750	18.745	455ms	618ms	1,0s	Όχι
4.000	19.994	477ms	654ms	1,0s	Όχι
4.250	21.248	534ms	717ms	1,0s	Όχι
4.500	22.492	531ms	718ms	1,0s	Όχι
4.750	23.751	562ms	760ms	1,0s	Όχι
5.000	24.998	637ms	854ms	1,0s	Ναι

Η στήλη Κατοικίες δίνει τον αριθμό κατοικιών ανά pod.
Η στήλη RPS δίνει τον παρατηρούμενο ρυθμό μηνυμάτων (records/s).
Οι στήλες Μέσος κύκλος και Μέγ. κύκλος δίνουν τον μέσο και μέγιστο χρόνο ολοκλήρωσης ενός κύκλου παραγωγής δεδομένων.
Η στήλη Διάρκεια κύκλου δίνει το ρυθμιστικό διάστημα-στόχο ενός κύκλου.
Η στήλη Κορεσμός δίνει τη σημαία κορεσμού του παραγωγού, δηλαδή «Ναι» εάν ο κύκλος υπερβαίνει τον στόχο του 1,0s.
Με έντονη γραφή σηματοδοτείται το επιλεγμένο κατώφλι ανάπτυξης των πειραμάτων.

8.2.3 Διφασική εκτέλεση πειράματος ομαδοποιώντας τα μηνύματα των παραγωγών

Κάθε βαθμίδα εκτελείται σε δύο διαδοχικές υπο-φάσεις. Στη μη ομαδοποιημένη Φάση Α, στο εξής Nonbatched, οι παραγωγοί IoT αποστέλλουν κάθε μήνυμα μεμονωμένα στο Kafka, χωρίς συσσώρευση, με την παράμετρο linger.ms ίση με μηδέν. Στη ομαδοποιημένη Φάση Β, στο εξής Batched, οι παραγωγοί χρησιμοποιούν ομαδοποίηση μηνυμάτων, με την παράμετρο linger.ms μεγαλύτερη του μηδενός και τη batch.size βελτιστοποιημένη, μειώνοντας έτσι τον αριθμό μηνυμάτων προς τους brokers [137]. Μεταξύ των δύο φάσεων εκτελείται πλήρης επαναφορά περιβάλλοντος, η οποία περιλαμβάνει διαγραφή των δεδομένων Paimon και επανεκκίνηση του Flink job, ώστε κάθε φάση να εκτελείται σε καθαρές συνθήκες.

8.2.4 Ορισμός βαθμίδων

Ο Πίνακας 8.4 συνοψίζει τις βαθμίδες φόρτου, καθεμία από τις οποίες συμβολίζεται με το γράμμα Τ ακολουθούμενο από αύξοντα αριθμό. Κάθε βαθμίδα αντιστοιχεί σε έναν αριθμό προσομοιούμενων κατοικιών, ο οποίος συμβολίζεται με το γράμμα Η και καθορίζει τον αριθμό pods παραγωγών καθώς και τον θεωρητικά εκτιμώμενο ρυθμό παραγωγής μηνυμάτων $H \times 5$ msg/s.

Πίνακας 8.4: *Βαθμίδες πειραμάτων φόρτου.*

Βαθμίδα	**Κατοικίες**	**Pods παραγ.**	**msg/s (θεωρ.)**
T1	50	1	250
T2	500	1	2.500
T3	1.500	1	7.500
T4	3.500	1	17.500
T5	5.500	2	27.500
T6	7.500	3	37.500
T7	10.000	3	50.000
T8	12.500	4	62.500
T9	15.000	5	75.000
T10	17.500	5	87.500

Η στήλη Κατοικίες δίνει τον αριθμό προσομοιούμενων κατοικιών της βαθμίδας.
Η στήλη Pods παραγ. δίνει τον αριθμό pods παραγωγών.
Η στήλη msg/s (θεωρ.) δίνει τον θεωρητικά εκτιμώμενο ρυθμό μηνυμάτων ανά δευτερόλεπτο, ίσο με $H \times 5$.

8.3 Ερωτήματα Αναφοράς Trino

Τα ερωτήματα αναφοράς είναι συνολικά πέντε και αριθμούνται διαδοχικά από το πρώτο έως το πέμπτο ως Q1-Q5. Σχεδιάστηκαν ώστε να αξιολογούν αναλυτικές λειτουργίες κλίμακας. Κάθε ερώτημα εκτελείται 5 φορές και αναφέρεται η τιμή p95 [24, 205]. Παρακάτω παρατίθενται σε απλοποιημένη μορφή που αναδεικνύει τον αναλυτικό πυρήνα τους, ενώ τα πλήρη ερωτήματα, με τις εντολές WHERE, ORDER BY και LIMIT, παρατίθενται στο Παράρτημα Ε΄.

1. **Q1, Πλήθος γραμμών (COUNT) στον ακατέργαστο πίνακα**. Κλιμακώνει γραμμικά ως προς τον όγκο δεδομένων και αποτελεί το βασικό metric αναφοράς.

```
SELECT COUNT(*) FROM sensor_data_raw
```

2. **Q2, Ομαδοποίηση (GROUP BY) με πολλαπλές συναρτήσεις συγκέντρωσης στον ακατέργαστο πίνακα**. Ομαδοποίηση κατά home_id/plug_id με μέσο όρο (AVG), μέγιστο (MAX) και πλήθος (COUNT). Κλιμακώνεται με τον όγκο δεδομένων μέσω πλήρους προσπέλασης και συγκέντρωσης.

```
SELECT  home_id,
        plug_id,
        AVG(voltage)                    AS avg_voltage,
        AVG(current_amp)                AS avg_current,
```

```
        MAX(voltage * current_amp * power_factor) AS peak_power_w,
        COUNT(*)                                  AS samples
FROM    sensor_data_raw
GROUP BY home_id, plug_id
```

3. **Q3, Συνάρτηση παραθύρου (κυλιόμενος μέσος όρος) στον ακατέργαστο πίνακα**. Βαρύ υπολογιστικά, καθώς απαιτεί PARTITION και ταξινόμηση (SORT) ολόκληρου του συνόλου δεδομένων.

```
SELECT  home_id,
        ts,
        AVG(voltage) OVER (
            PARTITION BY home_id
            ORDER BY ts
            ROWS BETWEEN 59 PRECEDING AND CURRENT ROW
        ) AS rolling_avg_voltage
FROM    sensor_data_raw
```

4. **Q4, Σύνδεση (JOIN) μεταξύ ακατέργαστου και δεκάλεπτου πίνακα**. Σύνδεση sensor_data_raw με sensor_data_10min βάσει home_id, plug_id και dt, υπολογίζοντας απόκλιση τάσης (voltage drift) μεταξύ ακατέργαστων και δεκάλεπτων συγκεντρωτικών δεδομένων. Επιδεικνύει αναλυτικό πρότυπο σύνδεσης (JOIN) σε κλίμακα.

```
SELECT  r.home_id,
        COUNT(*)                          AS raw_samples,
        t.energy_kwh,
        AVG(r.voltage)                    AS raw_avg_voltage,
        t.avg_voltage                     AS tenmin_avg_voltage,
        ABS(AVG(r.voltage) - t.avg_voltage) AS voltage_drift
FROM    sensor_data_raw   r
JOIN    sensor_data_10min t
     ON r.home_id = t.home_id
    AND r.plug_id = t.plug_id
    AND r.dt      = t.dt
WHERE   r.plug_id = 'general'
    AND t.samples >= (SELECT CAST(MAX(samples) AS DOUBLE) * 0.8
                      FROM sensor_data_10min)
GROUP BY r.home_id, t.energy_kwh, t.avg_voltage
ORDER BY voltage_drift DESC
LIMIT 20
```

5. **Q5, Στατιστική ανάλυση εκατοστημορίων στον ακατέργαστο πίνακα**. Βαρύς στατιστικός υπολογισμός που κλιμακώνεται με τον όγκο.

```
SELECT  home_id,
        approx_percentile(voltage, 0.50) AS p50_voltage,
        approx_percentile(voltage, 0.95) AS p95_voltage,
```

```
        approx_percentile(voltage, 0.99) AS p99_voltage,
        stddev(voltage)                  AS stddev_voltage
FROM    sensor_data_raw
GROUP BY home_id
```

Τα ερωτήματα Q2-Q5 σχεδιάστηκαν ώστε να δοκιμάζουν τις αναλυτικές δυνατότητες του Trino σε κλίμακα, εμφανίζοντας υπεργραμμική αύξηση χρόνου εκτέλεσης στις υψηλές βαθμίδες λόγω πλήρους προσπέλασης αρχείων Parquet στο ADLS [205].

8.4 Δείκτες απόδοσης

8.4.1 Ρυθμός μηνυμάτων ανά δευτερόλεπτο

Ο ρυθμός μηνυμάτων ανά δευτερόλεπτο, Records Per Second, στο εξής RPS και ισοδύναμα γνωστός στη διεθνή βιβλιογραφία ως throughput, εκφράζει το πραγματικό πλήθος μηνυμάτων που εισέρχονται στο Kafka ανά μονάδα χρόνου και μετράται μέσω της διαφοράς Kafka offsets σύμφωνα με την παρακάτω σχέση.

$$\text{throughput} = \frac{\Delta\text{offset}_{T_0+30} - \Delta\text{offset}_{T_0}}{30 \times 60} \quad [\text{msg/s}]$$

Στην παραπάνω σχέση, ο όρος throughput εκφράζει τον ζητούμενο ρυθμό μηνυμάτων ανά δευτερόλεπτο. Η χρονική στιγμή T_0 συμβολίζει την έναρξη κάθε παραθύρου μέτρησης διάρκειας 30 λεπτών, ενώ ο $T_0 + 30$ συμβολίζει τη χρονική στιγμή λήξης του ίδιου παραθύρου. Ο όρος $\Delta\text{offset}_{T_0}$ δηλώνει το αθροιστικό offset του Kafka topic τη χρονική στιγμή T_0 και αντίστοιχα ο όρος $\Delta\text{offset}_{T_0+30}$ δηλώνει το αθροιστικό offset τη χρονική στιγμή $T_0 + 30$. Ο παρονομαστής 30×60 μετατρέπει το διάστημα των 30 λεπτών σε 1800 δευτερόλεπτα, ώστε ο τελικός ρυθμός να εκφράζεται στη μονάδα msg/s, δηλαδή σε μηνύματα ανά δευτερόλεπτο. Η μέτρηση γίνεται ανά Kafka topic, τόσο του iot-sensor-general όσο και του iot-sensor-plug και αθροίζεται. Ο δείκτης αυτός αντανακλά τον πραγματικό ρυθμό εισαγωγής στο Kafka, ανεξάρτητα από τον ρυθμό κατανάλωσης του Flink [137]. Η μεθοδολογία μέτρησης ακολουθεί τις αρχές που περιγράφονται στη βιβλιογραφία [143], όπου τονίζεται ότι σε συστήματα επεξεργασίας ροών τα κυρίαρχα σημεία συμφόρησης εντοπίζονται στην πλευρά της επεξεργασίας και όχι της εισόδου/εξόδου, καθιστώντας απαραίτητη τη διάκριση μεταξύ ρυθμού εισαγωγής και ρυθμού κατανάλωσης.

8.4.2 Καθυστέρηση άκρο-σε-άκρο (end-to-end latency)

Η καθυστέρηση από άκρο σε άκρο (στο εξής αναφερόμενη και ως E2E latency ή απλώς E2E) μετρείται ως η χρονική διαφορά μεταξύ της πιο πρόσφατης χρονοσφραγίδας γεγονότος, MAX(ts), στον πίνακα raw και την ένδειξη του ρολογιού κατά τη στιγμή μέτρησης. Το metric αυτό αντανακλά τη «φρεσκάδα δεδομένων» (data freshness), δηλαδή πόσο πρόσφατα είναι τα δεδομένα που έχουν γίνει ορατά μέσω Trino [149].

8.4.3 Παρατήρηση δικτύου

Η εισερχόμενη και εξερχόμενη κίνηση ανά pod και ανά namespace μετρείται μέσω των metrics του cAdvisor που συλλέγονται από το Prometheus. Συγκεκριμένα, η μετρική container_network_receive_bytes_total καταγράφει το αθροιστικό πλήθος bytes που λαμβάνονται από τη δικτυακή διεπαφή του container, ενώ η μετρική container_network_transmit_bytes_total καταγράφει αντίστοιχα το αθροιστικό πλήθος bytes που μεταδίδονται προς τα έξω. Συμπληρωματικά, οι μετρικές BytesReceived και BytesSent του Azure Monitor σε επίπεδο VNet χρησιμοποιούνται ως διασταύρωση, καθώς αποτυπώνουν αντίστοιχα τη συνολική εισερχόμενη και εξερχόμενη κίνηση στο εικονικό δίκτυο [117].

8.5 Αποτελέσματα ρυθμού μηνυμάτων και κλιμάκωσης

8.5.1 Συνοπτικά αποτελέσματα

Ο Πίνακας 8.5 συνοψίζει τα κύρια αποτελέσματα ανά βαθμίδα για τις δύο στρατηγικές παραγωγών.

Πίνακας 8.5: *Συνοπτικά αποτελέσματα ανά βαθμίδα φόρτου για τις δύο στρατηγικές παραγωγών, με τον ρυθμό μηνυμάτων, την καθυστέρηση E2E και το πλήθος αστοχιών checkpoint και Trino.*

Βαθμ.	Αναμ.	Παρατ. RPS		E2E (s)		Ckpt	Trino
	RPS	N	B	N	B	fail	fail
T1	250	260	262	46,6	44,4	0	0
T2	2.500	2.602	2.602	47,2	31,8	0	0
T3	7.500	7.995	7.733	33,4	59,8	0	0
T4	17.500	18.356	18.452	67,5	69,5	0	0
T5	27.500	29.366	29.459	44,3	66,0	0	0
T6	37.500	40.057	39.414	39,7	66,8	0	0
T7	50.000	54.365	53.444	172,6	63,6	3	0
T8	62.500	70.701	68.258	89,9	109,2	1	0
T9	75.000	86.909	82.091	335,2	201,8	23	4
T10	87.500	103.281	94.902	178,2	354,6	14	5

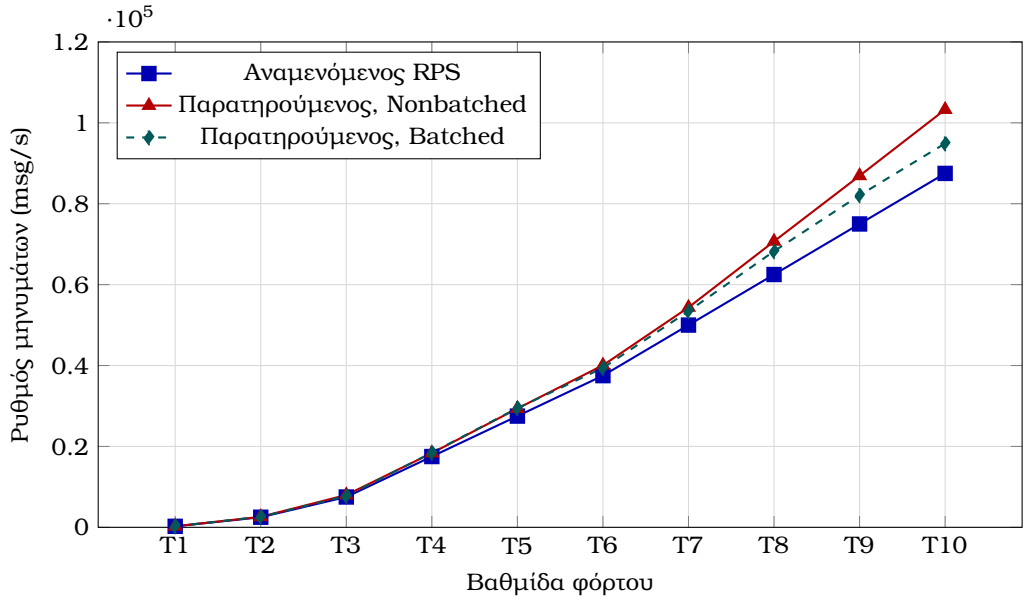
Με Βαθμ. συμβολίζεται η βαθμίδα φόρτου.
Με Αναμ. RPS συμβολίζεται ο αναμενόμενος ρυθμός μηνυμάτων ανά δευτερόλεπτο.
Με Παρατ. RPS συμβολίζεται ο παρατηρούμενος ρυθμός μηνυμάτων ανά δευτερόλεπτο.
Με N συμβολίζεται η φάση Nonbatched με μη ομαδοποιημένους παραγωγούς.
Με B συμβολίζεται η φάση Batched με ομαδοποιημένους παραγωγούς.
Η στήλη E2E δίνει την καθυστέρηση άκρο-σε-άκρο σε δευτερόλεπτα.
Η στήλη Ckpt fail δίνει το αθροιστικό πλήθος αποτυχημένων checkpoints στις δύο φάσεις κάθε βαθμίδας.
Η στήλη Trino fail δίνει το αθροιστικό πλήθος αστοχιών ερωτημάτων Trino στις δύο φάσεις κάθε βαθμίδας.

Ο παρατηρούμενος ρυθμός μηνυμάτων υπερβαίνει τον αναμενόμενο κατά 3 έως 18%, καθώς ο προσομοιωτής δεδομένων IoT ολοκληρώνει τους κύκλους αποστολής ταχύτερα από 1 s. Συγκεκριμένα, ο μέσος χρόνος κύκλου στους μη ομαδοποιημένους παραγωγούς είναι μεταξύ 0,003 και 0,42 s. Ο λόγος εισαγωγής, ingestion ratio, κυμαίνεται από 1,04 στη T1 έως 1,18 στη N10. Κατάσταση κορεσμού παραγωγού εμφανίζεται μόνο στη N10, όπου ο μέγιστος χρόνος κύκλου φτάνει τα 1,40 s και υπερβαίνει το κατώφλι του 1 s.

Ως προς τη σταθερότητα του συστήματος, οι βαθμίδες T1 έως T6 ολοκληρώνονται χωρίς αστοχίες και με καθυστέρηση E2E κάτω από 70 δευτερόλεπτα. Οι πρώτες αστοχίες checkpoint εμφανίζονται στη T7 και στις δύο φάσεις και ανακάμπτουν μέσω failover, ενώ η T8 παρουσιάζει μία μεμονωμένη αστοχία. Στις βαθμίδες T9 και T10 οι αστοχίες checkpoint πολλαπλασιάζονται και εμφανίζονται για πρώτη φορά αστοχίες ερωτημάτων Trino, με χαρακτηριστικότερη την αποτυχία του ερωτήματος Q3 στη N10. Αντίθετα, η B10 ολοκληρώνει όλα τα ερωτήματα επιτυχώς. Η καθυστέρηση E2E ακολουθεί αντίστοιχη πορεία. Στους μη ομαδοποιημένους παραγωγούς κορυφώνεται στη T9, όπου η τιμή 335,2 s της N9 προέρχεται από επιβεβαιωτική επανεκτέλεση, καθώς η αρχική εκτέλεση είχε καταγράψει 1.060,8 s, ενώ στους ομαδοποιημένους αυξάνεται προοδευτικά μετά τη T7 και κορυφώνεται στη T10. Η αναλυτική εξέταση των αστοχιών ανά βαθμίδα παρουσιάζεται στην ενότητα 8.10.

8.5.2 Ρυθμός μηνυμάτων ανά βαθμίδα

Το Σχήμα 8.2 απεικονίζει τη σύγκριση μεταξύ αναμενόμενου και παρατηρούμενου ρυθμού μηνυμάτων για τις δύο στρατηγικές παραγωγών.



Σχήμα 8.2: *Αναμενόμενος έναντι παρατηρούμενου RPS ανά βαθμίδα φόρτου. Ο παρατηρούμενος ρυθμός υπερβαίνει του αναμενόμενο κατά 3 έως 18%, χωρίς ουσιαστική διαφοροποίηση μεταξύ nonbatched και batched παραγωγών.*

Ένα αξιοσημείωτο εύρημα είναι ότι ο παρατηρούμενος ρυθμός δεν πέφτει κάτω από τον αναμενόμενο σε καμία βαθμίδα, σε αντίθεση με τα θεωρητικά αναμενόμενα όρια κορεσμού. Αυτό οφείλεται στο ότι ο παρατηρούμενος ρυθμός μετρά την παραγωγή μηνυμάτων στο Kafka, δηλαδή στην πλευρά του παραγωγού, ενώ τα bottlenecks εκδηλώνονται στην πλευρά κατανάλωσης και της επεξεργασίας, δηλαδή του Flink, ως αυξανόμενη καθυστέρηση E2E και αστοχίες checkpoint. Ειδικότερα, η καθυστέρηση E2E κορυφώνεται στη T9 nonbatched με διάρκεια 335,2 s, αντανακλώντας επαναλαμβανόμενους κύκλους ανάκαμψης checkpoint (13 αποτυχίες, 9 επαναφορές) που καθυστερούν την κατανάλωση Kafka μηνυμάτων [149].

Ο Πίνακας 8.6 παρουσιάζει αναλυτικά τον ρυθμό μηνυμάτων και την κατάσταση κορε-

σμού ανά κλίμακα.

Πίνακας 8.6: *Ρυθμός μηνυμάτων (RPS) και κατάσταση κορεσμού παραγωγών ανά κλίμακα.*

Tier	Κατοικ.	Pods	Αναμ.	Παρ. N	Παρ. B	Κορ. N	SLO N	SLO B
T1	50	1	250	260	262	FALSE	0,99	1,07
T2	500	1	2.500	2.602	2.602	FALSE	0,99	0,99
T3	1.500	1	7.500	7.995	7.733	FALSE	1,00	1,00
T4	3.500	1	17.500	18.356	18.452	FALSE	0,99	1,01
T5	5.500	2	27.500	29.366	29.459	FALSE	1,00	1,00
T6	7.500	3	37.500	40.057	39.414	FALSE	1,00	0,98
T7	10.000	3	50.000	54.365	53.444	FALSE	1,01	1,00
T8	12.500	4	62.500	70.701	68.258	FALSE	1,00	0,98
T9	15.000	5	75.000	86.909	82.091	FALSE	1,01	1,00
T10	17.500	5	87.500	103.281	94.902	TRUE	1,01	1,01

Με Κατοικ. συμβολίζεται ο αριθμός προσομοιούμενων κατοικιών.
Η στήλη Pods δίνει τον αριθμό pods παραγωγών.
Η στήλη Αναμ. δίνει τον αναμενόμενο ρυθμό μηνυμάτων ανά δευτερόλεπτο.
Οι στήλες Παρ. N και Παρ. B δίνουν τον παρατηρούμενο ρυθμό μηνυμάτων ανά δευτερόλεπτο για τις φάσεις Nonbatched και Batched αντίστοιχα.
Η στήλη Κορ. N δηλώνει αν εμφανίστηκε κορεσμός παραγωγού στη φάση Nonbatched.
Οι στήλες SLO N και SLO B δίνουν τον λόγο εγγραφών Paimon προς μηνύματα Kafka για τις δύο φάσεις αντίστοιχα.

8.6 Δείκτες απόδοσης ερωτημάτων Trino

8.6.1 Χρόνοι ερωτημάτων αναφοράς στη Nonbatched φάση

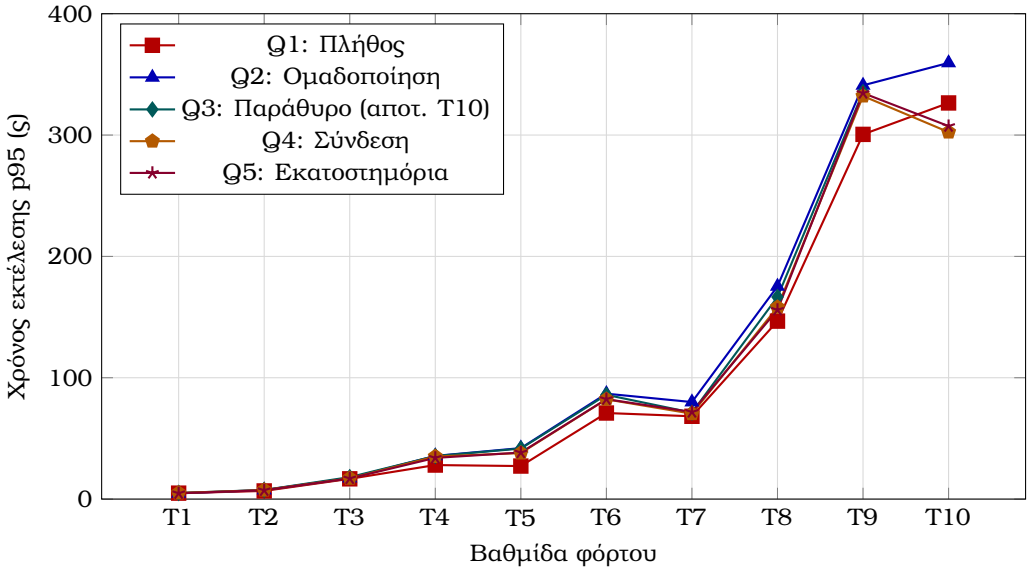
Το Σχήμα 8.3 παρουσιάζει τους χρόνους εκτέλεσης (p95) των πέντε ερωτημάτων Trino για τους μη ομαδοποιημένους παραγωγούς. Παρατηρείται δραματική αύξηση σε όλα τα ερωτήματα. Συγκεκριμένα το Q1 (πλήθος) αυξάνεται 67 φορές (4,9 s → 326 s), ενώ το Q2 (ομαδοποίηση) αυξάνεται 76 φορές (4,75 s → 359 s). Το ερώτημα Q3 (συνάρτηση παραθύρου) αποτυγχάνει στη βαθμίδα T10 λόγω υπέρβασης χρονικού ορίου (timeout).

Ο Πίνακας 8.7 παρουσιάζει αναλυτικά τους χρόνους εκτέλεσης (p95) ανά ερώτημα και κλίμακα.

Πίνακας 8.7: *Χρόνος ερωτημάτων Trino (p95, σε δευτερόλεπτα) ανά κλίμακα, μη ομαδοποιημένοι παραγωγοί.*

Tier	Q1 (s)	Q2 (s)	Q3 (s)	Q4 (s)	Q5 (s)	Αποτ.[†]
T1	4,90	4,75	5,03	5,01	4,78	0
T2	6,72	7,47	7,51	7,40	7,23	0
T3	16,70	17,35	18,09	17,10	17,00	0
T4	28,04	35,59	35,47	35,08	33,96	0
T5	27,22	42,02	41,65	37,95	38,36	0
T6	70,93	86,87	85,91	82,20	82,35	0
T7	68,24	79,96	71,25	70,36	71,75	0
T8	146,65	175,30	166,54	158,08	155,93	0
T9	300,56	340,96	335,43	332,15	334,61	4
T10	326,46	359,46	-*	302,44	307,37	5

[†]Η στήλη Αποτ. δίνει το πλήθος αποτυχημένων εκτελέσεων ερωτημάτων Trino στη βαθμίδα φόρτου, επί συνόλου 25 εκτελέσεων (5 ερωτήματα × 5 εκτελέσεις).
*Q3 απέτυχε λόγω υπέρβασης μνήμης σε ~200M εγγραφές.



Σχήμα 8.3: *Χρόνοι ερωτημάτων Trino (p95) ανά βαθμίδα, μη ομαδοποιημένοι παραγωγοί. Το Q1 αυξάνεται 67 φορές (4,9 s → 326 s). Το Q3 (συνάρτηση παραθύρου) αποτυγχάνει στη βαθμίδα T10. Τα Q2–Q5 εμφανίζουν υπεργραμμική κλιμάκωση στις βαθμίδες T8–T10.*

8.6.2 Χρόνοι ερωτημάτων αναφοράς στη batched φάση

Το Σχήμα 8.4 παρουσιάζει τα αντίστοιχα αποτελέσματα για τους ομαδοποιημένους παραγωγούς. Οι χρόνοι batched είναι συγκρίσιμοι ή ελαφρώς χαμηλότεροι από τους nonbatched.

Ο Πίνακας 8.8 παρουσιάζει τα αντίστοιχα αποτελέσματα σε μορφή πίνακα.

Πίνακας 8.8: *Χρόνος ερωτημάτων Trino (p95, σε δευτερόλεπτα) ανά κλίμακα, ομαδοποιημένοι παραγωγοί.*

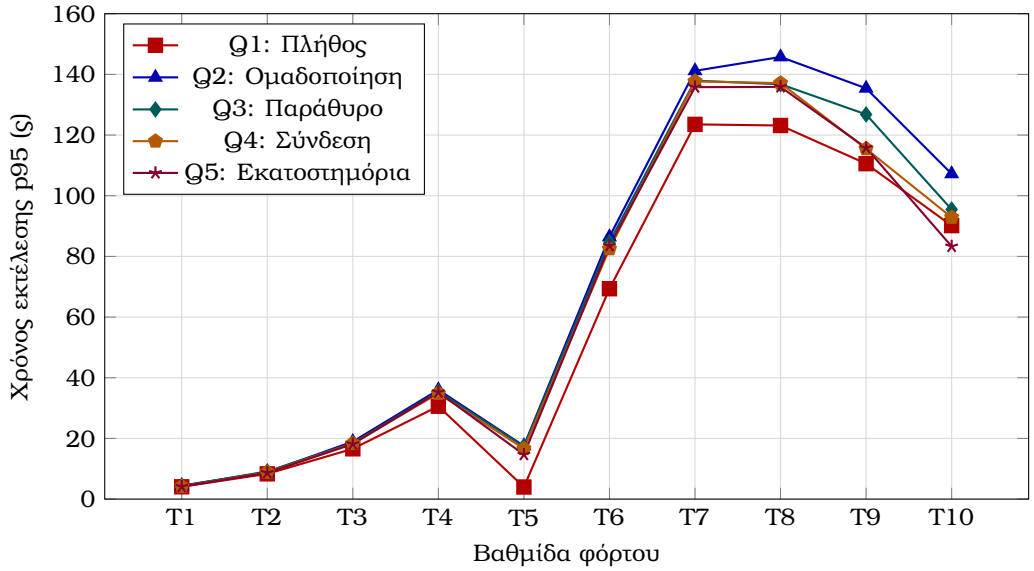
Tier	Q1 (s)	Q2 (s)	Q3 (s)	Q4 (s)	Q5 (s)	Αποτ.[†]
T1	4,06	4,43	4,31	4,15	4,10	0
T2	8,33	8,88	9,13	8,81	8,66	0
T3	16,58	18,95	18,48	18,32	18,15	0
T4	30,61	36,01	35,50	34,84	35,25	0
T5	3,98[*]	17,53	17,36	16,62	14,66	0
T6	69,35	86,42	84,73	82,48	83,50	0
T7	123,52	141,17	137,93	137,69	135,79	0
T8	123,15	145,74	136,76	137,12	135,88	0
T9	110,57	135,41	126,80	115,48	115,79	0
T10	90,14	107,20	95,47	92,85	83,29	0

[†]Η στήλη Αποτ. δίνει το πλήθος αποτυχημένων εκτελέσεων ερωτημάτων Trino στη βαθμίδα φόρτου, επί συνόλου 25 εκτελέσεων (5 ερωτήματα × 5 εκτελέσεις).

[*]Η τιμή Q1 στη B5 είναι ασυνήθιστα χαμηλή τιμή σε σχέση με την υπόλοιπη σειρά (T4=30,61 s, T6=69,35 s) πιθανόν να αποτυπώθηκε από το πρώτο πέρασμα του ερωτήματος μετά από cache miss/file pruning και να αντανακλά μη αντιπροσωπευτικό cold-start χαρακτηριστικό.

8.6.3 Ανάλυση κλιμάκωσης ερωτημάτων

Τα ερωτήματα Q1-Q5 αποκαλύπτουν σημαντικές διαφορές στη συμπεριφορά κλιμάκωσης. Το Q1, κατά το οποίο μετράται το πλήθος γραμμών του ακατέργαστου πίνακα, εμφανίζει σχεδόν γραμμική αύξηση, καθώς σαρώνει μόνο μεταδεδομένα αρχείων Parquet. Τα Q2 και



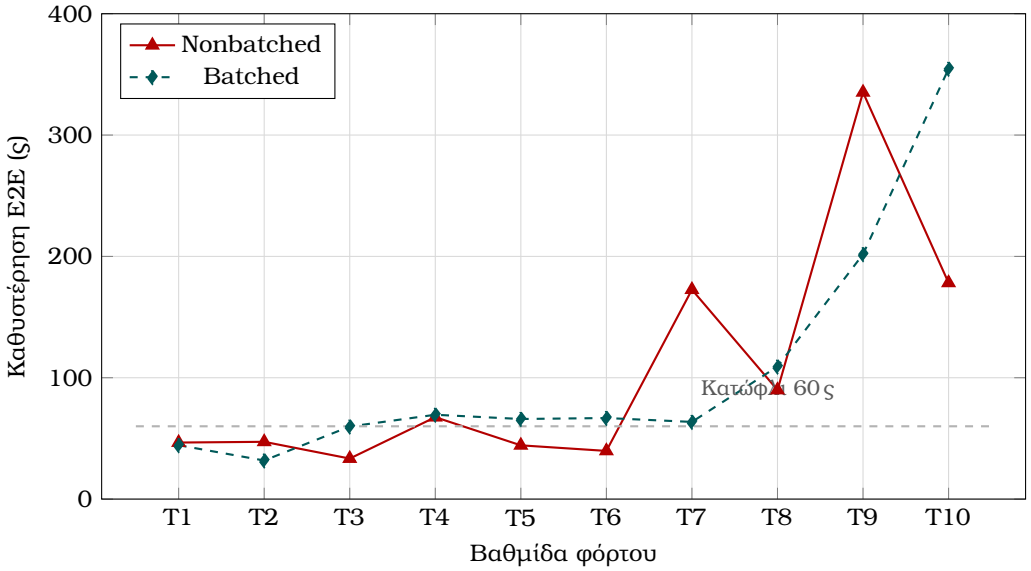
Σχήμα 8.4: *Χρόνοι ερωτημάτων Trino (p95) ανά βαθμίδα, ομαδοποιημένοι παραγωγοί. Όλα τα ερωτήματα Q1–Q5 ολοκληρώνονται επιτυχώς σε όλες τις βαθμίδες, συμπεριλαμβανομένου του Q3 στη T10 όπου η μη ομαδοποιημένη φάση αποτυγχάνει. Οι χρόνοι batched είναι 20–70% χαμηλότεροι από τους nonbatched στις βαθμίδες T9–T10.*

Q5, ερωτήματα ομαδοποίησης και διαλογής εκατοστημόριων, εμφανίζουν υπεργραμμική κλιμάκωση στις βαθμίδες T8-T10, καθώς ο όγκος δεδομένων υπερβαίνει τη χωρητικότητα heap των Trino workers (2GB) και αυξάνεται η χρήση δίσκου (spill-to-disk) [24]. Το Q3, στο οποίο εφαρμόζεται συνάρτηση παραθύρου, αποδεικνύεται το βαρύτερο ερώτημα καθώς απαιτεί ταξινόμηση και partition (PARTITION BY home_id ORDER BY ts) ολόκληρου του συνόλου δεδομένων, με αποτέλεσμα πλήρη κορεσμό μνήμης στη βαθμίδα N10, αφού υπάρχουν περίπου 212M (εκατομμύρια) εγγραφές ακατέργαστων δεδομένων, και αποτυχία εκτέλεσης στους μη ομαδοποιημένους παραγωγούς, ενώ στους ομαδοποιημένους (B10) ολοκληρώνεται επιτυχώς (95,47 s). Επιπλέον, στη βαθμίδα T9 nonbatched καταγράφονται 4 αποτυχίες Trino κατά την εκτέλεση των ερωτημάτων αναφοράς, αντανακλώντας την παρατεταμένη πίεση μνήμης στους workers κατά τη διάρκεια του πειράματος. Τέλος, το Q4 (JOIN) κλιμακώνεται ικανοποιητικά στις χαμηλές βαθμίδες αλλά αυξάνεται δραστικά στις υψηλές, καθώς ο ακατέργαστος πίνακας μεγαλώνει ενώ ο δεκάλεπτος παραμένει μικρός.

8.7 Καθυστέρηση από άκρο σε άκρο

Το Σχήμα 8.5 παρουσιάζει την καθυστέρηση από άκρο σε άκρο, E2E latency, ανά βαθμίδα για τις δύο στρατηγικές παραγωγών. Το metric αυτό αποκαλύπτει το bottleneck κατανάλωσης, το οποίο δεν είναι ορατό στον ρυθμό μηνυμάτων.

Στους μη ομαδοποιημένους παραγωγούς, η καθυστέρηση E2E παραμένει κάτω από 90 s στις βαθμίδες T1-T6 και T8. Στη T7 όμως εκτοξεύεται στα 172,6 s καθώς το Flink δεν προλαβαίνει να καταναλώσει τα μηνύματα με τον ρυθμό που φτάνουν στο Kafka, με αποτέλεσμα να συσσωρεύεται μεγάλη ουρά αδιάβαστων μηνυμάτων, δηλαδή consumer lag, ενώ ταυτόχρονα ένα αποτυχημένο checkpoint αναγκάζει την εργασία Flink να επανέλθει στην τελευταία



Σχήμα 8.5: *Καθυστέρηση από άκρο σε άκρο (E2E) ανά βαθμίδα. Στους μη ομαδοποιημένους παραγωγούς, η καθυστέρηση κορυφώνεται στη T9 (335,2 s) και υποχωρεί στη T10 (178,2 s). Οι ομαδοποιημένοι παραγωγοί παρουσιάζουν περιορισμένη αλλά σταθερή αύξηση (≤354,6 s στη T10).*

έγκυρη αποθηκευμένη κατάσταση και να επανεπεξεργαστεί δεδομένα. Η καθυστέρηση κορυφώνεται στη T9 στα 335,2 s, όπου το ίδιο φαινόμενο επαναλαμβάνεται σε μεγαλύτερη κλίμακα καθώς σημειώνονται 13 αποτυχημένα checkpoint και 9 ανακάμψεις έπειτα από αυτά, και κάθε ανάκαμψη επαναφέρει την εργασία σε προηγούμενο σημείο και σταματά προσωρινά την κατανάλωση των μηνυμάτων Kafka [149].

Στη T10 η καθυστέρηση υποχωρεί στα 178,2 s, χάρη στον κορεσμό του παραγωγού. Ο όρος «κορεσμός» εδώ σημαίνει ότι ο παραγωγός έχει φτάσει το όριο της επεξεργαστικής του ικανότητας και δεν μπορεί πλέον να παράγει μηνύματα ταχύτερα αφού ο χρόνος κάθε κύκλου παραγωγής σταθεροποιείται γύρω στα 0,42 s κατά μέσο όρο, χωρίς να ξεπερνά το 1,40 s. Έτσι ο ρυθμός με τον οποίο εισέρχονται δεδομένα στο Kafka παύει να αυξάνεται και σταθεροποιείται σε επίπεδο που το Flink μπορεί να εξυπηρετήσει, οπότε δεν συσσωρεύεται άλλη καθυστέρηση.

Αντίθετα, οι ομαδοποιημένοι παραγωγοί εμφανίζουν πιο ομαλή κλιμάκωση της καθυστέρησης, με τιμές 109,2 s στη T8, 201,8 s στη T9 και 354,6 s στη T10. Η ομαδοποίηση συμπιέζει πολλά μηνύματα σε λίγα αιτήματα του πρωτοκόλλου Transmission Control Protocol (TCP) και έτσι ασκεί μικρότερη πίεση στο Kafka. Το όφελος αυτό όμως αντισταθμίζεται από απότομες αυξήσεις της χρήσης μνήμης στο Flink TaskManager που τροφοδοτεί τα partitions του Paimon sink.

Ο Πίνακας 8.9 συνοψίζει τη σύγκριση καθυστέρησης μεταξύ των δύο στρατηγικών.

8.8 Κατανάλωση πόρων ανά υποσύστημα

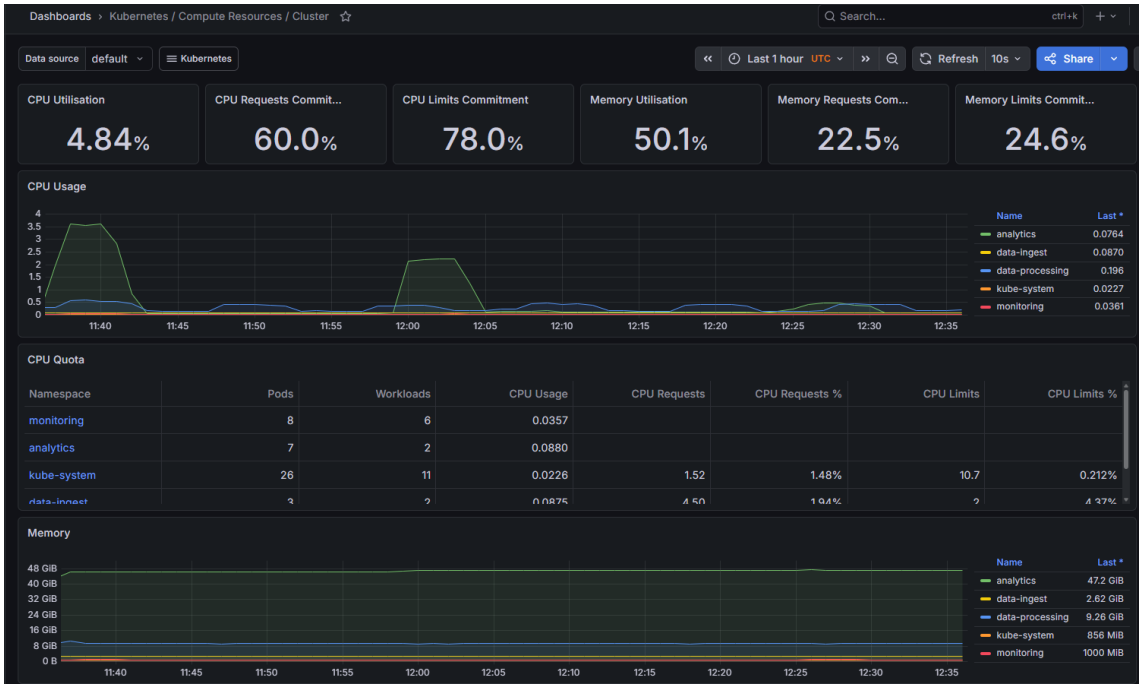
Οι μετρήσεις που παρουσιάζονται στις επόμενες υποενότητες αντλούνται από τα ενσωματωμένα dashboards του kube-prometheus-stack. Οι Εικόνες 8.1 και 8.2 παρουσιάζουν

Πίνακας 8.9: *Σύγκριση της καθυστέρησης από άκρο σε άκρο (E2E) του pipeline, μετρημένη σε δευτερόλεπτα ανά βαθμίδα φόρτου, μεταξύ μη ομαδοποιημένων (N) και ομαδοποιημένων (B) παραγωγών δεδομένων. Η στήλη «Μείωση» δείχνει την ποσοστιαία μείωση της καθυστέρησης που επιτυγχάνουν οι ομαδοποιημένοι παραγωγοί, όπου υπάρχει.*

Tier	E2E N (s)	E2E B (s)	Μείωση (%)
T1	46,6	44,4	5
T2	47,2	31,8	33
T3	33,4	59,8	-
T4	67,5	69,5	-
T5	44,3	66,0	-
T6	39,7	66,8	-
T7	172,6	63,6	63
T8	89,9	109,2	-
T9	335,2	201,8	40
T10	178,2	354,6	-

Η παύλα (‘-’) δηλώνει βαθμίδες στις οποίες οι ομαδοποιημένοι παραγωγοί δεν μείωσαν την καθυστέρηση, δηλαδή αυτή ήταν συγκρίσιμη ή μεγαλύτερη από εκείνη των μη ομαδοποιημένων.

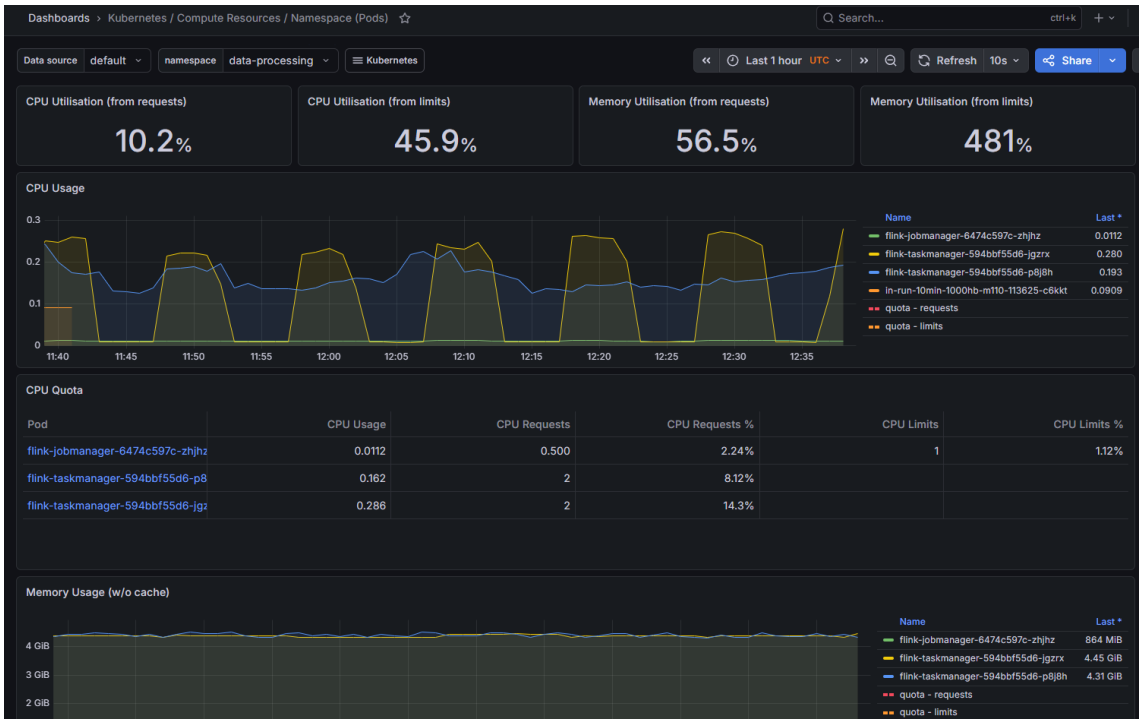
ενδεικτικά παραδείγματα στιγμιοτύπων τέτοιων πινάκων σε επίπεδο cluster και σε επίπεδο pod, αντίστοιχα.



Εικόνα 8.1: *Πίνακας ελέγχου kube-prometheus-stack (Compute Resources / Cluster) με τη χρήση CPU και μνήμης ανά namespace, όπου το data-processing κυριαρχεί.*

8.8.1 Σύγκριση Kafka CPU μεταξύ μη ομαδοποιημένων και ομαδοποιημένων παραγωγών

Το Σχήμα 8.6 απεικονίζει τη συνολική κατανάλωση CPU των δύο Kafka brokers. Η διαφορά μεταξύ των δύο στρατηγικών παραγωγής δεδομένων είναι δραματική. Συγκεκριμένα στη βαθμίδα T10, οι μη ομαδοποιημένοι παραγωγοί απαιτούν 2,46 πυρήνες Kafka CPU, ενώ



Εικόνα 8.2: *Πίνακας ελέγχου kube-prometheus-stack (Compute Resources / Pod) με τη χρήση CPU και μνήμης των pods Flink TaskManager εντός του namespace data-processing.*

οι ομαδοποιημένοι μόλις 0,15 πυρήνες, δηλαδή μείωση περίπου 16 φορές.

Η σημαντική μείωση CPU, περίπου 16 φορές, στους ομαδοποιημένους παραγωγούς εξηγείται από τη μείωση του αριθμού μηνυμάτων TCP αφού αντί να στέλνεται κάθε μήνυμα ξεχωριστά, η ομαδοποίηση συγκεντρώνει πολλαπλά μηνύματα σε ένα μήνυμα, μειώνοντας τον φόρτο επεξεργασίας TCP/TLS (Transport Layer Security) [234] στους brokers [21].

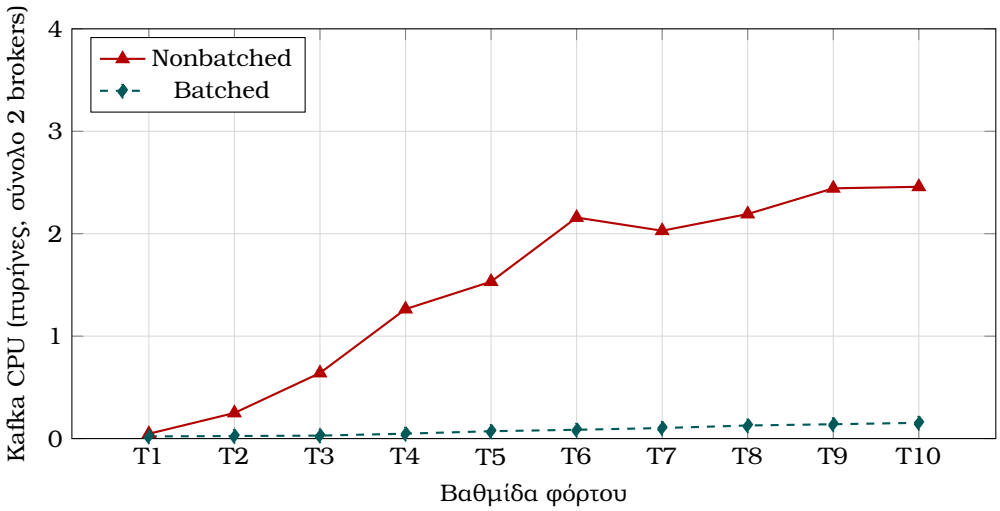
Ο Πίνακας 8.10 παρουσιάζει αναλυτικά τους μέσους πόρους του Kafka broker-0, ενός εκ των δύο brokers, ανά κλίμακα για τη μη ομαδοποιημένη φάση. Η χρήση CPU αυξάνεται σχεδόν γραμμικά μέχρι τη βαθμίδα T6 (από 0,025 σε 1,22 πυρήνες) και στη συνέχεια σταθεροποιείται μεταξύ 1,1 και 1,3 πυρήνων στις βαθμίδες T7 έως T10, καθώς το κόστος επεξεργασίας ανά αίτημα αποσβένεται στα μεγαλύτερα φορτία. Αντίθετα, η μνήμη, η δικτυακή κίνηση και ο αποθηκευτικός χώρος PVC συνεχίζουν να κλιμακώνονται με τον ρυθμό μηνυμάτων έως τη μέγιστη βαθμίδα, όπως αναλύεται και στην επόμενη υποενότητα.

8.8.2 Μνήμη και αποθηκευτικός χώρος Kafka

Η χρήση μνήμης στο Kafka αυξάνεται γραμμικά από 1,35GiB (T1) σε 21,34GiB (N10) και 17,98GiB (B10) στον broker-0. Η χρήση αποθηκευτικού χώρου PVC ακολουθεί ανάλογη αύξηση από 0,1GiB σε 67,4GiB (N10) και 41,2GiB (B10).

8.8.3 Flink

Οι TMs αποτελούν τους μεγαλύτερους καταναλωτές πόρων. Η χρήση CPU κυμαίνεται μεταξύ 0,02 και 3,12 πυρήνων ανά TM στους μη ομαδοποιημένους παραγωγούς. Στη βαθμίδα T10, ο πιο φορτωμένος TM χρησιμοποιεί 3,12 πυρήνες (TM-1 N10), υπερβαίνοντας το

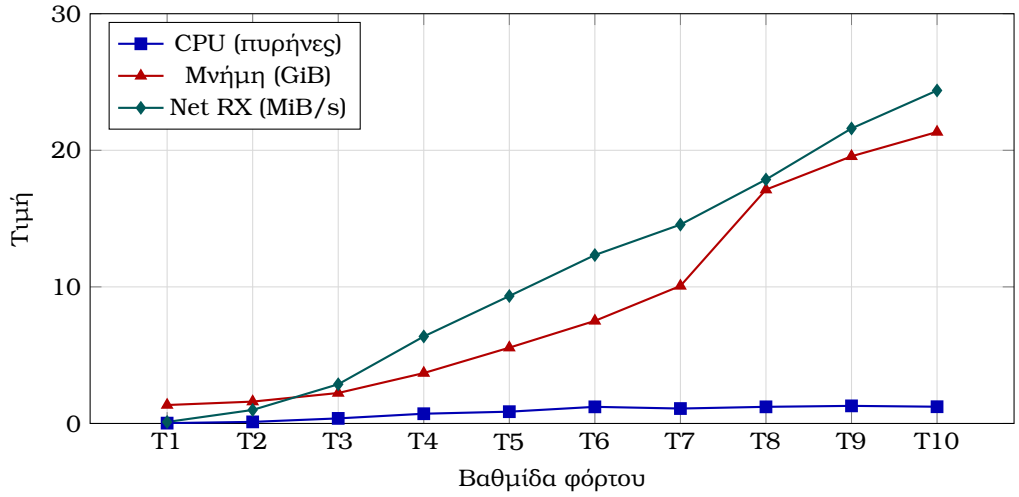


Σχήμα 8.6: *Κατανάλωση CPU Kafka (σύνολο 2 brokers) ανά βαθμίδα. Οι ομαδοποιημένοι παραγωγοί μειώνουν τη χρήση CPU κατά περίπου 16 φορές στη βαθμίδα T10 (2,46 → 0,15 πυρήνες), λόγω λιγότερων αιτημάτων TCP.*

Πίνακας 8.10: *Μέσοι πόροι του Kafka broker-0 ανά κλίμακα (παράθυρο 30 min, μη ομαδοποιημένοι παραγωγοί).*

Tier	CPU (πυρ.)	Μνήμη (GiB)	Net RX (MiB/s)	Net TX (MiB/s)	PVC (GiB)
T1	0,025	1,35	0,11	0,11	0,1
T2	0,116	1,60	0,99	0,87	0,9
T3	0,371	2,23	2,87	2,59	2,8
T4	0,713	3,69	6,37	5,82	6,9
T5	0,863	5,55	9,33	8,50	12,0
T6	1,217	7,51	12,33	11,23	17,8
T7	1,093	10,07	14,56	13,02	27,0
T8	1,218	17,12	17,86	16,17	40,9
T9	1,288	19,56	21,60	18,00	53,1
T10	1,226	21,34	24,38	20,87	67,4

Η στήλη CPU δίνει τη μέση χρήση επεξεργαστή σε πυρήνες.
Η στήλη Μνήμη δίνει τη μέση χρήση μνήμης σε GiB.
Οι στήλες Net RX και Net TX δίνουν τη μέση εισερχόμενη και εξερχόμενη δικτυακή κίνηση σε MiB/s αντίστοιχα.
Η στήλη PVC δίνει τον κατειλημμένο αποθηκευτικό χώρο του persistent volume σε GiB στο τέλος του παραθύρου μέτρησης.



Σχήμα 8.7: *Κατανάλωση πόρων ανά βαθμίδα σε έναν Kafka broker σε επίπεδο CPU, μνήμης και εισερχόμενης δικτυακής κίνησης.*

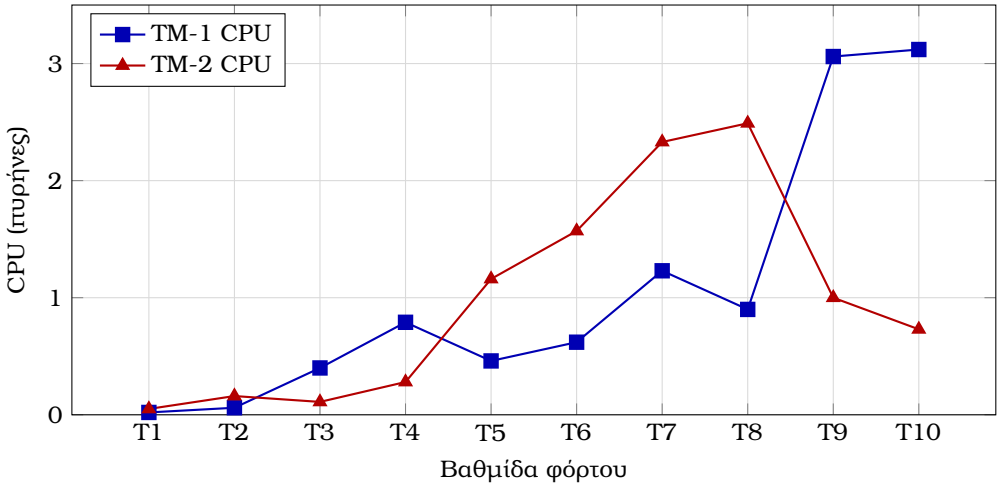
αίτημα (request) 2 πυρήνων χάρη στη burstable ρύθμιση, δηλαδή χωρίς αυστηρά ορισμένο άνω όριο (limit) στους δεσμευμένους πόρους (βλ. §9.1.1). Η μνήμη παραμένει σχετικά σταθερή στα 3,2-4,6 GiB ανά TM, λόγω σταθερού μεγέθους της περιοχής μνήμης εκτός σωρού JVM (off-heap), η οποία δεσμεύεται στατικά κατά την εκκίνηση και δεν μεταβάλλεται με τον φόρτο, 512 m αφορούν την απευθείας ανακατεύθυνση δεδομένων μεταξύ τελεστών εργασίας και 256 m δεσμεύονται για τα εσωτερικά του πλαισίου Flink (framework). Το κύριο bottleneck εντοπίζεται στην εγγραφή στο Paimon όπου τα checkpoints πυροδοτούν flush στο ADLS, και σε υψηλό ρυθμό εισαγωγής, ο χρόνος checkpoint αυξάνεται σημαντικά [149].

Ο Πίνακας 8.11 παρουσιάζει αναλυτικά τους πόρους των δύο TaskManagers ανά κλίμακα. Η ασύμμετρη κατανομή φόρτου μεταξύ τους, η οποία εναλλάσσεται από βαθμίδα σε βαθμίδα, οφείλεται στην ανακατανομή των partitions του Kafka σε κάθε επανεκκίνηση της εργασίας εισαγωγής, με αποτέλεσμα ο εκάστοτε πιο φορτωμένος TM να συγκεντρώνει το μεγαλύτερο μέρος της CPU και της εισερχόμενης κίνησης.

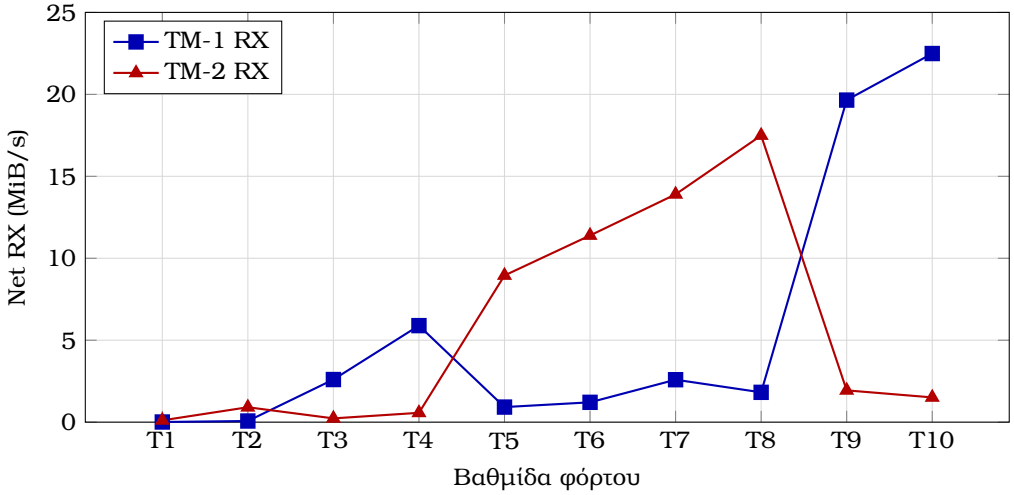
Πίνακας 8.11: *Μέσοι πόροι των Flink TM-1 και TM-2 ανά κλίμακα στους μη ομαδοποιημένους παραγωγούς.*

	TM-1			TM-2		
Tier	**CPU**	**Μνήμ.**	**RX**	**CPU**	**Μνήμ.**	**RX**
T1	0,02	2,63	0,01	0,05	4,38	0,11
T2	0,06	2,89	0,07	0,16	3,94	0,91
T3	0,40	4,07	2,60	0,11	3,06	0,23
T4	0,79	4,21	5,89	0,28	3,22	0,57
T5	0,46	3,48	0,92	1,16	4,25	8,95
T6	0,62	3,56	1,21	1,57	4,29	11,39
T7	1,23	3,74	2,59	2,33	4,57	13,90
T8	0,90	3,87	1,82	2,49	4,31	17,49
T9	3,06	4,35	19,65	1,00	3,91	1,94
T10	3,12	4,33	22,49	0,73	3,89	1,51

Η στήλη CPU δίνει τη μέση χρήση επεξεργαστή σε πυρήνες.
Η στήλη Μνήμ. δίνει τη μέση χρήση μνήμης σε GiB.
Η στήλη RX δίνει τη μέση εισερχόμενη δικτυακή κίνηση σε MiB/s.



Σχήμα 8.8: *Χρήση CPU των δύο Flink TaskManagers ανά κλίμακα.*



Σχήμα 8.9: *Εισερχόμενη δικτυακή κίνηση (Net RX) των δύο Flink TaskManagers ανά κλίμακα. Η εναλλασσόμενη κατανομή φόρτου μεταξύ TM-1 και TM-2 οφείλεται στην ανακατανομή partitions Kafka.*

8.8.4 Trino

Η κατανάλωση πόρων του Trino εξαρτάται από τη συχνότητα ερωτημάτων δηλαδή κυρίως από το ρυθμό ανανέωσης των Grafana dashboards. Σε κατάσταση ηρεμίας, η κατανάλωση CPU είναι ελάχιστη (< 100m), ενώ κατά την εκτέλεση ερωτημάτων SQL αυξάνεται σημαντικά [24].

8.8.5 Go παραγωγοί IoT δεδομένων

Η χρήση πόρων ανά pod παραγωγού είναι μικρή, περίπου 50 m CPU και 30 Mi (Mebibytes [227]) μνήμης. Ωστόσο, καθώς στις υψηλές βαθμίδες λειτουργούν ταυτόχρονα πέντε pods παραγωγών, η κατανάλωση αθροίζεται και η συνολική χρήση CPU φτάνει περίπου τα 250 m.

Ο Πίνακας 8.12 παρουσιάζει τον μέσο και μέγιστο χρόνο κύκλου ανά κλίμακα. Ο μέσος χρόνος κύκλου παραμένει σαφώς κάτω από το κατώφλι του 1 s σε όλες τις βαθμίδες, από 0,003 s στη T1 έως 0,42 s στη T10. Μόνο στη T10 ο μέγιστος χρόνος κύκλου (1,401 s) υπερβαίνει το κατώφλι, υποδεικνύοντας κορεσμό του παραγωγού στη μέγιστη κλίμακα. Σημειώνεται ότι το φορτίο ανά pod στη T10 (3.500 κατοικίες) είναι ίδιο με εκείνο της T4, η οποία δεν εμφανίζει κορεσμό, γεγονός που υποδηλώνει ότι ο κορεσμός σχετίζεται με τον συνολικό φόρτο του συστήματος και όχι μόνο με το φορτίο ανά pod.

Πίνακας 8.12: *Μέσος και μέγιστος χρόνος κύκλου των Go παραγωγών ανά κλίμακα (μη ομαδοποιημένοι παραγωγοί).*

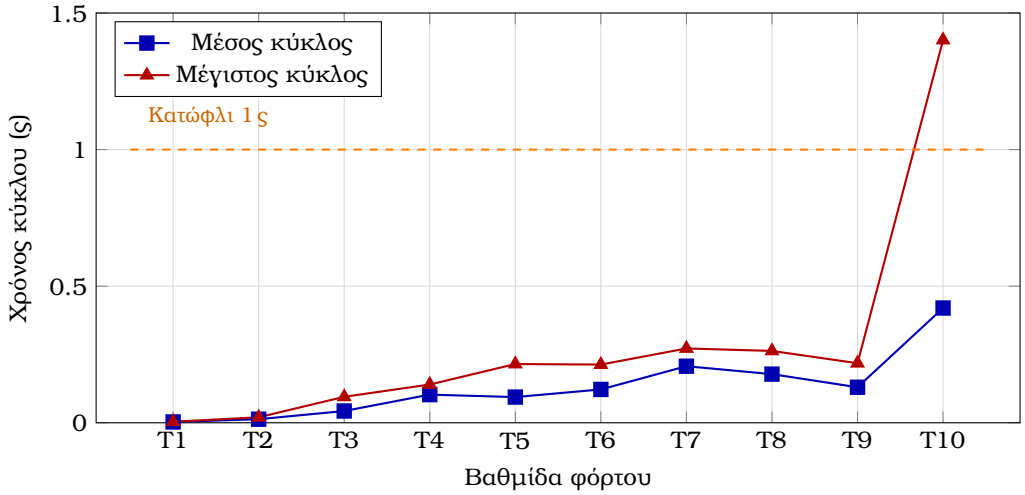
Tier	Κατοικ. / Pod	Μέσος κύκλος (s)	Μέγ. κύκλος (s)
T1	50	0,003	0,004
T2	500	0,013	0,020
T3	1.500	0,043	0,095
T4	3.500	0,103	0,140
T5	2.750	0,094	0,215
T6	2.500	0,122	0,213
T7	3.333	0,207	0,272
T8	3.125	0,178	0,263
T9	3.000	0,130	0,218
T10	3.500	0,420	1,401

Η στήλη Κατοικ. / Pod δίνει τον αριθμό κατοικιών που εξυπηρετεί κάθε pod παραγωγού.
Οι στήλες Μέσος κύκλος και Μέγ. κύκλος δίνουν τον μέσο και μέγιστο χρόνο ολοκλήρωσης ενός κύκλου παραγωγής δεδομένων σε δευτερόλεπτα, με κατώφλι κορεσμού το 1 s.

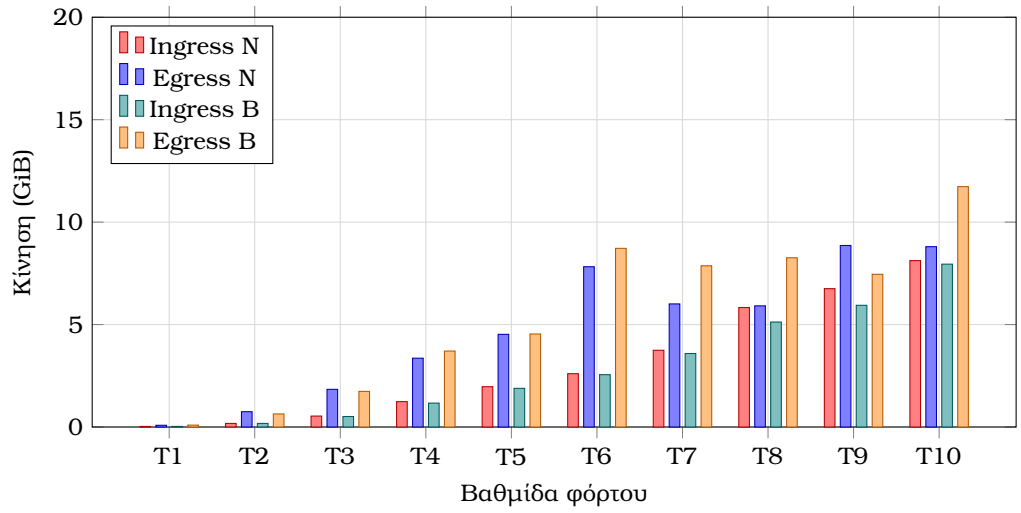
8.8.6 Azure Storage Account metrics

Το Σχήμα 8.11 παρουσιάζει τη χρήση του ADLS ανά βαθμίδα. Η εισερχόμενη κίνηση (ingress) αντανακλά τις εγγραφές Paimon (checkpoint flush), ενώ η εξερχόμενη (egress) αντανακλά τις αναγνώσεις Trino.

Η καθυστέρηση E2E του Azure Storage αυξάνεται από 22,8 ms (T1) σε 81,5 ms (N6) και 66,4 ms (N10), αντανακλώντας τη συμφόρηση λόγω ταυτόχρονων αιτημάτων PUT και GET. Το φαινόμενο αυτό είναι χαρακτηριστικό της φύσης του ADLS ως object storage, καθώς σε αντίθεση με ένα τοπικό σύστημα αρχείων, κάθε λειτουργία δεν εκτελείται τοπικά



Σχήμα 8.10: *Μέσος και μέγιστος χρόνος κύκλου των Go παραγωγών ανά κλίμακα (μη ομαδοποιημένοι). Η διακεκομμένη γραμμή σηματοδοτεί το κατώφλι του 1 s, πέρα από το οποίο η βαθμίδα βρίσκεται σε κατάσταση κορεσμού. Μόνο η βαθμίδα T10 εμφανίζει κορεσμό (max_cycle=1,401 s).*



Σχήμα 8.11: *Δικτυακή κίνηση Azure Storage Account (ADLS) ανά βαθμίδα. Εισερχόμενη (ingress) κίνηση από εγγραφές Paimon και εξερχόμενη (egress) κίνηση από αναγνώσεις Trino.*

αλλά υλοποιείται ως ξεχωριστή κλήση HTTP προς ένα REST API, με το αίτημα PUT να αντιστοιχεί στην εγγραφή δεδομένων και το αίτημα GET στην ανάγνωσή τους. Όσο αυξάνεται ο αριθμός των ταυτόχρονων αυτών αιτημάτων υπό υψηλό φόρτο, τόσο επιβαρύνεται ο χρόνος εξυπηρέτησής τους. Η αιχμή στη N6 (81,5 ms) συμπίπτει με ταυτόχρονη εκτέλεση εργασιών δεκάλεπτης συγκέντρωσης κατά τη διάρκεια του παραθύρου μέτρησης.

Οι Πίνακες 8.13 και 8.14 παρουσιάζουν αναλυτικά τις μετρήσεις Azure Storage Account ανά κλίμακα για τις δύο στρατηγικές παραγωγών. Και στις δύο φάσεις ο εισερχόμενος όγκος δεδομένων κλιμακώνεται σχεδόν γραμμικά με τη βαθμίδα φόρτου, ενώ ο εξερχόμενος αυξάνεται λιγότερο ομοιόμορφα, καθώς εξαρτάται από τον όγκο που σαρώνουν τα ερωτήματα Trino σε κάθε παράθυρο. Το πλήθος συναλλαγών παραμένει σχετικά σταθερό σε όλες τις βαθμίδες, περίπου μεταξύ 11.600 και 18.700, επειδή καθορίζεται κυρίως από τη συχνότητα των checkpoints και των ερωτημάτων και όχι από τον όγκο δεδομένων, ενώ η διαθεσιμότητα της υπηρεσίας διατηρείται πρακτικά στο 100%.

Πίνακας 8.13: *Μετρήσεις Azure Storage Account ανά κλίμακα (30 min, μη ομαδοποιημένοι παραγωγοί).*

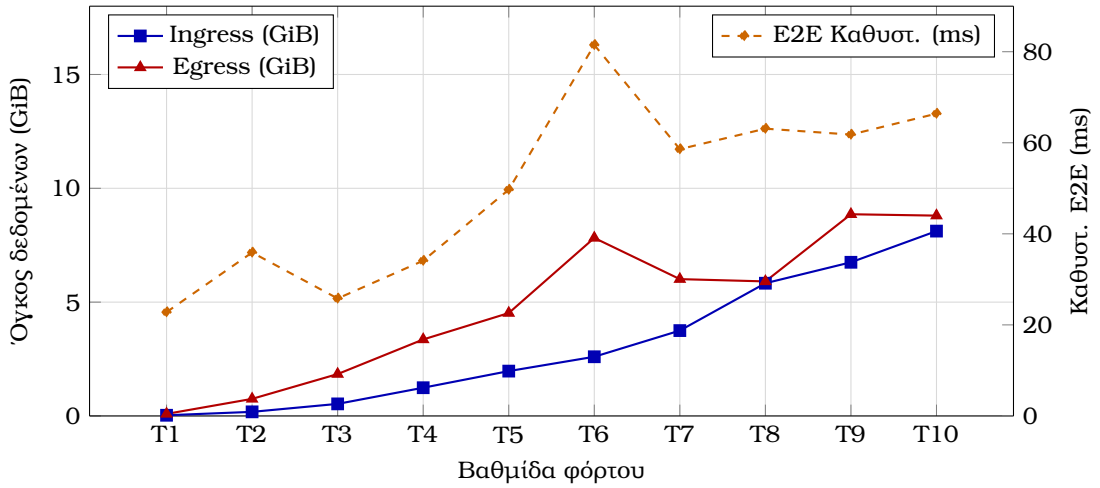
Tier	Συναλ.	Ingress (GiB)	Egress (GiB)	E2E (ms)	Διαθ. (%)
T1	12.376	0,03	0,09	22,8	100,0
T2	13.026	0,18	0,75	35,9	100,0
T3	16.015	0,53	1,84	25,8	100,0
T4	12.067	1,24	3,36	34,1	100,0
T5	12.017	1,97	4,52	49,7	100,0
T6	15.132	2,60	7,82	81,5	100,0
T7	11.603	3,75	6,01	58,6	100,0
T8	13.546	5,83	5,91	63,1	100,0
T9	15.536	6,75	8,86	61,8	100,0
T10	15.843	8,12	8,80	66,4	100,0

Η στήλη Συναλ. δίνει το πλήθος συναλλαγών (αιτήματα REST API) στο παράθυρο μέτρησης.
Οι στήλες Ingress και Egress δίνουν τον εισερχόμενο και εξερχόμενο όγκο δεδομένων σε GiB αντίστοιχα.
Η στήλη E2E δίνει τη μέση καθυστέρηση εξυπηρέτησης αιτημάτων από άκρο σε άκρο σε ms.
Η στήλη Διαθ. δίνει τη διαθεσιμότητα της υπηρεσίας αποθήκευσης ως ποσοστό.

Πίνακας 8.14: *Μετρήσεις Azure Storage Account ανά κλίμακα (30 min, ομαδοποιημένοι παραγωγοί).*

Tier	Συναλ.	Ingress (GiB)	Egress (GiB)	E2E (ms)	Διαθ. (%)
T1	15.805	0,03	0,09	19,3	100,0
T2	16.085	0,18	0,64	24,6	100,0
T3	17.291	0,51	1,74	25,4	100,0
T4	14.301	1,17	3,71	37,5	100,0
T5	12.752	1,89	4,54	54,0	100,0
T6	16.777	2,56	8,72	52,6	99,99
T7	13.705	3,59	7,87	55,9	100,0
T8	16.167	5,12	8,26	52,3	100,0
T9	14.675	5,94	7,45	51,6	100,0
T10	18.698	7,95	11,73	54,1	100,0

Η στήλη Συναλ. δίνει το πλήθος συναλλαγών (αιτήματα REST API) στο παράθυρο μέτρησης.
Οι στήλες Ingress και Egress δίνουν τον εισερχόμενο και εξερχόμενο όγκο δεδομένων σε GiB αντίστοιχα.
Η στήλη E2E δίνει τη μέση καθυστέρηση εξυπηρέτησης αιτημάτων από άκρο σε άκρο σε ms.
Η στήλη Διαθ. δίνει τη διαθεσιμότητα της υπηρεσίας αποθήκευσης ως ποσοστό.



Σχήμα 8.12: *Μετρήσεις Azure Storage Account ανά βαθμίδα. Εισερχόμενα και εξερχόμενα δεδομένα (μπλέ και κόκκινη γραμμή αντίστοιχα) και καθυστέρηση E2E (κίτρινη διακοπτόμενη γραμμή).*

8.9 Σταθερότητα checkpoints Flink

Οι αστοχίες checkpoint εμφανίζονται από τη βαθμίδα T7 και άνω, και επιδεινώνονται μονοτονικά μέχρι την T9. Η σημαντικότερη περίπτωση είναι η N9 (15.000 κατοικίες, μη ομαδοποιημένοι παραγωγοί), με 16 αποτυχημένα checkpoints και 12 επαναφορές, σε συνδυασμό με καθυστέρηση E2E 1.060,8 s, ενώ το παράθυρο μέτρησης διαρκεί μόνο 1.800 s. Η συμπεριφορά αυτή υποδεικνύει επαναλαμβανόμενη αυτόματη ανάκαμψη, failover, με ελάχιστο πραγματικό χρόνο εξυπηρέτησης. Η B9 εμφανίζει 7 αποτυχίες και 5 επαναφορές, η N10 9 αποτυχίες και 4 επαναφορές σε συνδυασμό με κορεσμό παραγωγού, και η B10 5 αποτυχίες και 5 επαναφορές. Οι N7 και B7 εμφανίζουν επίσης αστοχίες μικρότερης κλίμακας [149].

Ο Πίνακας 8.15 παρουσιάζει τη συμπεριφορά checkpoints ανά βαθμίδα.

Πίνακας 8.15: *Σημεία ελέγχου Flink ανά κλίμακα (παράθυρο 30 min).*

	Nonbatched				**Batched**			
Tier	**OK**	**Αποτ.**	**Επαν.**	**FO**	**OK**	**Αποτ.**	**Επαν.**	**FO**
T1	33	0	0	N	12	0	0	N
T2	35	0	0	N	34	0	0	N
T3	36	0	0	N	36	0	0	N
T4	37	0	0	N	38	0	0	N
T5	41	0	0	N	39	0	0	N
T6	46	0	0	N	45	0	0	N
T7	53	**1**	**5**	**Y**	47	**2**	**2**	**Y**
T8	64	**1**	0	N	57	0	0	N
T9	45	**16**	**12**	**Y**	55	**7**	**5**	**Y**
T10	67	**9**	**4**	**Y**	55	**5**	**5**	**Y**

Η στήλη OK δίνει το πλήθος επιτυχημένων checkpoints.
Η στήλη Αποτ. δίνει το πλήθος αποτυχημένων checkpoints.
Η στήλη Επαν. δίνει το πλήθος επαναφορών (restores) από checkpoint.
Η στήλη FO δηλώνει αν ενεργοποιήθηκε failover, με Y για ναι και N για όχι.
Οι τιμές αντιστοιχούν στο παράθυρο μέτρησης 30 min με διάστημα checkpoint 60 s.

8.10 Οριακές περιπτώσεις και αστοχίες

8.10.1 Ταυτόχρονες αστοχίες checkpoint στη βαθμίδα T7

Σε αντίθεση με προηγούμενες εκτελέσεις, στη βαθμίδα T7 (10.000 κατοικίες) εμφανίζονται αστοχίες checkpoint και στις δύο υπο-φάσεις. Συγκεκριμένα, το N7 εμφανίζει 1 αποτυχία και 5 επαναφορές και το B7 2 αποτυχίες και 2 επαναφορές, με τη στήλη FO του Πίνακα 8.15 να λαμβάνει τιμή Y και στις δύο περιπτώσεις, υποδεικνύοντας ότι οι αποτυχίες checkpoint πυροδότησαν αυτόματη επανεκκίνηση (failover) της εργασίας Flink. Η ενεργοποίηση failover και στις δύο φάσεις υποδηλώνει ότι το ζεύγος Flink TaskManagers φτάνει σε όριο πίεσης κοντά στις 50.000 msg/s ανεξαρτήτως στρατηγικής παραγωγού. Παρά τις αστοχίες, ο δείκτης SLO παρέμεινε σε επίπεδα 1,01-1,07 για τους ακατέργαστους πίνακες Paimon, αποδεικνύοντας ότι ο μηχανισμός αυτόματης ανάκαμψης του Flink λειτούργησε αποτελεσματικά. Η μεγάλη καθυστέρηση E2E στη N7 (172,6 s) σε σύγκριση με τη B7 (63,6 s) επιβεβαιώνει ότι το βάρος των αστοχιών μετατοπίζεται προς την μη ομαδοποιημένη πλευρά.

8.10.2 Αστοχίες checkpoint, failover και κορεσμός στην υπο-φάση N9

Η καθυστέρηση E2E των 1.060,8 s, σχεδόν το 59% του παραθύρου μέτρησης (1.800 s), υποδεικνύει ότι η εργασία Flink παρέμεινε σχεδόν στατική για το μεγαλύτερο μέρος του παραθύρου, αποτυγχάνοντας να καταναλώσει τα νέα μηνύματα Kafka όσο διαρκούσαν οι κύκλοι ανάκαμψης. Παρόλα αυτά, ο λόγος εγγραφών Paimon raw προς μηνύματα Kafka (στήλη SLO του Πίνακα 8.6) παρέμεινε 1,0056, δηλαδή το 100,56% των μηνυμάτων εγγράφηκε επιτυχώς στον πίνακα ακατέργαστων δεδομένων χωρίς καμία απώλεια εγγραφής κατά τους κύκλους ανάκαμψης. Η διατήρηση αυτής της ακεραιότητας οφείλεται στη σημασιολογία exactly-once (βλ. §4.4.2) και στον μηχανισμό deduplication του Paimon [149]. Επιπλέον, στη N9 καταγράφηκαν 4 αποτυχίες Trino κατά την εκτέλεση των ερωτημάτων αναφοράς, αντανακλώντας πίεση μνήμης στους Trino workers κατά τη διάρκεια του πειράματος.

Επιβεβαιωτική επανεκτέλεση Με δεδομένη την ακραία τιμή E2E των 1.060,8 s, η βαθμίδα N9 επανεκτελέστηκε με την ίδια διαμόρφωση, μεμονωμένα από την υπόλοιπη ακολουθία βαθμίδων. Η επανεκτέλεση παρήγαγε καθυστέρηση E2E 335,2 s με 13 αποτυχημένα checkpoints και 9 επαναφορές, έναντι 16 αποτυχημένων checkpoints και 12 επαναφορών της αρχικής εκτέλεσης. Η αναπαραγωγή του προτύπου αστοχιών checkpoint επιβεβαιώνει ότι το stress της βαθμίδας αποτελεί δομικό χαρακτηριστικό της κλίμακας φόρτου και όχι τυχαίο συμβάν. Αντίθετα, η μείωση της καθυστέρησης E2E κατά 68% υποδεικνύει ότι η αρχική τιμή 1.060,8 s δεν αντιπροσωπεύει σταθερή συμπεριφορά υπό φόρτο, αλλά μια παρατεταμένη εκδοχή των επαναλαμβανόμενων κύκλων αστοχίας και ανάκαμψης που περιγράφηκαν παραπάνω. Στην αρχική εκτέλεση η εργασία Flink παρέμεινε επί μεγάλο μέρος του παραθύρου σε διαδοχικές επανεκκινήσεις χωρίς ουσιαστική πρόοδο κατανάλωσης, πιθανώς λόγω προσωρινά αυξημένης καθυστέρησης του ADLS τη συγκεκριμένη χρονική περίοδο. Στην επανεκτέλεση δεν καταγράφηκαν αποτυχίες Trino, ενώ ο χρόνος ερωτήματος Q1 p95 μειώθηκε σε 252,8 s έναντι 300,6 s της αρχικής εκτέλεσης. Όλες οι αναφορές σε τιμή E2E T9 στο Σχήμα 8.5 και στους Πίνακες 8.5, 8.9 και 8.16 χρησιμοποιούν την επιβεβαιωμένη τιμή

της επανεκτέλεσης, δηλαδή 335,2 s. Η αρχική τιμή 1.060,8 s διατηρείται μόνο στην παρούσα αναλυτική περιγραφή των αστοχιών (§8.10.2) ως ιστορική αναφορά της πρώτης εκτέλεσης που οδήγησε στην επαλήθευση. Με την ίδια λογική, οι αναφορές σε αστοχίες checkpoint της N9 στο κεφάλαιο Συμπερασμάτων χρησιμοποιούν τις τιμές της επανεκτέλεσης (13 αποτυχίες, 9 επαναφορές), ενώ ο Πίνακας 8.15 διατηρεί τις τιμές της αρχικής εκτέλεσης (16 αποτυχίες και 12 επαναφορές), καθώς αποτυπώνει την πλήρη διαδοχική σειρά του πειράματος.

8.10.3 Κορεσμός παραγωγού και αποτυχία Q3 στη βαθμίδα N10

Στη βαθμίδα T10 (17.500 κατοικίες), η μη ομαδοποιημένη φάση N10 εμφάνισε δύο διακριτά συμπτώματα. Πρώτον, κορεσμό παραγωγού, με μέσο χρόνο κύκλου 0,42 s και μέγιστο 1,40 s, τιμή που υπερβαίνει το κατώφλι κορεσμού του 1 s. Δεύτερον, 5 αποτυχίες στο Trino, με χαρακτηριστικότερη την αποτυχία του ερωτήματος παραθύρου Q3 λόγω υπέρβασης της μνήμης heap, σταθερά ορισμένης στα 2 GB ανά worker, κατά την ταξινόμηση και partition περίπου 212 εκατομμυρίων εγγραφών ακατέργαστων δεδομένων. Αντίθετα, η ομαδοποιημένη φάση B10 ολοκλήρωσε επιτυχώς όλα τα ερωτήματα, συμπεριλαμβανομένου του Q3 σε 95,47 s, χωρίς αστοχίες στο Trino, παρά τις 5 αποτυχίες checkpoint και 5 επαναφορές. Αύξηση του heap ή χρήση spill-to-disk θα μετρίαζε το πρόβλημα [24].

8.11 Συζήτηση αποτελεσμάτων και εντοπισμός bottlenecks

8.11.1 Ανάλυση κλιμάκωσης

Σε αντίθεση με τις αρχικά αναμενόμενες τιμές, ο παρατηρούμενος ρυθμός μηνυμάτων δεν πέφτει κάτω από τον θεωρητικό σε καμία βαθμίδα, καθώς το Kafka δέχεται γραμμικά αυξανόμενο φόρτο με λόγο εισαγωγής 1,04 έως 1,18, επιβεβαιώνοντας ότι το επίπεδο ingestion δεν αποτελεί σημείο συμφόρησης. Το κύριο bottleneck εντοπίζεται στο επίπεδο αναλυτικής επεξεργασίας, δηλαδή στα ερωτήματα Trino, όπου οι χρόνοι εκτέλεσης στη μη ομαδοποιημένη φάση αυξάνονται κατά 67 φορές από τη βαθμίδα T1 έως τη T10, ενώ στην υψηλότερη βαθμίδα αποτυγχάνει το βαρύτερο ερώτημα Q3 [205]. Παράλληλα, η καθυστέρηση E2E εμφανίζει σημαντική αύξηση σε βαθμίδες άνω της T7 για τις μη ομαδοποιημένες φάσεις, κορυφούμενη στη N9 με την επιβεβαιωμένη τιμή 335,2 s, καθώς το Flink εισέρχεται σε επαναλαμβανόμενους κύκλους αστοχίας και ανάκαμψης checkpoint [151]. Αξιοσημείωτο εύρημα είναι ότι η ομαδοποιημένη φάση B10 ολοκληρώνει το Q3 σε 95,47 s, ταχύτερα από όλες τις βαθμίδες B6-B9, υποδηλώνοντας ότι στη μέγιστη κλίμακα η ομαδοποίηση δεδομένων οδηγεί σε βελτιωμένη οργάνωση Parquet με μεγαλύτερα και πιο αποδοτικά αρχεία.

8.11.2 Εντοπισμός bottlenecks

Η συνδυαστική ανάλυση των παραπάνω μετρήσεων εντοπίζει τρία σημεία συμφόρησης στο σύστημα, ενώ αποκλείει ένα τέταρτο υποψήφιο.

Το πρώτο σημείο αφορά τους πόρους των Flink TMs. Η διαμόρφωση 2 TMs × 4 slots παρέχει 8 διαθέσιμες θέσεις εργασιών (task slots), εκ των οποίων 4 δεσμεύονται από τη μόνιμη εργασία εισαγωγής ροής (streaming ingest), που καταναλώνει τα μηνύματα του Kafka

και τα γράφει ακατέργαστα στο Paimon, και 4 από τις περιοδικές εργασίες συγκέντρωσης (batch aggregation) που παράγουν τα συγκεντρωτικά επίπεδα. Καθεμία δηλαδή από τις δύο εργασίες εκτελείται με βαθμό παραλληλισμού (parallelism) 4, αξιοποιώντας 4 από τις 8 συνολικά θέσεις. Ωστόσο, η ανάλυση των αστοχιών checkpoint δείχνει ότι ο κυρίαρχος περιορισμός δεν είναι οι θέσεις εργασιών αυτές καθαυτές, αλλά το I/O προς το ADLS στο οποίο προσκρούουν.

Το δεύτερο, και σημαντικότερο, σημείο συμφόρησης είναι η διαδρομή εγγραφής Paimon/ADLS. Κάθε checkpoint του Flink πυροδοτεί εγγραφή αρχείων Parquet στο ADLS, με τον όγκο εγγραφών να φτάνει τα 8,1 GiB ανά παράθυρο 30 min στη βαθμίδα N10, ενώ η καθυστέρηση E2E του ADLS αυξάνεται από 22,8 ms στη T1 έως και 81,5 ms στις υψηλότερες βαθμίδες (Σχήμα 8.12). Η αυξανόμενη αυτή καθυστέρηση εγγραφής συμπίπτει χρονικά με τις αστοχίες checkpoint των βαθμίδων T7 και άνω.

Αντίθετα, η χωρητικότητα του Kafka δεν αποτελεί bottleneck [140]. Η διαμόρφωση 2 brokers με 10 partitions απορρόφησε τον γραμμικά αυξανόμενο φόρτο όλων των βαθμίδων με μηδενικές αποτυχίες παραγωγής. Καθοριστικό ρόλο σε αυτό διαδραμάτισε το γεγονός ότι η μνήμη των brokers είναι ρυθμισμένη ως burstable, δηλαδή με αίτημα (request) 4 GiB αλλά χωρίς άνω όριο (limit). Έτσι, κάθε broker μπόρεσε να δεσμεύσει μνήμη πολύ πάνω από το αίτημα, φτάνοντας τα 21,3 GiB στη μέγιστη βαθμίδα χωρίς να τερματιστεί βίαια λόγω υπέρβασης ορίου (Out-Of-Memory kill).

Τέλος, στο επίπεδο των ερωτημάτων, η μνήμη του Trino αναδεικνύεται σε περιοριστικό παράγοντα, καθώς το σταθερό heap των 2 GB ανά worker αποδεικνύεται ανεπαρκές για ερωτήματα παραθύρου σε σύνολα δεδομένων άνω των 200 εκατομμυρίων εγγραφών [24], όπως κατέδειξε η αποτυχία του Q3 στη N10.

8.11.3 Σύγκριση μη ομαδοποιημένων και ομαδοποιημένων παραγωγών

Η διφασική εκτέλεση αποκαλύπτει σημαντικές διαφορές μεταξύ των δύο στρατηγικών, οι οποίες συνοψίζονται στον Πίνακα 8.16.

Χαρακτηριστική περίπτωση αποτελεί η βαθμίδα T10, στην οποία οι δύο στρατηγικές αποκλίνουν έντονα ως προς τον κορεσμό παραγωγού. Ενώ η μη ομαδοποιημένη φάση εισέρχεται σε κορεσμό, η ομαδοποιημένη παραμένει άνετα εντός ορίων, με μέγιστη διάρκεια κύκλου (max_cycle) μόλις 0,125 s. Η σταθερότητα της ομαδοποιημένης φάσης σε αυτή την υψηλή κλίμακα επιβεβαιώνει ότι η ομαδοποίηση μηνυμάτων μετατοπίζει το όριο κλιμάκωσης πέρα από τα 87.500 μηνύματα ανά δευτερόλεπτο στην παρούσα διαμόρφωση. Παράλληλα, η ομαδοποιημένη φάση αποδίδει 67-80% χαμηλότερους χρόνους ερωτημάτων Trino στις T9-T10. Η διαφορά αυτή οφείλεται στον τρόπο με τον οποίο ο ρυθμός εισαγωγής επηρεάζει τη φυσική διάταξη των δεδομένων στον αποθηκευτικό χώρο. Όταν τα δεδομένα φθάνουν με ομαλό και σταθερό ρυθμό, κάθε κύκλος δέσμευσης του Flink προς το Paimon συσσωρεύει περισσότερες εγγραφές και τις αποθηκεύει σε λιγότερα και μεγαλύτερα αρχεία Parquet, με μεγαλύτερες ομάδες γραμμών, τα λεγόμενα row groups, ανά αρχείο. Αντίθετα, η ασταθής εισαγωγή της μη ομαδοποιημένης φάσης διασπά τον ίδιο όγκο δεδομένων σε πλήθος μικρών αρχείων, αναπαράγοντας το πρόβλημα των μικρών αρχείων που περιγράφηκε στο Κεφάλαιο 4.5.4. Καθώς το Trino πρέπει να εντοπίσει, να ανοίξει και να διαβάσει τα μεταδεδομένα κάθε αρ-

Πίνακας 8.16: *Σύγκριση μη ομαδοποιημένων (N) και ομαδοποιημένων (B) παραγωγών.*

Metric	**Μη ομαδ. (N)**	**Ομαδ. (B)**	**Βελτίωση**
CPU Kafka (T10)	2,46 πυρήνες	0,15 πυρήνες	~16πλάσια μείωση [21]
Καθυστ. E2E (T9)	335,2s	201,8s	40% χαμηλότερη
Καθυστ. E2E (T10)	178,2s	354,6s	Κορεσμός παραγωγού N10
Αστοχίες ckpt (T7)	1 (failover)	2 (failover)	Αμφότερες ασταθείς
Αστοχίες ckpt (T9)	16 (failover)	7 (failover)	56% λιγότερες
Χρόνοι Trino (T9–T10)	326–453s	90-111s	−67 έως −80%
Αποτυχίες Q3 (T10)	5 αποτυχίες	0 αποτυχίες	Επιτυχής εκτέλεση μόνο B10
Ρυθμός μηνυμάτων	Ισοδύναμος		Χωρίς απώλειες εισαγωγής

Η στήλη Metric δίνει το συγκρινόμενο μέγεθος και, σε παρένθεση, τη βαθμίδα φόρτου στην οποία αναφέρεται η μέτρηση.
Οι στήλες Μη ομαδ. (N) και Ομαδ. (B) δίνουν την τιμή του metric για τη μη ομαδοποιημένη και την ομαδοποιημένη φάση αντίστοιχα.
Η στήλη Βελτίωση δίνει τη μεταβολή της ομαδοποιημένης φάσης έναντι της μη ομαδοποιημένης ή σχετική ποιοτική παρατήρηση.
Η γραμμή CPU Kafka δίνει τη χρήση CPU των Kafka brokers σε πυρήνες.
Οι γραμμές Αστοχίες ckpt δίνουν το πλήθος αποτυχημένων checkpoints του Flink καθώς η ένδειξη failover δηλώνει ότι οι αστοχίες συνοδεύτηκαν από επανεκκίνηση της εργασίας (βλ. § 8.10.2).
Η γραμμή Χρόνοι Trino δίνει το εύρος χρόνων εκτέλεσης p95 των ερωτημάτων Q1-Q5.
Η γραμμή Αποτυχίες Q3 δίνει το πλήθος αποτυχημένων εκτελέσεων του ερωτήματος Q3 επί συνόλου 5 εκτελέσεων.
Ο συμβολισμός N10 και B10 δηλώνει τη φάση N ή B στη βαθμίδα T10.

χείου ξεχωριστά, τα πολλά μικρά αρχεία πολλαπλασιάζουν τα αιτήματα προς το ADLS και το διαχειριστικό κόστος εκτέλεσης του ερωτήματος, ενώ τα λιγότερα και μεγαλύτερα αρχεία επιτρέπουν συνεχόμενες, αποδοτικές αναγνώσεις και, συνεπώς, ταχύτερη εκτέλεση των ερωτημάτων.

8.11.4 Χαρτογράφηση κόστους

Τα μετρούμενα μεγέθη κοστολόγησης στο cloud όπως ώρες που είναι ενεργοί οι nodes, χωρητικότητα σε GB των δίσκων, μέγεθος σε GB της εξερχόμενης κίνησης και αιτήματα προς ACR χαρτογραφούνται στην εκτίμηση κόστους μέσω του Azure Pricing Calculator του Κεφαλαίου 6.1.10, επαληθεύοντας ότι η εκτίμηση κόστους αποκλίνει λιγότερο από 15% σε σχέση με τα πραγματικά δεδομένα λειτουργίας. Η κύρια πηγή απόκλισης προέρχεται από τη δικτυακή κίνηση προς το ADLS, η οποία υποεκτιμήθηκε αρχικά λόγω της υψηλής συχνότητας checkpoints του Flink.

Κεφάλαιο 9

Βελτιστοποίηση Πόρων, Εφαρμογής και Κόστους

Στο κεφάλαιο αυτό αναλύονται οι στρατηγικές βελτιστοποίησης που εφαρμόστηκαν ή προτείνονται, βασισμένες στα ευρήματα του Κεφαλαίου 8. Η ανάλυση εστιάζει σε τέσσερις άξονες οι οποίοι είναι η ρύθμιση ελάχιστων και μέγιστων δεσμευμένων πόρων, οι στρατηγικές κλιμάκωσης υποδομής και εφαρμογών, η επίδραση ρυθμού ανανέωσης των dashboards και οι οδηγίες λειτουργίας [99].

9.1 Ρύθμιση αίτησης κατώτατων και ανώτατων ορίων πόρων και επίδρασή τους στη σταθερότητα της εφαρμογής

9.1.1 Requests και limits

Στο Kubernetes, οι αιτήσεις απαιτούμενων πόρων (resource requests) καθορίζουν την ελάχιστα εγγυημένη υποδομή ανά pod, ενώ τα limits ορίζουν το ανώτατο αποδεκτό όριό τους. Η κατηγορία Quality of Service (QoS) κάθε pod εξαρτάται από τη σχέση μεταξύ αυτών των δύο ορίων [95, 227]:

Πίνακας 9.1: *Κατηγορίες QoS στο Kubernetes ως προς requests και limits.*

Κατηγορία	Σχέση requests/limits	Σύντομη ερμηνεία
Guaranteed	requests = limits	Για CPU και μνήμη. Προτεραιότητα στη διατήρηση.
Burstable	requests < limits	Μπορεί να χρησιμοποιήσει υπολειπόμενους πόρους.
BestEffort	requests = limits = ∅	Όταν δεν είναι ορισμένο κανένα από τα δύο τότε υπάρχει χαμηλότερη προτεραιότητα στην κατανομή πόρων.

Σημείωση: Η Burstable κατηγορία μπορεί να χρησιμοποιήσει υπολειπόμενους πόρους, φτάνοντας έως το άνω ορισμένο όριό της και, όταν δεν έχει οριστεί άνω όριο, έως την πλήρωση ολόκληρου του cluster.

9.1.2 Ρυθμίσεις requests και limits στην τρέχουσα υλοποίηση

Ο Πίνακας 9.2 συνοψίζει τις ρυθμίσεις πόρων της τρέχουσας υλοποίησης. Για κάθε υπηρεσία της αρχιτεκτονικής αναγράφονται τα requests και τα limits για CPU και μνήμη, όπως ορίζονται στα αντίστοιχα manifests αρχεία διαμόρφωσης.

Πίνακας 9.2: *Κάτω όρια (requests) και άνω όρια (limits) πόρων ανά υπηρεσία.*

Υπηρεσία	**CPU req**	**CPU lim**	**Mem req**	**Mem lim**
Flink JM	500m	1	1Gi	2Gi
Flink TM ($\times 2$)	2	-	8Gi	-
Kafka ($\times 2$)	2	-	4Gi	-
Trino coord.	-	-	-	-
Trino worker ($\times 2$)	-	-	-	-
IoT producer ($\times n$)	500m	2	256Mi	1Gi

Η στήλη Υπηρεσία δίνει την υπηρεσία και, όπου υπάρχει, τον αριθμό αντιγράφων της, π.χ. $\times 2$.
Οι στήλες CPU req και CPU lim δίνουν το κάτω και το άνω όριο CPU αντίστοιχα, σε πυρήνες ή millicores (m).
Οι στήλες Mem req και Mem lim δίνουν το κάτω και το άνω όριο μνήμης αντίστοιχα.
Η παύλα (‘-’) δηλώνει ότι το αντίστοιχο όριο δεν έχει οριστεί για την υπηρεσία.

Με βάση τις ρυθμίσεις του Πίνακα 9.2, οι υπηρεσίες κατατάσσονται σε τρεις ομάδες ως προς την κατηγορία QoS στην οποία ανήκουν.

- Ο Flink JobManager και οι IoT producers ανήκουν στην κατηγορία Burstable με ρητά requests και limits, προσφέροντας εγγυημένη βάση πόρων με ανώτατο όριο κατανάλωσης.

- Οι Flink TaskManagers και ο Kafka ορίζουν requests χωρίς limits, ανήκοντας στην κατηγορία Burstable χωρίς άνω όριο. Αυτό εξασφαλίζει εγγυημένη τοποθέτηση από τον χρονοπρογραμματιστή scheduler, επιτρέποντας παράλληλα προσωρινή αύξηση κατανάλωσης πόρων, όπως κατά το checkpoint flush ή σε κορύφωση εισερχόμενων μηνυμάτων.

- Το Trino, τόσο ο coordinator όσο και οι workers, δεν ορίζει requests ούτε limits και κατατάσσεται στην κατηγορία BestEffort. Αποτελεί τον πιο ευάλωτο κόμβο σε συνθήκες πίεσης μνήμης, καθώς ο kubelet agent, ο οποίος είναι υπεύθυνος για την υγεία των pods σε έναν node, το επιλέγει πρώτο για έξωση (eviction) [235].

Συνολικά, μόνο το Trino παραμένει BestEffort, ενώ οι υπόλοιπες υπηρεσίες έχουν τουλάχιστον requests ορισμένα. Αυτό μειώνει τον κίνδυνο ανεξέλεγκτης αποδόμησης των κρίσιμων στοιχείων της αρχιτεκτονικής.

9.1.3 Πρόταση βελτιστοποίησης των requests και limits

Βάσει των metrics κατανάλωσης κατά την εκτέλεση των δοκιμών (Κεφάλαιο 8), προτείνεται η ρύθμιση που παρουσιάζεται στον Πίνακα 9.3, όπου για κάθε υπηρεσία παρατίθενται οι τρέχοντες και οι προτεινόμενοι πόροι μαζί με τη σχετική αιτιολόγηση.

Πίνακας 9.3: *Προτεινόμενες βελτιστοποιήσεις ορίων πόρων ανά υπηρεσία*

	Τρέχοντες πόροι		**Προτεινόμενοι πόροι**		
Υπηρεσία	**CPU**	**Μνήμη**	**CPU**	**Μνήμη**	**Αιτιολόγηση**
Kafka	req: 2 lim: -	req: 4 Gi lim: -	req: 2 lim: 4	req: 4 Gi lim: 8 Gi	Αποφυγή ανεξέλεγκτης κατανάλωσης κατά κορύφωση εισερχόμενων μηνυμάτων και προστασία γειτονικών pods από OOM kill [7].
Flink TM	req: 2 lim: -	req: 8 Gi lim: -	req: 2 lim: 4	req: 8 Gi lim: 12 Gi	Περιορισμός burst κατά checkpoint flush. Η μνήμη limits ορίζεται ανώτερη των requests για αποφυγή OOM.
Trino workers	- (BestEffort)		req: 500 m lim: 2	req: 2 Gi lim: 4 Gi	Η ρύθμιση JVM heap (2 GB) περιορίζει ήδη τη μνήμη. Τα limits προσθέτουν δεύτερο επίπεδο ασφάλειας και αποτρέπουν eviction του coordinator.
IoT producers	req: 500 m lim: 2	req: 256 Mi lim: 1 Gi	Αμετάβλητοι		Βαθμονομημένοι από τα πειράματα (Πίνακας 8.3). Ελάχιστο προφίλ για σταθερή λειτουργία στις 3.500 κατοικίες ανά pod.

Σε κάθε κελί, το req δίνει το αίτημα (κάτω όριο) και το lim το άνω όριο πόρων για CPU και μνήμη.
Η παύλα ('-') δηλώνει όριο που δεν έχει οριστεί στην τρέχουσα διαμόρφωση.

9.2 Στρατηγικές κλιμάκωσης και προτάσεις αυτοματοποίησης

9.2.1 Οριζόντια κλιμάκωση

Ο μηχανισμός Horizontal Pod Autoscaler (HPA) [86] του Kubernetes ρυθμίζει δυναμικά τον αριθμό αντιγράφων (replicas) ενός Deployment βάσει metrics χρήσης [118]. Για τη συγκεκριμένη αρχιτεκτονική, προσφέρονται οι παρακάτω ευκαιρίες κλιμάκωσης:

- Trino workers: HPA βασισμένο σε μέση χρήση CPU. Με κατώφλι το 70%, ο αριθμός των workers μπορεί να αυξάνεται από 2 σε 4 κατά τη διάρκεια πειραμάτων υψηλού φόρτου.
- Flink TaskManagers: Οι TMs αναπτύσσονται ως Deployment με 2 replicas και 4 slots ανά pod. Σε σενάριο αυξημένου ρυθμού εισερχόμενων δεδομένων (π.χ. προσθήκη νέων πηγών IoT ή αύξηση συχνότητας μετρήσεων), η χρήση CPU των TM αυξάνεται. Μέσω HPA με κατώφλι χρήσης CPU ίση με 70%, ο αριθμός TM pods μπορεί να κλιμακωθεί αυτόματα, παρέχοντας επιπλέον task slots και αυξάνοντας τον διαθέσιμο παραλληλισμό επεξεργασίας χωρίς επανεκκίνηση των εργασιών.

Η κλιμάκωση του Kafka δεν υποστηρίζεται αυτόματα μέσω HPA, καθώς η αύξηση brokers απαιτεί αναδιανομή partitions, μια λειτουργία που δεν είναι αυτόματη στην τυπική εγκατάσταση [137].

9.2.2 Κατακόρυφη κλιμάκωση

Ο Vertical Pod Autoscaler (VPA) [87] προσαρμόζει δυναμικά τα requests ενός pod βάσει ιστορικής χρήσης. Υποστηρίζει τέσσερις λειτουργίες [117], οι οποίες συνοψίζονται στον Πίνακα 9.4.

Πίνακας 9.4: *Λειτουργίες του VPA.*

Λειτουργία	**Περιγραφή**
Off	Ο VPA υπολογίζει συστάσεις χωρίς να εφαρμόζει αλλαγές. Χρήσιμο για παρατήρηση και σύγκριση με τα τρέχοντα requests.
Initial	Οι συστάσεις εφαρμόζονται μόνο κατά τη δημιουργία νέων pods και δεν τροποποιούνται εν λειτουργία.
Auto	Ο VPA ενημερώνει τα requests αυτόματα, προκαλώντας επανεκκίνηση pods όταν απαιτείται.
Recreate	Όμοια με Auto, αλλά εγγυάται ότι οι αλλαγές εφαρμόζονται μέσω αναδημιουργίας του pod.

Στο πλαίσιο της παρούσας εργασίας, η λειτουργία Auto ή Recreate δεν ενδείκνυται κατά τη φάση των πειραμάτων, καθώς η αυτόματη αλλαγή requests μεταξύ εκτελέσεων θα αλλοίωνε τις μετρήσεις. Αντίθετα, η λειτουργία Off είναι η πλέον κατάλληλη καθώς επιτρέπει τη συλλογή συστάσεων βάσει πραγματικής χρήσης, οι οποίες μπορούν στη συνέχεια να αξιοποιηθούν για χειροκίνητη ρύθμιση των requests, όπως αυτή που περιγράφεται στον Πίνακα 9.3. Σε μελλοντικό παραγωγικό περιβάλλον, η μετάβαση σε Auto θα αυτοματοποιούσε πλήρως τη διαδικασία.

9.2.3 Αυτόματη κλιμάκωση cluster

Το AKS Cluster Autoscaler μπορεί να προσθέσει nodes όταν pods που βρίσκονται σε εκκρεμότητα δρομολόγησης δεν μπορούν να τοποθετηθούν (κατάσταση Pending) σε κάποιο node. Στα πειράματα, η κλιμάκωση των nodes του AKS ρυθμίστηκε στο εύρος $[2, 2]$, δηλαδή σταθερά δύο (2) nodes για τις εφαρμογές της λύσης. Σε παραγωγικό φόρτο, η αύξηση σε $[2, 6]$, δηλαδή έως έξι (6) nodes πάλι όμως ξεκινώντας από δύο (2), θα επέτρεπε αυτόματη ανταπόκριση σε αυξημένη ζήτηση, με ισοδύναμη αύξηση κόστους. Η χρήση Spot VMs (low-priority nodes) σε μη κρίσιμους φόρτους (π.χ. Trino workers) μειώνει το κόστος τους κατά 60-80% [14].

9.3 Επίδραση ρυθμού ανανέωσης των Grafana dashboards στο Trino

9.3.1 Πρόβλημα

Το Grafana εκτελεί ερωτήματα στο Trino ανά τακτά διαστήματα (ρυθμός ανανέωσης). Κάθε dashboard περιέχει πολλαπλά panels, καθένα με ξεχωριστό ερώτημα. Ένα dashboard με 20 panels σε ρυθμό ανανέωσης 10 δευτερολέπτων αντιστοιχεί σε 120 ερωτήματα ανά λεπτό μόνο από αυτόν τον πίνακα. Επιπλέον, χωρίς ενεργοποιημένο lazy loading, το Grafana εκτελεί ταυτόχρονα τα ερωτήματα όλων των panels κατά τη φόρτωση κάθε πίνακα ελέγχου, ακόμα και των panels που δεν είναι ορατά στην οθόνη, πολλαπλασιάζοντας άσκοπα τον φόρτο στο Trino [20].

9.3.2 Αναμενόμενη επίδραση

Αν και δεν διενεργήθηκαν αποκλειστικές δοκιμές ρυθμού ανανέωσης των Grafana dashboards, κατά τα πειράματα του Κεφαλαίου 8 (βλ. § 8.8.4) παρατηρήθηκε ποιοτικά ότι η κατανάλωση CPU του Trino σε κατάσταση ηρεμίας ήταν ελάχιστη, αλλά αυξανόταν αισθητά κατά την εκτέλεση ερωτημάτων. Η παρατήρηση αυτή υποδεικνύει ότι ο ρυθμός ανανέωσης επηρεάζει άμεσα τρεις παράγοντες. Πρώτον, τη χρήση CPU του coordinator. Δεύτερον, τον χρόνο απόκρισης των ερωτημάτων, λόγω ουράς αναμονής. Τρίτον, τη δικτυακή κίνηση προς το ADLS, καθώς κάθε ερώτημα σαρώνει αρχεία Paimon/Parquet, πλησιάζοντας πιθανώς τα όρια throttling του Storage Account.

9.3.3 Σύσταση

Για dashboards που απεικονίζουν ωριαία metrics, όπως για παράδειγμα IoT Energy KPIs, ένας ρυθμός ανανέωσης της τάξεως των 60s ή και μεγαλύτερος κρίνεται επαρκής, δεδομένου ότι τα υποκείμενα δεδομένα μεταβάλλονται αργά. Αντίθετα, dashboards πραγματικού χρόνου όπως ο IoT Live RAW ωφελούνται από ρυθμό 30s ή δυναμική ανανέωση (auto), ώστε να αντικατοπτρίζουν τις τελευταίες εγγραφές. Παράλληλα, η ενεργοποίηση του lazy loading στα dashboards περιορίζει την εκτέλεση ερωτημάτων μόνο στα panels που είναι ορατά στην οθόνη κατά τη φόρτωση, μειώνοντας σημαντικά τον αρχικό φόρτο στον Trino. Σε παραγωγικό περιβάλλον, η εισαγωγή caching σε επίπεδο Trino ή η χρήση ενδιάμεσης βάσης δεδομένων (π.χ. PostgreSQL materialized views) μπορεί να μετριάσει περαιτέρω τον αντίκτυπο στους υπολογιστικούς πόρους [24].

9.4 Οδηγίες λειτουργίας

9.4.1 Διάρκεια ζωής δεδομένων και διαχείριση στιγμιοτύπων

Ο πίνακας ακατέργαστων δεδομένων (sensor_data_raw) αξιοποιεί τη λήξη partition με περίοδο διατήρησης 7 ημερών, ώστε η διαγραφή παλαιών δεδομένων να γίνεται αυτόματα. Η παράμετρος partition.expiration-check-interval = 1h εξασφαλίζει ωριαίο έλεγχο από το Paimon χωρίς υπερβολική επιβάρυνση στο metastore [23]. Αντίστοιχα, η διατήρηση στιγμιοτύπων (snapshot retention) ορίζεται σε 30 ημέρες μέσω της παραμέτρου snapshot.time-retained = 30d και εφαρμόζεται ομοίως στον πίνακα δεδομένων δεκαλέπτου (sensor_data_10min), στον πίνακα δεδομένων τριάντα λεπτών (sensor_data_30min) και στον πίνακα ωριαίων δεδομένων (sensor_data_hourly). Η τιμή αυτή χρειάζεται περιοδική αξιολόγηση, καθώς μεγάλος αριθμός στιγμιοτύπων αυξάνει τον όγκο των μεταδεδομένων στο metastore. Τέλος, η περιοδική εκτέλεση της λειτουργίας COMPACT στους πίνακες Paimon συγχωνεύει τα μικρά αρχεία που δημιουργούνται κατά τη ροή εγγραφών, βελτιώνοντας την απόδοση ανάγνωσης από το Trino.

9.4.2 Παρακολούθηση και ειδοποιήσεις

Η αξιόπιστη λειτουργία της πλατφόρμας προϋποθέτει τον ορισμό κανόνων ειδοποίησης (alerting rules) στο Prometheus για κρίσιμα metrics, όπως η υπέρβαση της διάρκειας λήψης

checkpoint του Flink άνω των 120s, η καθυστέρηση καταναλωτή Kafka πέραν των 100.000 offsets και η χρήση μνήμης του node του AKS cluster πάνω από 90%. Συμπληρωματικά, η ρύθμιση ειδοποιήσεων στο Grafana με πηγή δεδομένων Prometheus καλύπτει τα metrics επίδοσης του Kubernetes και λειτουργεί παράλληλα με τα dashboards που έχουν πηγή το Trino. Ως δεύτερο επίπεδο παρακολούθησης, το Azure Monitor παρέχει ανεξάρτητη αποθήκευση metrics και εξασφαλίζει ορατότητα ακόμα και σε περίπτωση αστοχίας εντός του cluster.

9.4.3 Ασφάλεια

Τα κλειδιά του Azure Storage Account θα πρέπει να μεταφερθούν σε ένα αποθετήριο διαχείρισης κωδικών (Azure Key Vault, KV), δηλαδή μια υπηρεσία που αποθηκεύει κρυπτογραφημένα κλειδιά, πιστοποιητικά και κωδικούς (secrets) [236]. Για να τα ανακτήσει το AKS cluster από το KV, χρησιμοποιείται ο οδηγός Secrets Store CSI (Container Storage Interface), ο οποίος προσαρτά τα μυστικά απευθείας στα pods ως volumes [237]. Με τον τρόπο αυτό αντικαθίστανται τα Kubernetes Secrets, τα οποία αποθηκεύονται στο etcd σε απλή, αναστρέψιμη κωδικοποίηση base64 [238] και όχι σε πραγματική κρυπτογράφηση. Επιπλέον, η ταυτοποίηση μέσω Azure Managed Identity [239], δηλαδή ταυτότητας που παρέχεται και εναλλάσσεται αυτόματα από το Azure χωρίς αποθήκευση διαπιστευτηρίων, σε συνδυασμό με τον έλεγχο πρόσβασης βάσει ρόλων (Azure Role Based Access Control, RBAC) [240], εξαλείφει τον κίνδυνο διαρροής κλειδιών.

Όσον αφορά τον έλεγχο της εισερχόμενης δικτυακής κίνησης, η εισαγωγή ενός ελεγκτή εισόδου (Ingress Controller), ο οποίος δρομολογεί την εξωτερική κίνηση HTTP/HTTPS προς τις υπηρεσίες του cluster, επιτρέπει την κεντρική διαχείρισή της [241]. Για υπηρεσίες εσωτερικής χρήσης, όπως το Flink UI και το Trino UI, αρκεί ένα εσωτερικό Ingress (π.χ. μέσω NGINX Ingress [242]) σε συνδυασμό με πολιτικές δικτύου (Network Policies) [243], δηλαδή κανόνες που περιορίζουν τη δικτυακή επικοινωνία μεταξύ pods, ώστε η πρόσβαση να παραμένει εντός του VNet. Για υπηρεσίες με εξωτερική πρόσβαση, όπως το Grafana, προτείνεται η χρήση Azure Application Gateway με ενεργοποιημένο τείχος προστασίας εφαρμογών ιστού (Web Application Firewall, WAF) [244], ώστε να παρέχεται προστασία από κοινές κακόβουλες επιθέσεις, όπως οι δέκα σημαντικότεροι κίνδυνοι ασφάλειας διαδικτυακών εφαρμογών (OWASP Top 10) [245].

9.4.4 Εκτίμηση κόστους και βελτιστοποίηση δαπανών

Η κύρια πηγή κόστους της πλατφόρμας είναι οι υπολογιστικοί πόροι των nodes του AKS cluster, οι οποίοι αντιστοιχούν περίπου στο 84% του μηνιαίου κόστους (βλ. Ενότητα 6.1.10). Η δέσμευση του εκπτωτικού προγράμματος Reserved Instances μπορεί να μειώσει αυτό το κόστος κατά περίπου 37% για δέσμευση ενός έτους και 56% για τριών ετών. Από την πλευρά του αποθηκευτικού χώρου, το ADLS (βλ. Ενότητα 6.1.7) σε hot tier κοστίζει αναλογικά με τον όγκο δεδομένων. Η μετακίνηση ωριαίων δεδομένων παλαιότερων του ενός μήνα σε cool tier μειώνει το κόστος αποθήκευσης κατά περίπου 50%.

Μέρος III

Επίλογος

Κεφάλαιο 10

Συμπεράσματα και Μελλοντική Εργασία

Στο κεφάλαιο αυτό συνοψίζονται τα αποτελέσματα της εργασίας μέσω απαντήσεων στα ερευνητικά ερωτήματα, εξάγονται τα κύρια συμπεράσματα και προτείνονται κατευθύνσεις μελλοντικής έρευνας.

10.1 Απαντήσεις στα ερευνητικά ερωτήματα

10.1.1 Ισορροπία απόδοσης και κόστους

Ποια αρχιτεκτονική επιτυγχάνει ικανοποιητική ισορροπία μεταξύ απόδοσης επεξεργασίας (throughput) και λειτουργικού κόστους σε public cloud περιβάλλοντα, όταν χρησιμοποιούνται αποκλειστικά τεχνολογίες ανοιχτού κώδικα πάνω στην πλατφόρμα Kubernetes;

Η αρχιτεκτονική Kafka → Flink → Paimon → Trino, αναπτυγμένη σε AKS cluster, επιτυγχάνει σταθερή εισαγωγή μηνυμάτων σε όλες τις βαθμίδες φόρτου και ο παρατηρούμενος ρυθμός υπερβαίνει τον αναμενόμενο κατά 3 έως 18% σε όλο το εύρος δοκιμών από τη βαθμίδα T1 έως τη T10. Η χρήση αποκλειστικά τεχνολογιών ανοιχτού κώδικα εξαλείφει τα τέλη αδειοδότησης, ενώ η μορφή data lakehouse με τον πίνακα Paimon αποθηκευμένο πάνω σε ADLS [6] αξιοποιεί χαμηλού κόστους αποθηκευτικό χώρο αντί ακριβών εγκαταστάσεων data warehouse.

Η ισορροπία κόστους-απόδοσης εξαρτάται καθοριστικά από δύο παράγοντες, τη ρύθμιση checkpoints στο Flink και τη στρατηγική batching των μηνυμάτων των παραγωγών των δεδομένων. Η αύξηση του διαστήματος checkpointing μειώνει τον φόρτο εγγραφών στο ADLS [149], ενώ η ομαδοποίηση των μηνυμάτων πριν την αποστολή τους μειώνει τη χρήση επεξεργαστή του Kafka, το οποίο τα υποδέχεται, κατά περίπου 16 φορές (2,46 → 0,15 πυρήνες στη βαθμίδα T10 του πειράματος), χωρίς μείωση του ρυθμού εισαγωγής. Το εύρημα αυτό υποδεικνύει ότι η ομαδοποίηση μηνυμάτων αποτελεί σημαντικό μοχλό βελτιστοποίησης κόστους [137]. Τα πειράματα επαληθεύουν ότι η εκτίμηση κόστους, υπολογισμένη μέσω Azure Pricing Calculator, αποκλίνει λιγότερο από 15% από τα πραγματικά δεδομένα λειτουργίας [14].

10.1.2 Επίδραση επιλογών υποδομής στην καθυστέρηση από άκρο σε άκρο

Πώς επηρεάζουν οι επιλογές υποδομής, όπως, πρώτον, το πλήθος και το μέγεθος των nodes ενός cluster και, δεύτερον, οι ρυθμίσεις ελάχιστων και μέγιστων ορίων πόρων επε-

ξεργαστή και μνήμης (requests/limits), την καθυστέρηση από άκρο σε άκρο (end-to-end latency) της ροής δεδομένων;

Η καθυστέρηση από άκρο σε άκρο μετρά τον χρόνο από τη στιγμή που παράγεται ένα δεδομένο, ακριβώς πριν αποσταλεί στο Kafka, μέχρι τη φόρτωσή του στον πίνακα Paimon. Η καθυστέρηση αυτή εξαρτάται σημαντικά τόσο από τη βαθμίδα φόρτου, δηλαδή το πλήθος των μηνυμάτων που μεταφέρονται ανά δευτερόλεπτο, όσο και από τη στρατηγική ομαδοποίησης των μηνυμάτων πριν την αποστολή τους. Στους μη ομαδοποιημένους παραγωγούς η καθυστέρηση παραμένει χαμηλή στις βαθμίδες T1 έως T6, κυμαινόμενη από 33,4 έως 67,5 δευτερόλεπτα (s). Στη βαθμίδα T7 αυξάνεται απότομα και φτάνει τα 172,6s, καθώς συσσωρεύεται εκτεταμένη καθυστέρηση στην πλευρά του καταναλωτή του Kafka και το Flink αναγκάζεται σε επαναφορά checkpoint. Η καθυστέρηση κορυφώνεται στη βαθμίδα T9 (μη ομαδοποιημένη εκτέλεση) στα 335,2s, τιμή που προέκυψε από επιβεβαιωμένη επανεκτέλεση του πειράματος όπως αναλύεται στην §8.10.2. Στη βαθμίδα αυτή καταγράφονται 13 αποτυχίες checkpoint και 9 επαναφορές, οι οποίες υποδεικνύουν επαναλαμβανόμενους κύκλους ανάκαμψης του Flink [13]. Στη βαθμίδα T10 η καθυστέρηση μειώνεται στα 178,2s, διότι ο παραγωγός φτάνει σε κορεσμό, με τον μέγιστο χρόνο ανά κύκλο παραγωγής να ανέρχεται στα 1,40s, και έτσι ο ρυθμός εισαγωγής σταθεροποιείται σε επίπεδο που μπορεί να εξυπηρετήσει το Flink. Αντίθετα, στους ομαδοποιημένους παραγωγούς η καθυστέρηση κλιμακώνεται πιο ομαλά, με 109,2s στη βαθμίδα T8, 201,8s στη βαθμίδα T9 και 354,6s στη βαθμίδα T10, αποφεύγοντας έτσι τις απότομες αυξήσεις της μη ομαδοποιημένης περίπτωσης χάρη στη μειωμένη πίεση επεξεργαστή στο Kafka.

Οι βασικοί παράγοντες επιρροής παραμένουν το μέγεθος της συνολικής μνήμης RAM του cluster, όπου η διπλάσια μνήμη ανά node μεγεθύνει τον διαθέσιμο χώρο προσωρινής μνήμης (buffer pool) του Flink, μειώνοντας την ανάγκη προσωρινής εκφόρτωσης δεδομένων από τη μνήμη στον δίσκο (spill-to-disk) και βελτιώνοντας τον χρόνο που απαιτείται για τη λήψη ενός checkpoint, καθώς και ο αριθμός nodes, αφού η προσθήκη τρίτου node θα μείωνε τον ανταγωνισμό επεξεργαστή και μνήμης μεταξύ Flink, Kafka και Trino. Όσον αφορά τη μνήμη, οι Flink TaskManagers περιορίζονται σε επίπεδο JVM μέσω της ρύθμισης `taskmanager.memory.process.size=8192m` και τα πειραματικά δεδομένα επιβεβαιώνουν σχετικά σταθερή κατανάλωση από 3,2 έως 4,6GiB ανά TaskManager σε όλες τις βαθμίδες. Αντίθετα, οι Kafka brokers δεν διαθέτουν limits μνήμης σε επίπεδο Kubernetes και η κατανάλωσή τους αυξάνεται από περίπου 1,4GiB στη βαθμίδα T1 σε περίπου 21,3GiB στη βαθμίδα T10, υπερβαίνοντας κατά πολύ τα requests των 4GiB και αυξάνοντας τον κίνδυνο eviction σε συνθήκες πίεσης μνήμης του node [7]. Ωστόσο, η στρατηγική ομαδοποιημένων παραγωγών (batched producers) σταθεροποιεί την καθυστέρηση στις υψηλές βαθμίδες, με μείωση 40% στη T9 (335,2 → 201,8s) και αποφυγή των επαναλαμβανόμενων κύκλων αστοχίας και ανάκαμψης που εμφανίζονται στη μη ομαδοποιημένη φάση.

10.1.3 Όρια κλιμάκωσης και αστοχίες

Ποια είναι τα πρακτικά όρια κλιμάκωσης της προτεινόμενης αρχιτεκτονικής, και σε ποιο σημείο εμφανίζονται αστοχίες (bottlenecks) που επηρεάζουν τη σταθερότητα του συστήματος;

Σε αντίθεση με τα αρχικά αναμενόμενα αποτελέσματα, η αρχιτεκτονική δεν εμφανίζει

κορεσμό ρυθμού μηνυμάτων σε καμία βαθμίδα καθώς το Kafka δέχεται γραμμικά αυξανόμενο φόρτο έως περίπου 100.000msg/s στη βαθμίδα T10. Αυτό επιτυγχάνεται από τη burstable μνήμη που ορίστηκε για το Kafka. Το bottleneck εμφανίζεται αντ' αυτού στην πλευρά κατανάλωσης και ανάλυσης. Πρώτον, η καταχώρηση εγγραφών Paimon στο ADLS αποτελεί βασικό σημείο συμφόρησης καθώς η καθυστέρηση στο REST API αυξάνεται από 22,8ms (T1) σε 66,4ms (T10), με ενδιάμεση ακραία τιμή ίση με 81,5ms στη T6, οδηγώντας σε αυξανόμενη καθυστέρηση καταναλωτή (consumer lag) στις υψηλές βαθμίδες [149], ενώ η βαθμίδα T9 στη μη ομαδοποιημένη εκτέλεση (86.909 msg/s) συγκεντρώνει τις περισσότερες αστοχίες checkpoint (§8.10.2), με αποτέλεσμα επαναλαμβανόμενους κύκλους ανάκαμψης που καθυστερούν την κατανάλωση Kafka μηνυμάτων [149]. Δεύτερον, στη βαθμίδα T7 εμφανίζονται λιγότερες αστοχίες checkpoint (1 αποτυχία στη μη ομαδοποιημένη και 2 στην ομαδοποιημένη εκτέλεση, με failover), υποδεικνύοντας ότι το ζεύγος Flink TMs φτάνει σε όριο πίεσης κοντά στις 50.000 msg/s ανεξαρτήτως στρατηγικής παραγωγού. Τρίτον, στη βαθμίδα T10 (87.500 msg/s) το Trino ερώτημα Q3 (συνάρτηση παραθύρου) απέτυχε στη μη ομαδοποιημένη φάση και στις πέντε προσπάθειες εκτέλεσης, λόγω υπέρβασης μνήμης heap 2GB κατά την ταξινόμηση περίπου 212 εκατομμυρίων εγγραφών [24], ενώ στην ομαδοποιημένη φάση ολοκληρώθηκε επιτυχώς σε 95,47s χάρη στη βελτιωμένη οργάνωση αρχείων Parquet. Τέλος, με 8 διαθέσιμα slots (2×4) και παραλληλισμό εργασιών ίσο με 4 για την εισαγωγή ροής, τα υπόλοιπα 4 slots διατίθενται για τις περιοδικές εργασίες συγκέντρωσης, ενώ ο κυρίαρχος περιορισμός παραμένει το ADLS I/O.

10.2 Συμπεράσματα και μαθήματα από την υλοποίηση

10.2.1 Κύρια συμπεράσματα

Η αρχιτεκτονική data lakehouse πάνω σε Kubernetes καθίσταται βιώσιμη για φόρτους εργασίας IoT μεσαίας κλίμακας, δηλαδή περίπου 50.000 με 100.000 msg/s, χρησιμοποιώντας αποκλειστικά τεχνολογίες ανοιχτού κώδικα σε public cloud περιβάλλοντα, με τον ρυθμό εισαγωγής Kafka να κλιμακώνεται γραμμικά σε όλες τις βαθμίδες. Η χρήση πίνακα μορφής Paimon με αποθήκευση σε ADLS παρέχει τα πλεονεκτήματα τόσο του data lake (χαμηλό κόστος, ελεύθερο σχήμα) όσο και του data warehouse (ACID transactions, ανάγνωση μέσω SQL) [6]. Το Flink σε λειτουργία session είναι επαρκές όταν φιλοξενείται σε Kubernetes χωρίς τον Flink Kubernetes Operator, αλλά η απουσία αυτόματης ανάκαμψης εργασιών αποτελεί περιορισμό σε παραγωγικό περιβάλλον, όπως φανερώνουν οι επαναλαμβανόμενοι κύκλοι αστοχίας και ανάκαμψης στις βαθμίδες με πολλές αποτυχίες checkpoint. Η ρύθμιση checkpoints (interval, timeout, unaligned) συνιστά τον βασικό μοχλό απόδοσης, με τις αστοχίες να εμφανίζονται από τη βαθμίδα T7 και άνω και να επιδεινώνονται μονοτονικά μέχρι την T9, καθώς σοβαρές αστοχίες προκύπτουν μόνο υπό υψηλό φόρτο [149]. Η αρχιτεκτονική medallion [52] σε πίνακες Paimon επιβεβαιώνεται ως πρακτική, επιτρέποντας ξεχωριστές πολιτικές διατήρησης TTL των δεδομένων και βελτιώνοντας τους χρόνους περάτωσης των αναλυτικών ερωτημάτων στο Trino.

Δύο ευρήματα ξεχωρίζουν ιδιαίτερα. Η ομαδοποίηση μηνυμάτων (batching) αναδεικνύεται ως ο καθοριστικότερος μοχλός για τη σταθερότητα του συστήματος στις υψηλές βαθμίδες:

οι ομαδοποιημένοι παραγωγοί επιτυγχάνουν ισοδύναμο ρυθμό εισαγωγής αλλά με περίπου 16 φορές χαμηλότερη χρήση επεξεργαστή Kafka, σταθερότερα checkpoints (ενδεικτικά 56% λιγότερες αστοχίες στη T9) και έως 40% χαμηλότερη καθυστέρηση E2E στη T9 (335,2 → 201,8s, επιβεβαιωμένη τιμή επανεκτέλεσης), αποφεύγοντας τους κύκλους ανάκαμψης που εμφανίζει η μη ομαδοποιημένη φάση. Επιπλέον, η σουίτα ερωτημάτων (Q1-Q5) αποκαλύπτει δραματική υποβάθμιση απόδοσης στις υψηλές βαθμίδες. Ο χρόνος ολοκλήρωσης του ερωτήματος Q1 (πλήθος) αυξάνεται 67 φορές (4,9s → 326s), ενώ του Q2 (ομαδοποίηση) 76 φορές (4,75s → 359s) από τη βαθμίδα T1 στη T10. Τα ερωτήματα Q4 (σύνδεση ακατέργαστου και δεκάλεπτου πίνακα) και Q5 (στατιστική ανάλυση εκατοστημορίων) εμφανίζουν αντίστοιχη υπεργραμμική κλιμάκωση, με αύξηση 60 και 64 φορές αντίστοιχα (5,01s → 302s και 4,78s → 307s), ενώ οι ομαδοποιημένοι παραγωγοί βελτιώνουν τους χρόνους κατά 20-70% στις βαθμίδες T9-T10 λόγω καλύτερης οργάνωσης αρχείων Parquet [205]. Τέλος, η αστοχία του ερωτήματος Q3 (συνάρτηση παραθύρου) στη μη ομαδοποιημένη φάση της T10 καταδεικνύει ότι το Trino με 2GB heap δεν επαρκεί για αναλυτικές λειτουργίες παραθύρου σε περισσότερες από 200 εκατομμύρια εγγραφές.

10.2.2 Μετατόπιση bottleneck από εισαγωγή σε ανάλυση

Ένα κεντρικό εύρημα της εργασίας είναι η αντίθεση μεταξύ γραμμικής κλιμάκωσης εισαγωγής και υπεργραμμικής κλιμάκωσης ανάλυσης. Ενώ ο ρυθμός εισαγωγής Kafka κλιμακώνεται γραμμικά σε όλες τις βαθμίδες (λόγος παρατηρούμενου/αναμενόμενου 1,03-1,18), οι χρόνοι ερωτημάτων Trino αυξάνονται κατά μία τάξη μεγέθους (60-76 φορές). Αυτό υποδεικνύει ότι σε αρχιτεκτονικές data lakehouse το bottleneck μετατοπίζεται από την πλευρά εισαγωγής στην πλευρά κατανάλωσης και ανάλυσης καθώς αυξάνεται ο φόρτος, και ο σχεδιασμός πόρων πρέπει να λαμβάνει υπόψη αυτή τη μη γραμμική συμπεριφορά [205].

10.2.3 Η αναλογία (ratio) Kafka/Paimon ως δείκτης υγείας

Η αναλογία μεταξύ εγγραφών Kafka και εγγραφών Paimon αποτελεί πρακτικό δείκτη κορεσμού κατανάλωσης. Στα πειράματα η αναλογία παραμένει σταθερά κοντά στο 100%, καθώς κυμαίνεται μεταξύ 0,98 και 1,07 σε όλες τις βαθμίδες, εντός του εύρους SLO το οποίο είναι 0,90 έως 1,10, χάρη στους μηχανισμούς deduplication και exactly-once εγγραφών του Paimon, οι οποίοι διατηρούν την ακεραιότητα δεδομένων ακόμη και μετά από επαναλαμβανόμενες αστοχίες checkpoint (όπως στη μη ομαδοποιημένη βαθμίδα T9). Σε ένα παραγωγικό περιβάλλον χωρίς ανάλογες εγγυήσεις, η παρακολούθηση αυτής της αναλογίας σε πραγματικό χρόνο μπορεί να λειτουργήσει ως σήμα πρόληψης πριν εμφανιστούν αστοχίες checkpoint ή κορεσμός παραγωγού.

10.2.4 Οριοθέτηση αναλυτικών ερωτημάτων Trino

Τα πειράματα παρέχουν έναν πρακτικό κανόνα οριοθέτησης (dimensioning) για τον Trino σε περιβάλλον lakehouse. Με 2GB heap ανά worker, απλά ερωτήματα ομαδοποίησης (Q1, Q2) και σύνδεσης (Q4) ολοκληρώνονται σε αποδεκτούς χρόνους έως περίπου 50 εκατομμύρια εγγραφές (<100s). Πάνω από αυτό το όριο, η χρήση εκφόρτωσης στον δίσκο (spill-to-disk)

αυξάνεται δραστικά και οι χρόνοι υπερβαίνουν τα 300s [24]. Ερωτήματα που απαιτούν ταξινόμηση ολόκληρου του συνόλου δεδομένων (Q3, συνάρτηση παραθύρου) αποτυγχάνουν εντελώς σε περισσότερες από 200 εκατομμύρια εγγραφές. Η αύξηση heap ή η χρήση αρχιτεκτονικής medallion (ερωτήματα στον ωριαίο πίνακα αντί του ακατέργαστου) αποτελούν τις δύο βασικές στρατηγικές αντιμετώπισης.

10.2.5 Μεθοδολογική συνεισφορά

Η αυτοματοποιημένη πειραματική μεθοδολογία που αναπτύχθηκε αποτελεί αφ' εαυτής συνεισφορά της εργασίας ως αναπαράξιμο πλαίσιο αξιολόγησης συστημάτων επεξεργασίας ροών δεδομένων σε Kubernetes. Το σενάριο (script) το οποίο είναι γραμμένο σε γλώσσα σεναρίων PowerShell (βλ. Παράρτημα ΣΤ) αυτοματοποιεί τον πλήρη κύκλο εκτέλεσης του πειράματος (εκκίνηση, παρακολούθηση, συλλογή, εκκαθάριση) και εξασφαλίζει επαναληψιμότητα μεταξύ βαθμίδων [30]. Τρία στοιχεία της μεθοδολογίας αποδείχθηκαν καθοριστικά. Ο έλεγχος υγείας του Flink αποτρέπει τη συλλογή δεδομένων από αποτυχημένες εκτελέσεις, η περίοδος εκκαθάρισης (120s flush cooldown) εξασφαλίζει την πλήρη εγγραφή δεδομένων στο Paimon και η επικύρωση πριν την εκτέλεση εντοπίζει λανθασμένες ρυθμίσεις προτού σπαταληθούν πόροι. Η διφασική εκτέλεση, με τους μη ομαδοποιημένους και τους ομαδοποιημένους παραγωγούς να τρέχουν εντός του ίδιου κύκλου, εξασφαλίζει δίκαιη σύγκριση στρατηγικών υπό σταθερές εξωτερικές συνθήκες.

10.3 Μελλοντικές επεκτάσεις

10.3.1 Αυτόματη κλιμάκωση πλήρους στοίβας

Πέρα από τις στρατηγικές κλιμάκωσης που αναλύονται στο Κεφάλαιο 9 (HPA, VPA, Cluster Autoscaler), δύο επεκτάσεις θα ενίσχυαν σημαντικά την αυτονομία της πλατφόρμας. Πρώτον, η χρήση του Kubernetes Event-Driven Autoscaling (KEDA) [246], ενός μηχανισμού κλιμάκωσης βάσει γεγονότων, θα επέτρεπε κλιμάκωση των Flink TaskManagers βάσει της καθυστέρησης καταναλωτή (consumer lag) του Kafka [118], αντί της στατικής μετρικής χρήσης επεξεργαστή που προτείνεται στην οριζόντια αυτόματη κλιμάκωση (HPA). Δεύτερον, η εγκατάσταση μέσω του Flink Kubernetes Operator θα παρείχε αυτόματη ανάκαμψη και κλιμάκωση μέσω αντίδρασης (reactive scaling) εργασιών [247], δεδομένου ότι οι επαναλαμβανόμενοι κύκλοι αστοχίας και ανάκαμψης στη μη ομαδοποιημένη T9 υπογραμμίζουν πόσο εύθραυστη είναι η χειροκίνητη διαχείριση εργασιών Flink.

10.3.2 Βελτιωμένη παρακολούθηση

Στο Κεφάλαιο 9 προτείνονται κανόνες ειδοποίησης (alarms) στο Prometheus και η χρήση Azure Monitor ως δεύτερου επιπέδου παρακολούθησης. Ως επέκταση, το Prometheus με τον Alertmanager θα επέτρεπε την αυτόματη προώθηση ειδοποιήσεων σε κανάλια επικοινωνίας της ομάδας διαχείρισης, όπως Slack [248], Microsoft Teams [249] ή ηλεκτρονικό ταχυδρομείο, ώστε τα κρίσιμα γεγονότα (π.χ. αστοχίες checkpoint, κορεσμός παραγωγού, υπέρβαση μνήμης) να γίνονται αντιληπτά χωρίς χειροκίνητη παρακολούθηση. Παράλλη-

λα, η κατασκευή ενός ενοποιημένου dashboard “Site Reliability Engineer (SRE) Overview” στο Grafana, βασισμένου στους τέσσερις χρυσούς δείκτες (golden signals), δηλαδή καθυστέρηση, κίνηση, σφάλματα, κορεσμός [117], θα παρείχε συνολική εικόνα της υγείας της πλατφόρμας σε πραγματικό χρόνο.

10.3.3 Ασφάλεια και συμμόρφωση

Οι βασικές προτάσεις ασφάλειας (χρήση Azure Key Vault, Managed Identity, Network Policies, Ingress Controller) αναλύονται στο Κεφάλαιο 9. Ως επιπλέον βήματα, η υιοθέτηση Workload Identity Federation [250] θα εξάλειφε πλήρως τη χρήση στατικών κλειδιών, ενώ η κρυπτογράφηση δεδομένων κατά τη μεταφορά (data-in-transit) μέσω TLS, τόσο μεταξύ των Kafka brokers όσο και μεταξύ Flink και Kafka, θα ολοκλήρωνε την ασφάλεια της ροής δεδομένων.

10.3.4 Επέκταση σε πραγματικά δεδομένα IoT

Μια φυσική επέκταση της εργασίας θα ήταν η μετάβαση από προσομοιωμένα σε πραγματικά δεδομένα IoT. Μια τέτοια μετάβαση θα απαιτούσε την αντιμετώπιση ζητημάτων ποιότητας δεδομένων, όπως ο χειρισμός ελλιπών μετρήσεων, διπλότυπων εγγραφών και γεγονότων εκτός χρονολογικής σειράς [28]. Επιπλέον, θα προέκυπτε η ανάγκη υποστήριξης εξέλιξης του σχήματος και του μοντέλου (schema evolution), καθώς το Paimon υποστηρίζει αλλαγές σχήματος αλλά η σύνδεση με το Trino απαιτεί ρητή ενημέρωση του καταλόγου (catalog). Τέλος, θα ήταν απαραίτητη η ενσωμάτωση πρωτοκόλλων IoT (MQTT) [122] μέσω μιας γέφυρας MQTT-προς-Kafka (Kafka MQTT bridge) [251] ή μέσω Azure IoT Hub [252], ώστε η πλατφόρμα να δέχεται δεδομένα απευθείας από φυσικούς αισθητήρες.

10.3.5 Μετάβαση στην SaaS πλατφόρμα Microsoft Fabric

Μια ολοκληρωμένη εναλλακτική, οργανωμένη ως υπηρεσία λογισμικού ως υπηρεσίας (Software as a Service, SaaS) [57], αποτελεί το Microsoft Fabric [253], μια πρόσφατα διαθέσιμη αναλυτική πλατφόρμα η οποία συνδυάζει σε ένα περιβάλλον τρεις δυνατότητες που αντιστοιχούν στην παρούσα υλοποίηση. Η πρώτη είναι η αποθήκευση δεδομένων μέσω της λίμνης δεδομένων OneLake, σε μορφή Delta Lake πάνω σε ADLS Gen2 [254]. Η δεύτερη είναι η αρχιτεκτονική lakehouse με αυτόματα παρεχόμενο SQL analytics endpoint [255], που αποτελεί ευθεία αντιστοιχία με τη σύζευξη των Paimon και Trino της παρούσας υλοποίησης. Η τρίτη είναι η ροή δεδομένων σε πραγματικό χρόνο μέσω της δυνατότητας Real-Time Intelligence [256], όπου τα Eventstream [257] υποστηρίζουν εγγενώς πηγές Azure IoT Hub, MQTT και το πρωτόκολλο Apache Kafka, επιτρέποντας έτσι την επαναχρησιμοποίηση των υφιστάμενων παραγωγών δεδομένων χωρίς αλλαγή κώδικα. Μια συστηματική σύγκριση με την παρούσα υλοποίηση θα μπορούσε να ποσοτικοποιήσει την αντιστάθμιση (trade-off) σε τρεις άξονες. Ο πρώτος άξονας είναι το συνολικό κόστος ιδιοκτησίας (Total Cost of Ownership, TCO) [57], όπου αντιπαραβάλλεται το χρεωστικό μοντέλο μονάδων χωρητικότητας Fabric (Capacity Units) [258] με τους πόρους AKS και ADLS του παρόντος πειράματος. Ο δεύτερος άξονας είναι ο χρόνος υλοποίησης και η λειτουργική επιβάρυνση, καθώς η SaaS

προσέγγιση καταργεί την ανάγκη ρητής ρύθμισης Flink checkpoints, TaskManagers και πινάκων Paimon. Ο τρίτος άξονας είναι ο βαθμός ελέγχου και ευελιξίας, δεδομένου ότι η αυτοδιαχειριζόμενη υλοποίηση πάνω σε Kubernetes επιτρέπει βαθύτερη παραμετροποίηση και αποφεύγει τον εγκλωβισμό σε προμηθευτή (vendor lock-in) [14]. Δεδομένου ότι το OneLake βασίζεται στο ADLS Gen2 [254], υφίσταται η δυνατότητα σταδιακής μετάβασης μέσω των OneLake shortcuts [259] προς τα υπάρχοντα δεδομένα Paimon χωρίς αντιγραφή, διευκολύνοντας έναν υβριδικό σχεδιασμό κατά τη φάση αξιολόγησης.

10.3.6 Εναλλακτικές αρχιτεκτονικές

Η σύγκριση με εναλλακτικούς σχεδιασμούς θα μπορούσε να ενισχύσει τις αποφάσεις αρχιτεκτονικής σχεδίασης. Στο επίπεδο αποθήκευσης πινάκων, η αξιολόγηση του Apache Iceberg ή του Delta Lake [167] ως αντικαταστάτη του Paimon [6] θα αποκάλυπτε διαφορές σε επιδόσεις upsert, δηλαδή στη λειτουργία ενημέρωσης μιας υπάρχουσας ή εισαγωγής μιας νέας εγγραφής (update or insert), και σε συμβατότητα με το Flink. Η συγκριτική ανάλυση της βιβλιογραφίας [169] αποτελεί χρήσιμο πλαίσιο αναφοράς για μία τέτοια σύγκριση, καθώς αναδεικνύει ότι το Iceberg υπερτερεί σε κλιμακωσιμότητα μεταδεδομένων για μεγάλους πίνακες, το Delta Lake σε αυστηρότητα ACID εγγυήσεων και το Hudi σε φόρτους εντατικής εισαγωγής δεδομένων ροών (stream ingestion), ωστόσο κανένα σύστημα δεν υπερέχει σε όλες τις διαστάσεις. Στο επίπεδο εισαγωγής και αποθήκευσης streaming δεδομένων, το Apache Fluss [260] αποτελεί ενδιαφέρουσα εναλλακτική, καθώς σχεδιάστηκε ως ενιαίο επίπεδο αποθήκευσης streaming δεδομένων ειδικά για το Flink και θα μπορούσε να αντικαταστήσει το Kafka ως ενδιάμεση ουρά μηνυμάτων, ενώ παράλληλα αυτοματοποιεί τη μεταφορά δεδομένων σε πίνακες Paimon, εξαλείφοντας την ανάγκη ξεχωριστών εργασιών εισαγωγής. Στο επίπεδο επεξεργασίας ροών, η χρήση του Apache Spark Structured Streaming [45] ως αντικαταστάτη του Flink θα επέτρεπε σύγκριση μεταξύ επεξεργασίας μικρο-δεσμίδων (micro-batch) και συνεχούς επεξεργασίας (continuous processing) σε ίδιο φόρτο. Τέλος, η χρήση διαχειριζόμενων υπηρεσιών ανά επίπεδο [57] όπως το Azure Event Hubs [261] αντί του Kafka, ή το Azure Synapse [262] αντί του Flink και του Trino θα παρείχε εκτίμηση της αντιστάθμισης μεταξύ κόστους και βαθμού ελέγχου σε σχέση με το Microsoft Fabric που αναφέρθηκε στην προηγούμενη υποενότητα.

Παραρτήματα

Παράρτημα Α΄

Υποδομή Terraform

Σε αυτό το παράρτημα παρατίθενται τα βασικότερα αποσπάσματα κώδικα Terraform που ορίζουν την υποδομή Azure του έργου. Για λόγους αναγνωσιμότητας παρουσιάζονται αντιπροσωπευτικά αποσπάσματα ενώ ο πλήρης κώδικας είναι διαθέσιμος στο GitHub [263].

Α΄.1 Σύνθεση modules

Η υποδομή Azure ορίζεται σε Terraform με τρεις επαναχρησιμοποιήσιμες ενότητες (modules): `aks` (Kubernetes cluster με δύο node pools - systempool για υπηρεσίες ελέγχου και userpool για φόρτους εργασίας), `adls` (Azure Storage Account) και `acr` (Azure Container Registry). Το αρχείο `infra/main.tf` ενορχηστρώνει τα τρια modules και θέτει αυτόματα στο `kubelet_identity` του `aks` τον απαραίτητο ρόλο ώστε το cluster να αποκτήσει δικαίωμα AcrPull στο `acr` χωρίς χειροκίνητη ρύθμιση.

```
module "aks" {
  source              = "./modules/aks"
  resource_group_name = var.resource_group_name
  cluster_name        = var.cluster_name
  location            = var.location
  system_node_pool = {
    name                = "systempool"
    vm_size             = var.system_vm_size
    node_count          = var.system_node_count
    min_count           = var.system_min_nodes
    max_count           = var.system_max_nodes
    enable_auto_scaling = true
  }
  user_node_pool = {
    name                = "userpool"
    vm_size             = var.user_vm_size
    node_count          = var.user_node_count
    min_count           = var.user_min_nodes
    max_count           = var.user_max_nodes
    enable_auto_scaling = true
  }
}
```

```
module "adls" {
  source               = "./modules/adls"
  resource_group_name  = var.resource_group_name
  location             = var.location
  storage_account_name = var.storage_account_name
}

module "acr" {
  source              = "./modules/acr"
  resource_group_name = var.resource_group_name
  location            = var.location
  acr_name            = var.acr_name
  kubelet_identity    = module.aks.kubelet_identity
}
```

Κάθε module εκθέτει outputs (π.χ. `kubelet_identity`, `storage_account_key`) που χρησιμοποιούνται ως είσοδοι σε άλλες modules. Η παραμετροποίηση ανά περιβάλλον γίνεται μέσω αρχείων `terraform.tfvars` / `terraform.secondary.tfvars`. Σημειώνεται ότι για τις ανάγκες της εργασίας χρησιμοποιήθηκαν δύο διαθέσιμα Azure Subscriptions.

Α΄.2 AKS module

Το module `aks` δημιουργεί το Kubernetes cluster (`azurerm_kubernetes_cluster`) με ταυτότητα `SystemAssigned` και δύο node pools, το systempool ως default node pool και το userpool ως δευτερεύον node pool κατάλληλο για φόρτους εργασίας. Και τα δύο υποστηρίζουν αυτόματη κλιμάκωση (autoscaling):

```
resource "azurerm_kubernetes_cluster" "aks" {
  name                = var.cluster_name
  location            = var.location
  resource_group_name = var.resource_group_name
  dns_prefix          = "${var.cluster_name}-dns"

  default_node_pool {
    name                = var.system_node_pool.name
    node_count          = var.system_node_pool.node_count
    vm_size             = var.system_node_pool.vm_size
    enable_auto_scaling = var.system_node_pool.enable_auto_scaling
    min_count           = var.system_node_pool.min_count
    max_count           = var.system_node_pool.max_count
    type                = "VirtualMachineScaleSets"
  }

  identity { type = "SystemAssigned" }
}

resource "azurerm_kubernetes_cluster_node_pool" "user" {
```

```
  name                = var.user_node_pool.name
  kubernetes_cluster_id = azurerm_kubernetes_cluster.aks.id
  vm_size             = var.user_node_pool.vm_size
  node_count          = var.user_node_pool.node_count
  enable_auto_scaling = var.user_node_pool.enable_auto_scaling
  min_count           = var.user_node_pool.min_count
  max_count           = var.user_node_pool.max_count
  mode                = "User"
  os_type             = "Linux"
}
```

Τα modules `adls` και `acr` ακολουθούν πανομοιότυπη δομή (ορισμός πόρου, μεταβλητές εισόδου, εξαγόμενες τιμές), αλλάζοντας μόνο τον τύπο πόρου Azure. Κάθε module περιλαμβάνει τα αρχεία `variables.tf` (ορισμοί μεταβλητών εισόδου) και `outputs.tf` (εξαγόμενες τιμές), τα οποία παραλείπονται για συντομία.

Παράρτημα Β΄

Dockerfiles και κώδικας προσομοιωτή IoT

Σε αυτό το παράρτημα παρατίθενται τα αρχεία Dockerfile που χρησιμοποιήθηκαν για τη δημιουργία container images, καθώς και ενδεικτικά αποσπάσματα κώδικα σε Go του προσομοιωτή IoT. Για λόγους αναγνωσιμότητας παρουσιάζονται αντιπροσωπευτικά αποσπάσματα ενώ ο πλήρης κώδικας είναι διαθέσιμος στο GitHub [263].

Β΄.1 Dockerfile - Flink με Paimon και Kafka

Το προσαρμοσμένο image Flink βασίζεται στο επίσημο `apache/flink:1.20.2` και προσθέτει τα JAR αρχεία Paimon, Azure FS και τον Kafka connector.

```
FROM apache/flink:1.20.2-scala_2.12

COPY lib/ /opt/flink/lib/

RUN set -e; \
    mkdir -p /opt/flink/plugins/flink-azure-fs-hadoop; \
    if [ -f /opt/flink/lib/flink-azure-fs-hadoop-1.20.2.jar ]; \
    then \
      mv /opt/flink/lib/flink-azure-fs-hadoop-1.20.2.jar \
         /opt/flink/plugins/flink-azure-fs-hadoop/; \
    fi
```

Β΄.2 Dockerfile - Προσομοιωτής IoT σε Go

Το image του προσομοιωτή βασίζεται στο image `golang:1.23-alpine`. Πρόκειται για ένα ελαφρύ image βασισμένο στη διανομή Alpine Linux, το οποίο περιλαμβάνει προεγκατεστημένη την αλυσίδα εργαλείων της Go (έκδοση 1.23) για τη μεταγλώττιση του κώδικα. Το Dockerfile με βάση το οποίο χτίζεται το image αντιγράφει τον πηγαίο κώδικα του προσομοιωτή, `go-iot-sim-producer.Dockerfile`, για την παραλλαγή χωρίς ομαδοποίηση.

```
FROM golang:1.23-alpine

WORKDIR /app
```

```
COPY docker/go/go_iot_sim_data_producer.go .

RUN go mod init go_iot_sim_data_producer \
    && go get github.com/segmentio/kafka-go@latest \
    && go mod tidy

RUN go build -o go_iot_sim_data_producer \
    go_iot_sim_data_producer.go

ENV KFK_BOOTSTRAP_SERVERS=kafka:9092 \
    KFK_TOPIC_HOME=iot-sensor-general \
    KFK_TOPIC_PLUG=iot-sensor-plug \
    PRODUCER_STREAM=both \
    SIM_PLUGS_PER_HOME=4 \
    SIM_SLEEP_SEC=1.0 \
    SIM_HOME_ID_START=1 \
    SIM_HOME_ID_END=42

ENTRYPOINT ["./go_iot_sim_data_producer"]
```

Το `trino-paimon.Dockerfile` και η παραλλαγή του παραπάνω για ομαδοποιημένους παραγωγούς (`go-iot-sim-producer-batched.Dockerfile`) ακολουθούν πανομοιότυπη δομή.

Β΄.3 Προσομοιωτής IoT - δομές δεδομένων

Ο προσομοιωτής γράφτηκε σε γλώσσα προγραμματισμού Go και τρέχει μία ρουτίνα (goroutine) ανά εγκατάσταση (κατοικία). Κάθε κατοικία παράγει μία εγγραφή ‘κατοικίας’ και μία εγγραφή ανά ‘πρίζα’, κάθε 1 δευτερόλεπτο. Παρακάτω φαίνονται οι δύο δομές τύπου JSON.

```
type HomeVal struct {
    HomeID      string  `json:"home_id"`
    PlugID      *string `json:"plug_id"`
    Voltage     float64 `json:"voltage"`
    Current     float64 `json:"current"`
    PowerFactor float64 `json:"power_factor"`
    PowerW      float64 `json:"power_w"`
    EnergyWh    float64 `json:"energy_wh"`
    Timestamp   string  `json:"timestamp"`
}

type PlugVal struct {
    HomeID      string  `json:"home_id"`
    PlugID      string  `json:"plug_id"`
    Voltage     float64 `json:"voltage"`
    Current     float64 `json:"current"`
    PowerFactor float64 `json:"power_factor"`
    PowerW      float64 `json:"power_w"`
```

```
    EnergyWh    float64 `json:"energy_wh"`
    Timestamp   string  `json:"timestamp"`
}
```

Β΄.4 Προσομοιωτής IoT - βρόχος homeSimulator

Η συνάρτηση `homeSimulator` εκτελεί τον κεντρικό βρόχο παραγωγής για μία εγκατάσταση (κατοικία). Κάθε κύκλος δημιουργεί τάση κανονικής κατανομής, διαχωρίζει τυχαία την ισχύ στις πρίζες, ενημερώνει τους σωρευτικούς (κεντρικούς) μετρητές ενέργειας και αποστέλλει τα μηνύματα μορφής JSON στα αντίστοιχα Kafka topics.

```
func homeSimulator(ctx context.Context, homeID string,
    plugIDs []string, sleepSec float64,
    producerStream string,
    writerHome, writerPlug *kafka.Writer) {

    rng := rand.New(rand.NewSource(
        time.Now().UnixNano() + int64(hashStr(homeID))))
    homeEnergy := 0.0
    plugEnergy := make(map[string]float64, len(plugIDs))

    ticker := time.NewTicker(
        time.Duration(sleepSec * float64(time.Second)))
    defer ticker.Stop()

    for {
        select {
        case <-ctx.Done(): return
        case <-ticker.C:
        }
        cycleStart := time.Now()
        voltage := roundF(rng.NormFloat64()*2+230, 1)
        pf      := roundF(0.8+rng.Float64()*0.2, 2)
        totalCurrent := roundF(0.2+rng.Float64()*19.8, 2)
        totalW  := voltage * totalCurrent * pf
        ts := time.Now().UTC().Format(
            "2006-01-02T15:04:05.000Z")

        // ... split power to plugs, send messages ...
        cycles.record(homeID, time.Since(cycleStart))
    }
}
```

Β΄.5 Kafka Writer - ρυθμίσεις

Β΄.5.1 Μη ομαδοποιημένη αποστολή (Unbatched Writer)

Κάθε μήνυμα αποστέλλεται χωρίς αναμονή επιβεβαίωσης (ACK) (`RequireNone`) και χωρίς ομαδοποίηση (`BatchSize: 1`), ώστε η κατανομή φόρτου να αντικατοπτρίζει ρεαλιστικό σενάριο πραγματικού κόσμου.

```
writerHome := &kafka.Writer{
    Addr:         kafka.TCP(bootstrapServers...),
    Topic:        kfkTopicHome,
    Balancer:     &kafka.Hash{},
    RequiredAcks: kafka.RequireNone,
    BatchSize:    1,
}
```

Β΄.5.2 Ομαδοποιημένη αποστολή (Batched Writer)

Στην ομαδοποιημένη παραλλαγή, τα μηνύματα συσσωρεύονται σε δέσμες (batches) πριν αποσταλούν στον Kafka broker. Το μέγεθος δέσμης (`BatchSize`) και το χρονικό όριο αναμονής δέσμης (`BatchTimeout`) είναι παραμετροποιήσιμα. Η δέσμη αποστέλλεται όταν συμπληρωθεί το μέγεθος ή λήξει το χρονικό όριο, όποιο συμβεί πρώτο.

```
writerHome := &kafka.Writer{
    Addr:                   kafka.TCP(bootstrapServers...),
    Topic:                  kfkTopicHome,
    Balancer:               &kafka.Hash{},
    RequiredAcks:           kafka.RequireNone,
    BatchSize:              batchSizeHome,
    BatchTimeout:           batchTimeout,
    AllowAutoTopicCreation: true,
}
```

Παράρτημα Γ΄

Kubernetes Deployments, ConfigMaps, Helm και SQL

Σε αυτό το παράρτημα παρατίθενται τα βασικότερα αρχεία Kubernetes Deployments (YAML manifests, ConfigMaps, αρχεία ρυθμίσεων Helm) καθώς και τα ερωτήματα Flink SQL που υλοποιούν την αλυσίδα εισαγωγής και συγκέντρωσης των δεδομένων. Για λόγους αναγνωσιμότητας παρουσιάζονται αντιπροσωπευτικά αποσπάσματα ενώ ο πλήρης κώδικας είναι διαθέσιμος στο Github repository [263] της εργασίας.

Γ΄.1 Kafka StatefulSet και αρχικοποίηση topics

Το Kafka εγκαθίσταται ως StatefulSet δύο κόμβων, στο namespace `data-ingest`. Κάθε pod αναλαμβάνει ταυτόχρονα ρόλο broker και controller και εξυπηρετείται από headless Service (`kafka-svc`) που επιτρέπει προσδιορισμό ανά pod μέσω DNS (`kafka-0.kafka-svc.data-ingest.svc.cluster.local`). Κάθε pod διαθέτει αποκλειστικό αποθηκευτικό χώρο PersistentVolumeClaim 128 GiB σε δίσκο διαχειριζόμενο από το Azure.

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: kafka
  namespace: data-ingest
spec:
  serviceName: kafka-svc
  replicas: 2
  template:
    spec:
      nodeSelector:
        agentpool: userpool
      containers:
      - name: kafka-container
        image: doughgle/kafka-kraft:latest
        resources:
          requests: { cpu: "2", memory: "4Gi" }
        env:
```

```
          - { name: REPLICAS,                   value: "2" }
          - { name: DEFAULT_REPLICATION_FACTOR,  value: "2" }
          - { name: DEFAULT_MIN_INSYNC_REPLICAS, value: "1" }
          - { name: CLUSTER_ID, value: "LRx92c9yQKws786HYosuBn" }
  volumeClaimTemplates:
    - metadata: { name: kafka-storage }
      spec:
        accessModes: ["ReadWriteOnce"]
        storageClassName: managed-csi
        resources:
          requests: { storage: 128Gi }
```

Μετά την εκκίνηση των brokers, μια εργασία τύπου Kubernetes Job (`kafka-init-topics`) αναμένει έως ότου αποκριθεί το broker API (έλεγχος ανά 2 s, μέχρι 90 προσπάθειες) και στη συνέχεια δημιουργεί τα δύο topics με παράγοντα αντιγραφής (`replication-factor`) 2 και 4 και 6 `partitions` αντίστοιχα, χρησιμοποιώντας το:

```
BOOT="kafka-0.kafka-svc.data-ingest.svc.cluster.local:9092,\
      kafka-1.kafka-svc.data-ingest.svc.cluster.local:9092"

/opt/kafka/bin/kafka-topics.sh --bootstrap-server "$BOOT" \
  --create --if-not-exists \
  --topic iot-sensor-general \
  --replication-factor 2 --partitions 4

/opt/kafka/bin/kafka-topics.sh --bootstrap-server "$BOOT" \
  --create --if-not-exists \
  --topic iot-sensor-plug \
  --replication-factor 2 --partitions 6
```

Το topic `iot-sensor-general` αποθηκεύει μετρήσεις επιπέδου κατοικίας, ενώ το topic `iot-sensor-plug` αποθηκεύει μετρήσεις επιπέδου πρίζας.

Γ΄.2 JobManager deployment σε Flink Session Mode

Ο Flink JobManager εκτελείται ως Deployment ενός αντιγράφου (`replicas: 1`) στο node pool `systempool` και παρέχει το REST API, στην πόρτα 8081, μέσω του οποίου τα sql-client pods υποβάλλουν streaming και batch εργασίες.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flink-jobmanager
  namespace: data-processing
spec:
  replicas: 1
  template:
    spec:
```

```
      nodeSelector:
        agentpool: systempool
      containers:
      - name: jobmanager
        image: acrdatalakeonk8swe01.azurecr.io/flink-paimon-kafka:latest
        args: ["jobmanager"]
        env:
          - name: AZURE_STORAGE_ACCOUNT_KEY
            valueFrom:
              secretKeyRef:
                name: azure-storage-secret
                key: storage-account-key
          - name: ENABLE_BUILT_IN_PLUGINS
            value: flink-azure-fs-hadoop-1.20.2.jar
        ports:
          - { containerPort: 6123, name: rpc }
          - { containerPort: 8081, name: webui }
          - { containerPort: 9249, name: metrics }
        resources:
          requests: { cpu: "500m", memory: "1Gi" }
          limits:  { cpu: "1",    memory: "2Gi" }
        volumeMounts:
          - name: flink-config-volume
            mountPath: /opt/flink/conf
      volumes:
      - name: flink-config-volume
        configMap:
          name: flink-config
```

Ο Flink TaskManager ακολουθεί πανομοιότυπη δομή με τρεις ουσιαστικές διαφορές:

- `args: ["taskmanager"]` όπου ορίζεται ο ρόλος του TaskManager
- 2 αντίγραφα (`replicas: 2`) τοποθετημένα στο `userpool`
- Αυξημένα αιτήματα πόρων της τάξης των 2 CPU και 8 GiB μνήμης, για υποστήριξη του παραλληλισμού εισαγωγής και εργασιών συγκέντρωσης

Γ΄.3 Flink ConfigMap - σημεία ελέγχου και μετρικές

Οι ρυθμίσεις Flink διανέμονται μέσω ConfigMap (`flink-config`) και προσαρτώνται στα pods των JobManager και TaskManager ως αρχεία στο φάκελο `/opt/flink/conf`. Το κλειδί του ADLS παρέχεται ξεχωριστά μέσω Kubernetes Secret και παραλείπεται από την παρακάτω λίστα ρυθμίσεων:

```
apiVersion: v1
kind: ConfigMap
metadata:
```

```
  name: flink-config
  namespace: data-processing
data:
  config.yaml: |+
    jobmanager.rpc.address: flink-jobmanager
    taskmanager.numberOfTaskSlots: 4
    jobmanager.memory.process.size: 1600m
    taskmanager.memory.process.size: 8192m
    parallelism.default: 4

    # Prometheus metrics reporter
    metrics.reporters: prom
    metrics.reporter.prom.factory.class: >-
      org.apache.flink.metrics.prometheus.PrometheusReporterFactory
    metrics.reporter.prom.port: 9249-9259

    # Checkpointing
    execution.checkpointing.mode: EXACTLY_ONCE
    execution.checkpointing.interval: 60 s
    execution.checkpointing.timeout: 600 s
    execution.checkpointing.min-pause: 30 s
    execution.checkpointing.unaligned.enabled: false
    execution.checkpointing.tolerable-failed-checkpoints: 5
    execution.checkpointing.dir: abfs://checkpoints@stdatalakeonk8swe01.dfs.core.windows.net
        /flink-checkpoints
```

Τα checkpoints αποθηκεύονται στο ADLS (`abfs://checkpoints@...`), παρέχοντας ανθεκτικότητα έναντι αποτυχιών node ή pod και δυνατότητα ανάκαμψης της ροής δεδομένων χωρίς απώλεια μηνυμάτων. Αυτή η προσέγγιση επιτρέπει την πλήρη αποκατάσταση της εργασίας Flink από τελευταίο γνωστό σημείο συνέπειας, εξασφαλίζοντας παράδοση των δεδομένων σε όλες τις συνθήκες αποτυχίας.

Γ΄.4 Flink SQL - αρχικοποίηση καταλόγου Paimon (`init.sql`)

Το αρχείο `init.sql` τοποθετείται στο ConfigMap `sql-session-init` και χρησιμοποιείται ως αρχείο αρχικοποίησης συνεδρίας (session initialization) μέσω της παραμέτρου `-i` του Flink SQL client, η οποία φορτώνει και εκτελεί ένα αρχείο SQL κατά την εκκίνηση της συνεδρίας, πριν από οποιαδήποτε άλλη εντολή. Δεν περιέχει διαπιστευτήρια καθώς αυτά παρέχονται μέσω παραμέτρων `-D` κατά την εκκίνηση, ώστε το ConfigMap να παραμένει ασφαλές για αποθήκευση στο Github repository.

```
CREATE CATALOG IF NOT EXISTS paimon WITH (
  'type'              = 'paimon',
  'warehouse'         = 'abfs://warehouse@
      stdatalakeonk8swe01.dfs.core.windows.net/paimon',
  'azure.account-name' = 'stdatalakeonk8swe01',
  'property-version' = '1'
```

```
);
USE CATALOG paimon;
CREATE DATABASE IF NOT EXISTS iot;
USE iot;
```

Η εντολή `USE CATALOG paimon` θέτει τον κατάλογο ως ενεργό για όλες τις επόμενες εντολές SQL της ίδιας συνεδρίας. Η εντολή `USE iot` επιλέγει τη βάση δεδομένων `iot` ώστε οι πίνακες να δημιουργούνται στο σωστό Paimon namespace.

Γ΄.5 Flink SQL - δημιουργία πινάκων Paimon (`bootstrap.sql`)

Το αρχείο `bootstrap.sql` τοποθετείται στο ConfigMap `paimon-bootstrap-sql` και εκτελείται εφάπαξ μέσω εργασίας τύπου Kubernetes Job `paimon-bootstrap` (στο namespace `data-processing`). Ο SQL client δέχεται πρώτα την παράμετρο `-i init.sql` (αρχικοποίηση καταλόγου) και στη συνέχεια εκτελεί το `bootstrap.sql` μέσω της παραμέτρου `-f`, η οποία εκτελεί ένα αρχείο SQL σε μη διαδραστική λειτουργία και τερματίζει τον client αυτόματα μετά την ολοκλήρωση. Τα διαπιστευτήρια Azure παρέχονται ως μεταβλητές περιβάλλοντος από το Kubernetes Secret `azure-storage-secret`.

```
command: ["/bin/bash","-lc"]
args:
  - |
    /opt/flink/bin/sql-client.sh embedded \
      -Drest.address=flink-jobmanager \
      -Drest.port=8081 \
      -Dfs.azure.account.key.<STORAGE_ACCOUNT>
         .dfs.core.windows.net=$AZURE_STORAGE_ACCOUNT_KEY \
      -Dazure.account-key=$AZURE_STORAGE_ACCOUNT_KEY \
      -i /sql/init.sql \
      -f /sql/bootstrap.sql
```

Η λογική SQL δημιουργεί δύο κύριους πίνακες Paimon οι οποίοι είναι ο `sensor_data_raw`, ο οποίος αποθηκεύει τις ακατέργαστες μετρήσεις σε επίπεδο δευτερολέπτου, και ο `sensor_data_hourly`, ο οποίος αποθηκεύει τα ωριαία αθροίσματα των μετρήσεων.

```
DROP TABLE IF EXISTS sensor_data_raw;

CREATE TABLE IF NOT EXISTS sensor_data_raw (
  dt STRING, home_id STRING, plug_id STRING,
  voltage DOUBLE, current_amp DOUBLE,
  power_factor DOUBLE, energy_wh DOUBLE,
  ts TIMESTAMP(3),
  WATERMARK FOR ts AS ts - INTERVAL '5' SECOND,
  PRIMARY KEY (dt, home_id, plug_id, ts) NOT ENFORCED
) PARTITIONED BY (dt) WITH (
  'bucket' = '6',
  'partition.expiration-time'          = '7 d',
  'partition.expiration-check-interval' = '1 h',
```

```
  'snapshot.time-retained'              = '7 d'
);

CREATE TABLE IF NOT EXISTS sensor_data_hourly (
  dt STRING, home_id STRING, plug_id STRING,
  window_start TIMESTAMP(3), window_end TIMESTAMP(3),
  avg_voltage DOUBLE, avg_current DOUBLE,
  avg_pf DOUBLE, energy_kwh DOUBLE,
  samples BIGINT, peak_power_w DOUBLE,
  PRIMARY KEY (dt, home_id, plug_id, window_start)
    NOT ENFORCED
) PARTITIONED BY (dt) WITH (
  'bucket' = '1',
  'snapshot.time-retained' = '30 d'
);
```

Οι πίνακες `sensor_data_30min` και `sensor_data_10min` ακολουθούν πανομοιότυπη δομή με τον ωριαίο πίνακα, αντικαθιστώντας μόνο τη χρονική ανάλυση παραθύρου.

Γ΄.6 Flink SQL - εισαγωγή Kafka → Paimon

Δημιουργούνται δύο προσωρινοί πίνακες-πηγές (Kafka JSON) για τα δύο topics (`iot-sensor-general`, `iot-sensor-plug`), οι οποίοι ενοποιούνται μέσω της εντολής `UNION ALL` και εισάγονται στον πίνακα ακατέργαστων δεδομένων `sensor_data_raw`.

```
CREATE TEMPORARY TABLE sensor_data_kafka_home (
  home_id STRING, plug_id STRING, voltage DOUBLE,
  `current` DOUBLE, power_factor DOUBLE,
  energy_wh DOUBLE, `timestamp` STRING,
  ts_ltz AS TRY_CAST(`timestamp` AS TIMESTAMP_LTZ(3)),
  WATERMARK FOR ts_ltz AS ts_ltz - INTERVAL '5' SECOND
) WITH (
  'connector' = 'kafka',
  'topic' = 'iot-sensor-general',
  'properties.bootstrap.servers' =
    'kafka-0.kafka-svc.data-ingest...:9092,...',
  'scan.startup.mode' = 'earliest-offset',
  'value.format' = 'json',
  'value.json.timestamp-format.standard' = 'ISO-8601',
  'value.json.ignore-parse-errors' = 'true'
);

-- UNION ALL of home + plug sources
INSERT INTO sensor_data_raw
SELECT
  DATE_FORMAT(ts, 'yyyy-MM-dd') AS dt,
  home_id,
  COALESCE(plug_id, 'general') AS plug_id,
```

```
  voltage, 'current' AS current_amp,
  power_factor, energy_wh, ts
FROM sensor_data_kafka_all
WHERE home_id IS NOT NULL;
```

Γ΄.7 Flink SQL - ωριαία συγκέντρωση

Η batch εργασία εκτελείται ωριαία μέσω CronJob και υπολογίζει μέσους όρους, σώρευση ενέργειας και μέγιστη ισχύ ανά κατοικία/πρίζα/ώρα. Για να παραμείνει σταθερό το κόστος ανάγνωσης, η σάρωση περιορίζεται στα δεδομένα των τελευταίων τριών (3) ωρών, ενώ στην έξοδο γράφονται μόνο τα παράθυρα που έχουν ήδη ολοκληρωθεί, ώστε να αποφεύγεται η εγγραφή ημιτελών αποτελεσμάτων.

```
CREATE TEMPORARY VIEW raw_no_wm AS
SELECT
  dt, home_id, plug_id, voltage, current_amp, power_factor, energy_wh,
  CAST(ts AS TIMESTAMP(3)) AS ts
FROM sensor_data_raw;

CREATE TEMPORARY VIEW raw_recent AS
SELECT * FROM raw_no_wm
WHERE dt >= DATE_FORMAT(
        CAST(CURRENT_TIMESTAMP AS TIMESTAMP(3))
          - INTERVAL '1' DAY, 'yyyy-MM-dd')
  AND ts >= CAST(CURRENT_TIMESTAMP AS TIMESTAMP(3))
          - INTERVAL '3' HOUR;

INSERT INTO sensor_data_hourly
SELECT
  DATE_FORMAT(window_start, 'yyyy-MM-dd')         AS dt,
  home_id,
  plug_id,
  window_start,
  window_start + INTERVAL '1' HOUR                AS window_end,
  AVG(voltage)                                    AS avg_voltage,
  AVG(current_amp)                                AS avg_current,
  AVG(power_factor)                               AS avg_pf,
  SUM(energy_wh) / 1000.0                         AS energy_kwh,
  COUNT(*)                                        AS samples,
  MAX(voltage * current_amp * power_factor)       AS peak_power_w
FROM (
  SELECT *,
    CAST(DATE_FORMAT(ts, 'yyyy-MM-dd HH:00:00') AS TIMESTAMP(3))
      AS window_start
  FROM raw_recent
) t
WHERE window_start + INTERVAL '1' HOUR
```

```
      <= CAST(CURRENT_TIMESTAMP AS TIMESTAMP(3))
GROUP BY home_id, plug_id, window_start;
```

Η ίδια λογική εφαρμόζεται και στις δεκάλεπτες και τριαντάλεπτες συγκεντρώσεις, με προσαρμογή του παραθύρου σάρωσης και της ευθυγράμμισης του window_start στα δέκα (10) και τριάντα (30) λεπτά αντίστοιχα.

Γ΄.8 Kubernetes CronJob - ωριαία batch συγκέντρωση

Η ωριαία συγκέντρωση εκτελείται από CronJob με χρονοπρογραμματισμό ¨10 * * * *’, δηλαδή 10 λεπτά μετά κάθε πλήρη ώρα, ώστε να δοθεί χρόνος στο streaming job να εγγράψει δεδομένα της τρέχουσας ώρας. Η παράμετρος `concurrencyPolicy: Forbid` εξασφαλίζει ότι δεν ξεκινά νέα εκτέλεση αν η προηγούμενη δεν έχει ολοκληρωθεί. Κάθε εκτέλεση εκκινεί pod `sql-client` που υποβάλλει το σενάριο SQL σε λειτουργία batch:

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hourly-aggregation
  namespace: data-processing
spec:
  schedule: "10 * * * *"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 1
  failedJobsHistoryLimit: 2
  jobTemplate:
    spec:
      backoffLimit: 1
      activeDeadlineSeconds: 900
      template:
        spec:
          restartPolicy: Never
          containers:
          - name: sql-client
            image: acrdatalakeonk8swe01.azurecr.io/flink-paimon-kafka:latest
            command: ["/bin/bash","-lc"]
            args:
              - |
                /opt/flink/bin/sql-client.sh embedded \
                  -Drest.address=flink-jobmanager \
                  -Drest.port=8081 \
                  -Dexecution.runtime-mode=batch \
                  -i /sql/init.sql \
                  -f /sql/hourly.sql
```

Σε αντίθεση με το streaming job εισαγωγής (που εκτελείται συνεχώς), το CronJob τερματίζεται μετά την ολοκλήρωση της batch εκτέλεσης, απελευθερώνοντας πόρους του TaskManager.

Γ΄.9 Τιμές Helm για Trino - κατάλογος Paimon

Η εγκατάσταση του Trino γίνεται μέσω Helm chart με προσαρμοσμένο αρχείο ρυθμίσεων (`trino-values.yaml`). Το τμήμα `image` ορίζει image από το ACR, το οποίο περιλαμβάνει τον Paimon connector. Το τμήμα `server` ορίζει δύο workers και ρυθμίζει το JVM με μέγιστο heap 2 GiB:

```
image:
  repository: <ACR_NAME>.azurecr.io/trino-paimon
  tag: "latest"
  pullPolicy: Always

server:
  workers: 2
  jvm:
    maxHeapSize: "2G"
    gcMethod:
      type: "G1"
      g1:
        heapRegionSize: "32M"
```

Ο coordinator και οι workers τοποθετούνται στο `userpool` μέσω `nodeSelector`. Η ρύθμιση `includeCoordinator` ίση με `true` επιτρέπει στον coordinator να εκτελεί και tasks, κάτι χρήσιμο σε μικρά clusters.

```
coordinator:
  nodeSelector:
    agentpool: userpool
  config:
    nodeScheduler:
      includeCoordinator: true
  additionalJVMConfig:
    - --add-opens=java.base/sun.nio.ch=ALL-UNNAMED
    - --add-opens=java.base/java.nio=ALL-UNNAMED

worker:
  nodeSelector:
    agentpool: userpool
  additionalJVMConfig:
    - --add-opens=java.base/sun.nio.ch=ALL-UNNAMED
    - --add-opens=java.base/java.nio=ALL-UNNAMED
```

Η σύνδεση του Trino με τον αποθηκευτικό χώρο Paimon ρυθμίζεται μέσω του τμήματος `catalogs`. Το native Azure filesystem (`fs.native-azure.enabled=true`) χρησιμοποιείται αντί του Hadoop FS, αποφεύγοντας τη σύγκρουση κλάσεων HDFS που εμφανίζεται με τον Paimon connector (Ενότητα 7.12). Το κλειδί Azure Storage παρέχεται μέσω μεταβλητής περιβάλλοντος από το `azure-storage-secret`.

```
catalogs:
  paimon: |-
    connector.name=paimon
    warehouse=abfs://warehouse@<STORAGE_ACCOUNT>.dfs.core.windows.net/paimon
    fs.native-azure.enabled=true
    fs.hadoop.enabled=false
    azure.auth-type=ACCESS_KEY
    azure.access-key=${ENV:AZURE_STORAGE_ACCOUNT_KEY}

env:
  - name: AZURE_STORAGE_ACCOUNT_KEY
    valueFrom:
      secretKeyRef:
        name: azure-storage-secret
        key: storage-account-key
```

Γ΄.10 Τιμές Helm για kube-prometheus-stack

Η παρακολούθηση της λύσης βασίζεται στο `kube-prometheus-stack`. Ενεργοποιούνται ServiceMonitor για το Flink στη θύρα 9249 και εκτελείται το Grafana με πρόσθετα dashboards μέσω ConfigMaps.

```
prometheus:
  prometheusSpec:
    retention: 15d
    storageSpec:
      volumeClaimTemplate:
        spec:
          storageClassName: managed-csi
          resources:
            requests: { storage: 50Gi }
    additionalScrapeConfigs:
      - job_name: flink-taskmanager
        kubernetes_sd_configs:
          - role: pod
            namespaces:
              names: [data-processing]
        relabel_configs:
          - source_labels: [__meta_kubernetes_pod_label_component]
            regex: taskmanager
            action: keep
          - source_labels: [__address__]
            regex: (.+):.*
            target_label: __address__
            replacement: $1:9249

grafana:
```

```
  service:
    type: LoadBalancer
    port: 3000
```

Γ΄.11 Σχήματα πινάκων Paimon

Παρακάτω παρατίθενται τα σχήματα των πινάκων Paimon που δημιουργούνται κατά τη φάση εκκίνησης (bootstrap).

Γ΄.11.1 Ακατέργαστος πίνακας (sensor_data_raw)

Ο πίνακας ακατέργαστων δεδομένων αποθηκεύει κάθε εγγραφή τηλεμετρίας σε επίπεδο δευτερολέπτου, με διακριτικό κλειδί το συνδυασμό [κατοικία/πρίζα]. Κατατέμνεται ανά ημέρα (dt) και εφαρμόζει αυτόματη λήξη διατήρησης των δεδομένων στις 7 ημέρες.

```
CREATE TABLE IF NOT EXISTS sensor_data_raw (
  dt            STRING,
  home_id       STRING,
  plug_id       STRING,
  voltage       DOUBLE,
  current_amp   DOUBLE,
  power_factor  DOUBLE,
  energy_wh     DOUBLE,
  ts            TIMESTAMP(3),
  WATERMARK FOR ts AS ts - INTERVAL '5' SECOND,
  PRIMARY KEY (dt, home_id, plug_id, ts) NOT ENFORCED
)
PARTITIONED BY (dt)
WITH (
  'bucket'                          = '6',
  'partition.expiration-time'       = '7 d',
  'partition.expiration-check-interval' = '1 h',
  'snapshot.time-retained'          = '7 d'
);
```

Γ΄.11.2 Πίνακας ωριαίων συγκεντρώσεων (sensor_data_hourly)

Ο πίνακας συγκεντρώσεων αποθηκεύει τα αποτελέσματα της ωριαίας batch εργασίας (μέση τάση, μέσο ρεύμα, μέσος συντελεστής ισχύος, ενέργεια σε kWh, αριθμός δειγμάτων, μέγιστη ισχύς).

```
CREATE TABLE IF NOT EXISTS sensor_data_hourly (
  dt            STRING,
  home_id       STRING,
  plug_id       STRING,
  window_start  TIMESTAMP(3),
  window_end    TIMESTAMP(3),
```

```
  avg_voltage   DOUBLE,
  avg_current   DOUBLE,
  avg_pf        DOUBLE,
  energy_kwh    DOUBLE,
  samples       BIGINT,
  peak_power_w  DOUBLE,
  PRIMARY KEY (dt, home_id, plug_id, window_start)
    NOT ENFORCED
)
PARTITIONED BY (dt)
WITH (
  'bucket'                = '1',
  'snapshot.time-retained' = '30 d'
);
```

Οι πίνακες δεκάλεπτων (sensor_data_10min) και τριαντάλεπτων (sensor_data_30min) συγκεντρώσεων έχουν πανομοιότυπο σχήμα με τον sensor_data_hourly (ίδιες στήλες, ίδιο πρωτεύον κλειδί, `'bucket' = '1'`, `'snapshot.time-retained' = '30 d'`), διαφέροντας μόνο ως προς το εύρος του χρονικού παραθύρου (10 και 30 λεπτά αντίστοιχα). Έτσι, ερωτήματα και dashboards μπορούν να εναλλάσσουν πίνακα συγκέντρωσης με μία μόνο μετονομασία.

Γ'.12 Deployment προσομοιωτή IoT (iot-sim-producers.yaml)

Ο προσομοιωτής αναπτύσσεται ως Deployment στο namespace `data-ingest`. Οι μεταβλητές περιβάλλοντος ορίζουν τον αριθμό κατοικιών, πριζών ανά κατοικία, ρυθμό αποστολής και τα Kafka topics:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: iot-sim-producer
  namespace: data-ingest
spec:
  replicas: 1
  selector:
    matchLabels: { app: iot-sim-producer }
  template:
    spec:
      containers:
      - name: producer
        image: <ACR_LOGIN_SERVER>/iot_sim_data_producer:latest
        env:
          - name: KFK_BOOTSTRAP_SERVERS
            value: "kafka-0.kafka-svc....:9092,
                    kafka-1.kafka-svc....:9092"
          - { name: PRODUCER_STREAM,    value: "both" }
          - { name: KFK_TOPIC_HOME,     value: "iot-sensor-general" }
          - { name: KFK_TOPIC_PLUG,     value: "iot-sensor-plug" }
```

```
          - { name: SIM_HOME_ID_START,  value: "1" }
          - { name: SIM_HOME_ID_END,    value: "100" }
          - { name: SIM_PLUGS_PER_HOME, value: "4" }
          - { name: SIM_SLEEP_SEC,      value: "1.0" }
        resources:
          requests: { cpu: "500m", memory: "256Mi" }
          limits:   { cpu: "2",    memory: "1Gi" }
```

Παράρτημα Δ΄

Σωληνώσεις (Pipelines) CI/CD - GitHub Actions

Σε αυτό το παράρτημα παρατίθενται τα βασικότερα αποσπάσματα των pipelines GitHub Actions που αυτοματοποιούν την κατασκευή, δοκιμή και ανάπτυξη της υποδομής και των εφαρμογών. Και τα δύο κύρια pipelines ενεργοποιούνται χειροκίνητα μέσω `workflow_dispatch` με επιλογή περιβάλλοντος ανάμεσα στα δύο διαθέσιμα Azure Subscriptions.

Δ΄.1 `apps-ci.yml` - Εγκατάσταση εφαρμογών

Το κύριο pipeline ενορχηστρώνει σε μία εκτέλεση την κατασκευή του Flink image, τη δημιουργία Kubernetes secrets, το deployment των manifests και την εγκατάσταση Helm charts (Trino, kube-prometheus-stack). Η επιλογή περιβάλλοντος εξυπηρετείται μέσω JSON lookup στις μεταβλητές `env`:

```
on:
  workflow_dispatch:
    inputs:
      environment:
        description: 'Select Azure Subscription/Environment'
        required: true
        default: 'theomper.com'
        type: choice
        options: [theomper.com, azure_sponsorship]

env:
  ACR_LOGIN_SERVER: ${{ fromJSON('{
    "azure_sponsorship":"acrdatalakeonk8swe01.azurecr.io",
    "theomper.com":"acrdatalakeonk8sdevsec01.azurecr.io"
    }')[inputs.environment] }}
  MANIFESTS_DIR: ${{ fromJSON('{
    "azure_sponsorship":"k8s/manifests",
    "theomper.com":"k8s/manifests-mct"
    }')[inputs.environment] }}
```

Τα βασικά βήματα εκτέλεσης είναι τα ακόλουθα:

```
steps:
  - name: Build and push Flink + Paimon image
    run: |
      docker build -t "$IMAGE_SHA" -t "$IMAGE_LATEST" \
        -f docker/flink-paimon-kafka.Dockerfile docker
      docker push "$IMAGE_SHA"
      docker push "$IMAGE_LATEST"

  - name: Deploy manifests
    run: |
      kubectl apply -f k8s/manifests/data-ingest.yaml
      kubectl apply -f $MANIFESTS_DIR/data-processing.yaml
      kubectl apply -f $FLINK_STANDALONE_DIR/flink-conf-configmap.yaml
      kubectl apply -f $IOT_SIM_MANIFEST

  - name: Deploy Trino with Helm
    run: |
      helm upgrade --install trino-standalone trino/trino \
        -n analytics -f $TRINO_VALUES_FILE

  - name: Deploy Prometheus + Grafana
    run: |
      helm upgrade --install kps \
        prometheus-community/kube-prometheus-stack \
        -n monitoring -f $KPS_VALUES_FILE
```

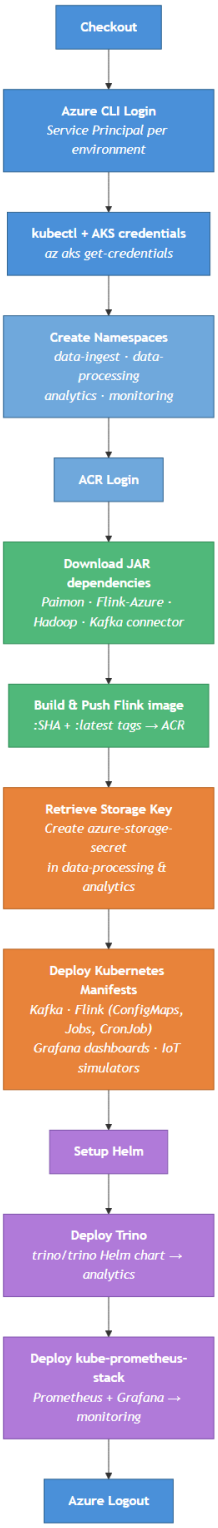
Δ΄.2 Διαγράμματα βημάτων διοχετεύσεων

Τα διαγράμματα που ακολουθούν απεικονίζουν τη σειρά εκτέλεσης βημάτων των δύο κύριων pipelines (για λεπτομερή περιγραφή σκοπού και εξαρτήσεων βλ. Ενότητα 7.11).

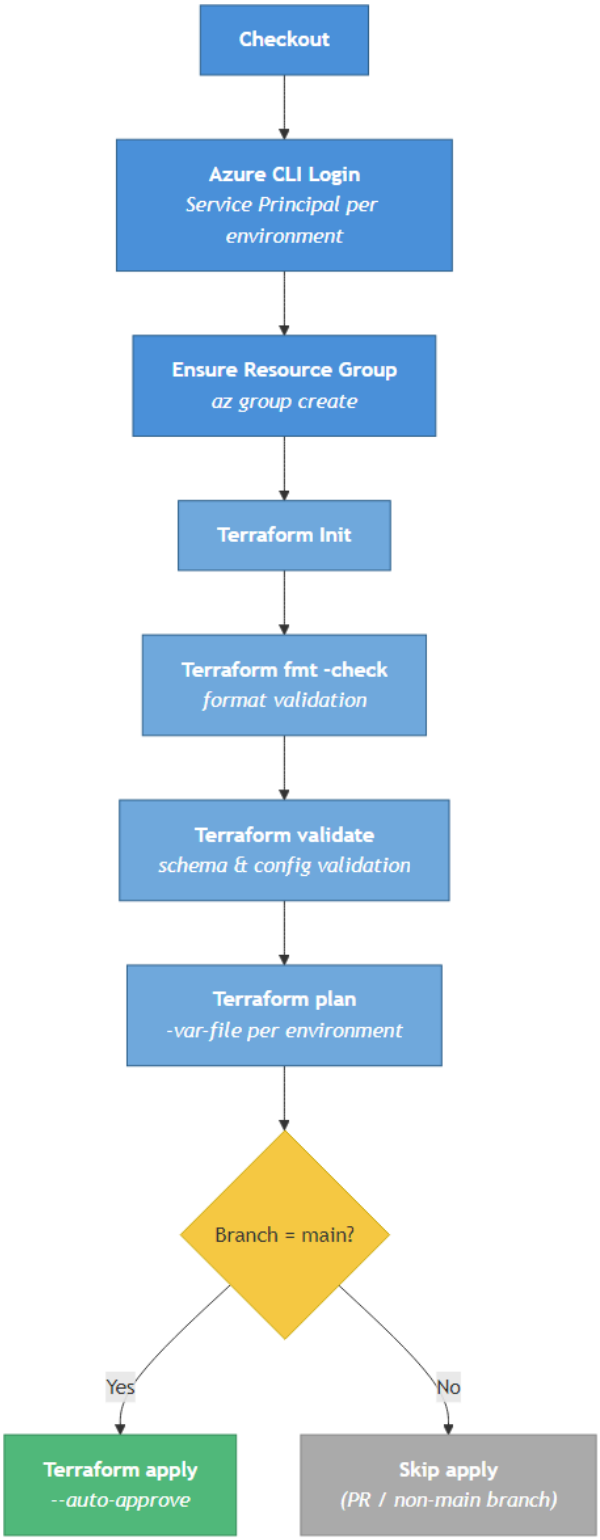
`apps-ci.yml` Ενορχηστρώνει σε μία εκτέλεση την κατασκευή του Flink image, τη δημιουργία Kubernetes secrets, το deployment των manifests και την εγκατάσταση Helm charts (Trino, kube-prometheus-stack).

`infra-ci.yml` Δημιουργεί την υποδομή στο Azure μέσω Terraform.

Τα pipelines `producer-ci.yml` και `trino-paimon-image.yml` ακολουθούν απλούστερη δομή και εξειδικεύονται στην κατασκευή και δημοσίευση container images για τον προσομοιωτή IoT και το Trino αντίστοιχα.



Εικόνα Δ΄.1: *Σειρά βημάτων του pipeline* `apps-ci.yml` *από τη σύνδεση στο Azure έως το deployment των εφαρμογών του συστήματος*



Εικόνα Δ'.2: *Σειρά βημάτων του pipeline* `infra-ci.yml` *για την επαλήθευση και εφαρμογή της Terraform*

Παράρτημα Ε΄

Ερωτήματα αναφοράς - Trino benchmark queries

Κατά την πειραματική αξιολόγηση εκτελέστηκαν πέντε ερωτήματα ανά κύκλο μέτρησης μέσω του Trino, με 5 επαναλήψεις το κάθε ένα, ώστε να μετρηθεί ο χρόνος απόκρισης (σε δευτερόλεπτα) υπό πραγματικό φόρτο εισαγωγής δεδομένων. Τα Q1-Q5 σχεδιάστηκαν ώστε να εκθέτουν αναλυτικές δυνατότητες και να παρουσιάζουν ευδιάκριτη κλιμάκωση με τον όγκο των δεδομένων.

Q1 - COUNT - μέτρηση γραμμών πίνακα ακατέργαστων δεδομένων

```
SELECT COUNT(*)
FROM paimon.iot.sensor_data_raw;
```

Επιστρέφει το συνολικό πλήθος εγγραφών του πίνακα ακατέργαστων δεδομένων. Πλήρης σάρωση χωρίς φιλτράρισμα. Αυξάνεται γραμμικά με τον όγκο δεδομένων και χρησιμοποιείται ως βασική μέτρηση αναφοράς.

Q2 - GROUP BY - ομαδοποίηση με πολλαπλές συναθροίσεις

```
SELECT   home_id, plug_id,
         AVG(voltage)                              AS avg_voltage,
         AVG(current_amp)                          AS avg_current,
         MAX(voltage * current_amp * power_factor) AS peak_power_w,
         COUNT(*)                                  AS samples
FROM     paimon.iot.sensor_data_raw
GROUP BY home_id, plug_id
ORDER BY peak_power_w DESC
LIMIT    50;
```

Επιστρέφει τους 50 συνδυασμούς κατοικίας και πρίζας με τη μεγαλύτερη στιγμιαία ισχύ, μαζί με τη μέση τάση, το μέσο ρεύμα και το πλήθος δειγμάτων. Πλήρης σάρωση με ομαδοποίηση μέσω της GROUP BY και πολλαπλές συναθροίσεις (AVG, MAX, COUNT) ανά κατοικία και πρίζα. Κλιμακώνεται με τον όγκο δεδομένων και δοκιμάζει αναλυτικό φόρτο.

Q3 - Συνάρτηση παραθύρου (rolling average) στον πίνακα ακατέργαστων δεδομένων

```
SELECT home_id, ts, voltage,
       AVG(voltage) OVER (
         PARTITION BY home_id
         ORDER BY ts
         ROWS BETWEEN 59 PRECEDING AND CURRENT ROW
       ) AS rolling_avg_60
FROM   paimon.iot.sensor_data_raw
WHERE  plug_id = 'general'
ORDER BY ts DESC
LIMIT  100;
```

Επιστρέφει τις τελευταίες 100 μετρήσεις τάσης ανά κατοικία, μαζί με τον κυλιόμενο μέσο όρο των 60 προηγούμενων δειγμάτων. Η φράση OVER ορίζει μια συνάρτηση παραθύρου, η οποία υπολογίζει μια τιμή σε σχέση με ένα σύνολο γραμμών γύρω από την τρέχουσα γραμμή, χωρίς να συμπτύσσει τα αποτελέσματα σε μία γραμμή όπως το GROUP BY. Το PARTITION BY χωρίζει τα δεδομένα σε ομάδες (εδώ ανά κατοικία), ενώ το ORDER BY καθορίζει τη σειρά των γραμμών μέσα σε κάθε ομάδα (εδώ χρονολογικά με βάση τη χρονοσφραγίδα ts). Βαρύ ερώτημα, καθώς ο συνδυασμός αυτός απαιτεί ταξινόμηση ολόκληρου του συνόλου δεδομένων. Παρουσιάζει δραματική κλιμάκωση με τον αριθμό εγγραφών.

Q4 - JOIN ακατέργαστου ↔ δεκάλεπτου πίνακα

```
SELECT   r.home_id,
         COUNT(*)                              AS raw_samples,
         t.energy_kwh,
         AVG(r.voltage)                        AS raw_avg_voltage,
         t.avg_voltage                         AS tenmin_avg_voltage,
         ABS(AVG(r.voltage) - t.avg_voltage)   AS voltage_drift
FROM     paimon.iot.sensor_data_raw    r
JOIN     paimon.iot.sensor_data_10min  t
         ON  r.home_id = t.home_id
         AND r.plug_id = t.plug_id
         AND r.dt      = t.dt
WHERE    r.plug_id = 'general'
         AND t.samples >= (SELECT CAST(MAX(samples) AS DOUBLE) * 0.8
                           FROM paimon.iot.sensor_data_10min)
GROUP BY r.home_id, t.energy_kwh, t.avg_voltage
ORDER BY voltage_drift DESC
LIMIT    20;
```

Επιστρέφει τις 20 κατοικίες με τη μεγαλύτερη απόκλιση τάσης μεταξύ ακατέργαστων και δεκάλεπτων δεδομένων, μαζί με τον αριθμό δειγμάτων και την κατανάλωση ενέργειας. Συνδέει τον πίνακα ακατέργαστων δεδομένων με τον πίνακα δεκάλεπτων συγκεντρωτικών δεδομένων για επαλήθευση συνέπειας δεδομένων. Κλιμακώνεται με τον

όγκο του πίνακα ακατέργαστων δεδομένων και αποδεικνύει αναλυτικό μοτίβο JOIN.

Q5 - Εκατοστημόρια και στατιστική ανάλυση

```
SELECT home_id,
       approx_percentile(voltage, 0.50) AS p50_voltage,
       approx_percentile(voltage, 0.95) AS p95_voltage,
       approx_percentile(voltage, 0.99) AS p99_voltage,
       stddev(voltage)                  AS stddev_voltage
FROM   paimon.iot.sensor_data_raw
WHERE  plug_id = 'general'
GROUP BY home_id
ORDER BY stddev_voltage DESC
LIMIT  20;
```

Επιστρέφει τις 20 κατοικίες με τη μεγαλύτερη τυπική απόκλιση τάσης, μαζί με τα εκατοστημόρια p50, p95 και p99. Τα εκατοστημόρια εκφράζουν την τιμή κάτω από την οποία βρίσκεται ένα συγκεκριμένο ποσοστό των παρατηρήσεων. Δηλαδή το p50 (διάμεσος) σημαίνει ότι το 50% των μετρήσεων είναι μικρότερο ή ίσο, το p95 ότι μόνο το 5% υπερβαίνει αυτή την τιμή, και το p99 ότι μόνο το 1% την υπερβαίνει. Η συνάρτηση `approx_percentile` υπολογίζει τα εκατοστημόρια προσεγγιστικά μέσω δομών σκίτσου (sketch), γεγονός που μειώνει τη χρήση μνήμης σε σύγκριση με ακριβή υπολογισμό. Βαρύ στατιστικό ερώτημα που κλιμακώνεται με τον αριθμό εγγραφών ανά κατοικία.

Παράρτημα ΣΤ΄

Σενάριο ενορχήστρωσης πειραμάτων

Το σενάριο `run-experiment-combined-improved.ps1` αυτοματοποιεί πλήρως τον κύκλο μέτρησης που περιγράφεται στην Ενότητα 8. Γράφτηκε σε γλώσσα σεναρίων PowerShell και εκτελεί σειριακά κάθε βαθμίδα φόρτου, με δύο υπο-φάσεις ανά βαθμίδα (μη ομαδοποιημένοι και ομαδοποιημένοι παραγωγοί). Παρακάτω παρουσιάζονται αντιπροσωπευτικά αποσπάσματα από τα κύρια τμήματα του σεναρίου.

ΣΤ΄.1 Παράμετροι και σταθερές

Το script δέχεται ως παραμέτρους τον πίνακα βαθμίδων, τους χρόνους προθέρμανσης και εκτέλεσης, καθώς και τον χρόνο εκκαθάρισης μετά τη διακοπή της παραγωγής δεδομένων.

```
[CmdletBinding()]
param(
    [Parameter(Mandatory = $true)]
    [int[]]$Tiers,
    [int]$WarmupMinutes        = 5,
    [int]$RunMinutes           = 30,
    [int]$FlushCooldownSeconds = 120,
    [string]$ExperimentTag     = (Get-Date -Format "yyyyMMdd_HHmmss"),
    [switch]$SkipCleanup,
    [switch]$ClusterShutdown
)
```

Οι σταθερές ορίζουν τα ονόματα υπηρεσιών εντός Kubernetes, τα Kafka topics, τις θύρες που ακούει το REST API του Flink και τον αριθμό κατοικιών (3.500) ανά pod προσομοιωτών IoT.

```
$KafkaPod            = "kafka-0"
$KafkaNamespace      = "data-ingest"
$TopicGeneral        = "iot-sensor-general"
$TopicPlug           = "iot-sensor-plug"
$FlinkJobManagerSvc  = "flink-jobmanager.data-processing.svc.cluster.local"
$FlinkRestPort       = 8081
$PlugsPerHome        = 4
$HomesPerPod         = 3500
```

Κάθε βαθμίδα αντιστοιχίζεται σε αναγνωριστικό εκτέλεσης (Run ID), ώστε τα αποτελέσματα να ενημερώνουν αυτόματα τον πίνακα αποτελεσμάτων σε μορφή Markdown:

```
$TierRunIdIndex = @{
    50 = 1; 500 = 2; 1500 = 3; 3500 = 4; 5500 = 5
    7500 = 6; 10000 = 7; 12500 = 8; 15000 = 9; 17500 = 10
}

function Get-RunId([int]$Homes, [string]$Phase) {
    $idx = $TierRunIdIndex[$Homes]
    if ($Phase -eq "nonbatched") { return "N$idx" }
    else { return "B$idx" }
}
```

ΣΤ΄.2 Επικύρωση πριν την εκτέλεση

Πριν ξεκινήσει ο κύκλος μετρήσεων, η συνάρτηση `Test-PreRunHealth` ελέγχει τέσσερα υποσυστήματα. Το Flink REST API, τα Kafka topics, το Trino και το Azure Storage. Εάν κάποιος έλεγχος αποτύχει, εμφανίζεται προειδοποίηση αλλά η εκτέλεση συνεχίζεται.

```
function Test-PreRunHealth {
    $allOk = $true

    # Flink REST API
    $flinkResp = kubectl exec -n data-processing `
        deployment/flink-jobmanager -- `
        curl -s -o /dev/null -w "%{http_code}" `
        "http://localhost:${FlinkRestPort}/overview"
    if (($flinkResp -join "").Trim() -ne "200") { $allOk = $false }

    # Kafka topics
    $topics = kubectl exec -n $KafkaNamespace $KafkaPod -- `
        /opt/kafka/bin/kafka-topics.sh `
        --bootstrap-server $KafkaBrokerInternal --list
    if ($topicList -notmatch $TopicGeneral) { $allOk = $false }

    # Trino health check
    $trinoCheck = Invoke-TrinoQuery "SELECT 1" "health-check"
    if ($trinoCheck.Trim() -ne "1") { $allOk = $false }

    # Azure Storage
    $azCheck = az storage container exists `
        --account-name $StorageAccountName `
        --name warehouse --auth-mode login --output tsv
    if ($azCheck -notmatch "True") { $allOk = $false }

    return $allOk
}
```

ΣΤ΄.3 Επαναφορά περιβάλλοντος

Κάθε υπο-φάση ξεκινά με πλήρη επαναφορά (full reset), ώστε τα αποτελέσματα να μην επηρεάζονται από προηγούμενα δεδομένα. Η συνάρτηση `Reset-Environment` εκτελεί τα ακόλουθα βήματα:

1. Διαγραφή των pods παραγωγών δεδομένων (`go-iot-sim-producer`), ώστε να σταματήσει η εισροή νέων δεδομένων.
2. Ακύρωση όλων των ενεργών εργασιών Flink μέσω του REST API του JobManager, διασφαλίζοντας ότι δεν θα υπάρχουν εν εξελίξει επεξεργασίες.
3. Αποκλιμάκωση των στιγμιοτύπων (`replicas`) του Flink στο μηδέν πριν από τον καθαρισμό του αποθηκευτικού χώρου, για να αποφευχθούν κλειδώματα αρχείων ή σφάλματα πρόσβασης.
4. Διαγραφή όλων των Paimon blobs από το ADLS, επαναφέροντας τη βάση δεδομένων σε κενή κατάσταση.
5. Αναδημιουργία των Kafka topics στις αρχικές τους καταστάσεις, έτοιμα για τη δεδομένη υπο-φάση του πειράματος.
6. Επανεκκίνηση του Flink cluster και αναμονή μέχρι να είναι έτοιμο να δεχθεί εργασίες.

```
function Reset-Environment {
    # 1.
    kubectl delete deployments -n $KafkaNamespace ‘
        -l "app=go-iot-sim-producer" --ignore-not-found
    kubectl delete deployments -n $KafkaNamespace ‘
        -l "app=go-iot-sim-producer-batched" --ignore-not-found

    # 2.                                Flink          REST API
    $jobs = (Get-FlinkJobsJson | ConvertFrom-Json).jobs
    foreach ($j in $jobs) {
        if ($j.status -in @("RUNNING","CREATED","RESTARTING")) {
            kubectl exec -n data-processing ‘
                deployment/flink-jobmanager -- curl -s -X PATCH ‘
                "http://localhost:${FlinkRestPort}/jobs/$($j.id)?mode=cancel"
        }
    }

    # 3.                    Flink

    kubectl scale deployment flink-taskmanager ‘
        -n data-processing --replicas=0
```

```
    kubectl scale deployment flink-jobmanager `
        -n data-processing --replicas=0

    # 4.                  Paimon blobs        ADLS
    az storage blob delete-batch `
        --account-name $StorageAccountName `
        --source warehouse --pattern "*" --auth-mode login

    # 5.                            Kafka topics
    kubectl exec -n $KafkaNamespace $KafkaPod -- `
        /opt/kafka/bin/kafka-topics.sh `
        --bootstrap-server $KafkaBrokerInternal `
        --create --topic $TopicGeneral `
        --replication-factor 2 --partitions 4

    # 6.                           Flink cluster
    kubectl scale deployment flink-jobmanager `
        -n data-processing --replicas=1
    kubectl scale deployment flink-taskmanager `
        -n data-processing --replicas=2
    Wait-FlinkClusterReady -timeoutSec 180
}
```

ΣΤ΄.4 Βρόχος εκτέλεσης ανά βαθμίδα

Ο κύριος βρόχος (loop) επαναλαμβάνεται για κάθε στοιχείο του πίνακα `$Tiers`. Για κάθε βαθμίδα εκτελούνται δύο πανομοιότυπες υπο-φάσεις (μη ομαδοποιημένη και ομαδοποιημένη παραγωγή δεδομένων), καθεμία με τον δικό της κύκλο επαναφοράς.

```
foreach ($homes in $Tiers) {
    # === SUB-PHASE A: NONBATCHED ===
    if (-not $SkipCleanup) { Reset-Environment }
    $flinkOk = Invoke-DeployWithRetry -maxAttempts 3
    if (-not $flinkOk) { continue }

    # A1.
    & $GenerateGoScript $homes
    kubectl apply -f $nbYamlFile

    # A2.                          (5 min)
    Start-Sleep -Seconds ($WarmupMinutes * 60)

    # A3.                     T0
    $nbT0 = Collect-Metrics $homes "T0-baseline" "nonbatched"
    $nbCkptT0 = Get-FlinkCheckpointCounts

    # A4.                                     (30 min)
```

```
    Start-Sleep -Seconds ($RunMinutes * 60)

    # A5.                                        (pods                          )
    $goStats = Parse-GoProducerStats "go-iot-sim-producer"

    # A6.                    T1 (                                              )
    $nbT1 = Collect-Metrics $homes "T1-final" "nonbatched"

    # A7.                                  +                      (120s)
    kubectl delete -f $nbYamlFile
    Start-Sleep -Seconds $FlushCooldownSeconds

    # A8.                                                 + checkpoints T1
    Invoke-ManualAggregation "${experimentLabel}-nb"
    $nbCkptT1 = Get-FlinkCheckpointCounts -JobId $nbFlinkJobId

    # A9. Prometheus, Azure SA, Trino benchmark
    $nbGranularProm = Collect-GranularPrometheus
    $nbAzSA = Get-AzureStorageMetrics $startUtc $endUtc
    $nbTrinoBench = Run-TrinoBenchmark -iterations 5

    # A10.
    Update-ExperimentMatrix -Homes $homes -Phase "nonbatched" ...

    # === SUB-PHASE B: BATCHED (                ) ===
    # ...
}
```

Η `Invoke-DeployWithRetry` εφαρμόζει τα δηλωτικά αρχεία επεξεργασίας, υποβάλλει τη streaming εργασία στο Flink και επαληθεύει ελέγχοντας μέσω τριών διαδοχικών κλήσεων στο REST API ότι η εργασία παραμένει σε κατάσταση RUNNING. Σε περίπτωση αποτυχίας επαναλαμβάνει έως τρεις φορές την προσπάθεια εκτέλεσης.

ΣΤ΄.5 Συλλογή μετρικών

Η συνάρτηση `Collect-Metrics` εκτελεί παράλληλα ερωτήματα προς το Trino, το Prometheus, το Flink REST API και το kubectl top, επιστρέφοντας έναν πίνακα με όλους τους δείκτες απόδοσης. Τα ερωτήματα στο Prometheus εκτελούνται παράλληλα μέσω `Start-Job`, ενσωματωμένου cmdlet του PowerShell που εκκινεί ασύγχρονες εργασίες σε ξεχωριστές διεργασίες, επιτρέποντας την ταυτόχρονη εκτέλεση πολλαπλών ερωτημάτων.

```
function Collect-GranularPrometheus([int]$windowMinutes) {
    $window = "${windowMinutes}m"
    $gm = [ordered]@{}

    $gm["kafka_cpu"] = Query-PromAvg '
        "avg_over_time(
```

```
            sum by (pod)(
              rate(container_cpu_usage_seconds_total{
                namespace='"data-ingest'",
                pod=~'"kafka.*'"}[5m])
            )[${window}:]
          )"
    $gm["kafka_mem"] = Query-PromAvg '
        "avg_over_time(
            sum by (pod)(
              container_memory_working_set_bytes{
                namespace='"data-ingest'",
                pod=~'"kafka.*'"}
            )[${window}:]
          )"
    # ... Flink CPU/MEM/NET, Trino CPU/MEM
    return $gm
}
```

Η συλλογή μετρικών από το Azure Storage χρησιμοποιεί το `az monitor metrics` εντολή του Azure CLI για τις μετρικές συναλλαγών (transactions), εισερχόμενης και εξερχόμενης κίνησης, καθυστέρησης (E2E και server) και χωρητικότητας.

```
function Get-AzureStorageMetrics($startUtc, $endUtc) {
    $sa = [ordered]@{}
    $resId = "/subscriptions/${subId}/.../storageAccounts/${StorageAccountName}"

    $txObj = az monitor metrics list --resource $resId '
        --metric Transactions Ingress Egress '
        --start-time $startStr --end-time $endStr '
        --interval PT30M --aggregation Total --output json |
        ConvertFrom-Json

    $latObj = az monitor metrics list --resource $resId '
        --metric SuccessE2ELatency SuccessServerLatency Availability '
        --interval PT30M --aggregation Average --output json |
        ConvertFrom-Json

    $capObj = az monitor metrics list --resource $resId '
        --metric UsedCapacity '
        --interval PT1H --aggregation Average --output json |
        ConvertFrom-Json

    return $sa
}
```

Τέλος, τα checkpoints του Flink καταγράφονται σε δύο χρονικά σημεία (T_0 και T_{30}) μέσω του REST API (`/jobs/{id}/checkpoints`), ώστε να υπολογιστεί η μεταβολή (delta) ολοκληρωμένων, αποτυχημένων και ανακτηθέντων checkpoints κατά τη διάρκεια του πειράματος.

ΣΤ΄.6 Σουίτα σημείων αναφοράς Trino

Η σουίτα εκτελεί τα πέντε ερωτήματα (Παράρτημα Ε΄) παράλληλα σε κάθε επανάληψη, αξιοποιώντας τους 3 workers του Trino μέσω `Start-Job`:

```
function Run-TrinoBenchmark([int]$iterations = 5) {
    $benchQueries = @(
        @{ Key="q1"; Sql="SELECT COUNT(*) FROM ...sensor_data_raw" },
        @{ Key="q2"; Sql="SELECT ... GROUP BY ... ORDER BY ..." },
        @{ Key="q3"; Sql="SELECT ... AVG(...) OVER (...) ..." },
        @{ Key="q4"; Sql="SELECT ... JOIN ...10min ... drift" },
        @{ Key="q5"; Sql="SELECT ... approx_percentile(...) ..." }
    )
    $allTimes = @{ q1=@(); q2=@(); q3=@(); q4=@(); q5=@() }

    for ($i = 0; $i -lt $iterations; $i++) {
        #                                    Q1-Q5          Start-Job
        $iterJobs = @()
        foreach ($q in $benchQueries) {
            $iterJobs += @{
                Key = $q.Key
                Job = Start-Job -ScriptBlock {
                    param($sql, $ns, $pod)
                    $sw = [Diagnostics.Stopwatch]::StartNew()
                    kubectl exec -n $ns $pod '
                        -- trino --execute $sql
                    $sw.Stop()
                    @{ elapsed_s = $sw.Elapsed.TotalSeconds }
                } -ArgumentList $q.Sql, $ns, $pod
            }
        }
        foreach ($j in $iterJobs) {
            $r = Receive-Job ($j.Job | Wait-Job)
            $allTimes[$j.Key] += $r.elapsed_s
        }
    }

    function Get-P95([double[]]$arr) {
        $sorted = $arr | Sort-Object
        $idx = [math]::Ceiling($sorted.Count * 0.95) - 1
        return [math]::Round($sorted[$idx], 2)
    }

    return @{
        q1_p95_s = Get-P95 $allTimes["q1"]
        q2_p95_s = Get-P95 $allTimes["q2"]
        q3_p95_s = Get-P95 $allTimes["q3"]
        q4_p95_s = Get-P95 $allTimes["q4"]
```

```
        q5_p95_s = Get-P95 $allTimes["q5"]
        failures = $failures
    }
}
```

Κάθε ερώτημα εκτελείται *iterations* φορές (προεπιλογή 5) και αναφέρεται η τιμή p95 ως αποτέλεσμα. Η παράλληλη εκτέλεση μειώνει τον συνολικό χρόνο αξιολόγησης ανά βαθμίδα, χωρίς να επηρεάζει τη μεμονωμένη καθυστέρηση κάθε ερωτήματος, δεδομένου ότι ο Trino κατανέμει τον φόρτο στους workers αυτόνομα.

Συντομογραφίες - Αρκτικόλεξα - Ακρωνύμια

ABFS	Azure Blob File System
ACID	Atomicity, Consistency, Isolation, Durability
ACL	Access Control List
ACR	Azure Container Registry
ADLS	Azure Data Lake Storage
AKS	Azure Kubernetes Service
AMQP	Advanced Message Queuing Protocol
API	Application Programming Interface
AWS	Amazon Web Services
CAP	Consistency, Availability, Partition tolerance
CapEx	Capital Expenditure
CDC	Change Data Capture
CI/CD	Continuous Integration / Continuous Delivery
CLI	Command-Line Interface
CNCF	Cloud Native Computing Foundation
CNI	Container Network Interface
CPU	Central Processing Unit
CRD	Custom Resource Definition
CRI	Container Runtime Interface
CSI	Container Storage Interface
CSV	Comma-Separated Values
DNS	Domain Name System
E2E	End-to-End
ETL	Extract-Transform-Load
GC	Garbage Collection
GCP	Google Cloud Platform
HDFS	Hadoop Distributed File System
HNS	Hierarchical Namespace
HPA	Horizontal Pod Autoscaler
HTTP	Hypertext Transfer Protocol
IaaS	Infrastructure as a Service
IaC	Infrastructure as Code
ISR	In-Sync Replicas (Kafka)
IIoT	Industrial Internet of Things
IOPS	Input/Output Operations Per Second

IoT	Internet of Things
IP	Internet Protocol
IT	Information Technology
JDBC	Java Database Connectivity
JM	JobManager (Flink)
JMX	Java Management Extensions
JAR	Java Archive
JSON	JavaScript Object Notation
JVM	Java Virtual Machine
KEDA	Kubernetes Event-Driven Autoscaling
KPI	Key Performance Indicator
KQL	Kusto Query Language
KRaft	Kafka Raft (consensus protocol)
KV	Key Vault
LB	Load Balancer
LRS	Locally Redundant Storage
LSM	Log-Structured Merge-tree
MQTT	Message Queuing Telemetry Transport
NAT	Network Address Translation
NSG	Network Security Group
OCI	Open Container Initiative
ODBC	Open Database Connectivity
OOM	Out of Memory
OpEx	Operational Expenditure
ORC	Optimized Row Columnar
OT	Operational Technology
PIP	Public IP Address
PV	Persistent Volume
PVC	Persistent Volume Claim
QoS	Quality of Service
RAM	Random Access Memory
RBAC	Role-Based Access Control
REST	Representational State Transfer
RF	Replication Factor (Kafka)
RFID	Radio-Frequency Identification
RG	Resource Group
RPC	Remote Procedure Call
RPS	Records Per Second
SaaS	Software as a Service
SKU	Stock Keeping Unit
SLA	Service Level Agreement
SLI	Service Level Indicator
SLO	Service Level Objective

SQL	Structured Query Language
SRE	Site Reliability Engineering
SSD	Solid-State Drive
TCO	Total Cost of Ownership
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TM	TaskManager (Flink)
TSDB	Time Series Database
TTL	Time To Live
UUID	Universally Unique Identifier
vCPU	virtual CPU
VM	Virtual Machine
VMSS	Virtual Machine Scale Set
VNet	Virtual Network
VPA	Vertical Pod Autoscaler
WAF	Web Application Firewall
YAML	YAML Ain't Markup Language
YARN	Yet Another Resource Negotiator

Απόδοση ξενόγλωσσων όρων

Απόδοση	Ξενόγλωσσος όρος
σωλήνωση δεδομένων	data pipeline
ακριβώς μία φορά	exactly-once
αστοχία / ανακατανομή	failover
ανοχή σφαλμάτων	fault tolerance
αντίγραφα	replicas
αρχιτεκτονική μεταλλίου	medallion architecture
αποθήκη δεδομένων	data warehouse
βάλτος δεδομένων	data swamp
αυτοΐαση	self-healing
αυτόματη κλιμάκωση	auto-scaling
ομάδα κόμβων	node pool
δοχείο	container
εικονικοποίηση	virtualization
εξέλιξη σχήματος	schema evolution
εξισορρόπηση φόρτου	load balancing
εξυπηρετητής	broker
ενορχήστρωση	orchestration
επεξεργασία δέσμης	batch processing
επεξεργασία ροής	stream processing
επίπεδο ελέγχου	control plane
θέμα	topic
καθυστέρηση	latency
καταναλωτής	consumer
γενεαλογία δεδομένων	data lineage
κατάλογος	catalog
κατάσταση	state
κατάτμηση	partition
κλιμάκωση / κλιμακωσιμότητα	scaling / scalability
κόμβος	node
λίμνη δεδομένων	data lake
μέσο λογισμικό	middleware
μεταδεδομένα	metadata
μετρική	metric
ορχήστρωση δοχείων	container orchestration

παραγωγός	producer
παράθυρο χρόνου	time window (tumble/slide)
παρατηρησιμότητα	observability
πίνακας ελέγχου	dashboard
ροή γεγονότων	event streaming
ρυθμός μηνυμάτων	throughput
σημείο αποθήκευσης	savepoint
σημείο ελέγχου	checkpoint
στενωπός	bottleneck
στιγμιότυπο	snapshot
συμπύκνωση	compaction
συστάδα	cluster
σχήμα	schema
τηλεμετρία	telemetry
υδατόσημο	watermark
υποδομή ως κώδικας	infrastructure as code
φιλοξενητής	hypervisor
φόρτος εργασίας	workload
χώρος ονομάτων	namespace

Βιβλιογραφία

[1] J. Gubbi, R. Buyya, S. Marusic και M. Palaniswami. *Internet of Things (IoT): A Vision, Architectural Elements, and Future Directions*. *Future Generation Computer Systems*, 29(7):1645–1660, 2013. doi: https://doi.org/10.1016/j.future.2013.01.010.

[2] Ioannis Papias, Vasilis Michalakopoulos, Elissaios Sarmas, Vangelis Marinakis, Gabriel Antonesi, Tudor Cioara και Ionut Anghel. *A data-driven framework for estimating residential energy flexibility for aggregated demand-side management*. *Sustainable Energy, Grids and Networks*, 43:101783, 2025. doi: https://doi.org/https://doi.org/10.1016/j.segan.2025.101783.

[3] Ioannis Papias, Vasilis Michalakopoulos, Elissaios Sarmas και Vangelis Marinakis. *Aggregated Flexibility Dynamic Forecasting of Residential Energy Prosumers for DR Programs*. *2024 15th International Conference on Information, Intelligence, Systems & Applications (IISA)*, σελίδες 1–9, 2024. doi: https://doi.org/10.1109/IISA62523.2024.10786677.

[4] Ioannis Papias, Loukas Papapetrou, Vasilis Michalakopoulos, Elissaios Sarmas, Vangelis Marinakis, Zoi Mylona και Costas Baslis. *Denoising Autoencoder for Appliance-Specific Energy Disaggregation in Residential Settings*. *2025 IEEE International Workshop on Metrology for Living Environment (MetroLivEnv)*, σελίδες 384–389, 2025. doi: https://doi.org/10.1109/MetroLivEnv64961.2025.11106991.

[5] Ioannis Papias, Vasilis Michalakopoulos, Dimitra Karoutsou, Elissaios Sarmas, Nikos Dimitropoulos, Vangelis Marinakis, Marion Paraschi και Dimitris I. Chatzigiannis. *Non-Intrusive Load Monitoring Using Cluster-Optimized Denoising Autoencoders*. *2025 16th International Conference on Information, Intelligence, Systems & Applications (IISA)*, σελίδες 1–8, 2025. doi: https://doi.org/10.1109/IISA66859.2025.11311198.

[6] M. Armbrust, A. Ghodsi, R. Xin και M. Zaharia. *Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics*. *Proceedings of the 11th Conference on Innovative Data Systems Research (CIDR)*, 2021.

[7] B. Burns, B. Grant, D. Oppenheimer, E. Brewer και J. Wilkes. *Borg, Omega, and Kubernetes*. *ACM Queue*, 14(1):70–93, 2016. doi: https://doi.org/10.1145/2890784.

[8] Ioannis Papias, Efstathios Sarantinopoulos, Vasilis Michalakopoulos, Elissaios Sarmas και Vangelis Marinakis. *FLEXiT: A platform for estimating residential energy flexibility for aggregated & disaggregated demand-side management*. *Softwa-*

reX, 32:102433, 2025. doi: https://doi.org/https://doi.org/10.1016/j.softx.2025.102433.

[9] Ioannis Papias, Alexandros Xenakis, Dimitra Karoutsou, Nikos Dimitropoulos, Apostolos Arsenopoulos και Vangelis Marinakis. *From Assessment to Action: Digital Tools for SRI Calculation and Smart Building Upgrade Planning*. *SSRN Electronic Journal*, 2026. Πρεπριντ. doi: https://doi.org/10.2139/ssrn.6740566.

[10] F. Nargesian, E. Zhu, R. J. Miller, K. Q. Pu και P. C. Arocena. *Data Lake Management: Challenges and Opportunities*. *Proceedings of the VLDB Endowment*, 12(12):1986–1989, 2019. doi: https://doi.org/10.14778/3352063.3352116.

[11] Filippos Serepas, Ioannis Papias, Konstantinos Christakis, Nikos Dimitropoulos και Vangelis Marinakis. *Lightweight Embedded IoT Gateway for Smart Homes Based on an ESP32 Microcontroller*. *Computers*, 14(9), 2025. doi: https://doi.org/10.3390/computers14090391.

[12] Filippos Serepas, Ioannis Papias, Nikolaos Bellos και Vangelis Marinakis. *A Comprehensive Approach to Real-Time and Batch Processing for Energy-Efficient IoT Homes: Leveraging Lambda Architecture and Data Lakes*. *2024 15th International Conference on Information, Intelligence, Systems & Applications (IISA)*, σελίδες 1–6, 2024. doi: https://doi.org/10.1109/IISA62523.2024.10786715.

[13] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi και K. Tzoumas. *Apache Flink: Stream and Batch Processing in a Single Engine*. *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, 38(4):28–38, 2015.

[14] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica και M. Zaharia. *A View of Cloud Computing*. *Communications of the ACM*, 53(4):50–58, 2010. doi: https://doi.org/10.1145/1721654.1721672.

[15] Filippos Serepas, Ioannis Papias, Konstantinos Christakis, Nikos Dimitropoulos και Vangelis Marinakis. *Decentralized IoT Data Marketplaces: Enabling Peer-to-Peer Exchange with Edge Computing*. *2025 IEEE International Workshop on Metrology for Living Environment (MetroLivEnv)*, σελίδες 306–311, 2025. doi: https://doi.org/10.1109/MetroLivEnv64961.2025.11107122.

[16] L. Atzori, A. Iera και G. Morabito. *The Internet of Things: A Survey*. *Computer Networks*, 54(15):2787–2805, 2010. doi: https://doi.org/10.1016/j.comnet.2010.05.010.

[17] I. Lee και K. Lee. *The Internet of Things (IoT): Applications, Investments, and Challenges for Enterprises*. *Business Horizons*, 58(4):431–440, 2015. doi: https://doi.org/10.1016/j.bushor.2015.03.001.

[18] Ioannis Papias, Vasilis Michalakopoulos, Elissaios Sarmas, Nikos Dimitropoulos και Vangelis Marinakis. *A Hybrid Data-Driven and Physics-Informed Framework for*

Residential Heating Flexibility Assessment. *SSRN Electronic Journal*, 2026. Ποστεδ 16 Θυνε 2026. doi: https://doi.org/10.2139/ssrn.6952655.

[19] N. Marz και J. Warren. *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning Publications, 2015.

[20] Grafana Labs. *Grafana: The Open Observability Platform*. https://grafana.com, 2021. Ημερομηνία πρόσβασης: 15-01-2026.

[21] J. Kreps, N. Narkhede και J. Rao. *Kafka: A Distributed Messaging System for Log Processing*. *Proceedings of the NetDB Workshop*, σελίδες 1–7, 2011.

[22] The Apache Software Foundation. *Apache Flink, Stateful Computations over Data Streams*. https://flink.apache.org/, 2024. Accessed: 2025-06-01.

[23] Apache Software Foundation. *Apache Paimon: Streaming Lakehouse Table Store*. https://paimon.apache.org, 2023. Ημερομηνία πρόσβασης: 15-01-2026.

[24] Trino Software Foundation. *Trino: A Distributed SQL Query Engine for Big Data*. https://trino.io, 2021. Ημερομηνία πρόσβασης: 15-01-2026.

[25] K. Morris. *Infrastructure as Code: Dynamic Systems for the Cloud Age*. O'Reilly Media, 2η έκδοση, 2020.

[26] HashiCorp. *Terraform: Infrastructure as Code*. https://www.terraform.io, 2023. Ημερομηνία πρόσβασης: 15-01-2026.

[27] J. Dizdarević, F. Carpio, A. Jukan και X. Masip-Bruin. *A Survey of Communication Protocols for Internet of Things and Related Challenges of Fog and Cloud Computing Integration*. *ACM Computing Surveys*, 51(6):1–29, 2019. doi: https://doi.org/10.1145/3292674.

[28] M. Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

[29] Donald D. Chamberlin και Raymond F. Boyce. *SEQUEL: A Structured English Query Language*. *Proceedings of the 1974 ACM SIGFIDET Workshop on Data Description, Access and Control*, σελίδες 249–264, 1974. doi: https://doi.org/10.1145/800296.811515.

[30] S. Henning και W. Hasselbring. *Theodolite: Scalability Benchmarking of Distributed Stream Processing Engines in Microservice Architectures*. *Big Data Research*, 25:100209, 2021. doi: https://doi.org/10.1016/j.bdr.2021.100209.

[31] Betsy Beyer, Chris Jones, Jennifer Petoff και Niall Richard Murphy. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.

[32] Betsy Beyer, Niall Richard Murphy, David K. Rensin, Kent Kawahara και Stephen Thorne. *The Site Reliability Workbook: Practical Ways to Implement SRE*. O'Reilly Media, 2018.

[33] S. Henning και W. Hasselbring. *Benchmarking Scalability of Stream Processing Frameworks Deployed as Microservices in the Cloud*. *Journal of Systems and Software*, 208:111879, 2024. doi: https://doi.org/10.1016/j.jss.2023.111879.

[34] Hazelcast. *Hazelcast Jet: Open Source Stream Processing Engine*. https://docs.hazelcast.com/hazelcast/latest/pipelines/overview, 2024. Ημερομηνία πρόσβασης: 31-05-2026.

[35] Apache Software Foundation. *Apache Beam: An Advanced Unified Programming Model*. https://beam.apache.org/, 2024. Ημερομηνία πρόσβασης: 31-05-2026.

[36] E. Sisinni, A. Saifullah, S. Han, U. Jennehag και M. Gidlund. *Industrial Internet of Things: Challenges, Opportunities, and Directions*. *IEEE Transactions on Industrial Informatics*, 14(11):4724–4734, 2018. doi: https://doi.org/10.1109/TII.2018.2852491.

[37] Cloud Native Computing Foundation. *Operator pattern, Kubernetes Documentation*. https://kubernetes.io/docs/concepts/extend-kubernetes/operator/, 2024. Ημερομηνία πρόσβασης: 07-06-2026.

[38] M. K. Geldenhuys, J. Will, B. J. J. Pfister, M. Haug, A. Scharmann και L. Thamsen. *Dependable IoT Data Stream Processing for Monitoring and Control of Urban Infrastructures*. *Proceedings of the IEEE International Conference on Cloud Engineering (IC2E), 1st International Workshop on Testing Distributed Internet of Things Systems*. IEEE, 2021. doi: https://doi.org/10.1109/IC2E52221.2021.00041.

[39] A. Shukla, S. Chaturvedi και Y. Simmhan. *RIoTBench: An IoT Benchmark for Distributed Stream Processing Systems*. *Concurrency and Computation: Practice and Experience*, 29(21):e4257, 2017. doi: https://doi.org/10.1002/cpe.4257.

[40] Ankit Toshniwal, Siddarth Taneja, Amit Shukla, Karthik Ramasamy, Jignesh M. Patel, Sanjeev Kulkarni, Jason Jackson, Krishna Gade, Maosong Fu, Jake Donham, Nikunj Bhagat, Sailesh Mittal και Dmitriy Ryaboy. *Storm@Twitter*. *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*, σελίδες 147–156. ACM, 2014. doi: https://doi.org/10.1145/2588555.2595641.

[41] P. Mell και T. Grance. *The NIST Definition of Cloud Computing*. Τεχνική Αναφορά με αριθμό SP 800-145, National Institute of Standards and Technology, 2011.

[42] M. Pöss, R. O. Nambiar, K. Kulkarni, C. Narasimhadevara, T. Rabl και H. Jacobsen. *Analysis of TPCx-IoT: The First Industry Standard Benchmark for IoT Gateway Systems*. *Proceedings of the 34th IEEE International Conference on Data Engineering (ICDE)*, σελίδες 1519–1530. IEEE, 2018. doi: https://doi.org/10.1109/ICDE.2018.00170.

[43] Z. Jerzak και H. Ziekow. *The DEBS 2014 Grand Challenge: Smart Homes*. *Proceedings of the 8th ACM International Conference on Distributed Event-Based Systems (DEBS)*, σελίδες 266–269. ACM, 2014. doi: https://doi.org/10.1145/2611286.2611333.

[44] G. Hesse, C. Matthies, M. Perscheid, M. Uflacker και H. Plattner. *ESPBench: The Enterprise Stream Processing Benchmark*. *Proceedings of the ACM/SPEC International Conference on Performance Engineering (ICPE)*, σελίδες 201–212. ACM, 2021. doi: https://doi.org/10.1145/3427921.3450242.

[45] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker και I. Stoica. *Apache Spark: A Unified Engine for Big Data Processing*. *Communications of the ACM*, 59(11):56–65, 2016. doi: https://doi.org/10.1145/2934664.

[46] J. Mostafa, S. Wehbi, S. Chilingaryan και A. Kopmann. *SciTS: A Benchmark for Time-Series Databases in Scientific Experiments and Industrial Internet of Things*. *Proceedings of the 34th International Conference on Scientific and Statistical Database Management (SSDBM)*, σελίδες 1–11. ACM, 2022. doi: https://doi.org/10.1145/3538712.3538723.

[47] ClickHouse Inc. *ClickHouse: Open-Source Column-Oriented DBMS for Online Analytical Processing*. https://clickhouse.com/docs, 2024. Ημερομηνία πρόσβασης: 31-05-2026.

[48] InfluxData. *InfluxDB: Time Series Data Platform*. https://docs.influxdata.com/influxdb/, 2024. Ημερομηνία πρόσβασης: 31-05-2026.

[49] Timescale Inc. *TimescaleDB: Time-Series Database Built on PostgreSQL*. https://docs.tigerdata.com/, 2024. Ημερομηνία πρόσβασης: 31-05-2026.

[50] The PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. https://www.postgresql.org/docs/16/, 2024. Ημερομηνία πρόσβασης: 30-05-2026.

[51] N. Bellos. *Development of an IoT Data Processing System Using Distributed Technologies Based on Kubernetes*. Διπλωματική εργασία, National Technical University of Athens, School of Electrical and Computer Engineering, 2024. http://artemis.cslab.ece.ntua.gr:8080/jspui/handle/123456789/19086.

[52] F. Ravat και Y. Zhao. *Data Lakes: Trends and Perspectives*. *Proceedings of the 20th International Conference on Database and Expert Systems Applications (DEXA)*, σελίδες 304–313. Springer, 2019.

[53] G. J. Popek και R. P. Goldberg. *Formal Requirements for Virtualizable Third Generation Architectures*. *Communications of the ACM*, 17(7):412–421, 1974. doi: https://doi.org/10.1145/355288.355289.

[54] Microsoft. *Microsoft Azure, Cloud Computing Services*. https://azure.microsoft.com, 2024. Ημερομηνία πρόσβασης: 15-01-2026.

[55] Amazon Web Services. *Amazon Web Services (AWS), Cloud Computing Services*. https://aws.amazon.com, 2024. Ημερομηνία πρόσβασης: 15-01-2026.

[56] Google. *Google Cloud Platform, Cloud Computing Services*. https://cloud.google.com, 2024. Ημερομηνία πρόσβασης: 15-01-2026.

[57] R. Buyya, C. S. Yeo, S. Venugopal, J. Broberg και I. Brandic. *Cloud Computing and Emerging IT Platforms: Vision, Hype, and Reality for Delivering Computing as the 5th Utility*. *Future Generation Computer Systems*, 25(6):599–616, 2009. doi: https://doi.org/10.1016/j.future.2009.05.007.

[58] P. Barham, B. Dragovic, K. Fraser, S. Hand, T. Harris, A. Ho, R. Neugebauer, I. Pratt και A. Warfield. *Xen and the Art of Virtualization*. *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP)*, σελίδες 164–177. ACM, 2003. doi: https://doi.org/10.1145/945445.945462.

[59] VMware by Broadcom. *VMware vSphere Hypervisor (ESXi)*. https://www.vmware.com/products/cloud-infrastructure/esxi-and-esx, 2024. Ημερομηνία πρόσβασης: 15-01-2026.

[60] Microsoft. *Hyper-V on Windows Server*. https://learn.microsoft.com/windows-server/virtualization/hyper-v/hyper-v-technology-overview, 2024. Ημερομηνία πρόσβασης: 15-01-2026.

[61] Oracle Corporation. *Oracle VM VirtualBox*. https://www.virtualbox.org, 2024. Ημερομηνία πρόσβασης: 15-01-2026.

[62] VMware by Broadcom. *VMware Workstation Pro*. https://www.vmware.com/products/desktop-hypervisor/workstation-and-fusion, 2024. Ημερομηνία πρόσβασης: 15-01-2026.

[63] Canonical. *Containerization vs. Virtualization: What is the Difference?* https://ubuntu.com/blog/containerization-vs-virtualization, 2024. Ημερομηνία πρόσβασης: 15-03-2026.

[64] S. Soltesz, H. Pötzl, M. E. Fiuczynski, A. Bavier και L. Peterson. *Container-Based Operating System Virtualization: A Scalable, High-Performance Alternative to Hypervisors*. *Proceedings of the 2nd ACM SIGOPS/EuroSys European Conference on Computer Systems*, σελίδες 275–287. ACM, 2007. doi: https://doi.org/10.1145/1272996.1273006.

[65] M. G. Xavier, M. V. Neves, F. D. Rossi, T. C. Ferreto, T. Lange και C. A. F. De Rose. *Performance Evaluation of Container-Based Virtualization for High Performance Computing Environments*. *Proceedings of the 21st Euromicro International Conference on Parallel, Distributed, and Network-Based Processing (PDP)*, σελίδες 233–240, 2013. doi: https://doi.org/10.1109/PDP.2013.41.

[66] W. Felter, A. Ferreira, R. Rajamony και J. Rubio. *An Updated Performance Comparison of Virtual Machines and Linux Containers*. *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, σελίδες 171–172. IEEE, 2015. doi: https://doi.org/10.1109/ISPASS.2015.7095802.

[67] C. Pahl. *Containerization and the PaaS Cloud*. *IEEE Cloud Computing*, 2(3):24–31, 2015. doi: https://doi.org/10.1109/MCC.2015.52.

[68] D. Merkel. *Docker: Lightweight Linux Containers for Consistent Development and Deployment*. *Linux Journal*, 2014(239):2, 2014.

[69] D. Bernstein. *Containers and Cloud: From LXC to Docker to Kubernetes*. *IEEE Cloud Computing*, 1(3):81–84, 2014. doi: https://doi.org/10.1109/MCC.2014.51.

[70] D. A. Patterson και J. L. Hennessy. *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*. Morgan Kaufmann, 1η έκδοση, 2017.

[71] J. Humble και D. Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.

[72] Carl Boettiger. *An introduction to Docker for reproducible research*. *ACM SIGOPS Operating Systems Review*, 49(1):71–79, 2015. doi: https://doi.org/10.1145/2723872.2723882.

[73] M. Al-Dhuraibi, F. Paraiso, N. Djarallah και P. Merle. *Elasticity in Cloud Computing: State of the Art and Research Challenges*. *IEEE Transactions on Services Computing*, 11(2):430–447, 2018. doi: https://doi.org/10.1109/TSC.2016.2520923.

[74] R. Kumar και S. Sharma. *Transforming the Software Development Lifecycle with Docker Containers: A Comparative Study*. *International Journal for Multidisciplinary Research (IJFMR)*, 4(1), 2022.

[75] Docker Inc. *Docker Hub Documentation*. https://docs.docker.com/docker-hub/, 2024. Ημερομηνία πρόσβασης: 20-06-2026.

[76] Microsoft. *Authenticate with Azure Container Registry (ACR) from Azure Kubernetes Service (AKS)*. https://learn.microsoft.com/azure/aks/cluster-container-registry-integration, 2025. Ημερομηνία πρόσβασης: 15-03-2026.

[77] Google. *Container Registry Documentation, Google Cloud*. https://cloud.google.com/container-registry/docs, 2024. Ημερομηνία πρόσβασης: 20-06-2026.

[78] Amazon Web Services. *What Is Amazon Elastic Container Registry?, AWS Documentation*. https://docs.aws.amazon.com/AmazonECR/latest/userguide/what-is-ecr.html, 2024. Ημερομηνία πρόσβασης: 20-06-2026.

[79] Docker Inc. *Docker Documentation*. https://docs.docker.com, 2024. Ημερομηνία πρόσβασης: 15-01-2026.

[80] Open Container Initiative. *Open Container Initiative*. https://opencontainers.org, 2024. Ημερομηνία πρόσβασης: 20-06-2026.

[81] M. Kerrisk. *namespaces(7) - Linux Programmer's Manual*. https://man7.org/linux/man-pages/man7/namespaces.7.html, 2024. Ημερομηνία πρόσβασης: 20-06-2026.

[82] M. Kerrisk. *cgroups(7) - Linux Programmer's Manual*. https://man7.org/linux/man-pages/man7/cgroups.7.html, 2024. Ημερομηνία πρόσβασης: 20-06-2026.

[83] D. P. Bovet και M. Cesati. *Understanding the Linux Kernel*. O'Reilly Media, 3η έκδοση, 2005.

[84] Cloud Native Computing Foundation. *Container Runtime Interface (CRI), Kubernetes Documentation*. https://kubernetes.io/docs/concepts/architecture/cri/, 2024. Ημερομηνία πρόσβασης: 20-06-2026.

[85] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune και J. Wilkes. *Large-Scale Cluster Management at Google with Borg*. *Proceedings of the 10th European Conference on Computer Systems (EuroSys)*, σελίδες 1–17. ACM, 2015. doi: https://doi.org/10.1145/2741948.2741964.

[86] Cloud Native Computing Foundation. *Horizontal Pod Autoscaling, Kubernetes Documentation*. https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/, 2024. Ημερομηνία πρόσβασης: 11-06-2026.

[87] Cloud Native Computing Foundation. *Vertical Pod Autoscaler, Kubernetes Autoscaler Repository*. https://github.com/kubernetes/autoscaler/tree/master/vertical-pod-autoscaler, 2024. Ημερομηνία πρόσβασης: 11-06-2026.

[88] P. Mockapetris. *Domain Names: Concepts and Facilities*. Τεχνική Αναφορά με αριθμό RFC 1034, Internet Engineering Task Force (IETF), 1987.

[89] Docker Inc. *Docker Swarm Overview*. https://docs.docker.com/engine/swarm/, 2024. Ημερομηνία πρόσβασης: 29-05-2026.

[90] B. Hindman, A. Konwinski, M. Zaharia, A. Ghodsi, A. D. Joseph, R. Katz, S. Shenker και I. Stoica. *Mesos: A Platform for Fine-Grained Resource Sharing in the Data Center*. *Proceedings of the 8th USENIX Conference on Networked Systems Design and Implementation (NSDI)*, σελίδες 295–308. USENIX Association, 2011.

[91] Mesosphere. *Marathon: A Container Orchestration Platform for Mesos*. https://mesosphere.github.io/marathon/, 2024. Ημερομηνία πρόσβασης: 29-05-2026.

[92] Apache Software Foundation. *Mesos: The Apache Attic*. https://attic.apache.org/projects/mesos.html, 2025. Ημερομηνία πρόσβασης: 29-05-2026.

[93] A. S. Tanenbaum και M. Van Steen. *Distributed Systems: Principles and Paradigms*. Pearson Education, 3η έκδοση, 2017.

[94] Cloud Native Computing Foundation. *Custom Resources: Kubernetes Documentation*. https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/, 2024. Ημερομηνία πρόσβασης: 15-01-2026.

[95] Cloud Native Computing Foundation. *Kubernetes Documentation*. https://kubernetes.io/docs, 2024. Ημερομηνία πρόσβασης: 15-01-2026.

[96] K. Hightower, B. Burns και J. Beda. *Kubernetes: Up and Running: Dive into the Future of Infrastructure*. O'Reilly Media, 2017.

[97] Cloud Native Computing Foundation. *Cloud Native Computing Foundation (CNCF)*. https://www.cncf.io, 2024. Ημερομηνία πρόσβασης: 29-05-2026.

[98] P. Srisuresh και K. Egevang. *Traditional IP Network Address Translator (Traditional NAT)*. Τεχνική Αναφορά με αριθμό RFC 3022, Internet Engineering Task Force, 2001. https://www.rfc-editor.org/rfc/rfc3022.

[99] B. Burns, J. Beda και K. Hightower. *Kubernetes: Up and Running*. O'Reilly Media, 2η έκδοση, 2019.

[100] D. Ongaro και J. Ousterhout. *In Search of an Understandable Consensus Algorithm*. *Proceedings of the USENIX Annual Technical Conference (USENIX ATC)*, σελίδες 305–320. USENIX Association, 2014.

[101] Tigera. *About Calico: Project Calico Documentation*. https://docs.tigera.io/calico/latest/about/, 2024. Ημερομηνία πρόσβασης: 15-01-2026.

[102] Cilium Authors. *High Performance Cloud Native Networking (CNI): Cilium*. https://cilium.io/use-cases/cni/, 2024. Ημερομηνία πρόσβασης: 15-01-2026.

[103] flannel-io. *flannel: A Network Fabric for Containers, Designed for Kubernetes*. https://github.com/flannel-io/flannel, 2024. Ημερομηνία πρόσβασης: 15-01-2026.

[104] Microsoft. *Azure Kubernetes Service (AKS) CNI Networking Overview*. https://learn.microsoft.com/azure/aks/concepts-network-cni-overview, 2025. Ημερομηνία πρόσβασης: 15-03-2026.

[105] Cloud Native Computing Foundation. *Controllers, Kubernetes Documentation*. https://kubernetes.io/docs/concepts/architecture/controller/, 2024. Ημερομηνία πρόσβασης: 30-05-2026.

[106] Cloud Native Computing Foundation. *Workload Management, Kubernetes Documentation*. https://kubernetes.io/docs/concepts/workloads/controllers/, 2024. Ημερομηνία πρόσβασης: 30-05-2026.

[107] Cloud Native Computing Foundation. *Deployments, Kubernetes Documentation*. https://kubernetes.io/docs/concepts/workloads/controllers/deployment/, 2024. Ημερομηνία πρόσβασης: 20-03-2026.

[108] Cloud Native Computing Foundation. *Persistent Volumes, Kubernetes Documentation*. https://kubernetes.io/docs/concepts/storage/persistent-volumes/, 2024. Ημερομηνία πρόσβασης: 30-05-2026.

[109] Cloud Native Computing Foundation. *StatefulSets, Kubernetes Documentation*. https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/, 2024. Ημερομηνία πρόσβασης: 20-03-2026.

[110] Cloud Native Computing Foundation. *Jobs, Kubernetes Documentation*. https://kubernetes.io/docs/concepts/workloads/controllers/job/, 2024. Ημερομηνία πρόσβασης: 20-03-2026.

[111] Cloud Native Computing Foundation. *CronJob, Kubernetes Documentation*. https://kubernetes.io/docs/concepts/workloads/controllers/cron-jobs/, 2024. Ημερομηνία πρόσβασης: 20-03-2026.

[112] Cloud Native Computing Foundation. *DaemonSet, Kubernetes Documentation*. https://kubernetes.io/docs/concepts/workloads/controllers/daemonset/, 2024. Ημερομηνία πρόσβασης: 20-03-2026.

[113] S. Gilbert και N. Lynch. *Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services*. *ACM SIGACT News*, 33(2):51–59, 2002. doi: https://doi.org/10.1145/564585.564601.

[114] Apache Software Foundation. *Apache Kafka Documentation: Replication*. https://kafka.apache.org/documentation/#replication, 2024. Ημερομηνία πρόσβασης: 30-05-2026.

[115] K. M. Chandy και L. Lamport. *Distributed Snapshots: Determining Global States of Distributed Systems*. *ACM Transactions on Computer Systems*, 3(1):63–75, 1985. doi: https://doi.org/10.1145/214451.214456.

[116] Cloud Native Computing Foundation. *Cluster Architecture: etcd, Kubernetes Documentation*. https://kubernetes.io/docs/concepts/architecture/#etcd, 2024. Ημερομηνία πρόσβασης: 30-05-2026.

[117] M. A. Rodriguez και R. Buyya. *Container Orchestration With Cost-Efficient Autoscaling in Cloud Computing Environments*. *Handbook of Research on Multimedia Cyber Security*, σελίδες 190–213. IGI Global, 2020.

[118] S. Hoque, M. S. De Brito, A. Willner, O. Keil και T. Magedanz. *Towards Container Orchestration in Fog Computing Infrastructures*. *Proceedings of the IEEE 41st Annual Computer Software and Applications Conference (COMPSAC)*, σελίδες 294–299, 2017. doi: https://doi.org/10.1109/COMPSAC.2017.79.

[119] K. Ashton. *That 'Internet of Things' Thing*. *RFID Journal*, 22(7):97–114, 2009.

[120] S. Madakam, R. Ramaswamy και S. Tripathi. *Internet of Things (IoT): A Literature Review*. *Journal of Computer and Communications*, 3(5):164–173, 2015. doi: https://doi.org/10.4236/jcc.2015.35021.

[121] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari και M. Ayyash. *Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications*. *IEEE Communications Surveys & Tutorials*, 17(4):2347–2376, 2015. doi: https://doi.org/10.1109/COMST.2015.2444095.

[122] A. Banks, E. Briggs, K. Borgendale και R. Gupta. *MQTT Version 5.0: OASIS Standard*. Τεχνική Αναφορά με αριθμό, OASIS, 2019. https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html.

[123] Z. Shelby, K. Hartke και C. Bormann. *The Constrained Application Protocol (CoAP)*. Τεχνική Αναφορά με αριθμό RFC 7252, Internet Engineering Task Force (IETF), 2014.

[124] R. Fielding, M. Nottingham και J. Reschke. *HTTP Semantics*. Τεχνική Αναφορά με αριθμό RFC 9110, Internet Engineering Task Force (IETF), 2022.

[125] OASIS. *OASIS Advanced Message Queuing Protocol (AMQP) Version 1.0*. Τεχνική Αναφορά με αριθμό, OASIS, 2012. https://docs.oasis-open.org/amqp/core/v1.0/os/amqp-core-overview-v1.0-os.html.

[126] Shadi Al-Sarawi, Mohammed Anbar, Rosni Abdullah και Ahmad B. Al Hawari. *Internet of Things Market Analysis Forecasts, 2020–2030. 2020 Fourth World Conference on Smart Trends in Systems, Security and Sustainability (WorldS4)*, σελίδες 449–453. IEEE, 2020. doi: https://doi.org/10.1109/WorldS451998.2020.9210375.

[127] Investopedia. *Compound Annual Growth Rate (CAGR): Formula and Calculation*. https://www.investopedia.com/terms/c/cagr.asp, 2024. Ημερομηνία πρόσβασης: 30-05-2026.

[128] J. A. Stankovic. *Research Directions for the Internet of Things. IEEE Internet of Things Journal*, 1(1):3–9, 2014. doi: https://doi.org/10.1109/JIOT.2014.2312291.

[129] GeeksforGeeks. *Introduction to Internet of Things (IoT)*. https://www.geeksforgeeks.org/computer-networks/introduction-to-internet-of-things-iot-set-1/, 2024. Ημερομηνία πρόσβασης: 15-03-2026.

[130] P. Sethi και S. R. Sarangi. *Internet of Things: Architectures, Protocols, and Applications. Journal of Electrical and Computer Engineering*, 2017:9324035, 2017. doi: https://doi.org/10.1155/2017/9324035.

[131] L. D. Xu, W. He και S. Li. *Internet of Things in Industries: A Survey. IEEE Transactions on Industrial Informatics*, 10(4):2233–2243, 2014. doi: https://doi.org/10.1109/TII.2014.2300753.

[132] E. Alreshidi. *Smart Sustainable Agriculture (SSA) Solution Underpinned by Internet of Things (IoT) and Artificial Intelligence (AI). International Journal of Advanced Computer Science and Applications*, 10(5), 2019.

[133] A. Zanella, N. Bui, A. Castellani, L. Vangelista και M. Zorzi. *Internet of Things for Smart Cities. IEEE Internet of Things Journal*, 1(1):22–32, 2014. doi: https://doi.org/10.1109/JIOT.2014.2306328.

[134] i-SCOOP. *Smart Manufacturing and Smart Industry in Context*. https://www.i-scoop.eu/industry-4-0/manufacturing-industry/, 2017. Ημερομηνία πρόσβασης: 22-03-2026.

[135] O. Givehchi, K. Landsdorf, P. Simoens και A. W. Colombo. *Interoperability for Industrial Cyber-Physical Systems: An Approach for Legacy Systems*. *IEEE Transactions on Industrial Informatics*, 13(6):3370–3378, 2017. doi: https://doi.org/10.1109/TII.2017.2717573.

[136] K. Goodhope, J. Koshy, J. Kreps, N. Narkhede, R. Park, J. Rao και V. Y. Ye. *Building LinkedIn's Real-time Activity Data Pipeline*. *IEEE Data Engineering Bulletin*, 35(2):33–45, 2012.

[137] N. Narkhede, G. Shapira και T. Palino. *Kafka: The Definitive Guide*. O'Reilly Media, 2017.

[138] Apache Software Foundation. *Apache Kafka: Use Cases*. https://kafka.apache.org/42/getting-started/uses/, 2024. Ημερομηνία πρόσβασης: 05-04-2026.

[139] Apache Software Foundation. *Apache Kafka: Introduction*. https://kafka.apache.org/42/getting-started/introduction/, 2024. Ημερομηνία πρόσβασης: 05-04-2026.

[140] G. Wang, J. Koshy, S. Subramanian, K. Paramasivam, M. Zadeh, N. Narkhede, J. Rao, J. Kreps και J. Stein. *Building a Replicated Logging System with Apache Kafka*. *Proceedings of the VLDB Endowment*, 8(12):1654–1655, 2015.

[141] Linux Foundation. *OpenMessaging Benchmark: Cloud-Native Benchmark for Messaging and Streaming Systems*. https://openmessaging.cloud/docs/benchmarks/, 2023. Ημερομηνία πρόσβασης: 15-01-2026.

[142] Apache Software Foundation. *Apache Flink Architecture*. https://nightlies.apache.org/flink/flink-docs-release-2.2/docs/concepts/flink-architecture/, 2024. Ημερομηνία πρόσβασης: 23-03-2026.

[143] S. Zeuch, B. del Monte, J. Karimov, C. Luber, M. Renz, J. Traub, S. Breß, T. Rabl και V. Markl. *Analyzing Efficient Stream Processing on Modern Hardware*. *Proceedings of the VLDB Endowment*, τόμος 12, σελίδες 516–530, 2019. doi: https://doi.org/10.14778/3303753.3303758.

[144] C. Lameter. *NUMA (Non-Uniform Memory Access): An Overview*. *ACM Queue*, 11(7):40–51, 2013. doi: https://doi.org/10.1145/2508834.2513149.

[145] V. K. Vavilapalli, A. C. Murthy, C. Douglas, S. Agarwal, M. Konar, R. Evans, T. Graves, J. Lowe, H. Shah, S. Seth, B. Saha, C. Curino, O. O'Malley, S. Radia, B. Reed και E. Baldeschwieler. *Apache Hadoop YARN: Yet Another Resource Negotiator*. *Proceedings of the 4th Annual Symposium on Cloud Computing (SoCC)*, σελίδες 1–16, 2013. doi: https://doi.org/10.1145/2523616.2523633.

[146] Oracle. *JAR File Overview*. https://docs.oracle.com/javase/8/docs/technotes/guides/jar/jarGuide.html, 2024. Ημερομηνία πρόσβασης: 30-05-2026.

[147] Apache Software Foundation. *Apache Flink, Stateful Stream Processing*. https://nightlies.apache.org/flink/flink-docs-release-2.2/docs/concepts/stateful-stream-processing/, 2024. Ημερομηνία πρόσβασης: 23-03-2026.

[148] Siying Dong, Andrew Kryczka, Yanqin Jin και Michael Stumm. *RocksDB: Evolution of Development Priorities in a Key-value Store Serving Large-scale Applications*. *Proceedings of the VLDB Endowment*, 14(12):3130–3142, 2021. doi: https://doi.org/10.14778/3476311.3476327.

[149] P. Carbone, S. Ewen, G. Fóra, S. Haridi, S. Richter και K. Tzoumas. *State Management in Apache Flink: Consistent Stateful Distributed Stream Processing*. *Proceedings of the VLDB Endowment*, 10(12):1718–1729, 2017. doi: https://doi.org/10.14778/3137765.3137777.

[150] Apache Software Foundation. *Apache Flink, Timely Stream Processing*. https://nightlies.apache.org/flink/flink-docs-release-2.2/docs/concepts/time/, 2024. Ημερομηνία πρόσβασης: 23-03-2026.

[151] T. Akidau, R. Bradshaw, C. Chambers, S. Chernyak, R. J. Fernandez-Moctezuma, R. Lax, S. McVeety, D. Mills, F. Perry, E. Schmidt και S. Whittle. *The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing*. *Proceedings of the VLDB Endowment*, 8(12):1792–1803, 2015. doi: https://doi.org/10.14778/2824032.2824076.

[152] Apache Software Foundation. *Apache Flink, Windows*. https://nightlies.apache.org/flink/flink-docs-release-2.2/docs/dev/datastream/operators/windows/, 2024. Ημερομηνία πρόσβασης: 23-03-2026.

[153] J. Dean και S. Ghemawat. *MapReduce: Simplified Data Processing on Large Clusters*. *Communications of the ACM*, 51(1):107–113, 2008. doi: https://doi.org/10.1145/1327452.1327492.

[154] E. Begoli, J. Camacho-Rodríguez, J. Hyde, M. J. Mior και D. Lemire. *Apache Calcite: A Foundational Framework for Optimized Query Processing Over Heterogeneous Data Sources*. *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, σελίδες 221–230. ACM, 2018. doi: https://doi.org/10.1145/3183713.3190662.

[155] T. Härder και A. Reuter. *Principles of Transaction-Oriented Database Recovery*. *ACM Computing Surveys*, 15(4):287–317, 1983. doi: https://doi.org/10.1145/289.291.

[156] N. Miloslavskaya και A. Tolstoy. *Big Data, Fast Data and Data Lake Concepts*. *Procedia Computer Science*, τόμος 88, σελίδες 300–305. Elsevier, 2016. doi: https://doi.org/10.1016/j.procs.2016.09.060.

[157] P. P. Khine και Z. S. Wang. *Data Lake: A New Ideology in Big Data Era*. *ITM Web of Conferences*, 17:03025, 2018. doi: https://doi.org/10.1051/itmconf/20181703025.

[158] I. G. Terrizzano, P. M. Schwarz, M. Roth και J. E. Colino. *Data Wrangling: The Challenging Journey from the Wild to the Lake*. *Proceedings of the 7th Biennial Conference on Innovative Data Systems Research (CIDR)*, 2015.

[159] R. Hai, S. Geisler και C. Quix. *Constance: An Intelligent Data Lake System*. *Proceedings of the 2016 International Conference on Management of Data (SIGMOD)*, σελίδες 2097–2100, 2016. doi: https://doi.org/10.1145/2882903.2899389.

[160] P. N. Sawadogo και J. Darmont. *On Data Lake Architectures and Metadata Management*. *Journal of Intelligent Information Systems*, 56(1):97–120, 2021. doi: https://doi.org/10.1007/s10844-019-00568-7.

[161] C. Giebler, C. Gröger, E. Hoos, H. Schwarz και B. Mitschang. *Leveraging the Data Lake: Current State and Challenges*. *Proceedings of the 21st International Conference on Big Data Analytics and Knowledge Discovery (DaWaK)*, σελίδες 179–188. Springer, 2019.

[162] Microsoft. *Introduction to Azure Data Lake Storage*. https://learn.microsoft.com/azure/storage/blobs/data-lake-storage-introduction, 2025. Ημερομηνία πρόσβασης: 15-03-2026.

[163] Amazon Web Services. *Amazon Simple Storage Service (Amazon S3)*. https://aws.amazon.com/s3/, 2025. Ημερομηνία πρόσβασης: 01-06-2026.

[164] Apache Software Foundation. *Apache Parquet: Columnar Storage Format*. https://parquet.apache.org/docs/, 2024. Ημερομηνία πρόσβασης: 30-05-2026.

[165] Apache Software Foundation. *Apache ORC: High-Performance Columnar Storage for Hadoop*. https://orc.apache.org/docs/, 2024. Ημερομηνία πρόσβασης: 25-06-2026.

[166] R. Leverich, R. Hopping και A. Cheung. *Apache Iceberg: An Open Table Format for Huge Analytic Datasets*. *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, 2021. Demo Paper.

[167] M. Armbrust, T. Das, L. Sun, B. Yavuz, S. Zhu, M. Murthy, J. Torres, H. van Hovell, A. Ionescu, A. Łuszczak, M. Switakowski, R. Sasaki, T. Li, H. Lu, S. Szpakowski, K. Lee και R. Xin. *Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores*. *Proceedings of the VLDB Endowment*, 13(12):3411–3424, 2020. doi: https://doi.org/10.14778/3415478.3415560.

[168] Apache Software Foundation. *Apache Hudi: Transactional Data Lake Platform*. https://hudi.apache.org/, 2024. Ημερομηνία πρόσβασης: 31-05-2026.

[169] P. Jain, P. Kraft, C. Guo, T. Condie και A. Behm. *Analyzing and Comparing Lakehouse Storage Systems*. *Proceedings of the 13th Conference on Innovative Data Systems Research (CIDR)*, 2023.

[170] Apache Software Foundation. *Apache Paimon: Overview*. https://paimon.apache.org/docs/1.3/concepts/overview/, 2024. Ημερομηνία πρόσβασης: 15-03-2026.

[171] Apache Software Foundation. *About the Apache Software Foundation*. https://www.apache.org/foundation/, 2024. Ημερομηνία πρόσβασης: 31-05-2026.

[172] Apache Software Foundation. *Apache Paimon: Primary Key Table Overview*. https://paimon.apache.org/docs/1.3/primary-key-table/overview/, 2024. Ημερομηνία πρόσβασης: 15-03-2026.

[173] Apache Software Foundation. *Apache Paimon: Append Table Overview*. https://paimon.apache.org/docs/1.3/append-table/overview/, 2024. Ημερομηνία πρόσβασης: 20-03-2026.

[174] A. Thusoo, J. S. Sarma, N. Jain, Z. Shao, P. Chakka, S. Anthony, H. Liu, P. Wyckoff και R. Murthy. *Hive: A Warehousing Solution over a Map-Reduce Framework*. *Proceedings of the VLDB Endowment*, τόμος 2, σελίδες 1626–1629, 2009. doi: https://doi.org/10.14778/1687553.1687609.

[175] Apache Software Foundation. *Apache Paimon: Merge Engine Overview*. https://paimon.apache.org/docs/1.3/primary-key-table/merge-engine/overview/, 2024. Ημερομηνία πρόσβασης: 20-03-2026.

[176] Apache Software Foundation. *Apache Paimon: Changelog Producer*. https://paimon.apache.org/docs/1.3/primary-key-table/changelog-producer/, 2024. Ημερομηνία πρόσβασης: 20-03-2026.

[177] StarRocks. *StarRocks: The World's Fastest Open Query Engine for Full Scenario Real-Time Analytics*. https://www.starrocks.io/, 2024. Ημερομηνία πρόσβασης: 30-05-2026.

[178] Apache Software Foundation. *Apache Doris: A Modern MPP Analytical Database Product*. https://doris.apache.org/, 2024. Ημερομηνία πρόσβασης: 30-05-2026.

[179] Apache Software Foundation. *Apache Paimon: Basic Concepts*. https://paimon.apache.org/docs/1.3/concepts/basic-concepts/, 2024. Ημερομηνία πρόσβασης: 15-03-2026.

[180] Ecma International. *ECMA-404: The JSON Data Interchange Standard*. Τεχνική Αναφορά με αριθμό ECMA-404, Ecma International, 2017.

[181] Apache Software Foundation. *Apache Paimon: Understand Files*. https://paimon.apache.org/docs/1.3/learn-paimon/understand-files/, 2024. Ημερομηνία πρόσβασης: 20-03-2026.

[182] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest και Clifford Stein. *Introduction to Algorithms*. MIT Press, 3η έκδοση, 2009.

[183] Konstantin Shvachko, Hairong Kuang, Sanjay Radia και Robert Chansler. *The Hadoop Distributed File System*. *Proceedings of the 2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)*, σελίδες 1–10. IEEE, 2010. doi: https://doi.org/10.1109/MSST.2010.5496972.

[184] Apache Software Foundation. *Apache Paimon: Consistency Guarantees and Concurrency Control*. https://paimon.apache.org/docs/1.3/concepts/basic-concepts/#consistency-guarantees, 2024. Ημερομηνία πρόσβασης: 15-03-2026.

[185] Oracle. *JDBC Overview*. https://www.oracle.com/java/technologies/javase/javase-tech-database.html, 2024. Ημερομηνία πρόσβασης: 02-07-2026.

[186] Apache Software Foundation. *Apache Paimon: SQL DDL and Catalog*. https://paimon.apache.org/docs/1.3/flink/sql-ddl/, 2024. Ημερομηνία πρόσβασης: 15-03-2026.

[187] Oracle. *MySQL 8.0 Reference Manual*. https://dev.mysql.com/doc/refman/8.0/en/, 2024. Ημερομηνία πρόσβασης: 30-05-2026.

[188] Charity Majors, Liz Fong-Jones και George Miranda. *Observability Engineering: Achieving Production Excellence*. O'Reilly Media, 2022.

[189] M. Usman, S. Ferlin, A. Brunström και J. Taheri. *A Survey on Observability of Distributed Edge & Container-Based Microservices*. *IEEE Access*, 10:86904–86919, 2022. doi: https://doi.org/10.1109/ACCESS.2022.3193102.

[190] S. Niedermaier, F. Koetter, A. Freymann και S. Wagner. *On Observability and Monitoring of Distributed Systems: An Industry Interview Study*. *Proceedings of the International Conference on Service-Oriented Computing (ICSOC 2019) Workshops*, τόμος 12019 στο *LNCS*, σελίδες 36–52. Springer, 2019. doi: https://doi.org/10.1007/978-3-030-33702-5_3.

[191] Google. *Error Budget Policy, The Site Reliability Workbook*. https://sre.google/workbook/error-budget-policy/, 2024. Ημερομηνία πρόσβασης: 01-06-2026.

[192] B. H. Sigelman, L. A. Barroso, M. Burrows, P. Stephenson, M. Plakal, D. Beaver, S. Jaspan και C. Shanbhag. *Dapper, a Large-Scale Distributed Systems Tracing Infrastructure*. Τεχνική Αναφορά με αριθμό, Google, 2010. https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/.

[193] OpenZipkin. *Zipkin: A Distributed Tracing System*. https://zipkin.io/, 2024. Ημερομηνία πρόσβασης: 31-05-2026.

[194] Cloud Native Computing Foundation. *Jaeger: Open Source, End-to-End Distributed Tracing*. https://www.jaegertracing.io/, 2024. Ημερομηνία πρόσβασης: 31-05-2026.

[195] Cloud Native Computing Foundation. *OpenTelemetry: High-Quality, Ubiquitous, and Portable Telemetry to Enable Effective Observability*. https://opentelemetry.io/docs/, 2024. Ημερομηνία πρόσβασης: 31-05-2026.

[196] James Turnbull. *Monitoring with Prometheus*. Turnbull Press, Brooklyn, NY, 2018.

[197] Etsy. *StatsD: Simple Daemon for Easy Stats Aggregation*. https://github.com/statsd/statsd, 2024. Ημερομηνία πρόσβασης: 31-05-2026.

[198] Prometheus Authors. *Prometheus: Monitoring System and Time Series Database*. https://prometheus.io/docs/, 2024. Ημερομηνία πρόσβασης: 15-01-2026.

[199] Prometheus Authors. *Prometheus: Overview*. https://prometheus.io/docs/introduction/overview/, 2024. Ημερομηνία πρόσβασης: 15-01-2026.

[200] T. Pelkonen, S. Franklin, P. Cavallaro, Q. Huang, J. Meza, J. Teller και K. Veeraraghavan. *Gorilla: A Fast, Scalable, In-Memory Time Series Database*. *Proceedings of the VLDB Endowment*, 8(12):1816–1827, 2015. doi: https://doi.org/10.14778/2824032.2824078.

[201] Prometheus Authors. *Prometheus: Metric Types*. https://prometheus.io/docs/concepts/metric_types/, 2024. Ημερομηνία πρόσβασης: 15-01-2026.

[202] Prometheus Authors. *Querying Prometheus: PromQL Basics*. https://prometheus.io/docs/prometheus/latest/querying/basics/, 2024. Ημερομηνία πρόσβασης: 31-05-2026.

[203] Cloud Native Computing Foundation. *Helm: The Package Manager for Kubernetes*. https://helm.sh/docs/, 2024. Ημερομηνία πρόσβασης: 31-05-2026.

[204] Oracle. *Java Management Extensions (JMX) Technology Overview*. https://docs.oracle.com/javase/8/docs/technotes/guides/jmx/overview.html, 2024. Ημερομηνία πρόσβασης: 31-05-2026.

[205] R. Sethi, M. Traverso, D. Sundstrom, D. Phillips, W. Xie, Y. Sun, N. Yegitbasi, H. Jin, E. Hwang, N. Shingte και C. Berner. *Presto: SQL on Everything*. *Proceedings of the 35th IEEE International Conference on Data Engineering (ICDE)*, σελίδες 1802–1813. IEEE, 2019. doi: https://doi.org/10.1109/ICDE.2019.00196.

[206] G. Graefe. *Query Evaluation Techniques for Large Databases*. *ACM Computing Surveys*, τόμος 25, σελίδες 73–169. ACM, 1993. doi: https://doi.org/10.1145/152610.152611.

[207] A. Sarikaya, M. Correll, L. Bartram, M. Tory και D. Fisher. *What Do We Talk About When We Talk About Dashboards?* *IEEE Transactions on Visualization and Computer Graphics*, 25(1):682–692, 2019. doi: https://doi.org/10.1109/TVCG.2018.2864903.

[208] Microsoft. *Open Database Connectivity (ODBC)*. https://learn.microsoft.com/en-us/sql/odbc/reference/odbc-overview, 2024. Ημερομηνία πρόσβασης: 08-07-2026.

[209] Software Freedom Conservancy. *Git: Distributed Version Control System*. https://git-scm.com/, 2024. Ημερομηνία πρόσβασης: 31-05-2026.

[210] Microsoft. *Microsoft Learn: Build Skills that Open Doors in your Career*. https://learn.microsoft.com/, 2024. Ημερομηνία πρόσβασης: 31-05-2026.

[211] Microsoft. *Baseline Architecture for an Azure Kubernetes Service (AKS) Cluster*. https://learn.microsoft.com/azure/architecture/reference-architectures/containers/aks/baseline-aks, 2025. Ημερομηνία πρόσβασης: 15-03-2026.

[212] Microsoft. *What is Microsoft Entra ID?* https://learn.microsoft.com/azure/active-directory/fundamentals/whatis, 2025. Ημερομηνία πρόσβασης: 17-04-2026.

[213] Microsoft. *Azure Subscriptions, Licenses, Accounts, and Tenants*. https://learn.microsoft.com/azure/cloud-adoption-framework/ready/landing-zone/design-area/azure-billing-ad-tenant, 2025. Ημερομηνία πρόσβασης: 17-04-2026.

[214] Microsoft. *Core Concepts for Azure Kubernetes Service (AKS)*. https://learn.microsoft.com/azure/aks/core-aks-concepts, 2025. Ημερομηνία πρόσβασης: 15-03-2026.

[215] Microsoft. *Manage System Node Pools in Azure Kubernetes Service (AKS)*. https://learn.microsoft.com/azure/aks/use-system-pools, 2025. Ημερομηνία πρόσβασης: 15-03-2026.

[216] Cloud Native Computing Foundation. *Using CoreDNS for Service Discovery: Kubernetes Documentation*. https://kubernetes.io/docs/tasks/administer-cluster/coredns/, 2024. Ημερομηνία πρόσβασης: 02-06-2026.

[217] Microsoft. *Azure Data Lake Storage Hierarchical Namespace*. https://learn.microsoft.com/azure/storage/blobs/data-lake-storage-namespace, 2025. Ημερομηνία πρόσβασης: 15-03-2026.

[218] Microsoft. *Access Tiers for Blob Data*. https://learn.microsoft.com/azure/storage/blobs/access-tiers-overview, 2025. Ημερομηνία πρόσβασης: 17-04-2026.

[219] Microsoft. *Use Container Storage Interface (CSI) Drivers on Azure Kubernetes Service (AKS)*. https://learn.microsoft.com/azure/aks/csi-storage-drivers, 2025. Ημερομηνία πρόσβασης: 15-03-2026.

[220] Amazon Web Services. *AWS Documentation*. https://docs.aws.amazon.com/, 2024. Ημερομηνία πρόσβασης: 16-06-2026.

[221] Google. *Google Cloud Documentation*. https://cloud.google.com/docs, 2024. Ημερομηνία πρόσβασης: 16-06-2026.

[222] Microsoft. *Azure Pricing Calculator*. https://azure.microsoft.com/pricing/calculator/, 2024. Ημερομηνία πρόσβασης: 31-05-2026.

[223] Amazon Web Services. *AWS Pricing Calculator*. https://calculator.aws/, 2024. Ημερομηνία πρόσβασης: 16-06-2026.

[224] Google. *Google Cloud Pricing Calculator*. https://cloud.google.com/products/calculator, 2024. Ημερομηνία πρόσβασης: 16-06-2026.

[225] Oracle. *The Java Virtual Machine Specification, Java SE 21 Edition*. https://docs.oracle.com/javase/specs/jvms/se21/html/index.html, 2024. Ημερομηνία πρόσβασης: 08-07-2026.

[226] HashiCorp. *State: Locking - Terraform Documentation*. https://developer.hashicorp.com/terraform/language/state/locking, 2024. Ημερομηνία πρόσβασης: 02-06-2026.

[227] Cloud Native Computing Foundation. *Resource Management for Pods and Containers, Kubernetes Documentation*. https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/, 2024. Ημερομηνία πρόσβασης: 14-06-2026.

[228] The Go Authors. *The Go Programming Language*. https://go.dev/, 2024. Ημερομηνία πρόσβασης: 31-05-2026.

[229] Trino Software Foundation. *Trino Release 440*. https://github.com/trinodb/trino/releases/tag/440, 2024. Ημερομηνία πρόσβασης: 02-06-2026.

[230] Google. *Guava: Google Core Libraries for Java*. https://github.com/google/guava, 2024. Ημερομηνία πρόσβασης: 02-06-2026.

[231] Apache Paimon Community. *Paimon Trino 440: HDFS Classpath Verification and Missing Dependencies*. https://github.com/apache/paimon-trino/issues/96, 2025. GitHub Issue #96, Ημερομηνία πρόσβασης: 07-04-2026.

[232] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen και V. Markl. *Benchmarking Distributed Stream Data Processing Systems*. *Proceedings of the 34th IEEE International Conference on Data Engineering (ICDE)*, σελίδες 1507–1518. IEEE, 2018. doi: https://doi.org/10.1109/ICDE.2018.00169.

[233] R. Nambiar, M. Poess, A. Masland, B. Janneck και D. Lemire. *Introducing TPCx-IoT: A New Benchmark for IoT Gateway Systems*. *Proceedings of the TPC Technology Conference on Performance Evaluation and Benchmarking (TPCTC)*, σελίδες 1–14. Springer, 2018. doi: https://doi.org/10.1007/978-3-030-11404-6_1.

[234] E. Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446, Internet Engineering Task Force (IETF), 2018. doi: https://doi.org/10.17487/RFC8446.

[235] Cloud Native Computing Foundation. *Scheduling, Preemption and Eviction, Kubernetes Documentation*. https://kubernetes.io/docs/concepts/scheduling-eviction/, 2024. Ημερομηνία πρόσβασης: 14-06-2026.

[236] Microsoft. *About Azure Key Vault*. https://learn.microsoft.com/azure/key-vault/general/overview, 2025. Ημερομηνία πρόσβασης: 08-07-2026.

[237] Microsoft. *Use the Azure Key Vault Provider for Secrets Store CSI Driver in an Azure Kubernetes Service (AKS) Cluster*. https://learn.microsoft.com/azure/aks/csi-secrets-store-driver, 2025. Ημερομηνία πρόσβασης: 08-07-2026.

[238] S. Josefsson. *The Base16, Base32, and Base64 Data Encodings*. RFC 4648, Internet Engineering Task Force (IETF), 2006. doi: https://doi.org/10.17487/RFC4648.

[239] Microsoft. *Managed Identities for Azure Resources*. https://learn.microsoft.com/entra/identity/managed-identities-azure-resources/overview, 2025. Ημερομηνία πρόσβασης: 08-07-2026.

[240] Microsoft. *What is Azure Role-Based Access Control (Azure RBAC)?* https://learn.microsoft.com/azure/role-based-access-control/overview, 2025. Ημερομηνία πρόσβασης: 08-07-2026.

[241] The Kubernetes Authors. *Ingress: Kubernetes Documentation*. https://kubernetes.io/docs/concepts/services-networking/ingress/, 2024. Ημερομηνία πρόσβασης: 08-07-2026.

[242] The Kubernetes Authors. *NGINX Ingress Controller Documentation*. https://kubernetes.github.io/ingress-nginx/, 2024. Ημερομηνία πρόσβασης: 08-07-2026.

[243] The Kubernetes Authors. *Network Policies: Kubernetes Documentation*. https://kubernetes.io/docs/concepts/services-networking/network-policies/, 2024. Ημερομηνία πρόσβασης: 08-07-2026.

[244] Microsoft. *What is Azure Web Application Firewall on Azure Application Gateway?* https://learn.microsoft.com/azure/web-application-firewall/ag/ag-overview, 2025. Ημερομηνία πρόσβασης: 08-07-2026.

[245] OWASP Foundation. *OWASP Top 10: The Ten Most Critical Web Application Security Risks*. https://owasp.org/www-project-top-ten/, 2021. Ημερομηνία πρόσβασης: 31-05-2026.

[246] The KEDA Authors. *KEDA: Kubernetes Event-driven Autoscaling*. https://keda.sh/docs/latest/concepts/, 2024. Ημερομηνία πρόσβασης: 08-07-2026.

[247] Apache Software Foundation. *Apache Flink Kubernetes Operator, Autoscaler*. https://nightlies.apache.org/flink/flink-kubernetes-operator-docs-stable/docs/custom-resource/autoscaler/, 2024. Ημερομηνία πρόσβασης: 08-07-2026.

[248] Slack Technologies. *Sending Messages Using Incoming Webhooks, Slack API Documentation*. https://api.slack.com/messaging/webhooks, 2024. Ημερομηνία πρόσβασης: 08-07-2026.

[249] Microsoft. *Microsoft Teams Documentation*. https://learn.microsoft.com/en-us/microsoftteams/, 2024. Ημερομηνία πρόσβασης: 08-07-2026.

[250] Microsoft. *Workload Identity Federation*. https://learn.microsoft.com/entra/workload-id/workload-identity-federation, 2025. Ημερομηνία πρόσβασης: 08-07-2026.

[251] Confluent. *MQTT Source and Sink Connector for Confluent Platform*. https://docs.confluent.io/kafka-connectors/mqtt/current/overview.html, 2024. Ημερομηνία πρόσβασης: 08-07-2026.

[252] Microsoft. *Azure IoT Hub Documentation*. https://learn.microsoft.com/azure/iot-hub/, 2024. Ημερομηνία πρόσβασης: 08-07-2026.

[253] Microsoft. *What is Microsoft Fabric?* https://learn.microsoft.com/fabric/fundamentals/microsoft-fabric-overview, 2026. Ημερομηνία πρόσβασης: 13-06-2026.

[254] Microsoft. *OneLake, the OneDrive for Data: Microsoft Fabric Documentation*. https://learn.microsoft.com/fabric/onelake/onelake-overview, 2026. Ημερομηνία πρόσβασης: 13-06-2026.

[255] Microsoft. *What is a Lakehouse in Microsoft Fabric?* https://learn.microsoft.com/fabric/data-engineering/lakehouse-overview, 2026. Ημερομηνία πρόσβασης: 13-06-2026.

[256] Microsoft. *What is Real-Time Intelligence in Microsoft Fabric?* https://learn.microsoft.com/fabric/real-time-intelligence/overview, 2026. Ημερομηνία πρόσβασης: 08-07-2026.

[257] Microsoft. *Overview of Microsoft Fabric Eventstreams*. https://learn.microsoft.com/fabric/real-time-intelligence/event-streams/overview, 2026. Ημερομηνία πρόσβασης: 13-06-2026.

[258] Microsoft. *Microsoft Fabric Capacity and Licenses*. https://learn.microsoft.com/fabric/enterprise/licenses, 2026. Ημερομηνία πρόσβασης: 08-07-2026.

[259] Microsoft. *OneLake Shortcuts: Microsoft Fabric Documentation*. https://learn.microsoft.com/fabric/onelake/onelake-shortcuts, 2026. Ημερομηνία πρόσβασης: 08-07-2026.

[260] Apache Software Foundation. *Apache Fluss (Incubating): Streaming Storage for Real-Time Analytics*. https://fluss.dev/, 2024. Ημερομηνία πρόσβασης: 19-04-2026.

[261] Microsoft. *Azure Event Hubs: A Big Data Streaming Platform and Event Ingestion Service*. https://learn.microsoft.com/azure/event-hubs/event-hubs-about, 2024. Ημερομηνία πρόσβασης: 08-07-2026.

[262] Microsoft. *What is Azure Synapse Analytics?* https://learn.microsoft.com/azure/synapse-analytics/overview-what-is, 2024. Ημερομηνία πρόσβασης: 08-07-2026.

[263] Theodoros Beretsos. *lakehouse-on-k8s: Πηγαίος κώδικας διπλωματικής εργασίας*. https://github.com/theomper/lakehouse-on-k8s, 2026. GitHub repository, Ημερομηνία πρόσβασης: 08-07-2026.